

1. Public API / ABI

Ada (spec)

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.ads (line 4)

[package] package DS_Buddy_Math with

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.ads (line 17)

[function] function Power_Of_Two (Exponent : Block_Size) return Phys_Addr with

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.ads (line 23)

[function] function Is_Aligned (Addr : Phys_Addr; Size_Exp : Block_Size) return Boolean with

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.ads (line 32)

[function] function Get_Buddy_Address (

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.ads (line 47)

[function] function Get_Left_Child_Address (

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.ads (line 55)

[function] function Get_Right_Child_Address (

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.ads (line 69)

[function] function Safe_Calculate_DMA_End (

kernel_Workspace/kernel/src/policy/ada/policy.ads (line 3)

[package] package Policy

kernel_Workspace/kernel/src/policy/ada/policy.ads (line 21)

[function] function Evaluate (Ring : Ring_Id;

kernel_Workspace/kernel/src/policy/ada/policy_c.ads (line 4)

[package] package Policy_C is

kernel_Workspace/kernel/src/policy/ada/policy_c.ads (line 6)

[function] function policy_evaluate (ring : Unsigned_8;

lib-ada/tg_alloc_buddy.ads (line 21)

[package] package Tg_Alloc_Buddy with SPARK_Mode => On is

lib-ada/tg_alloc_buddy.ads (line 48)

[function] function Order_Size (O : Order_Type) return Natural is (2O);**

lib-ada/tg_alloc_buddy.ads (line 51)

[function] function Aligned (I : Page_Index; O : Order_Type) return Boolean is

lib-ada/tg_alloc_buddy.ads (line 55)

[function] function In_Bounds (I : Page_Index; O : Order_Type) return Boolean is

lib-ada/tg_alloc_buddy.ads (line 59)

[function] function In_Block (I : Page_Index; O : Order_Type; P : Page_Index)

lib-ada/tg_alloc_buddy.ads (line 64)

[function] function Overlap (I1 : Page_Index; O1 : Order_Type;

lib-ada/tg_alloc_buddy.ads (line 73)

[function] function Valid (A : Buddy_Allocator) return Boolean;

lib-ada/tg_alloc_buddy.ads (line 76)

[function] function Used_Disjoint (A : Buddy_Allocator) return Boolean is

lib-ada/tg_alloc_buddy.ads (line 84)

[function] function Used_Pages (A : Buddy_Allocator) return Natural;

lib-ada/tg_alloc_buddy.ads (line 85)

[function] function Free_Pages (A : Buddy_Allocator) return Natural;

lib-ada/tg_alloc_buddy.ads (line 87)

[function] function Pages_Conserved (A : Buddy_Allocator) return Boolean is

lib-ada/tg_alloc_buddy.ads (line 91)

[procedure] procedure Allocate

lib-ada/tg_alloc_buddy.ads (line 110)

[procedure] procedure Free (A : in out Buddy_Allocator; I : Page_Index) with

lib-ada/tg_alloc_buddy.ads (line 117)

[procedure] procedure Lemma_Alloc_Disjoint

lib-ada/tg_alloc_buddy.ads (line 126)

[procedure] procedure Lemma_No_Double_Free

lib-ada/tg_comm.ads (line 12)

[package] package Tg_Comm is

lib-ada/tg_comm.ads (line 57)

[function] function Contains (Set, Subset : Rights) return Boolean is

lib-ada/tg_comm.ads (line 68)

[function] function Opcode (Cls : Op_Class; Op : Interfaces.Unsigned_16)

lib-ada/tg_comm.ads (line 119)

[function] function Tgcomm_Version return Interfaces.Unsigned_32;

lib-ada/tg_comm.ads (line 122)

[function] function Tgcomm_Msg_Size return Interfaces.Unsigned_32;

lib-ada/tg_comm.ads (line 125)

[function] function Tgcomm_Ring_Size (Slots : Interfaces.Unsigned_32)

lib-ada/tg_comm.ads (line 129)

[procedure] procedure Tgcomm_Msg_Init (Msg : in out Comm_Msg);

lib-ada/tg_comm.ads (line 132)

[function] function Tgcomm_Authorize (Table : System.Address; Msg : System.Address)

lib-ada/tg_comm.ads (line 136)

[procedure] procedure Tgcomm_Ring_Init (Ring : System.Address;

lib-ada/tg_comm.ads (line 140)

[function] function Tgcomm_Ring_Push (Ring : System.Address; Msg : System.Address)

lib-ada/tg_comm.ads (line 144)

**[function] function Tgcomm_Ring_Pop (Ring : System.Address; Out_Msg :
System.Address)**

lib-ada/tg_comm.ads (line 148)

[function] function Tgcomm_Ring_Available (Ring : System.Address)

lib-ada/tg_comm.ads (line 152)

[procedure] procedure Tgcomm_Cap_Table_Init (Table : System.Address);

lib-ada/tg_comm.ads (line 155)

[function] function Tgcomm_Cap_Insert (Table : System.Address;

lib-ada/tg_comm.ads (line 163)

[function] function Tgcomm_Cap_Check (Table : System.Address;

lib-ada/tg_comm.ads (line 169)

[function] function Tgcomm_Cap_Remove (Table : System.Address;

lib-ada/tg_comm_security.ads (line 23)

[package] package Tg_Comm_Security with SPARK_Mode => On is

lib-ada/tg_comm_security.ads (line 38)

[function] function Contains (Set, Subset : Rights_Mask) return Boolean is

lib-ada/tg_comm_security.ads (line 44)

[function] function Route_Allowed (From, To : Layer) return Boolean;

lib-ada/tg_comm_security.ads (line 53)

[function] function Matches (Req : Type_Req; Actual : Object_Type) return Boolean is

```

lib-ada/tg_comm_security.ads (line 73)
[function] function Required_Right (O : Opcode) return Rights_Mask;
lib-ada/tg_comm_security.ads (line 74)
[function] function Required_Type (O : Opcode) return Type_Req;
lib-ada/tg_comm_security.ads (line 97)
[function] function Empty_Table return Cap_Table is
lib-ada/tg_comm_security.ads (line 100)
[function] function Has_Cap (T : Cap_Table; C : Natural) return Boolean is
lib-ada/tg_comm_security.ads (line 103)
[function] function Rights_Of (T : Cap_Table; C : Natural) return Rights_Mask with
lib-ada/tg_comm_security.ads (line 106)
[function] function Type_Of (T : Cap_Table; C : Natural) return Object_Type with
lib-ada/tg_comm_security.ads (line 110)
[function] function Authorized (T : Cap_Table; M : Msg) return Boolean is
lib-ada/tg_comm_security.ads (line 121)
[function] function Authorize (T : Cap_Table; M : Msg) return Verdict with
lib-ada/tg_comm_security.ads (line 125)
[function] function Revokes (T, U : Cap_Table; C : Cap_Id) return Boolean is
lib-ada/tg_comm_security.ads (line 130)
[procedure] procedure Lemma_Empty_Table_Denies_All (M : Msg) with
lib-ada/tg_comm_security.ads (line 135)
[procedure] procedure Lemma_Revocation_Monotonic

```

Assembly

```

comgrub/src/boot.asm (line 30)
[label] global _start
dirtycode/mesa/src/util/blake3/blake3_avx2_x86-64_unix.S (line 21)
[label] .global _blake3_hash_many_avx2
dirtycode/mesa/src/util/blake3/blake3_avx2_x86-64_unix.S (line 22)
[label] .global blake3_hash_many_avx2
dirtycode/mesa/src/util/blake3/blake3_avx2_x86-64_windows_gnu.S (line 2)
[label] .global _blake3_hash_many_avx2
dirtycode/mesa/src/util/blake3/blake3_avx2_x86-64_windows_gnu.S (line 3)
[label] .global blake3_hash_many_avx2
dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_unix.S (line 26)
[label] .global _blake3_hash_many_avx512
dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_unix.S (line 27)
[label] .global blake3_hash_many_avx512
dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_unix.S (line 28)
[label] .global blake3_compress_in_place_avx512
dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_unix.S (line 29)
[label] .global _blake3_compress_in_place_avx512
dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_unix.S (line 30)
[label] .global blake3_compress_xof_avx512
dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_unix.S (line 31)
[label] .global _blake3_compress_xof_avx512
dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_unix.S (line 32)
[label] .global blake3_xof_many_avx512

```

dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_unix.S (line 33)
[label] .global _blake3_xof_many_avx512
dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_windows_gnu.S (line 3)
[label] .global _blake3_hash_many_avx512
dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_windows_gnu.S (line 4)
[label] .global blake3_hash_many_avx512
dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_windows_gnu.S (line 5)
[label] .global blake3_compress_in_place_avx512
dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_windows_gnu.S (line 6)
[label] .global _blake3_compress_in_place_avx512
dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_windows_gnu.S (line 7)
[label] .global blake3_compress_xof_avx512
dirtycode/mesa/src/util/blake3/blake3_avx512_x86-64_windows_gnu.S (line 8)
[label] .global _blake3_compress_xof_avx512
dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_unix.S (line 24)
[label] .global blake3_hash_many_sse2
dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_unix.S (line 25)
[label] .global _blake3_hash_many_sse2
dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_unix.S (line 26)
[label] .global blake3_compress_in_place_sse2
dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_unix.S (line 27)
[label] .global _blake3_compress_in_place_sse2
dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_unix.S (line 28)
[label] .global blake3_compress_xof_sse2
dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_unix.S (line 29)
[label] .global _blake3_compress_xof_sse2
dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_windows_gnu.S (line 2)
[label] .global blake3_hash_many_sse2
dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_windows_gnu.S (line 3)
[label] .global _blake3_hash_many_sse2
dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_windows_gnu.S (line 4)
[label] .global blake3_compress_in_place_sse2
dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_windows_gnu.S (line 5)
[label] .global _blake3_compress_in_place_sse2
dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_windows_gnu.S (line 6)
[label] .global blake3_compress_xof_sse2
dirtycode/mesa/src/util/blake3/blake3_sse2_x86-64_windows_gnu.S (line 7)
[label] .global _blake3_compress_xof_sse2
dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_unix.S (line 24)
[label] .global blake3_hash_many_sse41
dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_unix.S (line 25)
[label] .global _blake3_hash_many_sse41
dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_unix.S (line 26)
[label] .global blake3_compress_in_place_sse41
dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_unix.S (line 27)
[label] .global _blake3_compress_in_place_sse41
dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_unix.S (line 28)
[label] .global blake3_compress_xof_sse41

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_unix.S (line 29)
[label] .global _blake3_compress_xof_sse41

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_windows_gnu.S (line 2)
[label] .global blake3_hash_many_sse41

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_windows_gnu.S (line 3)
[label] .global _blake3_hash_many_sse41

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_windows_gnu.S (line 4)
[label] .global blake3_compress_in_place_sse41

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_windows_gnu.S (line 5)
[label] .global _blake3_compress_in_place_sse41

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_windows_gnu.S (line 6)
[label] .global blake3_compress_xof_sse41

dirtycode/mesa/src/util/blake3/blake3_sse41_x86-64_windows_gnu.S (line 7)
[label] .global _blake3_compress_xof_sse41

kernel_Workspace/kernel-bin/src/boot.asm (line 2)
[label] global kernel_bin_entry

kernel_Workspace/kernel/src/cpu/trampoline.s (line 5)
[label] .global trampoline_start

kernel_Workspace/kernel/src/cpu/trampoline.s (line 6)
[label] .global trampoline_end

kernel_Workspace/kernel/src/cpu/trampoline.s (line 7)
[label] .global trampoline_cr3

kernel_Workspace/kernel/src/cpu/trampoline.s (line 8)
[label] .global trampoline_entry

kernel_Workspace/kernel/src/cpu/trampoline.s (line 9)
[label] .global trampoline_stack

kernel_Workspace/kernel/src/cpu/trampoline.s (line 10)
[label] .global trampoline_arg

kernel_Workspace/kernel/src/cpu/trampoline.s (line 100)
[label] .global trampoline_end

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 3)
[label] .global tlb_asm_invlpg

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 9)
[label] .global tlb_asm_read_cr3

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 15)
[label] .global tlb_asm_write_cr3

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 21)
[label] .global tlb_asm_read_cr4

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 27)
[label] .global tlb_asm_write_cr4

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 34)
[label] .global tlb_asm_invpaid

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 40)
[label] .global tlb_asm_wbinvd

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 46)
[label] .global tlb_asm_clflush

kernel_Workspace/kernel/src/trampoline_rings/arch/aarch64/trampoline_rings.s (line 102)
[label] .global tr_init

```

kernel_Workspace/kernel/src/trampoline_rings/arch/aarch64/trampoline_rings.s (line 111)
[label] .global tr_restore_ctx
kernel_Workspace/kernel/src/trampoline_rings/arch/risc-v/trampoline_rings.s (line 9)
[label] .global trap_entry
kernel_Workspace/kernel/src/trampoline_rings/arch/risc-v/trampoline_rings.s (line 98)
[label] .global tr_init
kernel_Workspace/kernel/src/trampoline_rings/arch/risc-v/trampoline_rings.s (line 114)
[label] .global tr_restore_ctx
kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 3)
[label] .global tr_gdt
kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 12)
[label] .global tr_tss_desc
kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 21)
[label] .global tr_tss
kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 27)
[label] .global tr_init
kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 54)
[label] .global isr_hypercall
kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 94)
[label] .global isr_default
kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 99)
[label] .global tr_restore_ctx
triang-lang/out/x86_64/prog.asm (line 7)
[label] global main

```

C

```

comlimine/src/main.c (line 6)
[declaration] extern void kernel_main(uint64_t magic, void *info);
comlimine/src/main.c (line 28)
[function] void _start(void) {
comlimine/src/main.c (line 35)
[declaration] kernel_main(0, (void *)framebuffer.response);
dirtycode/alsa-lib/aserver/aserver.c (line 101)
[declaration] SYSErrorr("socket failed");
dirtycode/alsa-lib/aserver/aserver.c (line 112)
[declaration] SYSErrorr("bind failed");
dirtycode/alsa-lib/aserver/aserver.c (line 113)
[declaration] close(sock);
dirtycode/alsa-lib/aserver/aserver.c (line 128)
[declaration] SYSErrorr("socket failed");
dirtycode/alsa-lib/aserver/aserver.c (line 139)
[declaration] SYSErrorr("bind failed");
dirtycode/alsa-lib/aserver/aserver.c (line 140)
[declaration] close(sock);
dirtycode/alsa-lib/aserver/aserver.c (line 150)
[declaration] typedef int (*waiter_handler_t)(waiter_t *waiter, unsigned short events);
dirtycode/alsa-lib/aserver/aserver.c (line 262)
[function] if (n != 1) {

```

```

dirtycode/alsa-lib/aserver/aserver.c (line 263)
[declaration] SYSERROR("write failed");
dirtycode/alsa-lib/aserver/aserver.c (line 268)
[function] if (n != 1) {
dirtycode/alsa-lib/aserver/aserver.c (line 269)
[declaration] SYSERROR("read failed");
dirtycode/alsa-lib/aserver/aserver.c (line 336)
[declaration] SYSERROR("shmget failed");
dirtycode/alsa-lib/aserver/aserver.c (line 341)
[declaration] SYSERROR("shmctl IPC_SET failed");
dirtycode/alsa-lib/aserver/aserver.c (line 342)
[declaration] shmctl(shmid, IPC_RMID, 0);
dirtycode/alsa-lib/aserver/aserver.c (line 349)
[declaration] shmctl(shmid, IPC_RMID, 0);
dirtycode/alsa-lib/aserver/aserver.c (line 350)
[declaration] SYSERROR("shmat failed");
dirtycode/alsa-lib/aserver/aserver.c (line 368)
[declaration] del_waiter(client->device.pcm.fd);
dirtycode/alsa-lib/aserver/aserver.c (line 373)
[declaration] ERROR("snd_pcm_close");
dirtycode/alsa-lib/aserver/aserver.c (line 379)
[declaration] SYSERROR("shmdt failed");
dirtycode/alsa-lib/aserver/aserver.c (line 382)
[declaration] SYSERROR("shmctl IPC_RMID failed");
dirtycode/alsa-lib/aserver/aserver.c (line 423)
[declaration] return shm_ack_fd(client, rbptr->fd);
dirtycode/alsa-lib/aserver/aserver.c (line 451)
[function] if (ctrl->u.async.sig >= 0) {
dirtycode/alsa-lib/aserver/aserver.c (line 452)
[declaration] assert(client->async_sig < 0);
dirtycode/alsa-lib/aserver/aserver.c (line 457)
[declaration] assert(client->async_sig >= 0);
dirtycode/alsa-lib/aserver/aserver.c (line 458)
[declaration] snd_async_del_handler(client->async_handler);
dirtycode/alsa-lib/aserver/aserver.c (line 515)
[declaration] return shm_ack_fd(client, ctrl->u.channel_info.u.mmap.fd);
dirtycode/alsa-lib/aserver/aserver.c (line 552)
[declaration] return shm_ack_fd(client, _snd_pcm_poll_descriptor(pcm));
dirtycode/alsa-lib/aserver/aserver.c (line 557)
[declaration] return shm_rbp_ptr_fd(client, &pcm->hw);
dirtycode/alsa-lib/aserver/aserver.c (line 559)
[declaration] return shm_rbp_ptr_fd(client, &pcm->appl);
dirtycode/alsa-lib/aserver/aserver.c (line 561)
[declaration] ERROR("Bogus cmd: %x", ctrl->cmd);
dirtycode/alsa-lib/aserver/aserver.c (line 564)
[declaration] return shm_ack(client);
dirtycode/alsa-lib/aserver/aserver.c (line 580)
[function] if (n != 1) {

```

```

dirtycode/alsa-lib/aserver/aserver.c (line 581)
[declaration] SYSERROR("write failed");
dirtycode/alsa-lib/aserver/aserver.c (line 605)
[declaration] SYSERROR("shmget failed");
dirtycode/alsa-lib/aserver/aserver.c (line 610)
[declaration] SYSERROR("shmctl IPC_SET failed");
dirtycode/alsa-lib/aserver/aserver.c (line 611)
[declaration] shmctl(shmid, IPC_RMID, 0);
dirtycode/alsa-lib/aserver/aserver.c (line 618)
[declaration] shmctl(shmid, IPC_RMID, 0);
dirtycode/alsa-lib/aserver/aserver.c (line 619)
[declaration] SYSERROR("shmat failed");
dirtycode/alsa-lib/aserver/aserver.c (line 638)
[declaration] del_waiter(client->device.ctl.fd);
dirtycode/alsa-lib/aserver/aserver.c (line 643)
[declaration] ERROR("snd_ctl_close");
dirtycode/alsa-lib/aserver/aserver.c (line 649)
[declaration] SYSERROR("shmdt failed");
dirtycode/alsa-lib/aserver/aserver.c (line 652)
[declaration] SYSERROR("shmctl failed");
dirtycode/alsa-lib/aserver/aserver.c (line 677)
[function] if (ctrl->u.async.sig >= 0) {
dirtycode/alsa-lib/aserver/aserver.c (line 678)
[declaration] assert(client->async_sig < 0);
dirtycode/alsa-lib/aserver/aserver.c (line 683)
[declaration] assert(client->async_sig >= 0);
dirtycode/alsa-lib/aserver/aserver.c (line 684)
[declaration] snd_async_del_handler(client->async_handler);
dirtycode/alsa-lib/aserver/aserver.c (line 699)
[function] if (ctrl->u.element_list.space * sizeof(*ctrl->u.element_list.pids) >
maxsize) {
dirtycode/alsa-lib/aserver/aserver.c (line 760)
[declaration] return shm_ack_fd(client, _snd_ctl_poll_descriptor(ctl));
dirtycode/alsa-lib/aserver/aserver.c (line 762)
[declaration] ERROR("Bogus cmd: %x", ctrl->cmd);
dirtycode/alsa-lib/aserver/aserver.c (line 765)
[declaration] return shm_ack(client);
dirtycode/alsa-lib/aserver/aserver.c (line 783)
[declaration] SYSERROR("read failed");
dirtycode/alsa-lib/aserver/aserver.c (line 784)
[declaration] exit(1);
dirtycode/alsa-lib/aserver/aserver.c (line 797)
[declaration] SYSERROR("read failed");
dirtycode/alsa-lib/aserver/aserver.c (line 798)
[declaration] exit(1);
dirtycode/alsa-lib/aserver/aserver.c (line 807)
[function] if (!client->local) {
dirtycode/alsa-lib/aserver/aserver.c (line 811)
[function] switch (req.dev_type) {

```

dirtycode/alsa-lib/aserver/aserver.c (line 850)
[declaration] `SYSError("write failed");`
dirtycode/alsa-lib/aserver/aserver.c (line 851)
[declaration] `exit(1);`
dirtycode/alsa-lib/aserver/aserver.c (line 876)
[declaration] `close(client->ctrl_fd);`
dirtycode/alsa-lib/aserver/aserver.c (line 877)
[declaration] `del_waiter(client->ctrl_fd);`
dirtycode/alsa-lib/aserver/aserver.c (line 878)
[declaration] `list_del(&client->list);`
dirtycode/alsa-lib/aserver/aserver.c (line 879)
[declaration] `free(client);`
dirtycode/alsa-lib/aserver/aserver.c (line 885)
[declaration] `return snd_client_open(client);`
dirtycode/alsa-lib/aserver/aserver.c (line 910)
[declaration] `close(waiter->fd);`
dirtycode/alsa-lib/aserver/aserver.c (line 911)
[declaration] `list_del(&pending->list);`
dirtycode/alsa-lib/aserver/aserver.c (line 912)
[declaration] `free(pending);`
dirtycode/alsa-lib/aserver/aserver.c (line 946)
[declaration] `SYSError("accept failed");`
dirtycode/alsa-lib/aserver/aserver.c (line 953)
[declaration] `add_waiter(sock, POLLIN | POLLHUP, client_ctrl_handler, client);`
dirtycode/alsa-lib/aserver/aserver.c (line 954)
[declaration] `list_add_tail(&client->list, &clients);`
dirtycode/alsa-lib/aserver/aserver.c (line 965)
[declaration] `SYSError("accept failed");`
dirtycode/alsa-lib/aserver/aserver.c (line 971)
[declaration] `add_waiter(sock, POLLIN, inet_pending_handler, pending);`
dirtycode/alsa-lib/aserver/aserver.c (line 972)
[declaration] `list_add_tail(&pending->list, &inet_pendings);`
dirtycode/alsa-lib/aserver/aserver.c (line 988)
[declaration] `SYSError("sysconf failed");`
dirtycode/alsa-lib/aserver/aserver.c (line 998)
[function] `if (fcntl(sockn, F_SETFL, O_NONBLOCK) < 0) {`
dirtycode/alsa-lib/aserver/aserver.c (line 1000)
[declaration] `SYSError("fcntl O_NONBLOCK failed");`
dirtycode/alsa-lib/aserver/aserver.c (line 1003)
[function] `if (listen(sockn, 4) < 0) {`
dirtycode/alsa-lib/aserver/aserver.c (line 1005)
[declaration] `SYSError("listen failed");`
dirtycode/alsa-lib/aserver/aserver.c (line 1008)
[declaration] `add_waiter(sockn, POLLIN, local_handler, NULL);`
dirtycode/alsa-lib/aserver/aserver.c (line 1014)
[function] `if (fcntl(socki, F_SETFL, O_NONBLOCK) < 0) {`
dirtycode/alsa-lib/aserver/aserver.c (line 1016)
[declaration] `SYSError("fcntl failed");`

```

dirtycode/alsa-lib/aserver/aserver.c (line 1019)
[function] if (listen(socki, 4) < 0) {
dirtycode/alsa-lib/aserver/aserver.c (line 1021)
[declaration] SYSERROR("listen failed");
dirtycode/alsa-lib/aserver/aserver.c (line 1024)
[declaration] add_waiter(socki, POLLIN, inet_handler, NULL);
dirtycode/alsa-lib/aserver/aserver.c (line 1033)
[function] if (err < 0) {
dirtycode/alsa-lib/aserver/aserver.c (line 1034)
[declaration] SYSERROR("poll failed");
dirtycode/alsa-lib/aserver/aserver.c (line 1039)
[declaration] memcpy(pfds, pollfds, sizeof(*pfds) * pfds_count);
dirtycode/alsa-lib/aserver/aserver.c (line 1042)
[function] if (pfd->revents) {
dirtycode/alsa-lib/aserver/aserver.c (line 1048)
[declaration] ERROR("waiter handler failed");
dirtycode/alsa-lib/aserver/aserver.c (line 1054)
[declaration] close(sockn);
dirtycode/alsa-lib/aserver/aserver.c (line 1056)
[declaration] close(socki);
dirtycode/alsa-lib/aserver/aserver.c (line 1087)
[function] switch (c) {
dirtycode/alsa-lib/aserver/aserver.c (line 1089)
[declaration] usage();
dirtycode/alsa-lib/aserver/aserver.c (line 1092)
[declaration] fprintf(stderr, "Try '%s --help' for more information\n", command);
dirtycode/alsa-lib/aserver/aserver.c (line 1097)
[declaration] ERROR("you need to specify server name");
dirtycode/alsa-lib/aserver/aserver.c (line 1102)
[declaration] ERROR("cannot read configuration file");
dirtycode/alsa-lib/aserver/aserver.c (line 1108)
[declaration] ERROR("Missing definition for server %s", srvname);
dirtycode/alsa-lib/aserver/aserver.c (line 1112)
[declaration] snd_error(ASERVER, "Invalid type for server %s definition", srvname);
dirtycode/alsa-lib/aserver/aserver.c (line 1124)
[function] if (strcmp(id, "socket") == 0) {
dirtycode/alsa-lib/aserver/aserver.c (line 1126)
[function] if (err < 0) {
dirtycode/alsa-lib/aserver/aserver.c (line 1127)
[declaration] ERROR("Invalid type for %s", id);
dirtycode/alsa-lib/aserver/aserver.c (line 1132)
[function] if (strcmp(id, "port") == 0) {
dirtycode/alsa-lib/aserver/aserver.c (line 1134)
[function] if (err < 0) {
dirtycode/alsa-lib/aserver/aserver.c (line 1135)
[declaration] ERROR("Invalid type for %s", id);
dirtycode/alsa-lib/aserver/aserver.c (line 1140)
[function] if (strcmp(id, "gid") == 0) {

```

```

dirtycode/alsa-lib/aserver/aserver.c (line 1143)
[function] if (err < 0) {
dirtycode/alsa-lib/aserver/aserver.c (line 1144)
[declaration] ERROR("Invalid type for %s", id);
dirtycode/alsa-lib/aserver/aserver.c (line 1147)
[function] if (*group && parse_gid(group, &shm_gid) < 0) {
dirtycode/alsa-lib/aserver/aserver.c (line 1148)
[declaration] ERROR("unknown group: %s", group);
dirtycode/alsa-lib/aserver/aserver.c (line 1149)
[declaration] free(group);
dirtycode/alsa-lib/aserver/aserver.c (line 1152)
[declaration] free(group);
dirtycode/alsa-lib/aserver/aserver.c (line 1155)
[declaration] ERROR("Unknown field %s", id);
dirtycode/alsa-lib/aserver/aserver.c (line 1159)
[declaration] ERROR("either socket or port need to be defined");
dirtycode/alsa-lib/aserver/aserver.c (line 1164)
[function] if (!grp) {
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 86)
[declaration] return PyString_InternFromString(name);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 88)
[declaration] return PyUnicode_InternFromString(name);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 99)
[declaration] PyErr_Format(PyExc_TypeError, "missing '%s' attribute", attr);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 103)
[declaration] PyErr_Format(PyExc_TypeError, "'%s' attribute is not Int or Long",
attr);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 123)
[declaration] PyErr_Format(PyExc_TypeError, "missing '%s' attribute", attr);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 124)
[declaration] PyErr_Print();
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 125)
[declaration] PyErr_Clear();
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 126)
[declaration] Py_DECREF(args);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 132)
[declaration] PyErr_Print();
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 133)
[declaration] PyErr_Clear();
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 151)
[declaration] PyErr_Format(PyExc_TypeError, "wrong result from '%s'!", attr);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 152)
[declaration] PyErr_Print();
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 153)
[declaration] PyErr_Clear();
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 154)
[declaration] Py_DECREF(res);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 185)
[declaration] PyTuple_SET_ITEM(obj1, 0, PyInt_FromLong(dir));

```

```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 187)
[declaration] PyTuple_SET_ITEM(obj1, 1, PyInt_FromLong(val));
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 206)
[function] if (PyLong_Check(t1) && PyLong_Check(t2)) {
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 217)
[declaration] PyErr_Format(PyExc_TypeError, "wrong result (invalid tuple)");
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 218)
[declaration] PyErr_Print();
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 219)
[declaration] PyErr_Clear();
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 230)
[declaration] return get_x_range_ops(elem, dir, min, max, "opsGetRange");
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 243)
[declaration] return pcall(pymelem, "opsGetRange", obj1, NULL);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 260)
[function] if (PyLong_Check(t1)) {
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 269)
[declaration] PyErr_Format(PyExc_TypeError, "wrong result (invalid tuple)");
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 270)
[declaration] PyErr_Print();
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 271)
[declaration] PyErr_Clear();
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 282)
[declaration] return get_x_ops(elem, dir, channel, value, "opsGetVolume");
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 300)
[declaration] return get_x_ops(elem, dir, value, dbValue, "opsGetVolDB");
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 320)
[function] if (PyLong_Check(t1)) {
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 329)
[declaration] PyErr_Format(PyExc_TypeError, "wrong result (invalid tuple)");
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 330)
[declaration] PyErr_Print();
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 331)
[declaration] PyErr_Clear();
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 344)
[declaration] return get_x_ops(elem, dir, channel, value, "opsGetDB");
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 350)
[declaration] return get_x_range_ops(elem, dir, min, max, "opsGetDBRange");
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 363)
[declaration] return pcall(pymelem, "opsSetVolume", obj1, NULL);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 376)
[declaration] return pcall(pymelem, "opsSetSwitch", obj1, NULL);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 391)
[declaration] return pcall(pymelem, "opsSetDB", obj1, NULL);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 409)
[function] if (PyUnicode_Check(t1)) {
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 411)
[function] if (obj2) {

```

```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 416)
[declaration] memcpy(buf, s, len);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 418)
[declaration] Py_DECREF(obj2);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 428)
[declaration] memcpy(buf, s, len);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 433)
[declaration] PyErr_Format(PyExc_TypeError, "wrong result (invalid tuple)");
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 434)
[declaration] PyErr_Print();
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 435)
[declaration] PyErr_Clear();
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 456)
[function] if (PyLong_Check(t1)) {
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 465)
[declaration] PyErr_Format(PyExc_TypeError, "wrong result (invalid tuple)");
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 466)
[declaration] PyErr_Print();
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 467)
[declaration] PyErr_Clear();
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 485)
[declaration] return pcall(pymelem, "opsSetEnumItem", obj1, NULL);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 511)
[declaration] snd_mixer_selem_id_free(simple->id);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 519)
[declaration] return PyInt_FromLong((long)priv);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 525)
[declaration] return PyInt_FromLong(pymelem->selem.caps);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 531)
[declaration] return
PyUnicode_FromString(snd_mixer_selem_id_get_name(pymelem->selem.id));
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 537)
[declaration] return
PyInt_FromLong(snd_mixer_selem_id_get_index(pymelem->selem.id));
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 572)
[declaration] return PyInt_FromLong(-EIO);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 589)
[declaration] PyErr_Format(PyExc_RuntimeError, "Cannot attach hcontrol element to
mixer element: %s", snd_strerror(err));
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 609)
[declaration] PyErr_Format(PyExc_RuntimeError, "Cannot detach hcontrol element to
mixer element: %s", snd_strerror(err));
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 620)
[declaration] return PyInt_FromLong(snd_mixer_elem_info(pymelem->melem));
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 628)
[declaration] return PyInt_FromLong(snd_mixer_elem_value(pymelem->melem));
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 651)
[declaration] snd_mixer_selem_id_free(id);

```

```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 735)
[declaration] PyErr_Format(PyExc_RuntimeError, "Cannot attach hctl: %s",
snd_strerror(err));
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 740)
[declaration] PyErr_SetString(PyExc_RuntimeError, "No enough memory");
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 760)
[declaration] PyErr_Format(PyExc_RuntimeError, "Cannot register mixer: %s",
snd_strerror(err));
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 778)
[declaration] PyTuple_SET_ITEM(obj1, 0, (PyObject *)pymixer);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 779)
[declaration] Py_INCREF((PyObject *)pymixer);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 780)
[declaration] PyTuple_SET_ITEM(obj1, 1, PyUnicode_FromString(name));
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 781)
[declaration] PyTuple_SET_ITEM(obj1, 2, PyInt_FromLong(index));
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 782)
[declaration] PyTuple_SET_ITEM(obj1, 3, PyInt_FromLong(weight));
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 784)
[declaration] Py_XDECREF(obj1);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 785)
[function] if (obj2) {
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 788)
[function] if (melems == NULL) {
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 789)
[declaration] Py_DECREF(obj2);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 794)
[declaration] Py_INCREF(obj2);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 799)
[declaration] PyErr_Format(PyExc_RuntimeError, "Cannot find class '%s'", class);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 817)
[declaration] PyErr_Format(PyExc_RuntimeError, "Cannot add mixer element: %s",
snd_strerror(err));
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 847)
[declaration] snd_mixer_detach_hctl(self->mixer, self->hctl[idx*2]);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 848)
[declaration] Py_DECREF((PyObject *)self->hctl[idx*2+1]);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 851)
[declaration] free(self->hctl);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 855)
[declaration] Py_DECREF((PyObject *)self->helem[idx*2+1]);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 857)
[declaration] free(self->helem);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 861)
[declaration] Py_DECREF((PyObject *)self->melem[idx*2+1]);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 863)
[declaration] free(self->melem);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 913)
[function] if (pymixer->hctl[idx] == hctl) {

```

```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 923)
[declaration] PyTuple_SET_ITEM(obj1, 0, py_hctl);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 924)
[declaration] Py_INCREF(py_hctl);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 925)
[declaration] PyTuple_SET_ITEM(obj1, 1, PyFloat_FromDouble(1));
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 926)
[declaration] PyTuple_SET_ITEM(obj1, 2, PyInt_FromLong((long)helem));
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 928)
[function] if (obj2 == NULL) {
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 929)
[declaration] PyErr_Print();
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 930)
[declaration] PyErr_Clear();
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 932)
[declaration] Py_XDECREF(obj1);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 934)
[declaration] snd_error(MIXER, "Unable to create InternalMixer object");
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 940)
[function] if (helems == NULL) {
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 941)
[declaration] Py_DECREF(obj2);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 946)
[declaration] Py_INCREF(obj2);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 990)
[declaration] PyTuple_SET_ITEM(t, 0, (PyObject *)PyInt_FromLong(mask));
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 992)
[function] if (mask & SND_CTL_EVENT_MASK_ADD) {
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 998)
[declaration] PyTuple_SET_ITEM(t, 1, o);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 999)
[declaration] Py_INCREF(o);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1001)
[declaration] PyTuple_SET_ITEM(t, 2, o);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1002)
[declaration] Py_INCREF(o);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1004)
[declaration] Py_DECREF(t);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1005)
[function] if (r) {
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1006)
[function] if (PyLong_Check(r)) {
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1015)
[declaration] Py_DECREF(r);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1017)
[declaration] PyErr_Print();
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1018)
[declaration] PyErr_Clear();

```

```

dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1031)
[declaration] pymixer_free((struct pymixer *)priv->py_mixer);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1032)
[declaration] Py_DECREF(priv->py_mixer);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1035)
[declaration] Py_XDECREF(priv->py_event_func);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1036)
[declaration] Py_Finalize();
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1054)
[declaration] PyDict_SetItemString(mdict, "__file__", obj);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1058)
[declaration] PyDict_SetItemString(mdict, "device", obj);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1067)
[declaration] PyTuple_SET_ITEM(obj1, 0, PyInt_FromLong((long)class));
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1068)
[declaration] PyTuple_SET_ITEM(obj1, 1, PyInt_FromLong((long)mixer));
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1069)
[declaration] PyTuple_SET_ITEM(obj1, 2, mdict);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1070)
[declaration] Py_INCREF(mdict);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1072)
[declaration] Py_XDECREF(obj1);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1073)
[declaration] PyDict_SetItemString(mdict, "mixer", obj2);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1076)
[declaration] snd_error(MIXER, "Unable to create InternalMixer object");
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1082)
[declaration] PyErr_Print();
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1086)
[declaration] snd_error(MIXER, "Unable to find python function 'event'");
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1125)
[declaration] snd_dlopen(path, sizeof(path), SCRIPT);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1131)
[declaration] snd_error(MIXER, "Unable to find python module '%s'", file);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1137)
[function] PyType_Ready(&pymixer_type) < 0 {
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1138)
[declaration] fclose(fp);
dirtycode/alsa-lib/modules/mixer/simple/python.c (line 1151)
[declaration] alsa_mixer_simple_pyinit(priv, py_mod, fp, file, class, mixer,
device);
dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 37)
[declaration] static int selem_read(snd_mixer_elem_t *elem);
dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 85)
[function] list_for_each(pos, &s->helems) {
dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 87)
[declaration] printf("min = %li, max = %li\n", helem->min, helem->max);
dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 88)
[function] if (helem->caps & mask[dir]) {

```

```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 97)
[function] list_for_each(pos, &s->helems) {
dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 99)
[function] if (helem->caps & gmask[dir]) {
dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 121)
[function] list_for_each(pos, &s->helems) {
dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 308)
[declaration] snd_mixer_selem_id_free(simple->selem.id);
dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 311)
[declaration] free(hsimple);
dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 337)
[function] if (bsid->sids[ui].sid == sel->sid) {
dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 370)
[declaration] snd_mixer_selem_id_free(id);
dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 375)
[function] if (ctype != SND_CTL_ELEM_TYPE_BOOLEAN) {
dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 377)
[declaration] snd_mixer_selem_id_free(id);
dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 378)
[declaration] free(hsimple);
dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 396)
[function] if (!simple) {
dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 397)
[declaration] snd_mixer_selem_id_free(id);
dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 398)
[declaration] free(hsimple);
dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 403)
[declaration] INIT_LIST_HEAD(&simple->helems);
dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 408)
[function] if (err < 0) {
dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 409)
[declaration] snd_mixer_selem_id_free(id);
dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 410)
[declaration] free(hsimple);
dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 411)
[declaration] free(simple);
dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 417)
[declaration] snd_mixer_selem_id_free(id);
dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 450)
[declaration] snd_mixer_elem_free(melem);
dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 470)
[function] if (hsel->iface == iface && !strcmp(hsel->name, name) && hsel->index ==
index) {
dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 485)
[declaration] return simple_event_remove(helem, melem);
dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 504)
[function] if (err) {
dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 522)
[declaration] free(sel);

```

```

dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 526)
[declaration] free(sid);
dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 551)
[function] if (priv == NULL) {
dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 552)
[declaration] free(sel);
dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 573)
[function] if (priv == NULL) {
dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 574)
[declaration] free(sid);
dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 577)
[declaration] INIT_LIST_HEAD(&priv->selectors);
dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 578)
[declaration] INIT_LIST_HEAD(&priv->sids);
dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 579)
[declaration] snd_mixer_sbasic_set_private(class, priv);
dirtycode/alsa-lib/modules/mixer/simple/sbase.c (line 580)
[declaration] snd_mixer_sbasic_set_private_free(class, sbasic_cpriv_free);
dirtycode/alsa-lib/modules/mixer/simple/sbasedl.c (line 60)
[declaration] free(priv);
dirtycode/alsa-lib/modules/mixer/simple/sbasedl.c (line 68)
[declaration] snd_error(MIXER, "Unable to open library '%s': %s", xlib, errbuf);
dirtycode/alsa-lib/modules/mixer/simple/sbasedl.c (line 73)
[declaration] snd_error(MIXER, "Symbol 'alsa_mixer_sbasic_initpriv' was not found in '%s'", xlib);
dirtycode/alsa-lib/modules/mixer/simple/sbasedl.c (line 78)
[declaration] snd_error(MIXER, "Symbol 'alsa_mixer_sbasic_event' was not found in '%s'", xlib);
dirtycode/alsa-lib/modules/mixer/simple/sbasedl.c (line 83)
[declaration] snd_error(MIXER, "Symbol 'alsa_mixer_sbasic_selreg' was not found in '%s'", xlib);
dirtycode/alsa-lib/modules/mixer/simple/sbasedl.c (line 88)
[declaration] snd_error(MIXER, "Symbol 'alsa_mixer_sbasic_sidreg' was not found in '%s'", xlib);
dirtycode/alsa-lib/modules/mixer/simple/sbasedl.c (line 93)
[declaration] initpriv(class, priv);
dirtycode/alsa-lib/modules/mixer/simple/sbasedl.c (line 101)
[declaration] free(priv);
dirtycode/alsa-lib/modules/mixer/simple/sbasedl.c (line 103)
[declaration] snd_dlclose(h);
dirtycode/alsa-lib/src/async.c (line 40)
[declaration] void snd_async_init(void) __attribute__((constructor));
dirtycode/alsa-lib/src/async.c (line 46)
[declaration] snd_error(CORE, "Unable to find a RT signal to use for snd_async");
dirtycode/alsa-lib/src/async.c (line 47)
[declaration] exit(1);
dirtycode/alsa-lib/src/async.c (line 55)
[declaration] static LIST_HEAD(snd_async_handlers);
dirtycode/alsa-lib/src/async.c (line 131)
[declaration] memset(&act, 0, sizeof(act));

```

dirtycode/alsa-lib/src/async.c (line 134)
[declaration] sigemptyset(&act.sa_mask);
dirtycode/alsa-lib/src/async.c (line 135)
[declaration] assert(!previous_action.sa_sigaction);
dirtycode/alsa-lib/src/async.c (line 137)
[function] if (err < 0) {
dirtycode/alsa-lib/src/async.c (line 138)
[declaration] snd_errornum(CORE, "sigaction");
dirtycode/alsa-lib/src/async.c (line 156)
[function] switch (handler->type) {
dirtycode/alsa-lib/src/async.c (line 166)
[declaration] assert(0);
dirtycode/alsa-lib/src/async.c (line 169)
[declaration] list_del(&handler->hlist);
dirtycode/alsa-lib/src/async.c (line 172)
[function] switch (handler->type) {
dirtycode/alsa-lib/src/async.c (line 182)
[declaration] assert(0);
dirtycode/alsa-lib/src/async.c (line 187)
[declaration] list_del(&handler->glist);
dirtycode/alsa-lib/src/async.c (line 188)
[function] if (list_empty(&snd_async_handlers)) {
dirtycode/alsa-lib/src/async.c (line 190)
[function] if (err < 0) {
dirtycode/alsa-lib/src/async.c (line 191)
[declaration] snd_errornum(CORE, "sigaction");
dirtycode/alsa-lib/src/async.c (line 194)
[declaration] memset(&previous_action, 0, sizeof(previous_action));
dirtycode/alsa-lib/src/conf.c (line 547)
[function] list_for_each(pos, &fd1->include_paths) {
dirtycode/alsa-lib/src/conf.c (line 560)
[declaration] free(path);
dirtycode/alsa-lib/src/conf.c (line 580)
[declaration] list_del(&path->list);
dirtycode/alsa-lib/src/conf.c (line 582)
[declaration] free(path->dir);
dirtycode/alsa-lib/src/conf.c (line 583)
[declaration] free(path);
dirtycode/alsa-lib/src/conf.c (line 647)
[declaration] return snd_input_stdio_open(inputp, file, "r");
dirtycode/alsa-lib/src/conf.c (line 654)
[function] list_for_each(pos, ¤t->include_paths) {
dirtycode/alsa-lib/src/conf.c (line 659)
[declaration] snprintf(full_path, PATH_MAX, "%s/%s", path->dir, file);
dirtycode/alsa-lib/src/conf.c (line 722)
[declaration] snprintf(locstr, sizeof(locstr), "%s", saved_locale);
dirtycode/alsa-lib/src/conf.c (line 723)
[declaration] setlocale(LC_NUMERIC, "C");

```
dirtycode/alsa-lib/src/conf.c (line 731)
[declaration] uselocale(saved_locale);
dirtycode/alsa-lib/src/conf.c (line 732)
[declaration] freelocale(c_locale);
dirtycode/alsa-lib/src/conf.c (line 736)
[declaration] setlocale(LC_NUMERIC, locstr);
dirtycode/alsa-lib/src/conf.c (line 766)
[function] if (fd->next) {
dirtycode/alsa-lib/src/conf.c (line 767)
[declaration] snd_input_close(fd->in);
dirtycode/alsa-lib/src/conf.c (line 768)
[declaration] free(fd->name);
dirtycode/alsa-lib/src/conf.c (line 770)
[declaration] free(fd);
dirtycode/alsa-lib/src/conf.c (line 788)
[declaration] static int get_delimstring(char **string, int delim, input_t *input);
dirtycode/alsa-lib/src/conf.c (line 795)
[function] if (c == '<') {
dirtycode/alsa-lib/src/conf.c (line 804)
[function] if (!strncmp(str, "searchdir:", 10)) {
dirtycode/alsa-lib/src/conf.c (line 807)
[declaration] free(str);
dirtycode/alsa-lib/src/conf.c (line 813)
[function] if (!dirp) {
dirtycode/alsa-lib/src/conf.c (line 814)
[declaration] snd_error(CONFIG, "Invalid search dir %s", str);
dirtycode/alsa-lib/src/conf.c (line 815)
[declaration] free(str);
dirtycode/alsa-lib/src/conf.c (line 818)
[declaration] closedir(dirp);
dirtycode/alsa-lib/src/conf.c (line 821)
[function] if (err < 0) {
dirtycode/alsa-lib/src/conf.c (line 822)
[declaration] snd_error(CORE, "Cannot add search dir %s", str);
dirtycode/alsa-lib/src/conf.c (line 823)
[declaration] free(str);
dirtycode/alsa-lib/src/conf.c (line 826)
[declaration] free(str);
dirtycode/alsa-lib/src/conf.c (line 830)
[function] if (!strncmp(str, "confdir:", 8)) {
dirtycode/alsa-lib/src/conf.c (line 833)
[declaration] free(str);
dirtycode/alsa-lib/src/conf.c (line 842)
[function] if (err < 0) {
dirtycode/alsa-lib/src/conf.c (line 843)
[declaration] snd_error(CORE, "Cannot access file %s", str);
dirtycode/alsa-lib/src/conf.c (line 844)
[declaration] free(str);
```

```
dirtycode/alsa-lib/src/conf.c (line 848)
[function] if (!fd) {
dirtycode/alsa-lib/src/conf.c (line 849)
[declaration] free(str);
dirtycode/alsa-lib/src/conf.c (line 857)
[declaration] INIT_LIST_HEAD(&fd->include_paths);
dirtycode/alsa-lib/src/conf.c (line 863)
[function] while (1) {
dirtycode/alsa-lib/src/conf.c (line 881)
[function] switch (c) {
dirtycode/alsa-lib/src/conf.c (line 927)
[declaration] return get_hexachar(input);
dirtycode/alsa-lib/src/conf.c (line 935)
[function] if (c < '0' || c > '7') {
dirtycode/alsa-lib/src/conf.c (line 936)
[declaration] unget_char(c, input);
dirtycode/alsa-lib/src/conf.c (line 967)
[declaration] free(s->buf);
dirtycode/alsa-lib/src/conf.c (line 974)
[function] if (s->buf == s->tmpbuf) {
dirtycode/alsa-lib/src/conf.c (line 978)
[declaration] memcpy(s->buf, s->tmpbuf, s->alloc);
dirtycode/alsa-lib/src/conf.c (line 995)
[declaration] memcpy(dst, s->buf, s->idx);
dirtycode/alsa-lib/src/conf.c (line 1009)
[function] if (c < 0) {
dirtycode/alsa-lib/src/conf.c (line 1010)
[function] if (c == LOCAL_UNEXPECTED_EOF) {
dirtycode/alsa-lib/src/conf.c (line 1019)
[function] switch (c) {
dirtycode/alsa-lib/src/conf.c (line 1044)
[declaration] unget_char(c, input);
dirtycode/alsa-lib/src/conf.c (line 1051)
[function] if (add_char_local_string(&str, c) < 0) {
dirtycode/alsa-lib/src/conf.c (line 1071)
[function] if (c == '\\') {
dirtycode/alsa-lib/src/conf.c (line 1085)
[function] if (add_char_local_string(&str, c) < 0) {
dirtycode/alsa-lib/src/conf.c (line 1090)
[declaration] free_local_string(&str);
dirtycode/alsa-lib/src/conf.c (line 1118)
[declaration] unget_char(c, input);
dirtycode/alsa-lib/src/conf.c (line 1132)
[function] if (*id) {
dirtycode/alsa-lib/src/conf.c (line 1133)
[declaration] free(*id);
dirtycode/alsa-lib/src/conf.c (line 1144)
[declaration] INIT_LIST_HEAD(&n->u.compound.fields);
```

```

dirtycode/alsa-lib/src/conf.c (line 1171)
[function] if (len < 0) {
dirtycode/alsa-lib/src/conf.c (line 1194)
[declaration] free(s);
dirtycode/alsa-lib/src/conf.c (line 1201)
[function] if (err < 0) {
dirtycode/alsa-lib/src/conf.c (line 1204)
[function] if (err >= 0) {
dirtycode/alsa-lib/src/conf.c (line 1205)
[declaration] free(s);
dirtycode/alsa-lib/src/conf.c (line 1206)
[function] if (n) {
dirtycode/alsa-lib/src/conf.c (line 1207)
[function] if (n->type != SND_CONFIG_TYPE_REAL) {
dirtycode/alsa-lib/src/conf.c (line 1208)
[declaration] snd_error(CORE, "%s is not a real", *id);
dirtycode/alsa-lib/src/conf.c (line 1221)
[declaration] free(s);
dirtycode/alsa-lib/src/conf.c (line 1222)
[function] if (n) {
dirtycode/alsa-lib/src/conf.c (line 1223)
[function] if (n->type != SND_CONFIG_TYPE_INTEGER && n->type !=
SND_CONFIG_TYPE_INTEGER64) {
dirtycode/alsa-lib/src/conf.c (line 1224)
[declaration] snd_error(CORE, "%s is not an integer", *id);
dirtycode/alsa-lib/src/conf.c (line 1244)
[function] if (n->type != SND_CONFIG_TYPE_STRING) {
dirtycode/alsa-lib/src/conf.c (line 1245)
[declaration] snd_error(CORE, "%s is not a string", *id);
dirtycode/alsa-lib/src/conf.c (line 1246)
[declaration] free(s);
dirtycode/alsa-lib/src/conf.c (line 1260)
[declaration] static int parse_defs(snd_config_t *parent, input_t *input, int skip,
int override);
dirtycode/alsa-lib/src/conf.c (line 1261)
[declaration] static int parse_array_defs(snd_config_t *farther, input_t *input, int
skip, int override);
dirtycode/alsa-lib/src/conf.c (line 1272)
[function] while (1) {
dirtycode/alsa-lib/src/conf.c (line 1273)
[declaration] snprintf(static_id, sizeof(static_id), "%i", *idx);
dirtycode/alsa-lib/src/conf.c (line 1274)
[function] if (_snd_config_search(parent, static_id, -1, &n) == 0) {
dirtycode/alsa-lib/src/conf.c (line 1275)
[function] if (override) {
dirtycode/alsa-lib/src/conf.c (line 1276)
[declaration] snd_config_delete(n);
dirtycode/alsa-lib/src/conf.c (line 1301)
[function] if (!skip) {

```

```

dirtycode/alsa-lib/src/conf.c (line 1303)
[function] if (n) {
dirtycode/alsa-lib/src/conf.c (line 1304)
[function] if (n->type != SND_CONFIG_TYPE_COMPOUND) {
dirtycode/alsa-lib/src/conf.c (line 1305)
[declaration] snd_error(CORE, "%s is not a compound", id);
dirtycode/alsa-lib/src/conf.c (line 1311)
[declaration] assert(n == NULL);
dirtycode/alsa-lib/src/conf.c (line 1329)
[function] if (c < 0) {
dirtycode/alsa-lib/src/conf.c (line 1333)
[function] if (c != endchr) {
dirtycode/alsa-lib/src/conf.c (line 1335)
[declaration] snd_config_delete(n);
dirtycode/alsa-lib/src/conf.c (line 1342)
[declaration] unget_char(c, input);
dirtycode/alsa-lib/src/conf.c (line 1361)
[declaration] unget_char(c, input);
dirtycode/alsa-lib/src/conf.c (line 1383)
[function] switch (c) {
dirtycode/alsa-lib/src/conf.c (line 1398)
[declaration] unget_char(c, input);
dirtycode/alsa-lib/src/conf.c (line 1406)
[function] if (skip) {
dirtycode/alsa-lib/src/conf.c (line 1407)
[declaration] free(id);
dirtycode/alsa-lib/src/conf.c (line 1410)
[function] if (_snd_config_search(parent, id, -1, &n) == 0) {
dirtycode/alsa-lib/src/conf.c (line 1411)
[function] if (mode == DONT_OVERRIDE) {
dirtycode/alsa-lib/src/conf.c (line 1413)
[declaration] free(id);
dirtycode/alsa-lib/src/conf.c (line 1416)
[function] if (mode != OVERRIDE) {
dirtycode/alsa-lib/src/conf.c (line 1417)
[function] if (n->type != SND_CONFIG_TYPE_COMPOUND) {
dirtycode/alsa-lib/src/conf.c (line 1418)
[declaration] snd_error(CORE, "%s is not a compound", id);
dirtycode/alsa-lib/src/conf.c (line 1423)
[declaration] free(id);
dirtycode/alsa-lib/src/conf.c (line 1426)
[declaration] snd_config_delete(n);
dirtycode/alsa-lib/src/conf.c (line 1428)
[function] if (mode == MERGE) {
dirtycode/alsa-lib/src/conf.c (line 1429)
[declaration] snd_error(CORE, "%s does not exists", id);
dirtycode/alsa-lib/src/conf.c (line 1445)
[function] if (_snd_config_search(parent, id, -1, &n) == 0) {

```

```
dirtycode/alsa-lib/src/conf.c (line 1446)
[function] if (mode == DONT_OVERRIDE) {
dirtycode/alsa-lib/src/conf.c (line 1450)
[declaration] snd_config_delete(n);
dirtycode/alsa-lib/src/conf.c (line 1455)
[function] if (mode == MERGE) {
dirtycode/alsa-lib/src/conf.c (line 1456)
[declaration] snd_error(CORE, "%s does not exists", id);
dirtycode/alsa-lib/src/conf.c (line 1467)
[function] if (!skip) {
dirtycode/alsa-lib/src/conf.c (line 1468)
[function] if (n) {
dirtycode/alsa-lib/src/conf.c (line 1469)
[function] if (n->type != SND_CONFIG_TYPE_COMPOUND) {
dirtycode/alsa-lib/src/conf.c (line 1470)
[declaration] snd_error(CORE, "%s is not a compound", id);
dirtycode/alsa-lib/src/conf.c (line 1488)
[function] if (c < 0) {
dirtycode/alsa-lib/src/conf.c (line 1492)
[function] if (c != endchr) {
dirtycode/alsa-lib/src/conf.c (line 1494)
[declaration] snd_config_delete(n);
dirtycode/alsa-lib/src/conf.c (line 1501)
[declaration] unget_char(c, input);
dirtycode/alsa-lib/src/conf.c (line 1513)
[declaration] unget_char(c, input);
dirtycode/alsa-lib/src/conf.c (line 1527)
[declaration] unget_char(c, input);
dirtycode/alsa-lib/src/conf.c (line 1542)
[declaration] snd_output_puts(out, "");
dirtycode/alsa-lib/src/conf.c (line 1546)
[function] switch (*p) {
dirtycode/alsa-lib/src/conf.c (line 1587)
[function] switch (c) {
dirtycode/alsa-lib/src/conf.c (line 1589)
[declaration] snd_output_putc(out, '\\');
dirtycode/alsa-lib/src/conf.c (line 1590)
[declaration] snd_output_putc(out, 'n');
dirtycode/alsa-lib/src/conf.c (line 1593)
[declaration] snd_output_putc(out, '\\');
dirtycode/alsa-lib/src/conf.c (line 1594)
[declaration] snd_output_putc(out, 't');
dirtycode/alsa-lib/src/conf.c (line 1597)
[declaration] snd_output_putc(out, '\\');
dirtycode/alsa-lib/src/conf.c (line 1598)
[declaration] snd_output_putc(out, 'v');
dirtycode/alsa-lib/src/conf.c (line 1601)
[declaration] snd_output_putc(out, '\\');
```

```

dirtycode/alsa-lib/src/conf.c (line 1602)
[declaration] snd_output_putc(out, 'b');
dirtycode/alsa-lib/src/conf.c (line 1605)
[declaration] snd_output_putc(out, '\\');
dirtycode/alsa-lib/src/conf.c (line 1606)
[declaration] snd_output_putc(out, 'r');
dirtycode/alsa-lib/src/conf.c (line 1609)
[declaration] snd_output_putc(out, '\\');
dirtycode/alsa-lib/src/conf.c (line 1610)
[declaration] snd_output_putc(out, 'f');
dirtycode/alsa-lib/src/conf.c (line 1613)
[function] if (c == q) {
dirtycode/alsa-lib/src/conf.c (line 1614)
[declaration] snd_output_putc(out, '\\');
dirtycode/alsa-lib/src/conf.c (line 1615)
[declaration] snd_output_putc(out, c);
dirtycode/alsa-lib/src/conf.c (line 1618)
[declaration] snd_output_putc(out, c);
dirtycode/alsa-lib/src/conf.c (line 1620)
[declaration] snd_output_printf(out, "\\%04o", c);
dirtycode/alsa-lib/src/conf.c (line 1647)
[declaration] snd_output_printf(out, "%ld", n->u.integer);
dirtycode/alsa-lib/src/conf.c (line 1650)
[declaration] snd_output_printf(out, "%lld", n->u.integer64);
dirtycode/alsa-lib/src/conf.c (line 1653)
[declaration] snd_output_printf(out, "%-16g", n->u.real);
dirtycode/alsa-lib/src/conf.c (line 1656)
[declaration] string_print(n->u.string, 0, out);
dirtycode/alsa-lib/src/conf.c (line 1659)
[declaration] snd_error(CORE, "cannot save runtime pointer type");
dirtycode/alsa-lib/src/conf.c (line 1664)
[declaration] snd_output_putc(out, '\\n');
dirtycode/alsa-lib/src/conf.c (line 1668)
[declaration] level_print(out, level);
dirtycode/alsa-lib/src/conf.c (line 1669)
[declaration] snd_output_putc(out, array ? ']' : '}');
dirtycode/alsa-lib/src/conf.c (line 1678)
[declaration] assert(n->parent);
dirtycode/alsa-lib/src/conf.c (line 1679)
[declaration] id_print(n->parent, out, joins - 1);
dirtycode/alsa-lib/src/conf.c (line 1680)
[declaration] snd_output_putc(out, '.');
dirtycode/alsa-lib/src/conf.c (line 1701)
[declaration] level_print(out, level);
dirtycode/alsa-lib/src/conf.c (line 1702)
[function] if (!array) {
dirtycode/alsa-lib/src/conf.c (line 1703)
[declaration] id_print(n, out, joins);

```

```

dirtycode/alsa-lib/src/conf.c (line 1704)
[declaration] snd_output_putc(out, ' ');
dirtycode/alsa-lib/src/conf.c (line 1706)
[declaration] snd_output_putc(out, '=');
dirtycode/alsa-lib/src/conf.c (line 1715)
[declaration] snd_output_putc(out, '\n');
dirtycode/alsa-lib/src/conf.c (line 1753)
[function] snd_config_for_each(i, next, src) {
dirtycode/alsa-lib/src/conf.c (line 1762)
[declaration] free(dst->u.string);
dirtycode/alsa-lib/src/conf.c (line 1764)
[declaration] list_del(&src->list);
dirtycode/alsa-lib/src/conf.c (line 1881)
[declaration] return list_empty(&config->u.compound.fields);
dirtycode/alsa-lib/src/conf.c (line 1928)
[function] if (config->parent) {
dirtycode/alsa-lib/src/conf.c (line 1929)
[function] snd_config_for_each(i, next, config->parent) {
dirtycode/alsa-lib/src/conf.c (line 1967)
[declaration] return _snd_config_make(config, 0, SND_CONFIG_TYPE_COMPOUND);
dirtycode/alsa-lib/src/conf.c (line 2005)
[function] switch (err) {
dirtycode/alsa-lib/src/conf.c (line 2026)
[declaration] snd_error(CORE, "%s:%d:%d:%s", fd->name ? fd->name : "_toplevel_",
fd->line, fd->column, str);
dirtycode/alsa-lib/src/conf.c (line 2032)
[declaration] snd_error(CORE, "%s:%d:%d:Unexpected }", fd->name ? fd->name : "",
fd->line, fd->column);
dirtycode/alsa-lib/src/conf.c (line 2040)
[declaration] snd_input_close(fd->in);
dirtycode/alsa-lib/src/conf.c (line 2041)
[declaration] free(fd->name);
dirtycode/alsa-lib/src/conf.c (line 2042)
[declaration] free_include_paths(fd);
dirtycode/alsa-lib/src/conf.c (line 2043)
[declaration] free(fd);
dirtycode/alsa-lib/src/conf.c (line 2071)
[declaration] return _snd_config_load_with_include(config, in, 0, NULL);
dirtycode/alsa-lib/src/conf.c (line 2104)
[declaration] snd_input_close(input);
dirtycode/alsa-lib/src/conf.c (line 2110)
[declaration] snd_config_delete(dst);
dirtycode/alsa-lib/src/conf.c (line 2129)
[declaration] return _snd_config_load_with_include(config, in, 1, NULL);
dirtycode/alsa-lib/src/conf.c (line 2265)
[declaration] snd_config_remove(sn);
dirtycode/alsa-lib/src/conf.c (line 2266)
[declaration] snprintf(id, sizeof(id), "%d", index++);

```

```

dirtycode/alsa-lib/src/conf.c (line 2268)
[function] if (err < 0) {
dirtycode/alsa-lib/src/conf.c (line 2269)
[declaration] snd_config_delete(sn);
dirtycode/alsa-lib/src/conf.c (line 2273)
[declaration] list_add_tail(&sn->list, &dst->u.compound.fields);
dirtycode/alsa-lib/src/conf.c (line 2314)
[declaration] return snd_config_substitute(dst, src);
dirtycode/alsa-lib/src/conf.c (line 2317)
[declaration] return _snd_config_array_merge(dst, src, array);
dirtycode/alsa-lib/src/conf.c (line 2321)
[function] snd_config_for_each(di, dnext, dst) {
dirtycode/alsa-lib/src/conf.c (line 2323)
[function] if (strcmp(sn->id, dn->id) == 0) {
dirtycode/alsa-lib/src/conf.c (line 2339)
[function] if (!found) {
dirtycode/alsa-lib/src/conf.c (line 2341)
[declaration] snd_config_remove(sn);
dirtycode/alsa-lib/src/conf.c (line 2343)
[declaration] list_add_tail(&sn->list, &dst->u.compound.fields);
dirtycode/alsa-lib/src/conf.c (line 2367)
[declaration] list_del(&config->list);
dirtycode/alsa-lib/src/conf.c (line 2407)
[function] while (i != &config->u.compound.fields) {
dirtycode/alsa-lib/src/conf.c (line 2418)
[declaration] free(config->u.string);
dirtycode/alsa-lib/src/conf.c (line 2424)
[declaration] list_del(&config->list);
dirtycode/alsa-lib/src/conf.c (line 2498)
[declaration] return _snd_config_make(config, &id1, type);
dirtycode/alsa-lib/src/conf.c (line 2523)
[declaration] return snd_config_make(config, id, SND_CONFIG_TYPE_INTEGER);
dirtycode/alsa-lib/src/conf.c (line 2548)
[declaration] return snd_config_make(config, id, SND_CONFIG_TYPE_INTEGER64);
dirtycode/alsa-lib/src/conf.c (line 2570)
[declaration] return snd_config_make(config, id, SND_CONFIG_TYPE_REAL);
dirtycode/alsa-lib/src/conf.c (line 2595)
[declaration] return snd_config_make(config, id, SND_CONFIG_TYPE_STRING);
dirtycode/alsa-lib/src/conf.c (line 2617)
[declaration] return snd_config_make(config, id, SND_CONFIG_TYPE_POINTER);
dirtycode/alsa-lib/src/conf.c (line 2719)
[function] if (p) {
dirtycode/alsa-lib/src/conf.c (line 2721)
[function] if (err < 0) {
dirtycode/alsa-lib/src/conf.c (line 2726)
[declaration] strncpy(s, key, l);
dirtycode/alsa-lib/src/conf.c (line 2729)
[declaration] free(s);

```

```

dirtycode/alsa-lib/src/conf.c (line 2740)
[function] if (err == 0) {
dirtycode/alsa-lib/src/conf.c (line 2741)
[function] if ((*config)->type != SND_CONFIG_TYPE_COMPOUND) {
dirtycode/alsa-lib/src/conf.c (line 2742)
[function] if (override) {
dirtycode/alsa-lib/src/conf.c (line 2882)
[function] if (!tmp->u.string) {
dirtycode/alsa-lib/src/conf.c (line 2883)
[declaration] snd_config_delete(tmp);
dirtycode/alsa-lib/src/conf.c (line 2924)
[function] if (!tmp->u.string) {
dirtycode/alsa-lib/src/conf.c (line 2925)
[declaration] snd_config_delete(tmp);
dirtycode/alsa-lib/src/conf.c (line 3156)
[declaration] free(config->u.string);
dirtycode/alsa-lib/src/conf.c (line 3285)
[function] else if (config->type == SND_CONFIG_TYPE_STRING) {
dirtycode/alsa-lib/src/conf.c (line 3381)
[function] if (err < 0 || err == sizeof(res)) {
dirtycode/alsa-lib/src/conf.c (line 3382)
[declaration] assert(0);
dirtycode/alsa-lib/src/conf.c (line 3393)
[function] if (err < 0 || err == sizeof(res)) {
dirtycode/alsa-lib/src/conf.c (line 3394)
[declaration] assert(0);
dirtycode/alsa-lib/src/conf.c (line 3405)
[function] if (err < 0 || err == sizeof(res)) {
dirtycode/alsa-lib/src/conf.c (line 3406)
[declaration] assert(0);
dirtycode/alsa-lib/src/conf.c (line 3409)
[function] if (res[0]) { /* trim the string */
dirtycode/alsa-lib/src/conf.c (line 3450)
[declaration] return strcmp(config->id, id);
dirtycode/alsa-lib/src/conf.c (line 3478)
[declaration] return _snd_config_save_children(config, out, 0, 0, array);
dirtycode/alsa-lib/src/conf.c (line 3480)
[declaration] return _snd_config_save_node_value(config, out, 0);
dirtycode/alsa-lib/src/conf.c (line 3501)
[function] if (p) { \
dirtycode/alsa-lib/src/conf.c (line 3508)
[declaration] return _snd_config_search(config, key, -1, result); \
dirtycode/alsa-lib/src/conf.c (line 3519)
[function] if (config->type != SND_CONFIG_TYPE_COMPOUND) { \
dirtycode/alsa-lib/src/conf.c (line 3528)
[function] if (p) { \
dirtycode/alsa-lib/src/conf.c (line 3535)
[declaration] return _snd_config_search(config, key, -1, result); \

```

```

dirtycode/alsa-lib/src/conf.c (line 3551)
[function] if (err < 0) { \
dirtycode/alsa-lib/src/conf.c (line 3552)
[declaration] va_end(arg); \
dirtycode/alsa-lib/src/conf.c (line 3575)
[function] if (err < 0) { \
dirtycode/alsa-lib/src/conf.c (line 3576)
[declaration] va_end(arg); \
dirtycode/alsa-lib/src/conf.c (line 3595)
[function] if (old_key == NULL) { \
dirtycode/alsa-lib/src/conf.c (line 3601)
[function] if (err < 0) { \
dirtycode/alsa-lib/src/conf.c (line 3610)
[declaration] assert(key); \
dirtycode/alsa-lib/src/conf.c (line 3611)
[function] if (!first && (strcmp(key, old_key) == 0 || maxloop <= 0)) { \
dirtycode/alsa-lib/src/conf.c (line 3613)
[declaration] snd_error(CORE, "maximum loop count reached (circular
configuration?"); \
dirtycode/alsa-lib/src/conf.c (line 3615)
[declaration] snd_error(CORE, "key %s refers to itself", key); \
dirtycode/alsa-lib/src/conf.c (line 3620)
[declaration] free(old_key); \
dirtycode/alsa-lib/src/conf.c (line 3700)
[declaration] * snd_config_searcha(root, config, "a.b.c.d", &result);
dirtycode/alsa-lib/src/conf.c (line 3748)
[declaration] * snd_config_searchv(cfg, &res, "a", "b.c", "d.e", NULL);
dirtycode/alsa-lib/src/conf.c (line 3752)
[declaration] * snd_config_search(cfg, "a.b.c.d.e", &res);
dirtycode/alsa-lib/src/conf.c (line 3833)
[declaration] static int snd_config_hooks(snd_config_t *config, snd_config_t
*private_data);
dirtycode/alsa-lib/src/conf.c (line 4019)
[declaration] snd_error(CORE, "Field func is missing");
dirtycode/alsa-lib/src/conf.c (line 4024)
[declaration] snd_error(CORE, "Invalid type for field func");
dirtycode/alsa-lib/src/conf.c (line 4031)
[function] if (snd_config_get_type(func_conf) != SND_CONFIG_TYPE_COMPOUND) {
dirtycode/alsa-lib/src/conf.c (line 4032)
[declaration] snd_error(CORE, "Invalid type for func %s definition", str);
dirtycode/alsa-lib/src/conf.c (line 4036)
[function] snd_config_for_each(i, next, func_conf) {
dirtycode/alsa-lib/src/conf.c (line 4041)
[function] if (strcmp(id, "lib") == 0) {
dirtycode/alsa-lib/src/conf.c (line 4043)
[function] if (err < 0) {
dirtycode/alsa-lib/src/conf.c (line 4044)
[declaration] snd_error(CORE, "Invalid type for %s", id);

```

```

dirtycode/alsa-lib/src/conf.c (line 4049)
[function] if (strcmp(id, "func") == 0) {
dirtycode/alsa-lib/src/conf.c (line 4051)
[function] if (err < 0) {
dirtycode/alsa-lib/src/conf.c (line 4052)
[declaration] snd_error(CORE, "Invalid type for %s", id);
dirtycode/alsa-lib/src/conf.c (line 4057)
[declaration] snd_error(CORE, "Unknown field %s", id);
dirtycode/alsa-lib/src/conf.c (line 4063)
[function] if (! buf) {
dirtycode/alsa-lib/src/conf.c (line 4067)
[declaration] snprintf(buf, len, "snd_config_hook_%s", str);
dirtycode/alsa-lib/src/conf.c (line 4075)
[declaration] snd_error(CORE, "Cannot open shared library %s (%s)", lib, errbuf);
dirtycode/alsa-lib/src/conf.c (line 4078)
[declaration] snd_error(CORE, "symbol %s is not defined inside %s", func_name, lib);
dirtycode/alsa-lib/src/conf.c (line 4079)
[declaration] snd_dlclose(h);
dirtycode/alsa-lib/src/conf.c (line 4084)
[declaration] snd_config_delete(func_conf);
dirtycode/alsa-lib/src/conf.c (line 4089)
[declaration] snd_error(CORE, "function %s returned error: %s", func_name,
snd_strerror(err));
dirtycode/alsa-lib/src/conf.c (line 4090)
[declaration] snd_dlclose(h);
dirtycode/alsa-lib/src/conf.c (line 4112)
[function] snd_config_for_each(i, next, n) {
dirtycode/alsa-lib/src/conf.c (line 4117)
[function] if (err < 0) {
dirtycode/alsa-lib/src/conf.c (line 4118)
[declaration] snd_error(CORE, "id of field %s is not and integer", id);
dirtycode/alsa-lib/src/conf.c (line 4122)
[function] if (i == idx) {
dirtycode/alsa-lib/src/conf.c (line 4165)
[declaration] snd_input_close(in);
dirtycode/alsa-lib/src/conf.c (line 4167)
[declaration] snd_error(CORE, "%s may be old or corrupted: consider to remove or fix
it", filename);
dirtycode/alsa-lib/src/conf.c (line 4169)
[declaration] snd_error(CORE, "cannot access file %s", filename);
dirtycode/alsa-lib/src/conf.c (line 4183)
[declaration] snd_error(CORE, "cannot stat file/directory %s", fn);
dirtycode/alsa-lib/src/conf.c (line 4187)
[declaration] return config_file_open(root, fn);
dirtycode/alsa-lib/src/conf.c (line 4207)
[function] if (err >= 0) {
dirtycode/alsa-lib/src/conf.c (line 4210)
[declaration] snprintf(filename, sl, "%s/%s", fn, namelist[j]->d_name);

```

```

dirtycode/alsa-lib/src/conf.c (line 4214)
[declaration] free(filename);
dirtycode/alsa-lib/src/conf.c (line 4216)
[declaration] free(namelist[j]);
dirtycode/alsa-lib/src/conf.c (line 4218)
[declaration] free(namelist);
dirtycode/alsa-lib/src/conf.c (line 4232)
[declaration] return config_file_load(root, fn, errors);
dirtycode/alsa-lib/src/conf.c (line 4245)
[function] if ((err = snd_config_search(_file, "file", &file)) < 0) {
dirtycode/alsa-lib/src/conf.c (line 4246)
[declaration] snd_error(CORE, "Field file not found");
dirtycode/alsa-lib/src/conf.c (line 4249)
[function] if ((err = snd_config_search(_file, "root", &root)) >= 0) {
dirtycode/alsa-lib/src/conf.c (line 4251)
[function] if (err < 0) {
dirtycode/alsa-lib/src/conf.c (line 4252)
[declaration] snd_error(CORE, "Field root is bad");
dirtycode/alsa-lib/src/conf.c (line 4265)
[function] if (remain) {
dirtycode/alsa-lib/src/conf.c (line 4281)
[function] if (err == 0) {
dirtycode/alsa-lib/src/conf.c (line 4282)
[function] if (snd_config_get_type(root) == SND_CONFIG_TYPE_COMPOUND) {
dirtycode/alsa-lib/src/conf.c (line 4294)
[declaration] snd_config_delete(root);
dirtycode/alsa-lib/src/conf.c (line 4323)
[function] if (errors < 0) {
dirtycode/alsa-lib/src/conf.c (line 4324)
[declaration] snd_error(CORE, "Invalid bool value in field errors");
dirtycode/alsa-lib/src/conf.c (line 4329)
[declaration] snd_error(CORE, "Unable to find field files in the pre-load section");
dirtycode/alsa-lib/src/conf.c (line 4333)
[declaration] snd_error(CORE, "Unable to expand filenames in the pre-load section");
dirtycode/alsa-lib/src/conf.c (line 4337)
[declaration] snd_error(CORE, "Invalid type for field filenames");
dirtycode/alsa-lib/src/conf.c (line 4342)
[function] snd_config_for_each(i, next, n) {
dirtycode/alsa-lib/src/conf.c (line 4347)
[function] if (err < 0) {
dirtycode/alsa-lib/src/conf.c (line 4348)
[declaration] snd_error(CORE, "id of field %s is not and integer", id);
dirtycode/alsa-lib/src/conf.c (line 4352)
[function] if (i == idx) {
dirtycode/alsa-lib/src/conf.c (line 4372)
[declaration] int snd_determine_driver(int card, char **driver);
dirtycode/alsa-lib/src/conf.c (line 4388)
[declaration] snd_config_delete(v);

```

```

dirtycode/alsa-lib/src/conf.c (line 4396)
[declaration] snd_config_delete(v);
dirtycode/alsa-lib/src/conf.c (line 4415)
[declaration] snd_error(CORE, "Unable to expand table compound");
dirtycode/alsa-lib/src/conf.c (line 4419)
[function] snd_config_get_string(tn, &id) < 0) {
dirtycode/alsa-lib/src/conf.c (line 4420)
[declaration] snd_error(CORE, "Unable to find field table.id");
dirtycode/alsa-lib/src/conf.c (line 4421)
[declaration] snd_config_delete(n);
dirtycode/alsa-lib/src/conf.c (line 4425)
[function] snd_config_get_type(tn) != SND_CONFIG_TYPE_STRING) {
dirtycode/alsa-lib/src/conf.c (line 4426)
[declaration] snd_error(CORE, "Unable to find field table.value");
dirtycode/alsa-lib/src/conf.c (line 4427)
[declaration] snd_config_delete(n);
dirtycode/alsa-lib/src/conf.c (line 4432)
[declaration] snd_config_delete(tn);
dirtycode/alsa-lib/src/conf.c (line 4433)
[declaration] snd_config_delete(n);
dirtycode/alsa-lib/src/conf.c (line 4438)
[declaration] snd_config_delete(tn);
dirtycode/alsa-lib/src/conf.c (line 4467)
[function] if (card >= 0) {
dirtycode/alsa-lib/src/conf.c (line 4474)
[function] if (snd_config_search(root, fdriver, &n) >= 0) {
dirtycode/alsa-lib/src/conf.c (line 4475)
[function] if (snd_config_get_string(n, &driver) < 0) {
dirtycode/alsa-lib/src/conf.c (line 4476)
[function] if (snd_config_get_type(n) == SND_CONFIG_TYPE_COMPOUND) {
dirtycode/alsa-lib/src/conf.c (line 4483)
[function] while (1) {
dirtycode/alsa-lib/src/conf.c (line 4496)
[function] if (!private_data) {
dirtycode/alsa-lib/src/conf.c (line 4506)
[declaration] snd_config_delete(private_data);
dirtycode/alsa-lib/src/conf.c (line 4507)
[declaration] free(fdriver);
dirtycode/alsa-lib/src/conf.c (line 4562)
[function] if (!configs || !*configs) {
dirtycode/alsa-lib/src/conf.c (line 4565)
[declaration] strlen("alsa.conf") + 2);
dirtycode/alsa-lib/src/conf.c (line 4566)
[declaration] sprintf(s, "%s/alsa.conf", topdir);
dirtycode/alsa-lib/src/conf.c (line 4587)
[declaration] free(local);
dirtycode/alsa-lib/src/conf.c (line 4592)
[declaration] memcpy(name, c, l);

```

```

dirtycode/alsa-lib/src/conf.c (line 4609)
[function] if (stat64(lf->name, &st) >= 0) {
dirtycode/alsa-lib/src/conf.c (line 4614)
[declaration] snd_error(CORE, "Cannot access file %s", lf->name);
dirtycode/alsa-lib/src/conf.c (line 4615)
[declaration] free(lf->name);
dirtycode/alsa-lib/src/conf.c (line 4617)
[function] if (k < local->count) {
dirtycode/alsa-lib/src/conf.c (line 4618)
[declaration] memmove(&local->finfo[k], &local->finfo[k+1], sizeof(struct finfo) *
(local->count - k));
dirtycode/alsa-lib/src/conf.c (line 4642)
[function] if (top) {
dirtycode/alsa-lib/src/conf.c (line 4643)
[declaration] snd_config_delete(top);
dirtycode/alsa-lib/src/conf.c (line 4646)
[function] if (update) {
dirtycode/alsa-lib/src/conf.c (line 4647)
[declaration] snd_config_update_free(update);
dirtycode/alsa-lib/src/conf.c (line 4652)
[declaration] snd_config_update_free(local);
dirtycode/alsa-lib/src/conf.c (line 4659)
[declaration] snd_config_update_free(update);
dirtycode/alsa-lib/src/conf.c (line 4663)
[declaration] snd_config_delete(top);
dirtycode/alsa-lib/src/conf.c (line 4674)
[function] if (err >= 0) {
dirtycode/alsa-lib/src/conf.c (line 4676)
[declaration] snd_input_close(in);
dirtycode/alsa-lib/src/conf.c (line 4677)
[function] if (err < 0) {
dirtycode/alsa-lib/src/conf.c (line 4678)
[declaration] snd_error(CORE, "%s may be old or corrupted: consider to remove or fix
it", local->finfo[k].name);
dirtycode/alsa-lib/src/conf.c (line 4682)
[declaration] snd_error(CORE, "cannot access file %s", local->finfo[k].name);
dirtycode/alsa-lib/src/conf.c (line 4688)
[declaration] snd_error(CORE, "hooks failed, removing configuration");
dirtycode/alsa-lib/src/conf.c (line 4744)
[function] if (snd_config) {
dirtycode/alsa-lib/src/conf.c (line 4745)
[function] if (top) {
dirtycode/alsa-lib/src/conf.c (line 4788)
[declaration] snd_config_delete(cfg);
dirtycode/alsa-lib/src/conf.c (line 4803)
[declaration] free(update->finfo[k].name);
dirtycode/alsa-lib/src/conf.c (line 4823)
[declaration] snd_config_delete(snd_config);

```

```

dirtycode/alsa-lib/src/conf.c (line 4826)
[declaration] snd_config_update_free(snd_config_global_update);
dirtycode/alsa-lib/src/conf.c (line 4905)
[declaration] return list_entry(iterator, snd_config_t, list);
dirtycode/alsa-lib/src/conf.c (line 4942)
[function] snd_config_for_each(i, next, src) {
dirtycode/alsa-lib/src/conf.c (line 4950)
[function] if (err && d) {
dirtycode/alsa-lib/src/conf.c (line 4957)
[function] if (err <= 0) {
dirtycode/alsa-lib/src/conf.c (line 4960)
[declaration] snd_config_delete(*dst);
dirtycode/alsa-lib/src/conf.c (line 4990)
[function] switch (type) {
dirtycode/alsa-lib/src/conf.c (line 4995)
[declaration] assert(err >= 0);
dirtycode/alsa-lib/src/conf.c (line 4996)
[declaration] snd_config_set_integer(*dst, v);
dirtycode/alsa-lib/src/conf.c (line 5003)
[declaration] assert(err >= 0);
dirtycode/alsa-lib/src/conf.c (line 5004)
[declaration] snd_config_set_integer64(*dst, v);
dirtycode/alsa-lib/src/conf.c (line 5011)
[declaration] assert(err >= 0);
dirtycode/alsa-lib/src/conf.c (line 5012)
[declaration] snd_config_set_real(*dst, v);
dirtycode/alsa-lib/src/conf.c (line 5019)
[declaration] assert(err >= 0);
dirtycode/alsa-lib/src/conf.c (line 5026)
[declaration] assert(0);
dirtycode/alsa-lib/src/conf.c (line 5056)
[declaration] return snd_config_walk(src, NULL, dst, _snd_config_copy, NULL, NULL);
dirtycode/alsa-lib/src/conf.c (line 5066)
[declaration] return snd_config_copy(dst, val);
dirtycode/alsa-lib/src/conf.c (line 5090)
[function] switch (type) {
dirtycode/alsa-lib/src/conf.c (line 5095)
[declaration] assert(err >= 0);
dirtycode/alsa-lib/src/conf.c (line 5105)
[declaration] assert(err >= 0);
dirtycode/alsa-lib/src/conf.c (line 5115)
[declaration] assert(err >= 0);
dirtycode/alsa-lib/src/conf.c (line 5128)
[function] if (s && *s == '$') {
dirtycode/alsa-lib/src/conf.c (line 5135)
[function] if (err < 0) {
dirtycode/alsa-lib/src/conf.c (line 5136)
[declaration] snd_config_delete(*dst);

```

```

dirtycode/alsa-lib/src/conf.c (line 5147)
[declaration] assert(0);
dirtycode/alsa-lib/src/conf.c (line 5176)
[function] if (err < 0) {
dirtycode/alsa-lib/src/conf.c (line 5177)
[declaration] snd_error(CORE, "Invalid type for @func");
dirtycode/alsa-lib/src/conf.c (line 5180)
[declaration] assert(str);
dirtycode/alsa-lib/src/conf.c (line 5182)
[function] if (err >= 0) {
dirtycode/alsa-lib/src/conf.c (line 5184)
[function] if (snd_config_get_type(func_conf) != SND_CONFIG_TYPE_COMPOUND) {
dirtycode/alsa-lib/src/conf.c (line 5185)
[declaration] snd_error(CORE, "Invalid type for func %s definition", str);
dirtycode/alsa-lib/src/conf.c (line 5189)
[function] snd_config_for_each(i, next, func_conf) {
dirtycode/alsa-lib/src/conf.c (line 5194)
[function] if (strcmp(id, "lib") == 0) {
dirtycode/alsa-lib/src/conf.c (line 5196)
[function] if (err < 0) {
dirtycode/alsa-lib/src/conf.c (line 5197)
[declaration] snd_error(CORE, "Invalid type for %s", id);
dirtycode/alsa-lib/src/conf.c (line 5202)
[function] if (strcmp(id, "func") == 0) {
dirtycode/alsa-lib/src/conf.c (line 5204)
[function] if (err < 0) {
dirtycode/alsa-lib/src/conf.c (line 5205)
[declaration] snd_error(CORE, "Invalid type for %s", id);
dirtycode/alsa-lib/src/conf.c (line 5210)
[declaration] snd_error(CORE, "Unknown field %s", id);
dirtycode/alsa-lib/src/conf.c (line 5213)
[function] if (!func_name) {
dirtycode/alsa-lib/src/conf.c (line 5216)
[function] if (! buf) {
dirtycode/alsa-lib/src/conf.c (line 5220)
[declaration] snprintf(buf, len, "snd_func_%s", str);
dirtycode/alsa-lib/src/conf.c (line 5228)
[function] if (!h) {
dirtycode/alsa-lib/src/conf.c (line 5229)
[declaration] snd_error(CORE, "Cannot open shared library %s (%s)", lib, errbuf);
dirtycode/alsa-lib/src/conf.c (line 5233)
[declaration] snd_error(CORE, "symbol %s is not defined inside %s", func_name, lib);
dirtycode/alsa-lib/src/conf.c (line 5234)
[declaration] snd_dlclose(h);
dirtycode/alsa-lib/src/conf.c (line 5240)
[declaration] snd_config_delete(func_conf);
dirtycode/alsa-lib/src/conf.c (line 5241)
[function] if (err >= 0) {

```

```

dirtycode/alsa-lib/src/conf.c (line 5245)
[declaration] snd_error(CORE, "function %s returned error: %s", func_name,
snd_strerror(err));
dirtycode/alsa-lib/src/conf.c (line 5246)
[declaration] snd_dlclose(h);
dirtycode/alsa-lib/src/conf.c (line 5251)
[declaration] free(buf);
dirtycode/alsa-lib/src/conf.c (line 5275)
[declaration] return snd_config_walk(config, root, result, _snd_config_evaluate,
NULL, private_data);
dirtycode/alsa-lib/src/conf.c (line 5286)
[function] snd_config_for_each(f, fnext, def) {
dirtycode/alsa-lib/src/conf.c (line 5291)
[function] if (strcmp(id, "default") == 0) {
dirtycode/alsa-lib/src/conf.c (line 5298)
[function] if (err < 0) {
dirtycode/alsa-lib/src/conf.c (line 5299)
[declaration] snd_config_delete(deflt);
dirtycode/alsa-lib/src/conf.c (line 5303)
[function] if (err < 0) {
dirtycode/alsa-lib/src/conf.c (line 5304)
[declaration] snd_config_delete(deflt);
dirtycode/alsa-lib/src/conf.c (line 5309)
[declaration] snd_error(CORE, "Unknown field %s", id);
dirtycode/alsa-lib/src/conf.c (line 5319)
[function] switch (**ptr) {
dirtycode/alsa-lib/src/conf.c (line 5386)
[function] switch (**ptr) {
dirtycode/alsa-lib/src/conf.c (line 5413)
[function] switch (c) {
dirtycode/alsa-lib/src/conf.c (line 5415)
[declaration] snd_error(CORE, "Unterminated string");
dirtycode/alsa-lib/src/conf.c (line 5419)
[function] if (c < 0) {
dirtycode/alsa-lib/src/conf.c (line 5421)
[declaration] free(buf);
dirtycode/alsa-lib/src/conf.c (line 5427)
[function] if (c == delim) {
dirtycode/alsa-lib/src/conf.c (line 5431)
[declaration] memcpy(*val, buf, idx);
dirtycode/alsa-lib/src/conf.c (line 5434)
[declaration] free(buf);
dirtycode/alsa-lib/src/conf.c (line 5438)
[function] if (idx >= alloc) {
dirtycode/alsa-lib/src/conf.c (line 5441)
[function] if (old_alloc == bufsize) {
dirtycode/alsa-lib/src/conf.c (line 5445)
[declaration] memcpy(buf, _buf, old_alloc);

```

```

dirtycode/alsa-lib/src/conf.c (line 5448)
[function] if (!buf2) {
dirtycode/alsa-lib/src/conf.c (line 5449)
[declaration] free(buf);
dirtycode/alsa-lib/src/conf.c (line 5524)
[function] while (1) {
dirtycode/alsa-lib/src/conf.c (line 5525)
[function] switch (str[--len]) {
dirtycode/alsa-lib/src/conf.c (line 5543)
[declaration] snd_input_close(input);
dirtycode/alsa-lib/src/conf.c (line 5546)
[function] snd_config_for_each(i, next, subs) {
dirtycode/alsa-lib/src/conf.c (line 5551)
[function] if (err < 0) {
dirtycode/alsa-lib/src/conf.c (line 5552)
[declaration] snd_error(CORE, "Unknown parameter %s", id);
dirtycode/alsa-lib/src/conf.c (line 5572)
[function] if (varlen > 0) {
dirtycode/alsa-lib/src/conf.c (line 5573)
[declaration] assert(varlen < sizeof(buf));
dirtycode/alsa-lib/src/conf.c (line 5574)
[declaration] memcpy(buf, str, varlen);
dirtycode/alsa-lib/src/conf.c (line 5577)
[declaration] sprintf(buf, "%d", arg);
dirtycode/alsa-lib/src/conf.c (line 5580)
[function] if (err < 0) {
dirtycode/alsa-lib/src/conf.c (line 5581)
[declaration] snd_error(CORE, "Unknown parameter %s", var);
dirtycode/alsa-lib/src/conf.c (line 5584)
[function] if (snd_config_get_type(def) != SND_CONFIG_TYPE_COMPOUND) {
dirtycode/alsa-lib/src/conf.c (line 5585)
[declaration] snd_error(CORE, "Parameter %s definition is not correct", var);
dirtycode/alsa-lib/src/conf.c (line 5592)
[declaration] snd_config_delete(sub);
dirtycode/alsa-lib/src/conf.c (line 5595)
[function] if (err < 0) {
dirtycode/alsa-lib/src/conf.c (line 5597)
[declaration] snd_error(CORE, "Parameter %s definition is missing a valid type
info", var);
dirtycode/alsa-lib/src/conf.c (line 5603)
[function] if (strcmp(tmp, "integer") == 0) {
dirtycode/alsa-lib/src/conf.c (line 5609)
[function] if (err < 0) {
dirtycode/alsa-lib/src/conf.c (line 5610)
[declaration] snd_error(CORE, "Parameter %s must be an integer", var);
dirtycode/alsa-lib/src/conf.c (line 5622)
[function] if (err < 0) {
dirtycode/alsa-lib/src/conf.c (line 5623)
[declaration] snd_error(CORE, "Parameter %s must be an integer", var);

```

```

dirtycode/alsa-lib/src/conf.c (line 5635)
[function] if (err < 0) {
dirtycode/alsa-lib/src/conf.c (line 5636)
[declaration] snd_error(CORE, "Parameter %s must be a real", var);
dirtycode/alsa-lib/src/conf.c (line 5657)
[function] if (err < 0) {
dirtycode/alsa-lib/src/conf.c (line 5660)
[declaration] snd_config_delete(sub);
dirtycode/alsa-lib/src/conf.c (line 5661)
[declaration] free(val);
dirtycode/alsa-lib/src/conf.c (line 5664)
[declaration] free(val);
dirtycode/alsa-lib/src/conf.c (line 5703)
[declaration] snd_error(CORE, "Expand error (walk): %s", snd_strerror(err));
dirtycode/alsa-lib/src/conf.c (line 5734)
[function] if (args != NULL) {
dirtycode/alsa-lib/src/conf.c (line 5735)
[declaration] snd_error(CORE, "Unknown parameters %s", args);
dirtycode/alsa-lib/src/conf.c (line 5746)
[function] if (err < 0) {
dirtycode/alsa-lib/src/conf.c (line 5747)
[declaration] snd_error(CORE, "Load defaults error: %s", snd_strerror(err));
dirtycode/alsa-lib/src/conf.c (line 5751)
[function] if (err < 0) {
dirtycode/alsa-lib/src/conf.c (line 5752)
[declaration] snd_error(CORE, "Parse arguments error: %s", snd_strerror(err));
dirtycode/alsa-lib/src/conf.c (line 5756)
[function] if (err < 0) {
dirtycode/alsa-lib/src/conf.c (line 5757)
[declaration] snd_error(CORE, "Args evaluate error: %s", snd_strerror(err));
dirtycode/alsa-lib/src/conf.c (line 5761)
[function] if (err < 0) {
dirtycode/alsa-lib/src/conf.c (line 5762)
[declaration] snd_error(CORE, "Expand error (walk): %s", snd_strerror(err));
dirtycode/alsa-lib/src/conf.c (line 5768)
[declaration] snd_error(CORE, "Evaluate error: %s", snd_strerror(err));
dirtycode/alsa-lib/src/conf.c (line 5769)
[declaration] snd_config_delete(res);
dirtycode/alsa-lib/src/conf.c (line 5776)
[declaration] snd_config_delete(subs);
dirtycode/alsa-lib/src/conf.c (line 5820)
[declaration] memcpy(key, name, args - name - 1);
dirtycode/alsa-lib/src/conf.c (line 5832)
[declaration] snd_config_unlock();
dirtycode/alsa-lib/src/conf.c (line 5849)
[function] if (conf->hop >= SND_CONF_MAX_HOPS) {
dirtycode/alsa-lib/src/conf.c (line 5850)
[declaration] snd_error(CORE, "Too many definition levels (looped?)\n");

```

```

dirtycode/alsa-lib/src/conf.c (line 5861)
[declaration] void _snd_config_end(void) __attribute__((destructor));
dirtycode/alsa-lib/src/conf.c (line 5867)
[declaration] snd_config_delete(snd_config);
dirtycode/alsa-lib/src/conf.c (line 5870)
[declaration] free(files_info[k].name);
dirtycode/alsa-lib/src/confeval.c (line 49)
[function] if (*s == begin) {
dirtycode/alsa-lib/src/confeval.c (line 116)
[function] if (pos == END) {
dirtycode/alsa-lib/src/confeval.c (line 117)
[declaration] snd_error(CORE, "unexpected expression tail '%s'", s);
dirtycode/alsa-lib/src/confeval.c (line 120)
[function] if (pos == OP) {
dirtycode/alsa-lib/src/confeval.c (line 121)
[function] switch (c) {
dirtycode/alsa-lib/src/confeval.c (line 130)
[declaration] snd_error(CORE, "unknown operation '%c'", c);
dirtycode/alsa-lib/src/confeval.c (line 137)
[function] if (c == '(') {
dirtycode/alsa-lib/src/confeval.c (line 142)
[function] if (s[1] == '[') {
dirtycode/alsa-lib/src/confeval.c (line 151)
[declaration] memcpy(m, s + off, e - s - off);
dirtycode/alsa-lib/src/confeval.c (line 154)
[declaration] free(m);
dirtycode/alsa-lib/src/confeval.c (line 162)
[function] while (*e) {
dirtycode/alsa-lib/src/confeval.c (line 170)
[declaration] memcpy(m, s + 1, e - s - 1);
dirtycode/alsa-lib/src/confeval.c (line 173)
[declaration] free(m);
dirtycode/alsa-lib/src/confeval.c (line 176)
[function] if (tmp == NULL) {
dirtycode/alsa-lib/src/confeval.c (line 184)
[declaration] snd_config_delete(tmp);
dirtycode/alsa-lib/src/confeval.c (line 195)
[declaration] snd_error(CORE, "incomplete expression '%s'", save);
dirtycode/alsa-lib/src/confeval.c (line 200)
[function] switch (op) {
dirtycode/alsa-lib/src/confeval.c (line 213)
[declaration] return snd_config_imake_integer64(dst, NULL, left);
dirtycode/alsa-lib/src/confeval.c (line 215)
[declaration] return snd_config_imake_integer(dst, NULL, left);
dirtycode/alsa-lib/src/confeval.c (line 239)
[declaration] snd_error(CORE, "wrong expression '%s'", s);
dirtycode/alsa-lib/src/confmisc.c (line 128)
[function] if (v < 0 || v > 1) {

```

```

dirtycode/alsa-lib/src/confmisc.c (line 130)
[declaration] snd_error(CORE, "Invalid value for %s", id);
dirtycode/alsa-lib/src/confmisc.c (line 137)
[declaration] snd_error(CORE, "Invalid type for %s", id);
dirtycode/alsa-lib/src/confmisc.c (line 158)
[function] if (snd_config_get_string(conf, &str)) {
dirtycode/alsa-lib/src/confmisc.c (line 160)
[declaration] snd_error(CORE, "Invalid field %s", id);
dirtycode/alsa-lib/src/confmisc.c (line 164)
[function] if (err < 0) {
dirtycode/alsa-lib/src/confmisc.c (line 165)
[declaration] snd_error(CORE, "Cannot get card index for %s", str);
dirtycode/alsa-lib/src/confmisc.c (line 185)
[function] if (safe_strtol(ascii, &v) >= 0) {
dirtycode/alsa-lib/src/confmisc.c (line 214)
[function] if (v < 0 || v > SND_CTL_ELEM_IFACE_LAST) {
dirtycode/alsa-lib/src/confmisc.c (line 216)
[declaration] snd_error(CORE, "Invalid value for %s", id);
dirtycode/alsa-lib/src/confmisc.c (line 223)
[declaration] snd_error(CORE, "Invalid type for %s", id);
dirtycode/alsa-lib/src/confmisc.c (line 266)
[declaration] snd_error(CORE, "field vars not found");
dirtycode/alsa-lib/src/confmisc.c (line 271)
[declaration] snd_error(CORE, "error evaluating vars");
dirtycode/alsa-lib/src/confmisc.c (line 276)
[declaration] snd_error(CORE, "field default not found");
dirtycode/alsa-lib/src/confmisc.c (line 281)
[declaration] snd_error(CORE, "error evaluating default");
dirtycode/alsa-lib/src/confmisc.c (line 286)
[declaration] snd_error(CORE, "error getting field default");
dirtycode/alsa-lib/src/confmisc.c (line 291)
[function] snd_config_for_each(i, next, n) {
dirtycode/alsa-lib/src/confmisc.c (line 297)
[function] if (snd_config_get_type(n) != SND_CONFIG_TYPE_STRING) {
dirtycode/alsa-lib/src/confmisc.c (line 298)
[declaration] snd_error(CORE, "field %s is not a string", id);
dirtycode/alsa-lib/src/confmisc.c (line 303)
[function] if (err < 0) {
dirtycode/alsa-lib/src/confmisc.c (line 304)
[declaration] snd_error(CORE, "id of field %s is not an integer", id);
dirtycode/alsa-lib/src/confmisc.c (line 308)
[function] if (i == idx) {
dirtycode/alsa-lib/src/confmisc.c (line 311)
[function] if (err < 0) {
dirtycode/alsa-lib/src/confmisc.c (line 312)
[declaration] snd_error(CORE, "invalid string for id %s", id);
dirtycode/alsa-lib/src/confmisc.c (line 368)
[declaration] snd_config_delete(d);

```

```

dirtycode/alsa-lib/src/confmisc.c (line 373)
[declaration] snd_config_delete(d);
dirtycode/alsa-lib/src/confmisc.c (line 417)
[declaration] snd_error(CORE, "field strings not found");
dirtycode/alsa-lib/src/confmisc.c (line 422)
[declaration] snd_error(CORE, "error evaluating strings");
dirtycode/alsa-lib/src/confmisc.c (line 427)
[function] snd_config_for_each(i, next, n) {
dirtycode/alsa-lib/src/confmisc.c (line 435)
[function] if (err < 0) {
dirtycode/alsa-lib/src/confmisc.c (line 436)
[declaration] snd_error(CORE, "id of field %s is not an integer", id);
dirtycode/alsa-lib/src/confmisc.c (line 440)
[function] if (i == idx) {
dirtycode/alsa-lib/src/confmisc.c (line 443)
[function] if (err < 0) {
dirtycode/alsa-lib/src/confmisc.c (line 444)
[declaration] snd_error(CORE, "invalid ascii string for id %s", id);
dirtycode/alsa-lib/src/confmisc.c (line 450)
[function] if (tmp == NULL) {
dirtycode/alsa-lib/src/confmisc.c (line 451)
[declaration] free(ptr);
dirtycode/alsa-lib/src/confmisc.c (line 455)
[declaration] memcpy(tmp + len, ptr, len1);
dirtycode/alsa-lib/src/confmisc.c (line 456)
[declaration] free(ptr);
dirtycode/alsa-lib/src/confmisc.c (line 465)
[declaration] snd_error(CORE, "empty string is not accepted");
dirtycode/alsa-lib/src/confmisc.c (line 496)
[declaration] snd_error(CORE, "field integers not found");
dirtycode/alsa-lib/src/confmisc.c (line 501)
[declaration] snd_error(CORE, "error evaluating integers");
dirtycode/alsa-lib/src/confmisc.c (line 506)
[function] snd_config_for_each(i, next, n) {
dirtycode/alsa-lib/src/confmisc.c (line 513)
[function] if (err < 0) {
dirtycode/alsa-lib/src/confmisc.c (line 514)
[declaration] snd_error(CORE, "id of field %s is not an integer", id);
dirtycode/alsa-lib/src/confmisc.c (line 518)
[function] if (i == idx) {
dirtycode/alsa-lib/src/confmisc.c (line 521)
[function] if (err < 0) {
dirtycode/alsa-lib/src/confmisc.c (line 522)
[declaration] snd_error(CORE, "invalid integer for id %s", id);
dirtycode/alsa-lib/src/confmisc.c (line 526)
[function] switch (op) {
dirtycode/alsa-lib/src/confmisc.c (line 566)
[declaration] return snd_func_iops(dst, root, src, private_data, 0);

```

```
dirtycode/alsa-lib/src/confmisc.c (line 595)
[declaration] return snd_func_iops(dst, root, src, private_data, 1);
dirtycode/alsa-lib/src/confmisc.c (line 626)
[declaration] return snd_config_imake_string(dst, id, snd_config_topdir());
dirtycode/alsa-lib/src/confmisc.c (line 637)
[declaration] return snd_ctl_open(ctl, name, 0);
dirtycode/alsa-lib/src/confmisc.c (line 660)
[declaration] return snd_config_copy(dst, src);
dirtycode/alsa-lib/src/confmisc.c (line 669)
[declaration] snd_error(CORE, "field %s not found", id);
dirtycode/alsa-lib/src/confmisc.c (line 704)
[declaration] snd_error(CORE, "field string is not a string");
dirtycode/alsa-lib/src/confmisc.c (line 745)
[declaration] snd_error(CORE, "field integer is not a string");
dirtycode/alsa-lib/src/confmisc.c (line 768)
[declaration] snd_error(CORE, "could not open control for card %i", card);
dirtycode/alsa-lib/src/confmisc.c (line 773)
[declaration] snd_error(CORE, "snd_ctl_card_info error: %s", snd_strerror(err));
dirtycode/alsa-lib/src/confmisc.c (line 785)
[declaration] snd_ctl_close(ctl);
dirtycode/alsa-lib/src/confmisc.c (line 817)
[declaration] snd_error(CORE, "field card not found");
dirtycode/alsa-lib/src/confmisc.c (line 822)
[declaration] snd_error(CORE, "field card is not an integer");
dirtycode/alsa-lib/src/confmisc.c (line 846)
[declaration] snd_error(CORE, "field card not found");
dirtycode/alsa-lib/src/confmisc.c (line 851)
[declaration] snd_error(CORE, "error evaluating card");
dirtycode/alsa-lib/src/confmisc.c (line 856)
[declaration] snd_error(CORE, "field card is not an integer or a string");
dirtycode/alsa-lib/src/confmisc.c (line 861)
[declaration] snd_error(CORE, "cannot find card '%s'", str);
dirtycode/alsa-lib/src/confmisc.c (line 968)
[declaration] snd_error(CORE, "could not open control for card %i", card);
dirtycode/alsa-lib/src/confmisc.c (line 973)
[declaration] snd_error(CORE, "snd_ctl_card_info error: %s", snd_strerror(err));
dirtycode/alsa-lib/src/confmisc.c (line 979)
[declaration] snd_ctl_card_info_get_id(&info));
dirtycode/alsa-lib/src/confmisc.c (line 982)
[declaration] snd_ctl_close(ctl);
dirtycode/alsa-lib/src/confmisc.c (line 1019)
[declaration] snd_error(CORE, "could not open control for card %i", card);
dirtycode/alsa-lib/src/confmisc.c (line 1024)
[declaration] snd_error(CORE, "snd_ctl_card_info error: %s", snd_strerror(err));
dirtycode/alsa-lib/src/confmisc.c (line 1030)
[declaration] snd_ctl_card_info_get_name(&info));
dirtycode/alsa-lib/src/confmisc.c (line 1033)
[declaration] snd_ctl_close(ctl);
```

```

dirtycode/alsa-lib/src/confmisc.c (line 1084)
[declaration] snd_error(CORE, "field device not found");
dirtycode/alsa-lib/src/confmisc.c (line 1089)
[declaration] snd_error(CORE, "error evaluating device");
dirtycode/alsa-lib/src/confmisc.c (line 1094)
[declaration] snd_error(CORE, "field device is not an integer");
dirtycode/alsa-lib/src/confmisc.c (line 1099)
[function] if (err < 0) {
dirtycode/alsa-lib/src/confmisc.c (line 1100)
[declaration] snd_error(CORE, "error evaluating subdevice");
dirtycode/alsa-lib/src/confmisc.c (line 1104)
[function] if (err < 0) {
dirtycode/alsa-lib/src/confmisc.c (line 1105)
[declaration] snd_error(CORE, "field subdevice is not an integer");
dirtycode/alsa-lib/src/confmisc.c (line 1111)
[declaration] snd_error(CORE, "could not open control for card %li", card);
dirtycode/alsa-lib/src/confmisc.c (line 1118)
[declaration] snd_error(CORE, "snd_ctl_pcm_info error: %s", snd_strerror(err));
dirtycode/alsa-lib/src/confmisc.c (line 1124)
[declaration] snd_pcm_info_get_id(&info));
dirtycode/alsa-lib/src/confmisc.c (line 1127)
[declaration] snd_ctl_close(ctl);
dirtycode/alsa-lib/src/confmisc.c (line 1167)
[declaration] snd_error(CORE, "field class not found");
dirtycode/alsa-lib/src/confmisc.c (line 1172)
[declaration] snd_error(CORE, "error evaluating class");
dirtycode/alsa-lib/src/confmisc.c (line 1177)
[declaration] snd_error(CORE, "field class is not an integer");
dirtycode/alsa-lib/src/confmisc.c (line 1182)
[declaration] snd_error(CORE, "field index not found");
dirtycode/alsa-lib/src/confmisc.c (line 1187)
[declaration] snd_error(CORE, "error evaluating index");
dirtycode/alsa-lib/src/confmisc.c (line 1192)
[declaration] snd_error(CORE, "field index is not an integer");
dirtycode/alsa-lib/src/confmisc.c (line 1198)
[function] if (err < 0) {
dirtycode/alsa-lib/src/confmisc.c (line 1199)
[declaration] snd_error(CORE, "could not get next card");
dirtycode/alsa-lib/src/confmisc.c (line 1205)
[function] if (err < 0) {
dirtycode/alsa-lib/src/confmisc.c (line 1206)
[declaration] snd_error(CORE, "could not open control for card %i", card);
dirtycode/alsa-lib/src/confmisc.c (line 1210)
[function] while(1) {
dirtycode/alsa-lib/src/confmisc.c (line 1212)
[function] if (err < 0) {
dirtycode/alsa-lib/src/confmisc.c (line 1213)
[declaration] snd_error(CORE, "could not get next pcm for card %i", card);

```

```

dirtycode/alsa-lib/src/confmisc.c (line 1218)
[declaration] snd_pcm_info_set_device(&info, dev);
dirtycode/alsa-lib/src/confmisc.c (line 1226)
[declaration] snd_ctl_close(ctl);
dirtycode/alsa-lib/src/confmisc.c (line 1233)
[declaration] snd_ctl_close(ctl);
dirtycode/alsa-lib/src/confmisc.c (line 1238)
[declaration] snprintf(name, sizeof(name), "CARD=%i,DEV=%i", card, dev);
dirtycode/alsa-lib/src/confmisc.c (line 1274)
[declaration] return snd_config_copy(dst, src);
dirtycode/alsa-lib/src/confmisc.c (line 1277)
[declaration] snd_error(CORE, "field pcm_handle not found");
dirtycode/alsa-lib/src/confmisc.c (line 1283)
[declaration] snd_error(CORE, "field pcm_handle is not a pointer");
dirtycode/alsa-lib/src/confmisc.c (line 1288)
[declaration] snd_error(CORE, "snd_ctl_pcm_info error: %s", snd_strerror(err));
dirtycode/alsa-lib/src/confmisc.c (line 1294)
[declaration] snd_pcm_info_get_subdevice(&info));
dirtycode/alsa-lib/src/confmisc.c (line 1334)
[function] if (err < 0) {
dirtycode/alsa-lib/src/confmisc.c (line 1335)
[declaration] snd_error(CORE, "error evaluating file");
dirtycode/alsa-lib/src/confmisc.c (line 1339)
[function] if (err < 0) {
dirtycode/alsa-lib/src/confmisc.c (line 1340)
[declaration] snd_error(CORE, "file is not a string");
dirtycode/alsa-lib/src/confmisc.c (line 1347)
[function] if (err < 0) {
dirtycode/alsa-lib/src/confmisc.c (line 1348)
[declaration] snd_error(CORE, "error evaluating name");
dirtycode/alsa-lib/src/confmisc.c (line 1352)
[function] if (err < 0) {
dirtycode/alsa-lib/src/confmisc.c (line 1353)
[declaration] snd_error(CORE, "name is not a string");
dirtycode/alsa-lib/src/confmisc.c (line 1359)
[declaration] snd_error(CORE, "name is not specified");
dirtycode/alsa-lib/src/confmisc.c (line 1365)
[function] if (err < 0) {
dirtycode/alsa-lib/src/confmisc.c (line 1366)
[declaration] snd_error(CORE, "Unable to open file %s: %s", file,
snd_strerror(err));
dirtycode/alsa-lib/src/confmisc.c (line 1370)
[declaration] snd_input_close(input);
dirtycode/alsa-lib/src/confmisc.c (line 1383)
[declaration] snd_error(CORE, "Unable to find definition '%s'", name);
dirtycode/alsa-lib/src/confmisc.c (line 1389)
[function] if ((err = snd_config_copy(dst, n)) >= 0) {
dirtycode/alsa-lib/src/confmisc.c (line 1392)
[declaration] snd_config_delete(*dst);

```

```

dirtycode/alsa-lib/src/control/cards.c (line 49)
[function] if (ioctl(open_dev, SNDRV_CTL_IOCTL_CARD_INFO, &info) < 0) {
dirtycode/alsa-lib/src/control/cards.c (line 51)
[declaration] close(open_dev);
dirtycode/alsa-lib/src/control/cards.c (line 54)
[declaration] close(open_dev);
dirtycode/alsa-lib/src/control/cards.c (line 71)
[declaration] sprintf(aload, SND_FILE_LOAD, card);
dirtycode/alsa-lib/src/control/cards.c (line 117)
[function] if (snd_card_load(card)) {
dirtycode/alsa-lib/src/control/cards.c (line 162)
[declaration] return snd_card_load2(string);
dirtycode/alsa-lib/src/control/cards.c (line 171)
[function] if (snd_ctl_card_info(handle, &info) < 0) {
dirtycode/alsa-lib/src/control/cards.c (line 172)
[declaration] snd_ctl_close(handle);
dirtycode/alsa-lib/src/control/cards.c (line 175)
[declaration] snd_ctl_close(handle);
dirtycode/alsa-lib/src/control/cards.c (line 203)
[declaration] snd_ctl_close(handle);
dirtycode/alsa-lib/src/control/cards.c (line 233)
[declaration] snd_ctl_close(handle);
dirtycode/alsa-lib/src/control/control.c (line 241)
[declaration] snd_async_del_handler(h);
dirtycode/alsa-lib/src/control/control.c (line 578)
[declaration] snd_ctl_elem_id_set_numid(&data.id, numid + i);
dirtycode/alsa-lib/src/control/control.c (line 660)
[declaration] snd_ctl_elem_id_set_numid(&data.id, numid + i);
dirtycode/alsa-lib/src/control/control.c (line 722)
[declaration] return __snd_ctl_add_elem_set(ctl, info, element_count, member_count);
dirtycode/alsa-lib/src/control/control.c (line 798)
[declaration] strcpy(p, labels[i]);
dirtycode/alsa-lib/src/control/control.c (line 857)
[declaration] return __snd_ctl_add_elem_set(ctl, info, element_count, member_count);
dirtycode/alsa-lib/src/control/control.c (line 918)
[declaration] return snd_ctl_add_boolean_elem_set(ctl, &info, 1, member_count);
dirtycode/alsa-lib/src/control/control.c (line 1056)
[function] if (id->numid == 0) {
dirtycode/alsa-lib/src/control/control.c (line 1064)
[declaration] free(info);
dirtycode/alsa-lib/src/control/control.c (line 1370)
[declaration] snd_error(CONTROL, "Invalid poll_fds %d", npfds);
dirtycode/alsa-lib/src/control/control.c (line 1378)
[declaration] snd_check(CONTROL, "invalid poll descriptors %d", err);
dirtycode/alsa-lib/src/control/control.c (line 1421)
[function] if (err < 0) {
dirtycode/alsa-lib/src/control/control.c (line 1422)
[declaration] snd_async_del_handler(h);

```

```

dirtycode/alsa-lib/src/control/control.c (line 1461)
[declaration] snd_error(CONTROL, "Invalid type for CTL %s definition", name);
dirtycode/alsa-lib/src/control/control.c (line 1463)
[declaration] snd_error(CONTROL, "Invalid type for CTL definition");
dirtycode/alsa-lib/src/control/control.c (line 1468)
[declaration] snd_error(CONTROL, "type is not defined");
dirtycode/alsa-lib/src/control/control.c (line 1473)
[declaration] snd_error(CONTROL, "unable to get id");
dirtycode/alsa-lib/src/control/control.c (line 1478)
[declaration] snd_error(CONTROL, "Invalid type for %s", id);
dirtycode/alsa-lib/src/control/control.c (line 1483)
[function] if (snd_config_get_type(type_conf) != SND_CONFIG_TYPE_COMPOUND) {
dirtycode/alsa-lib/src/control/control.c (line 1484)
[declaration] snd_error(CONTROL, "Invalid type for CTL type %s definition", str);
dirtycode/alsa-lib/src/control/control.c (line 1488)
[function] snd_config_for_each(i, next, type_conf) {
dirtycode/alsa-lib/src/control/control.c (line 1495)
[function] if (strcmp(id, "lib") == 0) {
dirtycode/alsa-lib/src/control/control.c (line 1497)
[function] if (err < 0) {
dirtycode/alsa-lib/src/control/control.c (line 1498)
[declaration] snd_error(CONTROL, "Invalid type for %s", id);
dirtycode/alsa-lib/src/control/control.c (line 1503)
[function] if (strcmp(id, "open") == 0) {
dirtycode/alsa-lib/src/control/control.c (line 1505)
[function] if (err < 0) {
dirtycode/alsa-lib/src/control/control.c (line 1506)
[declaration] snd_error(CONTROL, "Invalid type for %s", id);
dirtycode/alsa-lib/src/control/control.c (line 1511)
[declaration] snd_error(CONTROL, "Unknown field %s", id);
dirtycode/alsa-lib/src/control/control.c (line 1518)
[function] if (buf == NULL) {
dirtycode/alsa-lib/src/control/control.c (line 1523)
[declaration] sprintf(buf, "_snd_ctl%s_open", str);
dirtycode/alsa-lib/src/control/control.c (line 1527)
[function] while (*build_in) {
dirtycode/alsa-lib/src/control/control.c (line 1532)
[function] if (*build_in == NULL) {
dirtycode/alsa-lib/src/control/control.c (line 1534)
[function] if (buf1 == NULL) {
dirtycode/alsa-lib/src/control/control.c (line 1539)
[declaration] sprintf(buf1, "libasound_module_ctl%s.so", str);
dirtycode/alsa-lib/src/control/control.c (line 1546)
[declaration] SND_DLSYM_VERSION(SND_CONTROL_DLSYM_VERSION), 1);
dirtycode/alsa-lib/src/control/control.c (line 1549)
[function] if (err >= 0) {
dirtycode/alsa-lib/src/control/control.c (line 1553)
[declaration] snd_dlobj_cache_put(open_func);

```

```

dirtycode/alsa-lib/src/control/control.c (line 1560)
[declaration] snd_config_delete(type_conf);
dirtycode/alsa-lib/src/control/control.c (line 1575)
[declaration] snd_error(CONTROL, "Invalid CTL %s", name);
dirtycode/alsa-lib/src/control/control.c (line 1581)
[declaration] snd_config_set_hop(ctl_conf, hop);
dirtycode/alsa-lib/src/control/control.c (line 1599)
[declaration] return snd_ctl_open_noupdate(pctl, root, str, mode, hop + 1);
dirtycode/alsa-lib/src/control/control.c (line 1600)
[declaration] return snd_ctl_open_conf(pctl, name, root, conf, mode);
dirtycode/alsa-lib/src/control/control.c (line 1645)
[declaration] return snd_ctl_open_noupdate(ctlp, lconf, name, mode, 0);
dirtycode/alsa-lib/src/control/control.c (line 1664)
[declaration] free((*ctlp)->name);
dirtycode/alsa-lib/src/control/control.c (line 1882)
[declaration] return sizeof(snd_ctl_elem_id_t);
dirtycode/alsa-lib/src/control/control.c (line 2131)
[declaration] return sizeof(snd_ctl_card_info_t);
dirtycode/alsa-lib/src/control/control.c (line 2294)
[declaration] return sizeof(snd_ctl_event_t);
dirtycode/alsa-lib/src/control/control.c (line 2357)
[declaration] return sizeof(snd_ctl_elem_list_t);
dirtycode/alsa-lib/src/control/control.c (line 2569)
[declaration] return sizeof(snd_ctl_elem_info_t);
dirtycode/alsa-lib/src/control/control.c (line 3171)
[declaration] return sizeof(snd_ctl_elem_value_t);
dirtycode/alsa-lib/src/control/control.c (line 3241)
[declaration] return memcmp(left, right, sizeof(*left));
dirtycode/alsa-lib/src/control/control_empty.c (line 86)
[function] if (strcmp(id, "child") == 0) {
dirtycode/alsa-lib/src/control/control_empty.c (line 90)
[declaration] snd_error(CONTROL, "Unknown field %s", id);
dirtycode/alsa-lib/src/control/control_empty.c (line 94)
[declaration] snd_error(CONTROL, "child is not defined");
dirtycode/alsa-lib/src/control/control_empty.c (line 97)
[declaration] return _snd_ctl_open_named_child(handlep, name, root, child, mode,
conf);
dirtycode/alsa-lib/src/control/control_ext.c (line 106)
[declaration] snd_ctl_elem_id_clear(ids);
dirtycode/alsa-lib/src/control/control_ext.c (line 176)
[declaration] sizeof(info->value.enumerated.name));
dirtycode/alsa-lib/src/control/control_ext.c (line 244)
[declaration] sizeof(control->value.bytes.data));
dirtycode/alsa-lib/src/control/control_ext.c (line 297)
[declaration] sizeof(control->value.bytes.data));
dirtycode/alsa-lib/src/control/control_ext.c (line 367)
[declaration] memcpy(tlv, ext->tlv.p, len);
dirtycode/alsa-lib/src/control/control_ext.c (line 419)
[declaration] memset(event, 0, sizeof(*event));

```

```

dirtycode/alsa-lib/src/control/control_ext.c (line 559)
[function] if (strcmp(id, "my_own_parameter") == 0) {
dirtycode/alsa-lib/src/control/control_ext.c (line 563)
[declaration] snd_error(CONTROL, "Unknown field %s", id);
dirtycode/alsa-lib/src/control/control_ext.c (line 583)
[declaration] myctl_free(myctl);
dirtycode/alsa-lib/src/control/control_ext.c (line 715)
[declaration] snd_error(CONTROL, "ctl_ext: Plugin version mismatch");
dirtycode/alsa-lib/src/control/control_ext.c (line 741)
[declaration] return snd_ctl_close(ext->handle);
dirtycode/alsa-lib/src/control/control_hw.c (line 74)
[declaration] snd_erronum(CONTROL, "F_GETFL failed");
dirtycode/alsa-lib/src/control/control_hw.c (line 82)
[declaration] snd_erronum(CONTROL, "F_SETFL for O_NONBLOCK failed");
dirtycode/alsa-lib/src/control/control_hw.c (line 95)
[declaration] snd_erronum(CONTROL, "F_GETFL failed");
dirtycode/alsa-lib/src/control/control_hw.c (line 103)
[declaration] snd_erronum(CONTROL, "F_SETFL for O_ASYNC failed");
dirtycode/alsa-lib/src/control/control_hw.c (line 109)
[declaration] snd_erronum(CONTROL, "F_SETSIG failed");
dirtycode/alsa-lib/src/control/control_hw.c (line 113)
[declaration] snd_erronum(CONTROL, "F_SETOWN failed");
dirtycode/alsa-lib/src/control/control_hw.c (line 123)
[declaration] snd_erronum(CONTROL, "SNDRV_CTL_IOCTL_SUBSCRIBE_EVENTS failed");
dirtycode/alsa-lib/src/control/control_hw.c (line 133)
[declaration] snd_erronum(CONTROL, "SNDRV_CTL_IOCTL_CARD_INFO failed");
dirtycode/alsa-lib/src/control/control_hw.c (line 246)
[declaration] free(xtlv);
dirtycode/alsa-lib/src/control/control_hw.c (line 252)
[function] if (size > tlv_size) {
dirtycode/alsa-lib/src/control/control_hw.c (line 253)
[declaration] free(xtlv);
dirtycode/alsa-lib/src/control/control_hw.c (line 256)
[declaration] memcpy(tlv, xtlv->tlv, size);
dirtycode/alsa-lib/src/control/control_hw.c (line 293)
[declaration] return __snd_pcm_info_eld_fixup(info);
dirtycode/alsa-lib/src/control/control_hw.c (line 379)
[declaration] sizeof(*event), res);
dirtycode/alsa-lib/src/control/control_hw.c (line 441)
[declaration] snd_check(CONTROL, "Invalid card index %d", card);
dirtycode/alsa-lib/src/control/control_hw.c (line 455)
[declaration] snd_card_load(card);
dirtycode/alsa-lib/src/control/control_hw.c (line 462)
[declaration] close(fd);
dirtycode/alsa-lib/src/control/control_hw.c (line 466)
[declaration] close(fd);
dirtycode/alsa-lib/src/control/control_hw.c (line 471)
[declaration] close(fd);

```

```

dirtycode/alsa-lib/src/control/control_hw.c (line 480)
[declaration] close(fd);
dirtycode/alsa-lib/src/control/control_hw.c (line 481)
[declaration] free(hw);
dirtycode/alsa-lib/src/control/control_hw.c (line 537)
[function] if (strcmp(id, "card") == 0) {
dirtycode/alsa-lib/src/control/control_hw.c (line 548)
[declaration] return snd_ctl_hw_open(handlep, name, card, mode);
dirtycode/alsa-lib/src/control/control_remap.c (line 132)
[declaration] static int remap_load_list(snd_ctl_remap_t *priv);
dirtycode/alsa-lib/src/control/control_remap.c (line 149)
[declaration] return remap_numid_temp(priv, numid_app);
dirtycode/alsa-lib/src/control/control_remap.c (line 168)
[declaration] memset(numid + priv->numid_alloc, 0, sizeof(*numid) * 16);
dirtycode/alsa-lib/src/control/control_remap.c (line 192)
[declaration] return remap_numid_new(priv, numid_child, numid_app);
dirtycode/alsa-lib/src/control/control_remap.c (line 203)
[declaration] return remap_numid_temp(priv, numid_child);
dirtycode/alsa-lib/src/control/control_remap.c (line 208)
[declaration] return remap_numid_child_new(priv, numid_child);
dirtycode/alsa-lib/src/control/control_remap.c (line 285)
[declaration] return remap_find_map_numid(priv, id->numid);
dirtycode/alsa-lib/src/control/control_remap.c (line 304)
[function] if (rid->id_app.numid == 0) {
dirtycode/alsa-lib/src/control/control_remap.c (line 306)
[function] if (numid) {
dirtycode/alsa-lib/src/control/control_remap.c (line 314)
[function] if (numid) {
dirtycode/alsa-lib/src/control/control_remap.c (line 337)
[function] if (err >= 0 && rid->id_app.numid == 0) {
dirtycode/alsa-lib/src/control/control_remap.c (line 346)
[function] if (err >= 0) {
dirtycode/alsa-lib/src/control/control_remap.c (line 378)
[declaration] return remap_find_sync_numid(priv, id->numid);
dirtycode/alsa-lib/src/control/control_remap.c (line 397)
[function] if (snd_ctl_elem_id_compare_set(id, &sync->control_ids[index2]) == 0) {
dirtycode/alsa-lib/src/control/control_remap.c (line 424)
[declaration] return remap_find_sync_switch_numid(priv, id->numid);
dirtycode/alsa-lib/src/control/control_remap.c (line 443)
[declaration] free(sync->control_ids);
dirtycode/alsa-lib/src/control/control_remap.c (line 448)
[declaration] free(map->controls[idx2].channel_map);
dirtycode/alsa-lib/src/control/control_remap.c (line 449)
[declaration] free(map->controls);
dirtycode/alsa-lib/src/control/control_remap.c (line 470)
[declaration] return snd_ctl_nonblock(priv->child, nonblock);
dirtycode/alsa-lib/src/control/control_remap.c (line 476)
[declaration] return snd_ctl_async(priv->child, sig, pid);

```

```

dirtycode/alsa-lib/src/control/control_remap.c (line 482)
[declaration] return snd_ctl_subscribe_events(priv->child, subscribe);
dirtycode/alsa-lib/src/control/control_remap.c (line 488)
[declaration] return snd_ctl_card_info(priv->child, info);
dirtycode/alsa-lib/src/control/control_remap.c (line 506)
[function] if (rid) {
dirtycode/alsa-lib/src/control/control_remap.c (line 507)
[declaration] assert(id->numid > 0);
dirtycode/alsa-lib/src/control/control_remap.c (line 514)
[function] if (rid) {
dirtycode/alsa-lib/src/control/control_remap.c (line 550)
[declaration] return _snd_ctl_remap_elem_list(priv, list);
dirtycode/alsa-lib/src/control/control_remap.c (line 609)
[declaration] snd_ctl_elem_info_clear(&info3);
dirtycode/alsa-lib/src/control/control_remap.c (line 611)
[declaration] debug_id(&info3.id, "%s controls[%zd]\n", __func__, item);
dirtycode/alsa-lib/src/control/control_remap.c (line 681)
[declaration] remap_update_sync_id(priv, &info->id);
dirtycode/alsa-lib/src/control/control_remap.c (line 682)
[declaration] return remap_id_to_app(priv, &info->id, rid, err);
dirtycode/alsa-lib/src/control/control_remap.c (line 701)
[declaration] snd_ctl_elem_value_clear(&control2);
dirtycode/alsa-lib/src/control/control_remap.c (line 703)
[declaration] debug_id(&control2.id, "%s controls[%zd]\n", __func__, item);
dirtycode/alsa-lib/src/control/control_remap.c (line 776)
[declaration] return remap_id_to_app(priv, &control->id, rid, err);
dirtycode/alsa-lib/src/control/control_remap.c (line 794)
[declaration] snd_ctl_elem_value_clear(&control2);
dirtycode/alsa-lib/src/control/control_remap.c (line 796)
[declaration] debug_id(&control2.id, "%s controls[%zd]\n", __func__, item);
dirtycode/alsa-lib/src/control/control_remap.c (line 806)
[function] if ((unsigned long)dst < ARRAY_SIZE(control->value.integer.value)) {
dirtycode/alsa-lib/src/control/control_remap.c (line 812)
[function] if ((unsigned long)dst < ARRAY_SIZE(control->value.integer.value)) {
dirtycode/alsa-lib/src/control/control_remap.c (line 822)
[function] if ((unsigned long)dst < ARRAY_SIZE(control->value.integer64.value)) {
dirtycode/alsa-lib/src/control/control_remap.c (line 828)
[function] if ((unsigned long)dst < ARRAY_SIZE(control->value.integer64.value)) {
dirtycode/alsa-lib/src/control/control_remap.c (line 840)
[function] if ((unsigned long)dst < ARRAY_SIZE(control->value.bytes.data)) {
dirtycode/alsa-lib/src/control/control_remap.c (line 846)
[declaration] debug_id(&control2.id, "%s changes %d\n", __func__, changes);
dirtycode/alsa-lib/src/control/control_remap.c (line 847)
[function] if (changes > 0) {
dirtycode/alsa-lib/src/control/control_remap.c (line 878)
[declaration] debug_id(&control2.id, "%s sync[%zd]\n", __func__, item);
dirtycode/alsa-lib/src/control/control_remap.c (line 884)
[declaration] return remap_id_to_app(priv, &control->id, NULL, 0);

```

```

dirtycode/alsa-lib/src/control/control_remap.c (line 904)
[declaration] return remap_id_to_app(priv, &control->id, rid, err);
dirtycode/alsa-lib/src/control/control_remap.c (line 918)
[declaration] return remap_id_to_app(priv, id, rid, err);
dirtycode/alsa-lib/src/control/control_remap.c (line 932)
[declaration] return remap_id_to_app(priv, id, rid, err);
dirtycode/alsa-lib/src/control/control_remap.c (line 985)
[function] if (err < 0) {
dirtycode/alsa-lib/src/control/control_remap.c (line 986)
[declaration] free(tlv2);
dirtycode/alsa-lib/src/control/control_remap.c (line 989)
[declaration] memset(tlv2, 0, tlv_size);
dirtycode/alsa-lib/src/control/control_remap.c (line 991)
[function] if (err < 0) {
dirtycode/alsa-lib/src/control/control_remap.c (line 992)
[declaration] free(tlv2);
dirtycode/alsa-lib/src/control/control_remap.c (line 995)
[function] if (memcmp(tlv, tlv2, tlv_size) != 0) {
dirtycode/alsa-lib/src/control/control_remap.c (line 996)
[declaration] free(tlv2);
dirtycode/alsa-lib/src/control/control_remap.c (line 1025)
[declaration] return snd_ctl_hwdep_next_device(priv->child, device);
dirtycode/alsa-lib/src/control/control_remap.c (line 1031)
[declaration] return snd_ctl_hwdep_info(priv->child, info);
dirtycode/alsa-lib/src/control/control_remap.c (line 1037)
[declaration] return snd_ctl_pcm_next_device(priv->child, device);
dirtycode/alsa-lib/src/control/control_remap.c (line 1043)
[declaration] return snd_ctl_pcm_info(priv->child, info);
dirtycode/alsa-lib/src/control/control_remap.c (line 1049)
[declaration] return snd_ctl_pcm_prefer_subdevice(priv->child, subdev);
dirtycode/alsa-lib/src/control/control_remap.c (line 1055)
[declaration] return snd_ctl_rawmidi_next_device(priv->child, device);
dirtycode/alsa-lib/src/control/control_remap.c (line 1061)
[declaration] return snd_ctl_rawmidi_info(priv->child, info);
dirtycode/alsa-lib/src/control/control_remap.c (line 1067)
[declaration] return snd_ctl_rawmidi_prefer_subdevice(priv->child, subdev);
dirtycode/alsa-lib/src/control/control_remap.c (line 1073)
[declaration] return snd_ctl_set_power_state(priv->child, state);
dirtycode/alsa-lib/src/control/control_remap.c (line 1079)
[declaration] return snd_ctl_get_power_state(priv->child, state);
dirtycode/alsa-lib/src/control/control_remap.c (line 1097)
[function] if (priv->event_queue[head].numid_app == numid_app) {
dirtycode/alsa-lib/src/control/control_remap.c (line 1128)
[function] if (mctl->id_child.numid == 0) {
dirtycode/alsa-lib/src/control/control_remap.c (line 1135)
[declaration] debug_id(&map->map_id, "%s found (all)\n", __func__);
dirtycode/alsa-lib/src/control/control_remap.c (line 1139)
[declaration] event_add(priv, &map->map_id, map->map_id.numid, event_mask);

```

```

dirtycode/alsa-lib/src/control/control_remap.c (line 1160)
[function] if (sid->numid == 0) {
dirtycode/alsa-lib/src/control/control_remap.c (line 1167)
[declaration] debug_id(sid, "%s found (all)\n", __func__);
dirtycode/alsa-lib/src/control/control_remap.c (line 1180)
[declaration] event_add(priv, sid, numid->numid_app, event_mask);
dirtycode/alsa-lib/src/control/control_remap.c (line 1196)
[declaration] _next_ptr(&priv->event_queue_head, priv->event_items);
dirtycode/alsa-lib/src/control/control_remap.c (line 1197)
[declaration] memset(event, 0, sizeof(*event));
dirtycode/alsa-lib/src/control/control_remap.c (line 1202)
[declaration] debug_id(&event->data.elem.id, "%s queue read\n", __func__);
dirtycode/alsa-lib/src/control/control_remap.c (line 1211)
[declaration] debug_id(&event->data.elem.id, "%s event mask 0x%x\n", __func__,
event->data.elem.mask);
dirtycode/alsa-lib/src/control/control_remap.c (line 1213)
[declaration] remap_event_for_all_map_controls(priv, &event->data.elem.id,
event->data.elem.mask);
dirtycode/alsa-lib/src/control/control_remap.c (line 1214)
[declaration] remap_event_for_all_sync_controls(priv, &event->data.elem.id,
event->data.elem.mask);
dirtycode/alsa-lib/src/control/control_remap.c (line 1216)
[function] if (rid) {
dirtycode/alsa-lib/src/control/control_remap.c (line 1217)
[function] if (rid->id_child.numid == 0) {
dirtycode/alsa-lib/src/control/control_remap.c (line 1232)
[declaration] remap_forget_numid_child(priv, numid_child);
dirtycode/alsa-lib/src/control/control_remap.c (line 1274)
[declaration] memset(rid + priv->remap_alloc, 0, sizeof(*rid) * 16);
dirtycode/alsa-lib/src/control/control_remap.c (line 1299)
[function] if (snd_config_get_string(n, &str) < 0) {
dirtycode/alsa-lib/src/control/control_remap.c (line 1300)
[declaration] snd_error(CONTROL, "expected string with the target control id!");
dirtycode/alsa-lib/src/control/control_remap.c (line 1303)
[declaration] snd_ctl_elem_id_clear(&app);
dirtycode/alsa-lib/src/control/control_remap.c (line 1305)
[function] if (err < 0) {
dirtycode/alsa-lib/src/control/control_remap.c (line 1306)
[declaration] snd_error(CONTROL, "unable to parse target id '%s'", str);
dirtycode/alsa-lib/src/control/control_remap.c (line 1309)
[function] if (remap_find_id_app(priv, &app)) {
dirtycode/alsa-lib/src/control/control_remap.c (line 1310)
[declaration] snd_error(CONTROL, "duplicate target id '%s'", id);
dirtycode/alsa-lib/src/control/control_remap.c (line 1313)
[declaration] snd_ctl_elem_id_clear(&child);
dirtycode/alsa-lib/src/control/control_remap.c (line 1315)
[function] if (err < 0) {
dirtycode/alsa-lib/src/control/control_remap.c (line 1316)
[declaration] snd_error(CONTROL, "unable to parse source id '%s'", id);

```

```

dirtycode/alsa-lib/src/control/control_remap.c (line 1319)
[function] if (remap_find_id_child(priv, &app)) {
dirtycode/alsa-lib/src/control/control_remap.c (line 1320)
[declaration] snd_error(CONTROL, "duplicate source id '%s'!", id);
dirtycode/alsa-lib/src/control/control_remap.c (line 1340)
[declaration] memset(map + priv->map_alloc, 0, sizeof(*map) * 16);
dirtycode/alsa-lib/src/control/control_remap.c (line 1363)
[declaration] memset(mctl + map->controls_alloc, 0, sizeof(*mctl) * 4);
dirtycode/alsa-lib/src/control/control_remap.c (line 1379)
[declaration] snd_error(CONTROL, "Wrong channel mapping (extra source channel?");
dirtycode/alsa-lib/src/control/control_remap.c (line 1407)
[function] if (safe_strtol(dst_id, &idx) || snd_config_get_integer(n, &chn)) {
dirtycode/alsa-lib/src/control/control_remap.c (line 1408)
[declaration] snd_error(CONTROL, "Wrong channel mapping (%ld -> %ld)", idx, chn);
dirtycode/alsa-lib/src/control/control_remap.c (line 1437)
[function] if (snd_config_is_array(n) > 0) {
dirtycode/alsa-lib/src/control/control_remap.c (line 1440)
[function] if (safe_strtol(id, &idx) || snd_config_get_integer(n, &chn)) {
dirtycode/alsa-lib/src/control/control_remap.c (line 1441)
[declaration] snd_error(CONTROL, "Wrong channel mapping (%ld -> %ld)", idx, chn);
dirtycode/alsa-lib/src/control/control_remap.c (line 1464)
[function] if (strcmp(id, "vindex") == 0) {
dirtycode/alsa-lib/src/control/control_remap.c (line 1485)
[declaration] snd_ctl_elem_id_clear(&cid);
dirtycode/alsa-lib/src/control/control_remap.c (line 1487)
[function] if (err < 0) {
dirtycode/alsa-lib/src/control/control_remap.c (line 1488)
[declaration] snd_error(CONTROL, "unable to parse control id '%s'!", id);
dirtycode/alsa-lib/src/control/control_remap.c (line 1516)
[declaration] snd_ctl_elem_id_clear(&eid);
dirtycode/alsa-lib/src/control/control_remap.c (line 1518)
[function] if (err < 0) {
dirtycode/alsa-lib/src/control/control_remap.c (line 1519)
[declaration] snd_error(CONTROL, "unable to parse id '%s'!", id);
dirtycode/alsa-lib/src/control/control_remap.c (line 1541)
[declaration] memset(sync + priv->sync_alloc, 0, sizeof(*sync) * 16);
dirtycode/alsa-lib/src/control/control_remap.c (line 1565)
[function] if (snd_config_get_string(n, &str) < 0) {
dirtycode/alsa-lib/src/control/control_remap.c (line 1566)
[declaration] snd_error(CONTROL, "strings are expected in sync array");
dirtycode/alsa-lib/src/control/control_remap.c (line 1570)
[declaration] snd_ctl_elem_id_clear(eid);
dirtycode/alsa-lib/src/control/control_remap.c (line 1572)
[function] if (err < 0) {
dirtycode/alsa-lib/src/control/control_remap.c (line 1573)
[declaration] snd_error(CONTROL, "unable to parse control id '%s'!", str);
dirtycode/alsa-lib/src/control/control_remap.c (line 1598)
[function] if (strcmp(id, "switch") == 0) {

```

```

dirtycode/alsa-lib/src/control/control_remap.c (line 1599)
[function] if (snd_config_get_string(n, &str) < 0) {
dirtycode/alsa-lib/src/control/control_remap.c (line 1600)
[declaration] snd_error(CONTROL, "String is expected for switch");
dirtycode/alsa-lib/src/control/control_remap.c (line 1604)
[function] if (err < 0) {
dirtycode/alsa-lib/src/control/control_remap.c (line 1605)
[declaration] snd_error(CONTROL, "unable to parse id '%s'!", str);
dirtycode/alsa-lib/src/control/control_remap.c (line 1610)
[function] if (strcmp(id, "controls") == 0) {
dirtycode/alsa-lib/src/control/control_remap.c (line 1612)
[function] if (count <= 0) {
dirtycode/alsa-lib/src/control/control_remap.c (line 1613)
[declaration] snd_error(CONTROL, "Array is expected for sync!");
dirtycode/alsa-lib/src/control/control_remap.c (line 1644)
[function] if (snd_config_get_type(n) == SND_CONFIG_TYPE_COMPOUND) {
dirtycode/alsa-lib/src/control/control_remap.c (line 1648)
[function] if (count <= 0) {
dirtycode/alsa-lib/src/control/control_remap.c (line 1649)
[declaration] snd_error(CONTROL, "Array is expected for sync!");
dirtycode/alsa-lib/src/control/control_remap.c (line 1711)
[declaration] remap_free(priv);
dirtycode/alsa-lib/src/control/control_remap.c (line 1713)
[declaration] free(child->name);
dirtycode/alsa-lib/src/control/control_remap.c (line 1860)
[function] if (strcmp(id, "remap") == 0) {
dirtycode/alsa-lib/src/control/control_remap.c (line 1864)
[function] if (strcmp(id, "map") == 0) {
dirtycode/alsa-lib/src/control/control_remap.c (line 1868)
[function] if (strcmp(id, "sync") == 0) {
dirtycode/alsa-lib/src/control/control_remap.c (line 1872)
[function] if (strcmp(id, "child") == 0) {
dirtycode/alsa-lib/src/control/control_remap.c (line 1876)
[declaration] snd_error(CONTROL, "Unknown field %s", id);
dirtycode/alsa-lib/src/control/control_remap.c (line 1880)
[declaration] snd_error(CONTROL, "child is not defined");
dirtycode/alsa-lib/src/control/control_remap.c (line 1888)
[declaration] snd_ctl_close(ctl);
dirtycode/alsa-lib/src/control/control_shm.c (line 63)
[declaration] snd_error(CONTROL, "Server has not done the cmd");
dirtycode/alsa-lib/src/control/control_shm.c (line 82)
[declaration] snd_error(CONTROL, "Server has not done the cmd");
dirtycode/alsa-lib/src/control/control_shm.c (line 115)
[declaration] return snd_ctl_shm_action(ctl);
dirtycode/alsa-lib/src/control/control_shm.c (line 136)
[declaration] return snd_ctl_shm_action(ctl);
dirtycode/alsa-lib/src/control/control_shm.c (line 307)
[declaration] return snd_ctl_shm_action(ctl);

```

```
dirtycode/alsa-lib/src/control/control_shm.c (line 344)
[declaration] return snd_ctl_shm_action(ctl);
dirtycode/alsa-lib/src/control/control_shm.c (line 353)
[declaration] return snd_ctl_shm_action(ctl);
dirtycode/alsa-lib/src/control/control_shm.c (line 428)
[declaration] close(sock);
dirtycode/alsa-lib/src/control/control_shm.c (line 451)
[declaration] snd_error(CONTROL, "server for socket %s is not running", sockname);
dirtycode/alsa-lib/src/control/control_shm.c (line 466)
[declaration] snd_error(CONTROL, "write error");
dirtycode/alsa-lib/src/control/control_shm.c (line 471)
[declaration] snd_error(CONTROL, "write size error");
dirtycode/alsa-lib/src/control/control_shm.c (line 477)
[declaration] snd_error(CONTROL, "read error");
dirtycode/alsa-lib/src/control/control_shm.c (line 482)
[declaration] snd_error(CONTROL, "read size error");
dirtycode/alsa-lib/src/control/control_shm.c (line 487)
[declaration] snd_error(CONTROL, "invalid data error");
dirtycode/alsa-lib/src/control/control_shm.c (line 519)
[declaration] snd_ctl_close(ctl);
dirtycode/alsa-lib/src/control/control_shm.c (line 528)
[declaration] close(sock);
dirtycode/alsa-lib/src/control/control_shm.c (line 530)
[declaration] shmctl(ctl);
dirtycode/alsa-lib/src/control/control_shm.c (line 552)
[function] if (strcmp(id, "server") == 0) {
dirtycode/alsa-lib/src/control/control_shm.c (line 554)
[function] if (err < 0) {
dirtycode/alsa-lib/src/control/control_shm.c (line 555)
[declaration] snd_error(CONTROL, "Invalid type for %s", id);
dirtycode/alsa-lib/src/control/control_shm.c (line 560)
[function] if (strcmp(id, "ctl") == 0) {
dirtycode/alsa-lib/src/control/control_shm.c (line 562)
[function] if (err < 0) {
dirtycode/alsa-lib/src/control/control_shm.c (line 563)
[declaration] snd_error(CONTROL, "Invalid type for %s", id);
dirtycode/alsa-lib/src/control/control_shm.c (line 568)
[declaration] snd_error(CONTROL, "Unknown field %s", id);
dirtycode/alsa-lib/src/control/control_shm.c (line 572)
[declaration] snd_error(CONTROL, "ctl is not defined");
dirtycode/alsa-lib/src/control/control_shm.c (line 576)
[declaration] snd_error(CONTROL, "server is not defined");
dirtycode/alsa-lib/src/control/control_shm.c (line 581)
[declaration] snd_error(CONTROL, "Unknown server %s", server);
dirtycode/alsa-lib/src/control/control_shm.c (line 585)
[declaration] snd_error(CONTROL, "Invalid type for server %s definition", server);
dirtycode/alsa-lib/src/control/control_shm.c (line 598)
[function] if (strcmp(id, "socket") == 0) {
```

```

dirtycode/alsa-lib/src/control/control_shm.c (line 600)
[function] if (err < 0) {
dirtycode/alsa-lib/src/control/control_shm.c (line 601)
[declaration] snd_error(CONTROL, "Invalid type for %s", id);
dirtycode/alsa-lib/src/control/control_shm.c (line 606)
[function] if (strcmp(id, "port") == 0) {
dirtycode/alsa-lib/src/control/control_shm.c (line 608)
[function] if (err < 0) {
dirtycode/alsa-lib/src/control/control_shm.c (line 609)
[declaration] snd_error(CONTROL, "Invalid type for %s", id);
dirtycode/alsa-lib/src/control/control_shm.c (line 614)
[declaration] snd_error(CONTROL, "Unknown field %s", id);
dirtycode/alsa-lib/src/control/control_shm.c (line 620)
[declaration] snd_error(CONTROL, "socket is not defined");
dirtycode/alsa-lib/src/control/ctlparse.c (line 127)
[declaration] numid, iface, snd_ctl_elem_id_get_name(id));
dirtycode/alsa-lib/src/control/ctlparse.c (line 130)
[declaration] iface, snd_ctl_elem_id_get_name(id));
dirtycode/alsa-lib/src/control/ctlparse.c (line 137)
[declaration] snprintf(buf1, sizeof(buf1), "index=%u", index);
dirtycode/alsa-lib/src/control/ctlparse.c (line 139)
[declaration] strcat(buf, buf1);
dirtycode/alsa-lib/src/control/ctlparse.c (line 142)
[declaration] snprintf(buf1, sizeof(buf1), "device=%u", device);
dirtycode/alsa-lib/src/control/ctlparse.c (line 144)
[declaration] strcat(buf, buf1);
dirtycode/alsa-lib/src/control/ctlparse.c (line 147)
[declaration] snprintf(buf1, sizeof(buf1), "subdevice=%u", subdevice);
dirtycode/alsa-lib/src/control/ctlparse.c (line 149)
[declaration] strcat(buf, buf1);
dirtycode/alsa-lib/src/control/ctlparse.c (line 151)
[declaration] return strdup(buf);
dirtycode/alsa-lib/src/control/ctlparse.c (line 171)
[function] if (!strncasecmp(str, "numid=", 6)) {
dirtycode/alsa-lib/src/control/ctlparse.c (line 175)
[function] while (*str && *str != ',') {
dirtycode/alsa-lib/src/control/ctlparse.c (line 176)
[function] if (size < (int)sizeof(buf) - 1) {
dirtycode/alsa-lib/src/control/ctlparse.c (line 185)
[function] if (l <= 0 || l >= INT32_MAX) {
dirtycode/alsa-lib/src/control/ctlparse.c (line 186)
[declaration] snd_error(CONTROL, "Invalid numid %ld (%s)", l, buf);
dirtycode/alsa-lib/src/control/ctlparse.c (line 189)
[declaration] snd_ctl_elem_id_set_numid(dst, (int)l);
dirtycode/alsa-lib/src/control/ctlparse.c (line 194)
[function] if (!strncasecmp(str, "card", 4)) {
dirtycode/alsa-lib/src/control/ctlparse.c (line 195)
[declaration] snd_ctl_elem_id_set_interface(dst, SND_CTL_ELEM_IFACE_CARD);

```

```

dirtycode/alsa-lib/src/control/ctlparse.c (line 198)
[declaration] snd_ctl_elem_id_set_interface(dst, SND_CTL_ELEM_IFACE_MIXER);
dirtycode/alsa-lib/src/control/ctlparse.c (line 201)
[declaration] snd_ctl_elem_id_set_interface(dst, SND_CTL_ELEM_IFACE_PCM);
dirtycode/alsa-lib/src/control/ctlparse.c (line 204)
[declaration] snd_ctl_elem_id_set_interface(dst, SND_CTL_ELEM_IFACE_RAWMIDI);
dirtycode/alsa-lib/src/control/ctlparse.c (line 207)
[declaration] snd_ctl_elem_id_set_interface(dst, SND_CTL_ELEM_IFACE_TIMER);
dirtycode/alsa-lib/src/control/ctlparse.c (line 210)
[declaration] snd_ctl_elem_id_set_interface(dst, SND_CTL_ELEM_IFACE_SEQUENCER);
dirtycode/alsa-lib/src/control/ctlparse.c (line 219)
[function] if (*str == '\\' || *str == '\"') {
dirtycode/alsa-lib/src/control/ctlparse.c (line 221)
[function] while (*str && *str != c) {
dirtycode/alsa-lib/src/control/ctlparse.c (line 222)
[function] if (size < (int)sizeof(buf) - 1) {
dirtycode/alsa-lib/src/control/ctlparse.c (line 231)
[function] while (*str && *str != ',') {
dirtycode/alsa-lib/src/control/ctlparse.c (line 232)
[function] if (size < (int)sizeof(buf) - 1) {
dirtycode/alsa-lib/src/control/ctlparse.c (line 240)
[declaration] snd_ctl_elem_id_set_name(dst, buf);
dirtycode/alsa-lib/src/control/ctlparse.c (line 243)
[declaration] snd_ctl_elem_id_set_index(dst, atoi(str));
dirtycode/alsa-lib/src/control/ctlparse.c (line 248)
[declaration] snd_ctl_elem_id_set_device(dst, atoi(str));
dirtycode/alsa-lib/src/control/ctlparse.c (line 253)
[declaration] snd_ctl_elem_id_set_subdevice(dst, atoi(str));
dirtycode/alsa-lib/src/control/ctlparse.c (line 257)
[function] if (*str == ',') {
dirtycode/alsa-lib/src/control/ctlparse.c (line 286)
[declaration] return __snd_ctl_ascii_elem_id_parse(dst, str, NULL);
dirtycode/alsa-lib/src/control/ctlparse.c (line 309)
[declaration] snd_ctl_elem_info_set_item(info, i);
dirtycode/alsa-lib/src/control/ctlparse.c (line 316)
[function] if (end == '\0' && (ptr[len] == '\0' || ptr[len] == ',' || ptr[len] ==
'\n')) {
dirtycode/alsa-lib/src/control/ctlparse.c (line 320)
[function] if (end != '\0' && ptr[len] == end) {
dirtycode/alsa-lib/src/control/ctlparse.c (line 335)
[declaration] return ARRAY_SIZE(value.value.integer.value);
dirtycode/alsa-lib/src/control/ctlparse.c (line 337)
[declaration] return ARRAY_SIZE(value.value.integer64.value);
dirtycode/alsa-lib/src/control/ctlparse.c (line 339)
[declaration] return ARRAY_SIZE(value.value.enumerated.item);
dirtycode/alsa-lib/src/control/ctlparse.c (line 341)
[declaration] return ARRAY_SIZE(value.value.bytes.data);
dirtycode/alsa-lib/src/control/ctlparse.c (line 381)
[function] switch (type) {

```

```

dirtycode/alsa-lib/src/control/ctlparse.c (line 403)
[declaration] snd_ctl_elem_value_set_boolean(dst, idx, tmp);
dirtycode/alsa-lib/src/control/ctlparse.c (line 408)
[declaration] snd_ctl_elem_info_get_max(info));
dirtycode/alsa-lib/src/control/ctlparse.c (line 409)
[declaration] snd_ctl_elem_value_set_integer(dst, idx, tmp);
dirtycode/alsa-lib/src/control/ctlparse.c (line 414)
[declaration] snd_ctl_elem_info_get_max64(info));
dirtycode/alsa-lib/src/control/ctlparse.c (line 415)
[declaration] snd_ctl_elem_value_set_integer64(dst, idx, tmp64);
dirtycode/alsa-lib/src/control/ctlparse.c (line 421)
[declaration] snd_ctl_elem_info_get_items(info) - 1);
dirtycode/alsa-lib/src/control/ctlparse.c (line 422)
[declaration] snd_ctl_elem_value_set_enumerated(dst, idx, tmp);
dirtycode/alsa-lib/src/control/ctlparse.c (line 426)
[declaration] snd_ctl_elem_value_set_byte(dst, idx, tmp);
dirtycode/alsa-lib/src/control/eld.c (line 56)
[declaration] snd_checknum(CONTROL, "Cannot open the associated CTL");
dirtycode/alsa-lib/src/control/eld.c (line 69)
[declaration] snd_checknum(CONTROL, "Cannot read ELD");
dirtycode/alsa-lib/src/control/eld.c (line 81)
[declaration] snd_error(CONTROL, "ELD decode failed, using old HDMI output names");
dirtycode/alsa-lib/src/control/eld.c (line 91)
[function] if (c <= ' ' || c >= 0x7f) {
dirtycode/alsa-lib/src/control/eld.c (line 102)
[declaration] snd_strncpy((char *)info->name, s, sizeof(info->name));
dirtycode/alsa-lib/src/control/eld.c (line 105)
[declaration] strncat((char *)info->name, " *", sizeof(info->name) - 1);
dirtycode/alsa-lib/src/control/hcontrol.c (line 79)
[declaration] snd_ctl_close(ctl);
dirtycode/alsa-lib/src/control/hcontrol.c (line 133)
[declaration] return snd_ctl_name(hctl->ctl);
dirtycode/alsa-lib/src/control/hcontrol.c (line 145)
[declaration] return snd_ctl_nonblock(hctl->ctl, nonblock);
dirtycode/alsa-lib/src/control/hcontrol.c (line 160)
[declaration] return snd_ctl_async(hctl->ctl, sig, pid);
dirtycode/alsa-lib/src/control/hcontrol.c (line 171)
[declaration] return snd_ctl_poll_descriptors_count(hctl->ctl);
dirtycode/alsa-lib/src/control/hcontrol.c (line 184)
[declaration] return snd_ctl_poll_descriptors(hctl->ctl, pfds, space);
dirtycode/alsa-lib/src/control/hcontrol.c (line 198)
[declaration] return snd_ctl_poll_descriptors_revents(hctl->ctl, pfds, nfds,
revents);
dirtycode/alsa-lib/src/control/hcontrol.c (line 222)
[function] if (!strcmp(*name, *names, strlen(*names))) {
dirtycode/alsa-lib/src/control/hcontrol.c (line 355)
[function] if (!h) {
dirtycode/alsa-lib/src/control/hcontrol.c (line 362)
[declaration] list_add_tail(&elem->list, &hctl->elems);

```

```

dirtycode/alsa-lib/src/control/hcontrol.c (line 366)
[declaration] assert(dir != 0);
dirtycode/alsa-lib/src/control/hcontrol.c (line 367)
[function] if (dir > 0) {
dirtycode/alsa-lib/src/control/hcontrol.c (line 368)
[declaration] list_add(&elem->list, &hctl->pelems[idx]->list);
dirtycode/alsa-lib/src/control/hcontrol.c (line 371)
[declaration] list_add_tail(&elem->list, &hctl->pelems[idx]->list);
dirtycode/alsa-lib/src/control/hcontrol.c (line 379)
[declaration] return snd_hctl_throw_event(hctl, SNDRV_CTL_EVENT_MASK_ADD, elem);
dirtycode/alsa-lib/src/control/hcontrol.c (line 394)
[declaration] m * sizeof(snd_hctl_elem_t *));
dirtycode/alsa-lib/src/control/hcontrol.c (line 405)
[declaration] snd_hctl_elem_remove(hctl, hctl->count - 1);
dirtycode/alsa-lib/src/control/hcontrol.c (line 439)
[declaration] list_add_tail(&hctl->pelems[k]->list, &hctl->elems);
dirtycode/alsa-lib/src/control/hcontrol.c (line 503)
[declaration] return list_entry(hctl->elems.next, snd_hctl_elem_t, list);
dirtycode/alsa-lib/src/control/hcontrol.c (line 516)
[declaration] return list_entry(hctl->elems.prev, snd_hctl_elem_t, list);
dirtycode/alsa-lib/src/control/hcontrol.c (line 529)
[declaration] return list_entry(elem->list.next, snd_hctl_elem_t, list);
dirtycode/alsa-lib/src/control/hcontrol.c (line 542)
[declaration] return list_entry(elem->list.prev, snd_hctl_elem_t, list);
dirtycode/alsa-lib/src/control/hcontrol.c (line 587)
[declaration] free(hctl->pelems);
dirtycode/alsa-lib/src/control/hcontrol.c (line 589)
[function] if (!hctl->pelems) {
dirtycode/alsa-lib/src/control/hcontrol.c (line 597)
[function] if (elem == NULL) {
dirtycode/alsa-lib/src/control/hcontrol.c (line 598)
[declaration] snd_hctl_free(hctl);
dirtycode/alsa-lib/src/control/hcontrol.c (line 606)
[declaration] list_add_tail(&elem->list, &hctl->elems);
dirtycode/alsa-lib/src/control/hcontrol.c (line 683)
[declaration] snd_error(CONTROL, "Invalid poll_fds %d", npfds);
dirtycode/alsa-lib/src/control/hcontrol.c (line 692)
[declaration] snd_check(CONTROL, "invalid poll descriptors %d", err);
dirtycode/alsa-lib/src/control/hcontrol.c (line 698)
[function] if (err_poll < 0) {
dirtycode/alsa-lib/src/control/hcontrol.c (line 747)
[declaration] snd_hctl_elem_remove(hctl, (unsigned int) res);
dirtycode/alsa-lib/src/control/hcontrol.c (line 811)
[declaration] return snd_ctl_elem_info(elem->hctl->ctl, info);
dirtycode/alsa-lib/src/control/hcontrol.c (line 826)
[declaration] return snd_ctl_elem_read(elem->hctl->ctl, value);
dirtycode/alsa-lib/src/control/hcontrol.c (line 843)
[declaration] return snd_ctl_elem_write(elem->hctl->ctl, value);

```

dirtycode/alsa-lib/src/control/hcontrol.c (line 858)

```
[declaration] return snd_ctl_elem_tlv_read(elem->hctl->ctl, &elem->id, tlv,  
tlv_size);
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 874)

```
[declaration] return snd_ctl_elem_tlv_write(elem->hctl->ctl, &elem->id, tlv);
```

dirtycode/alsa-lib/src/control/hcontrol.c (line 890)

```
[declaration] return snd_ctl_elem_tlv_command(elem->hctl->ctl, &elem->id, tlv);
```

dirtycode/alsa-lib/src/control/namehint.c (line 61)

```
[declaration] memset(n + list->allocated, 0, 10 * sizeof(*n));
```

dirtycode/alsa-lib/src/control/namehint.c (line 71)

```
[declaration] memcpy(x, "NAME", 4);
```

dirtycode/alsa-lib/src/control/namehint.c (line 72)

```
[declaration] strcpy(x + 4, name);
```

dirtycode/alsa-lib/src/control/namehint.c (line 73)

```
[function] if (description != NULL) {
```

dirtycode/alsa-lib/src/control/namehint.c (line 74)

```
[declaration] strcat(x, "|DESC");
```

dirtycode/alsa-lib/src/control/namehint.c (line 75)

```
[declaration] strcat(x, description);
```

dirtycode/alsa-lib/src/control/namehint.c (line 96)

```
[declaration] return hint_list_add(list, entry, NULL);
```

dirtycode/alsa-lib/src/control/namehint.c (line 129)

```
[declaration] snd_hwdep_info_set_device(&info, device);
```

dirtycode/alsa-lib/src/control/namehint.c (line 140)

```
[declaration] snd_pcm_info_set_device(&info, device);
```

dirtycode/alsa-lib/src/control/namehint.c (line 141)

```
[declaration] snd_pcm_info_set_stream(&info, stream ? SND_PCM_STREAM_CAPTURE :  
SND_PCM_STREAM_PLAYBACK);
```

dirtycode/alsa-lib/src/control/namehint.c (line 144)

```
[function] switch (snd_pcm_info_get_class(&info)) {
```

dirtycode/alsa-lib/src/control/namehint.c (line 159)

```
[declaration] snd_rawmidi_info_set_device(&info, device);
```

dirtycode/alsa-lib/src/control/namehint.c (line 160)

```
[declaration] snd_rawmidi_info_set_stream(&info, stream ? SND_RAWMIDI_STREAM_INPUT :  
SND_RAWMIDI_STREAM_OUTPUT);
```

dirtycode/alsa-lib/src/control/namehint.c (line 183)

```
[declaration] free(str1);
```

dirtycode/alsa-lib/src/control/namehint.c (line 187)

```
[function] if (str1 != NULL && str2 != NULL) {
```

dirtycode/alsa-lib/src/control/namehint.c (line 188)

```
[function] if (strcmp(str1, str2) == 0) {
```

dirtycode/alsa-lib/src/control/namehint.c (line 190)

```
[function] if (res != NULL) {
```

dirtycode/alsa-lib/src/control/namehint.c (line 191)

```
[declaration] strcpy(res, list->cardname);
```

dirtycode/alsa-lib/src/control/namehint.c (line 192)

```
[declaration] strcat(res, ", ");
```

dirtycode/alsa-lib/src/control/namehint.c (line 193)

```
[declaration] strcat(res, str2);
```

```

dirtycode/alsa-lib/src/control/namehint.c (line 197)
[function] if (res != NULL) {
dirtycode/alsa-lib/src/control/namehint.c (line 198)
[declaration] strcpy(res, list->cardname);
dirtycode/alsa-lib/src/control/namehint.c (line 199)
[declaration] strcat(res, ", ");
dirtycode/alsa-lib/src/control/namehint.c (line 200)
[declaration] strcat(res, str2);
dirtycode/alsa-lib/src/control/namehint.c (line 201)
[declaration] strcat(res, " / ");
dirtycode/alsa-lib/src/control/namehint.c (line 202)
[declaration] strcat(res, str1);
dirtycode/alsa-lib/src/control/namehint.c (line 205)
[declaration] free(str2);
dirtycode/alsa-lib/src/control/namehint.c (line 206)
[declaration] free(str1);
dirtycode/alsa-lib/src/control/namehint.c (line 209)
[function] if (str1 != NULL) {
dirtycode/alsa-lib/src/control/namehint.c (line 216)
[function] if (res == NULL) {
dirtycode/alsa-lib/src/control/namehint.c (line 217)
[declaration] free(str1);
dirtycode/alsa-lib/src/control/namehint.c (line 220)
[declaration] strcpy(res, list->cardname);
dirtycode/alsa-lib/src/control/namehint.c (line 221)
[declaration] strcat(res, ", ");
dirtycode/alsa-lib/src/control/namehint.c (line 222)
[declaration] strcat(res, str1);
dirtycode/alsa-lib/src/control/namehint.c (line 223)
[declaration] strcat(res, "|I0ID");
dirtycode/alsa-lib/src/control/namehint.c (line 224)
[declaration] strcat(res, str2);
dirtycode/alsa-lib/src/control/namehint.c (line 225)
[declaration] free(str1);
dirtycode/alsa-lib/src/control/namehint.c (line 232)
[declaration] return strdup(list->cardname);
dirtycode/alsa-lib/src/control/namehint.c (line 266)
[declaration] sprintf(buf, "%s: CARD=%s, DEV=%i", name,
snd_ctl_card_info_get_id(list->info), list->device);
dirtycode/alsa-lib/src/control/namehint.c (line 268)
[declaration] sprintf(buf, "%s: CARD=%s", name,
snd_ctl_card_info_get_id(list->info));
dirtycode/alsa-lib/src/control/namehint.c (line 270)
[declaration] strcpy(buf, name);
dirtycode/alsa-lib/src/control/namehint.c (line 286)
[declaration] fprintf(stderr, "***** PCM '%s':\n", buf);
dirtycode/alsa-lib/src/control/namehint.c (line 287)
[declaration] snd_output_stdio_attach(&out, stderr, 0);

```

```

dirtycode/alsa-lib/src/control/namehint.c (line 288)
[declaration] snd_config_save(res, out);
dirtycode/alsa-lib/src/control/namehint.c (line 289)
[declaration] snd_output_close(out);
dirtycode/alsa-lib/src/control/namehint.c (line 290)
[declaration] fprintf(stderr, "\n");
dirtycode/alsa-lib/src/control/namehint.c (line 300)
[function] strcmp(str, "hw") == 0) {
dirtycode/alsa-lib/src/control/namehint.c (line 301)
[function] if (snd_config_search(cfg1, "device", &cfg) >= 0) {
dirtycode/alsa-lib/src/control/namehint.c (line 302)
[function] if (snd_config_get_integer(cfg, &dev) < 0) {
dirtycode/alsa-lib/src/control/namehint.c (line 303)
[declaration] snd_error(CONTROL, "(%s) device must be an integer", buf);
dirtycode/alsa-lib/src/control/namehint.c (line 311)
[function] if (snd_config_get_type(cfg) != SND_CONFIG_TYPE_COMPOUND) {
dirtycode/alsa-lib/src/control/namehint.c (line 312)
[declaration] snd_error(CONTROL, "hint (%s) must be a compound", buf);
dirtycode/alsa-lib/src/control/namehint.c (line 326)
[function] snd_config_get_string(n, &str) >= 0) {
dirtycode/alsa-lib/src/control/namehint.c (line 328)
[function] if (buf1 == NULL) {
dirtycode/alsa-lib/src/control/namehint.c (line 333)
[function] if (snd_config_search(cfg, "device", &n) >= 0) {
dirtycode/alsa-lib/src/control/namehint.c (line 334)
[function] if (snd_config_get_integer(n, &dev) < 0) {
dirtycode/alsa-lib/src/control/namehint.c (line 335)
[declaration] snd_error(CONTROL, "(%s) device must be an integer", buf);
dirtycode/alsa-lib/src/control/namehint.c (line 342)
[function] if (snd_config_search(cfg, "device_input", &n) >= 0) {
dirtycode/alsa-lib/src/control/namehint.c (line 343)
[function] if (snd_config_get_integer(n, &list->device_input) < 0) {
dirtycode/alsa-lib/src/control/namehint.c (line 344)
[declaration] snd_error(CONTROL, "(%s) device_input must be an integer", buf);
dirtycode/alsa-lib/src/control/namehint.c (line 352)
[function] if (snd_config_search(cfg, "device_output", &n) >= 0) {
dirtycode/alsa-lib/src/control/namehint.c (line 353)
[function] if (snd_config_get_integer(n, &list->device_output) < 0) {
dirtycode/alsa-lib/src/control/namehint.c (line 354)
[declaration] snd_error(CONTROL, "(%s) device_output must be an integer", buf);
dirtycode/alsa-lib/src/control/namehint.c (line 380)
[function] snd_config_for_each(i, next, cfg) {
dirtycode/alsa-lib/src/control/namehint.c (line 401)
[function] if (str != NULL) {
dirtycode/alsa-lib/src/control/namehint.c (line 404)
[function] if (buf2 != NULL) {
dirtycode/alsa-lib/src/control/namehint.c (line 405)
[function] if (buf1 != NULL) {

```

```

dirtycode/alsa-lib/src/control/namehint.c (line 408)
[declaration] memmove(buf2 + (level - strlen(str)), str, strlen(str));
dirtycode/alsa-lib/src/control/namehint.c (line 412)
[declaration] memcpy((char *)str, buf1, strlen(buf1));
dirtycode/alsa-lib/src/control/namehint.c (line 414)
[declaration] free(buf1);
dirtycode/alsa-lib/src/control/namehint.c (line 418)
[declaration] free((char *)str);
dirtycode/alsa-lib/src/control/namehint.c (line 426)
[declaration] snd_config_delete(res);
dirtycode/alsa-lib/src/control/namehint.c (line 428)
[declaration] free(buf1);
dirtycode/alsa-lib/src/control/namehint.c (line 436)
[declaration] typedef int (*next_devices_t)(snd_ctl_t *, int *);
dirtycode/alsa-lib/src/control/namehint.c (line 475)
[function] if (next_devices[list->iface] != NULL) {
dirtycode/alsa-lib/src/control/namehint.c (line 483)
[function] while (err >= 0 && device >= 0) {
dirtycode/alsa-lib/src/control/namehint.c (line 503)
[function] if (err < 0) {
dirtycode/alsa-lib/src/control/namehint.c (line 634)
[declaration] add_software_devices(local_config, local_config_rw, &list);
dirtycode/alsa-lib/src/control/namehint.c (line 638)
[function] while (card >= 0) {
dirtycode/alsa-lib/src/control/namehint.c (line 653)
[function] snd_config_for_each(i, next, conf) {
dirtycode/alsa-lib/src/control/namehint.c (line 670)
[declaration] snd_device_name_free_hint((void **)list.list);
dirtycode/alsa-lib/src/control/namehint.c (line 677)
[declaration] snd_config_update_free(local_config_update);
dirtycode/alsa-lib/src/control/namehint.c (line 694)
[declaration] free(*h);
dirtycode/alsa-lib/src/control/namehint.c (line 724)
[function] if (memcmp(id, hint1, 4) != 0) {
dirtycode/alsa-lib/src/control/namehint.c (line 731)
[declaration] return strdup(hint1 + 4);
dirtycode/alsa-lib/src/control/namehint.c (line 734)
[function] if (res != NULL) {
dirtycode/alsa-lib/src/control/namehint.c (line 735)
[declaration] memcpy(res, hint1 + 4, size);
dirtycode/alsa-lib/src/control/setup.c (line 64)
[declaration] snd_ctl_elem_id_free(elem->id);
dirtycode/alsa-lib/src/control/setup.c (line 65)
[declaration] snd_ctl_elem_info_free(elem->info);
dirtycode/alsa-lib/src/control/setup.c (line 66)
[declaration] snd_ctl_elem_value_free(elem->val);
dirtycode/alsa-lib/src/control/setup.c (line 67)
[declaration] snd_ctl_elem_value_free(elem->mask);

```

```

dirtycode/alsa-lib/src/control/setup.c (line 68)
[declaration] snd_ctl_elem_value_free(elem->old);
dirtycode/alsa-lib/src/control/setup.c (line 69)
[declaration] list_del(&elem->list);
dirtycode/alsa-lib/src/control/setup.c (line 70)
[declaration] free(elem);
dirtycode/alsa-lib/src/control/setup.c (line 93)
[function] if (elem->lock) {
dirtycode/alsa-lib/src/control/setup.c (line 95)
[function] if (err < 0) {
dirtycode/alsa-lib/src/control/setup.c (line 96)
[declaration] snd_error(CONTROL, "Cannot lock ctl elem");
dirtycode/alsa-lib/src/control/setup.c (line 101)
[function] if (err < 0) {
dirtycode/alsa-lib/src/control/setup.c (line 102)
[declaration] snd_error(CONTROL, "Cannot read ctl elem");
dirtycode/alsa-lib/src/control/setup.c (line 107)
[function] switch (type) {
dirtycode/alsa-lib/src/control/setup.c (line 114)
[function] if (old) {
dirtycode/alsa-lib/src/control/setup.c (line 117)
[declaration] snd_ctl_elem_value_set_boolean(elem->val, k, val);
dirtycode/alsa-lib/src/control/setup.c (line 127)
[function] if (old) {
dirtycode/alsa-lib/src/control/setup.c (line 130)
[declaration] snd_ctl_elem_value_set_integer(elem->val, k, val);
dirtycode/alsa-lib/src/control/setup.c (line 140)
[function] if (old) {
dirtycode/alsa-lib/src/control/setup.c (line 143)
[declaration] snd_ctl_elem_value_set_enumerated(elem->val, k, val);
dirtycode/alsa-lib/src/control/setup.c (line 156)
[function] if (old) {
dirtycode/alsa-lib/src/control/setup.c (line 159)
[declaration] snd_ctl_elem_value_set_byte(elem->val, k, val);
dirtycode/alsa-lib/src/control/setup.c (line 164)
[declaration] assert(0);
dirtycode/alsa-lib/src/control/setup.c (line 168)
[function] if (err < 0) {
dirtycode/alsa-lib/src/control/setup.c (line 169)
[declaration] snd_error(CONTROL, "Cannot write ctl elem");
dirtycode/alsa-lib/src/control/setup.c (line 188)
[function] if (elem->lock) {
dirtycode/alsa-lib/src/control/setup.c (line 190)
[function] if (err < 0) {
dirtycode/alsa-lib/src/control/setup.c (line 191)
[declaration] snd_error(CONTROL, "Cannot unlock ctl elem");
dirtycode/alsa-lib/src/control/setup.c (line 205)
[function] if (elem->preserve && snd_ctl_elem_value_compare(elem->val, elem->old)) {

```

```

dirtycode/alsa-lib/src/control/setup.c (line 207)
[function] if (err < 0) {
dirtycode/alsa-lib/src/control/setup.c (line 208)
[declaration] snd_error(CONTROL, "Cannot restore ctl elem");
dirtycode/alsa-lib/src/control/setup.c (line 237)
[declaration] snd_ctl_elem_info_set_item(info, idx);
dirtycode/alsa-lib/src/control/setup.c (line 239)
[function] if (err < 0) {
dirtycode/alsa-lib/src/control/setup.c (line 240)
[declaration] snd_error(CONTROL, "Cannot obtain info for CTL elem");
dirtycode/alsa-lib/src/control/setup.c (line 266)
[function] switch (type) {
dirtycode/alsa-lib/src/control/setup.c (line 269)
[function] if (v >= 0) {
dirtycode/alsa-lib/src/control/setup.c (line 270)
[declaration] snd_ctl_elem_value_set_boolean(val, 0, v);
dirtycode/alsa-lib/src/control/setup.c (line 272)
[declaration] snd_ctl_elem_value_set_boolean(mask, 0, 1);
dirtycode/alsa-lib/src/control/setup.c (line 278)
[function] if (err == 0) {
dirtycode/alsa-lib/src/control/setup.c (line 279)
[declaration] snd_ctl_elem_value_set_integer(val, 0, v);
dirtycode/alsa-lib/src/control/setup.c (line 281)
[declaration] snd_ctl_elem_value_set_integer(mask, 0, ~0L);
dirtycode/alsa-lib/src/control/setup.c (line 287)
[function] if (v >= 0) {
dirtycode/alsa-lib/src/control/setup.c (line 288)
[declaration] snd_ctl_elem_value_set_enumerated(val, 0, v);
dirtycode/alsa-lib/src/control/setup.c (line 290)
[declaration] snd_ctl_elem_value_set_enumerated(mask, 0, ~0);
dirtycode/alsa-lib/src/control/setup.c (line 298)
[declaration] snd_error(CONTROL, "Unknown control type: %d", type);
dirtycode/alsa-lib/src/control/setup.c (line 310)
[function] if (err >= 0) {
dirtycode/alsa-lib/src/control/setup.c (line 314)
[function] if (len % 2 != 0 || len > count * 2) {
dirtycode/alsa-lib/src/control/setup.c (line 316)
[declaration] snd_error(CONTROL, "bad value content");
dirtycode/alsa-lib/src/control/setup.c (line 319)
[function] while (*buf) {
dirtycode/alsa-lib/src/control/setup.c (line 330)
[function] if (idx % 2 == 1) {
dirtycode/alsa-lib/src/control/setup.c (line 331)
[declaration] snd_ctl_elem_value_set_byte(val, idx / 2, c1 << 4 | c);
dirtycode/alsa-lib/src/control/setup.c (line 333)
[declaration] snd_ctl_elem_value_set_byte(mask, idx / 2, 0xff);
dirtycode/alsa-lib/src/control/setup.c (line 345)
[declaration] snd_error(CONTROL, "bad value type");

```

```

dirtycode/alsa-lib/src/control/setup.c (line 355)
[function] if (err < 0 || idx < 0 || (unsigned int) idx >= count) {
dirtycode/alsa-lib/src/control/setup.c (line 356)
[declaration] snd_error(CONTROL, "bad value index");
dirtycode/alsa-lib/src/control/setup.c (line 359)
[function] switch (type) {
dirtycode/alsa-lib/src/control/setup.c (line 364)
[declaration] snd_ctl_elem_value_set_boolean(val, idx, v);
dirtycode/alsa-lib/src/control/setup.c (line 366)
[declaration] snd_ctl_elem_value_set_boolean(mask, idx, 1);
dirtycode/alsa-lib/src/control/setup.c (line 372)
[declaration] snd_ctl_elem_value_set_integer(val, idx, v);
dirtycode/alsa-lib/src/control/setup.c (line 374)
[declaration] snd_ctl_elem_value_set_integer(mask, idx, ~0L);
dirtycode/alsa-lib/src/control/setup.c (line 380)
[declaration] snd_ctl_elem_value_set_enumerated(val, idx, v);
dirtycode/alsa-lib/src/control/setup.c (line 382)
[declaration] snd_ctl_elem_value_set_enumerated(mask, idx, ~0);
dirtycode/alsa-lib/src/control/setup.c (line 389)
[declaration] snd_ctl_elem_value_set_byte(val, idx, v);
dirtycode/alsa-lib/src/control/setup.c (line 391)
[declaration] snd_ctl_elem_value_set_byte(mask, idx, 0xff);
dirtycode/alsa-lib/src/control/setup.c (line 426)
[function] if (strcmp(id, "iface") == 0 || strcmp(id, "interface") == 0) {
dirtycode/alsa-lib/src/control/setup.c (line 428)
[function] if ((err = snd_config_get_string(n, &ptr)) < 0) {
dirtycode/alsa-lib/src/control/setup.c (line 429)
[declaration] snd_error(CONTROL, "field %s is not a string", id);
dirtycode/alsa-lib/src/control/setup.c (line 432)
[function] if ((err = snd_config_get_ctl_iface_ascii(ptr)) < 0) {
dirtycode/alsa-lib/src/control/setup.c (line 433)
[declaration] snd_error(CONTROL, "Invalid value for '%s'", id);
dirtycode/alsa-lib/src/control/setup.c (line 439)
[function] if (strcmp(id, "name") == 0) {
dirtycode/alsa-lib/src/control/setup.c (line 440)
[function] if ((err = snd_config_get_string(n, &name)) < 0) {
dirtycode/alsa-lib/src/control/setup.c (line 441)
[declaration] snd_error(CONTROL, "field %s is not a string", id);
dirtycode/alsa-lib/src/control/setup.c (line 446)
[function] if (strcmp(id, "index") == 0) {
dirtycode/alsa-lib/src/control/setup.c (line 447)
[function] if ((err = snd_config_get_integer(n, &index)) < 0) {
dirtycode/alsa-lib/src/control/setup.c (line 448)
[declaration] snd_error(CONTROL, "field %s is not an integer", id);
dirtycode/alsa-lib/src/control/setup.c (line 453)
[function] if (strcmp(id, "device") == 0) {
dirtycode/alsa-lib/src/control/setup.c (line 454)
[function] if ((err = snd_config_get_integer(n, &device)) < 0) {

```

```

dirtycode/alsa-lib/src/control/setup.c (line 455)
[declaration] snd_error(CONTROL, "field %s is not an integer", id);
dirtycode/alsa-lib/src/control/setup.c (line 460)
[function] if (strcmp(id, "subdevice") == 0) {
dirtycode/alsa-lib/src/control/setup.c (line 461)
[function] if ((err = snd_config_get_integer(n, &subdevice)) < 0) {
dirtycode/alsa-lib/src/control/setup.c (line 462)
[declaration] snd_error(CONTROL, "field %s is not an integer", id);
dirtycode/alsa-lib/src/control/setup.c (line 467)
[function] if (strcmp(id, "lock") == 0) {
dirtycode/alsa-lib/src/control/setup.c (line 474)
[function] if (strcmp(id, "preserve") == 0) {
dirtycode/alsa-lib/src/control/setup.c (line 481)
[function] if (strcmp(id, "value") == 0) {
dirtycode/alsa-lib/src/control/setup.c (line 485)
[function] if (strcmp(id, "mask") == 0) {
dirtycode/alsa-lib/src/control/setup.c (line 489)
[function] if (strcmp(id, "optional") == 0) {
dirtycode/alsa-lib/src/control/setup.c (line 496)
[function] if (strcmp(id, "skip_rest") == 0) {
dirtycode/alsa-lib/src/control/setup.c (line 503)
[declaration] snd_error(CONTROL, "Unknown field %s", id);
dirtycode/alsa-lib/src/control/setup.c (line 507)
[declaration] snd_error(CONTROL, "Missing control name");
dirtycode/alsa-lib/src/control/setup.c (line 512)
[declaration] snd_error(CONTROL, "Missing control value");
dirtycode/alsa-lib/src/control/setup.c (line 549)
[declaration] snd_error(CONTROL, "Cannot obtain info for CTL elem
(%s, '%s', %li, %li, %li): %s", snd_ctl_elem_iface_name(iface), name, index, device,
subdevice, snd_strerror(err));
dirtycode/alsa-lib/src/control/setup.c (line 578)
[declaration] snd_ctl_elem_id_free(elem->id);
dirtycode/alsa-lib/src/control/setup.c (line 580)
[declaration] snd_ctl_elem_info_free(elem->info);
dirtycode/alsa-lib/src/control/setup.c (line 582)
[declaration] snd_ctl_elem_value_free(elem->val);
dirtycode/alsa-lib/src/control/setup.c (line 584)
[declaration] snd_ctl_elem_value_free(elem->mask);
dirtycode/alsa-lib/src/control/setup.c (line 586)
[declaration] snd_ctl_elem_value_free(elem->old);
dirtycode/alsa-lib/src/control/setup.c (line 587)
[declaration] free(elem);
dirtycode/alsa-lib/src/control/setup.c (line 620)
[declaration] snd_ctl_close(handle);
dirtycode/alsa-lib/src/control/setup.c (line 629)
[function] if (err < 0) {
dirtycode/alsa-lib/src/control/setup.c (line 630)
[declaration] free_elems(h);

```

```

dirtycode/alsa-lib/src/control/setup.c (line 648)
[declaration] return free_elems(sctl);
dirtycode/alsa-lib/src/control/tlv.c (line 72)
[declaration] snd_error(CONTROL, "TLV stream too short");
dirtycode/alsa-lib/src/control/tlv.c (line 81)
[declaration] snd_error(CONTROL, "TLV size error");
dirtycode/alsa-lib/src/control/tlv.c (line 88)
[function] while (size > 0) {
dirtycode/alsa-lib/src/control/tlv.c (line 112)
[function] if (size < minsize) {
dirtycode/alsa-lib/src/control/tlv.c (line 113)
[declaration] snd_error(CONTROL, "Invalid dB_scale TLV size");
dirtycode/alsa-lib/src/control/tlv.c (line 116)
[function] if (size > MAX_TLV_RANGE_SIZE) {
dirtycode/alsa-lib/src/control/tlv.c (line 117)
[declaration] snd_error(CONTROL, "Too big dB_scale TLV size: %d", size);
dirtycode/alsa-lib/src/control/tlv.c (line 150)
[function] while (pos + 4 <= len) {
dirtycode/alsa-lib/src/control/tlv.c (line 162)
[function] if (pos > 2) {
dirtycode/alsa-lib/src/control/tlv.c (line 222)
[function] while (pos + 4 <= len) {
dirtycode/alsa-lib/src/control/tlv.c (line 249)
[function] if (volume <= rangemin || rangemax <= rangemin) {
dirtycode/alsa-lib/src/control/tlv.c (line 316)
[function] while (pos + 4 <= len) {
dirtycode/alsa-lib/src/control/tlv.c (line 329)
[function] else if (db_gain < dbmin) {
dirtycode/alsa-lib/src/control/tlv.c (line 403)
[declaration] pow(10.0, (double)min / 2000.0);
dirtycode/alsa-lib/src/dlmisc.c (line 95)
[function] if (env) {
dirtycode/alsa-lib/src/dlmisc.c (line 98)
[declaration] snd_dlnfo_origin(path, path_len);
dirtycode/alsa-lib/src/dlmisc.c (line 132)
[function] if (self == NULL) {
dirtycode/alsa-lib/src/dlmisc.c (line 161)
[declaration] snprintf(errbuf, errbuflen, "%s", dlerror());
dirtycode/alsa-lib/src/dlmisc.c (line 169)
[declaration] return INTERNAL(snd_dlopen)(name, mode, NULL, 0);
dirtycode/alsa-lib/src/dlmisc.c (line 193)
[declaration] return dlclose(handle);
dirtycode/alsa-lib/src/dlmisc.c (line 226)
[declaration] snd_error(CORE, "unable to verify version for symbol %s", name);
dirtycode/alsa-lib/src/dlmisc.c (line 255)
[function] while (link) {
dirtycode/alsa-lib/src/dlmisc.c (line 271)
[declaration] return dlsym(handle, name);

```

```

dirtycode/alsa-lib/src/dlmisc.c (line 308)
[declaration] static LIST_HEAD(pcm_dlobj_list);
dirtycode/alsa-lib/src/dlmisc.c (line 327)
[function] if (strcmp(c->name, name) == 0) {
dirtycode/alsa-lib/src/dlmisc.c (line 362)
[declaration] free((void *)c->name);
dirtycode/alsa-lib/src/dlmisc.c (line 363)
[declaration] free((void *)c->lib);
dirtycode/alsa-lib/src/dlmisc.c (line 364)
[declaration] free(c);
dirtycode/alsa-lib/src/dlmisc.c (line 366)
[declaration] snd_dlclose(dlobj);
dirtycode/alsa-lib/src/dlmisc.c (line 418)
[function] if (c->func == func) {
dirtycode/alsa-lib/src/dlmisc.c (line 422)
[declaration] snd_dlobj_unlock();
dirtycode/alsa-lib/src/dlmisc.c (line 440)
[declaration] list_del(p);
dirtycode/alsa-lib/src/dlmisc.c (line 441)
[declaration] snd_dlclose(c->dlobj);
dirtycode/alsa-lib/src/dlmisc.c (line 442)
[declaration] free((void *)c->name); /* shut up gcc warning */
dirtycode/alsa-lib/src/dlmisc.c (line 443)
[declaration] free((void *)c->lib); /* shut up gcc warning */
dirtycode/alsa-lib/src/dlmisc.c (line 444)
[declaration] free(c);
dirtycode/alsa-lib/src/error.c (line 37)
[declaration] static void snd_lib_error_default(const char *file, int line, const
char *function, int errcode, const char *fmt, ...);
dirtycode/alsa-lib/src/error.c (line 58)
[declaration] return (const char *) strerror(errno);
dirtycode/alsa-lib/src/error.c (line 195)
[function] if (colon) {
dirtycode/alsa-lib/src/error.c (line 206)
[function] strcmp(snd_ilog_interface_names[i], interface_name) == 0) {
dirtycode/alsa-lib/src/error.c (line 213)
[function] if (interface_num < 0) {
dirtycode/alsa-lib/src/error.c (line 223)
[function] strcmp(snd_log_prio_names[i], level_str) == 0) {
dirtycode/alsa-lib/src/error.c (line 230)
[function] if (level < 0) {
dirtycode/alsa-lib/src/error.c (line 247)
[function] strcmp(snd_log_prio_names[i], token) == 0) {
dirtycode/alsa-lib/src/error.c (line 254)
[function] if (level < 0) {
dirtycode/alsa-lib/src/error.c (line 297)
[declaration] static void snd_lib_error_vdefault(const char *file, int line, const
char *function, int errcode, const char *fmt, va_list arg);

```

```

dirtycode/alsa-lib/src/error.c (line 319)
[declaration] local_log(prio, interface, file, line, function, errcode, fmt, arg);
dirtycode/alsa-lib/src/error.c (line 323)
[declaration] local_error(file, line, function, errcode, fmt, arg);
dirtycode/alsa-lib/src/error.c (line 328)
[declaration] snd_lib_error_vdefault(file, line, function, errcode, fmt, arg);
dirtycode/alsa-lib/src/error.c (line 341)
[declaration] fprintf(stderr, "[%s.%s] ", text1 ? text1 : "", text2 ? text2 : "");
dirtycode/alsa-lib/src/error.c (line 345)
[declaration] fprintf(stderr, ": %s", snd_strerror(errcode));
dirtycode/alsa-lib/src/error.c (line 393)
[declaration] assert(0);
dirtycode/alsa-lib/src/error.c (line 527)
[declaration] memcpy(dst, src, len);
dirtycode/alsa-lib/src/error.c (line 554)
[declaration] memcpy(dst, src, len);
dirtycode/alsa-lib/src/hwdep/hwdep.c (line 57)
[declaration] snd_error(HWDEP, "Invalid type for HWDEP %s definition", name);
dirtycode/alsa-lib/src/hwdep/hwdep.c (line 59)
[declaration] snd_error(HWDEP, "Invalid type for HWDEP definition");
dirtycode/alsa-lib/src/hwdep/hwdep.c (line 64)
[declaration] snd_error(HWDEP, "type is not defined");
dirtycode/alsa-lib/src/hwdep/hwdep.c (line 69)
[declaration] snd_error(HWDEP, "unable to get id");
dirtycode/alsa-lib/src/hwdep/hwdep.c (line 74)
[declaration] snd_error(HWDEP, "Invalid type for %s", id);
dirtycode/alsa-lib/src/hwdep/hwdep.c (line 79)
[function] if (snd_config_get_type(type_conf) != SND_CONFIG_TYPE_COMPOUND) {
dirtycode/alsa-lib/src/hwdep/hwdep.c (line 80)
[declaration] snd_error(HWDEP, "Invalid type for HWDEP type %s definition", str);
dirtycode/alsa-lib/src/hwdep/hwdep.c (line 84)
[function] snd_config_for_each(i, next, type_conf) {
dirtycode/alsa-lib/src/hwdep/hwdep.c (line 91)
[function] if (strcmp(id, "lib") == 0) {
dirtycode/alsa-lib/src/hwdep/hwdep.c (line 93)
[function] if (err < 0) {
dirtycode/alsa-lib/src/hwdep/hwdep.c (line 94)
[declaration] snd_error(HWDEP, "Invalid type for %s", id);
dirtycode/alsa-lib/src/hwdep/hwdep.c (line 99)
[function] if (strcmp(id, "open") == 0) {
dirtycode/alsa-lib/src/hwdep/hwdep.c (line 101)
[function] if (err < 0) {
dirtycode/alsa-lib/src/hwdep/hwdep.c (line 102)
[declaration] snd_error(HWDEP, "Invalid type for %s", id);
dirtycode/alsa-lib/src/hwdep/hwdep.c (line 107)
[declaration] snd_error(HWDEP, "Unknown field %s", id);
dirtycode/alsa-lib/src/hwdep/hwdep.c (line 114)
[declaration] snprintf(buf, sizeof(buf), "_snd_hwdep_%s_open", str);

```

```

dirtycode/alsa-lib/src/hwdep/hwdep.c (line 124)
[declaration] snd_error(HWDEP, "Cannot open shared library %s (%s)", lib, errbuf);
dirtycode/alsa-lib/src/hwdep/hwdep.c (line 127)
[declaration] snd_error(HWDEP, "symbol %s is not defined inside %s", open_name,
lib);
dirtycode/alsa-lib/src/hwdep/hwdep.c (line 128)
[declaration] snd_dlclose(h);
dirtycode/alsa-lib/src/hwdep/hwdep.c (line 133)
[declaration] snd_config_delete(type_conf);
dirtycode/alsa-lib/src/hwdep/hwdep.c (line 136)
[function] if (err >= 0) {
dirtycode/alsa-lib/src/hwdep/hwdep.c (line 139)
[declaration] snd_dlclose(h);
dirtycode/alsa-lib/src/hwdep/hwdep.c (line 151)
[declaration] snd_error(HWDEP, "Unknown HwDep %s", name);
dirtycode/alsa-lib/src/hwdep/hwdep.c (line 198)
[declaration] return snd_hwdep_open_noupdate(hwdep, lconf, name, mode);
dirtycode/alsa-lib/src/hwdep/hwdep.c (line 215)
[declaration] snd_dlclose(hwdep->dl_handle);
dirtycode/alsa-lib/src/hwdep/hwdep.c (line 271)
[function] switch (hwdep->mode & 0_ACCMODE) {
dirtycode/alsa-lib/src/hwdep/hwdep.c (line 332)
[declaration] return sizeof(snd_hwdep_info_t);
dirtycode/alsa-lib/src/hwdep/hwdep.c (line 528)
[declaration] return sizeof(snd_hwdep_dsp_status_t);
dirtycode/alsa-lib/src/hwdep/hwdep.c (line 633)
[declaration] return sizeof(snd_hwdep_dsp_image_t);
dirtycode/alsa-lib/src/hwdep/hwdep_hw.c (line 120)
[declaration] snd_card_load(card);
dirtycode/alsa-lib/src/hwdep/hwdep_hw.c (line 127)
[declaration] close(fd);
dirtycode/alsa-lib/src/hwdep/hwdep_hw.c (line 131)
[declaration] close(fd);
dirtycode/alsa-lib/src/hwdep/hwdep_hw.c (line 136)
[declaration] close(fd);
dirtycode/alsa-lib/src/hwdep/hwdep_hw.c (line 162)
[function] if (strcmp(id, "card") == 0) {
dirtycode/alsa-lib/src/hwdep/hwdep_hw.c (line 169)
[function] if (strcmp(id, "device") == 0) {
dirtycode/alsa-lib/src/hwdep/hwdep_hw.c (line 175)
[declaration] snd_error(HWDEP, "Unexpected field %s", id);
dirtycode/alsa-lib/src/hwdep/hwdep_hw.c (line 180)
[declaration] return snd_hwdep_hw_open(hwdep, name, card, device, mode);
dirtycode/alsa-lib/src/input.c (line 130)
[declaration] fclose(stdio->fp);
dirtycode/alsa-lib/src/input.c (line 138)
[declaration] extern int vfscanf(FILE *, const char *, va_list);
dirtycode/alsa-lib/src/input.c (line 139)
[declaration] return vfscanf(stdio->fp, format, args);

```

dirtycode/alsa-lib/src/input.c (line 145)
[declaration] return fgets(str, (int) size, stdio->fp);

dirtycode/alsa-lib/src/input.c (line 151)
[declaration] return getc(stdio->fp);

dirtycode/alsa-lib/src/input.c (line 157)
[declaration] return ungetc(c, stdio->fp);

dirtycode/alsa-lib/src/input.c (line 189)
[declaration] free(stdio);

dirtycode/alsa-lib/src/input.c (line 219)
[declaration] fclose(fp);

dirtycode/alsa-lib/src/input.c (line 242)
[declaration] extern int vsscanf(const char *, const char *, va_list);

dirtycode/alsa-lib/src/input.c (line 245)
[declaration] return vsscanf((char *)buffer->ptr, format, args);

dirtycode/alsa-lib/src/input.c (line 316)
[declaration] free(buffer);

dirtycode/alsa-lib/src/input.c (line 323)
[declaration] free(input);

dirtycode/alsa-lib/src/input.c (line 324)
[declaration] free(buffer);

dirtycode/alsa-lib/src/mixer/bag.c (line 42)
[declaration] return list_empty(bag);

dirtycode/alsa-lib/src/mixer/bag.c (line 60)
[function] if (b->ptr == ptr) {

dirtycode/alsa-lib/src/mixer/bag.c (line 61)
[declaration] list_del(&b->list);

dirtycode/alsa-lib/src/mixer/bag.c (line 62)
[declaration] free(b);

dirtycode/alsa-lib/src/mixer/bag.c (line 75)
[declaration] list_del(&b->list);

dirtycode/alsa-lib/src/mixer/bag.c (line 76)
[declaration] free(b);

dirtycode/alsa-lib/src/mixer/mixer.c (line 104)
[declaration] return bag_add(&elem->helems, helem);

dirtycode/alsa-lib/src/mixer/mixer.c (line 140)
[declaration] return bag_empty(&elem->helems);

dirtycode/alsa-lib/src/mixer/mixer.c (line 151)
[function] bag_for_each_safe(i, n, bag) {

dirtycode/alsa-lib/src/mixer/mixer.c (line 161)
[declaration] assert(bag_empty(bag));

dirtycode/alsa-lib/src/mixer/mixer.c (line 162)
[declaration] bag_free(bag);

dirtycode/alsa-lib/src/mixer/mixer.c (line 168)
[function] bag_for_each_safe(i, n, bag) {

dirtycode/alsa-lib/src/mixer/mixer.c (line 190)
[declaration] snd_hctl_elem_set_callback(elem, hctl_elem_event_handler);

dirtycode/alsa-lib/src/mixer/mixer.c (line 191)
[declaration] snd_hctl_elem_set_callback_private(elem, bag);

```

dirtycode/alsa-lib/src/mixer/mixer.c (line 192)
[function] list_for_each(pos, &mixer->classes) {
dirtycode/alsa-lib/src/mixer/mixer.c (line 240)
[declaration] snd_hctl_close(hctl);
dirtycode/alsa-lib/src/mixer/mixer.c (line 245)
[declaration] snd_hctl_close(hctl);
dirtycode/alsa-lib/src/mixer/mixer.c (line 246)
[declaration] free(slave);
dirtycode/alsa-lib/src/mixer/mixer.c (line 268)
[function] if (strcmp(name, snd_hctl_name(s->hctl)) == 0) {
dirtycode/alsa-lib/src/mixer/mixer.c (line 269)
[declaration] snd_hctl_close(s->hctl);
dirtycode/alsa-lib/src/mixer/mixer.c (line 270)
[declaration] list_del(pos);
dirtycode/alsa-lib/src/mixer/mixer.c (line 271)
[declaration] free(s);
dirtycode/alsa-lib/src/mixer/mixer.c (line 292)
[function] if (hctl == s->hctl) {
dirtycode/alsa-lib/src/mixer/mixer.c (line 293)
[declaration] list_del(pos);
dirtycode/alsa-lib/src/mixer/mixer.c (line 294)
[declaration] free(s);
dirtycode/alsa-lib/src/mixer/mixer.c (line 314)
[function] if (strcmp(name, snd_hctl_name(s->hctl)) == 0) {
dirtycode/alsa-lib/src/mixer/mixer.c (line 421)
[function] if (!m) {
dirtycode/alsa-lib/src/mixer/mixer.c (line 428)
[declaration] list_add_tail(&elem->list, &mixer->elems);
dirtycode/alsa-lib/src/mixer/mixer.c (line 432)
[declaration] assert(dir != 0);
dirtycode/alsa-lib/src/mixer/mixer.c (line 433)
[function] if (dir > 0) {
dirtycode/alsa-lib/src/mixer/mixer.c (line 434)
[declaration] list_add(&elem->list, &mixer->pelems[idx]->list);
dirtycode/alsa-lib/src/mixer/mixer.c (line 437)
[declaration] list_add_tail(&elem->list, &mixer->pelems[idx]->list);
dirtycode/alsa-lib/src/mixer/mixer.c (line 445)
[declaration] return snd_mixer_throw_event(mixer, SND_CTL_EVENT_MASK_ADD, elem);
dirtycode/alsa-lib/src/mixer/mixer.c (line 468)
[declaration] snd_mixer_elem_detach(elem, helem);
dirtycode/alsa-lib/src/mixer/mixer.c (line 478)
[declaration] m * sizeof(snd_mixer_elem_t *));
dirtycode/alsa-lib/src/mixer/mixer.c (line 504)
[declaration] return snd_mixer_elem_throw_event(elem, SND_CTL_EVENT_MASK_INFO);
dirtycode/alsa-lib/src/mixer/mixer.c (line 516)
[declaration] return snd_mixer_elem_throw_event(elem, SND_CTL_EVENT_MASK_VALUE);
dirtycode/alsa-lib/src/mixer/mixer.c (line 540)
[function] while (elem) {

```

```

dirtycode/alsa-lib/src/mixer/mixer.c (line 565)
[declaration] snd_mixer_elem_remove(e);
dirtycode/alsa-lib/src/mixer/mixer.c (line 603)
[declaration] snd_hctl_free(s->hctl);
dirtycode/alsa-lib/src/mixer/mixer.c (line 619)
[declaration] snd_mixer_class_unregister(c);
dirtycode/alsa-lib/src/mixer/mixer.c (line 632)
[declaration] list_del(&s->list);
dirtycode/alsa-lib/src/mixer/mixer.c (line 633)
[declaration] free(s);
dirtycode/alsa-lib/src/mixer/mixer.c (line 667)
[declaration] list_add_tail(&mixer->pelems[k]->list, &mixer->elems);
dirtycode/alsa-lib/src/mixer/mixer.c (line 733)
[function] if (space >= (unsigned int) n) {
dirtycode/alsa-lib/src/mixer/mixer.c (line 786)
[declaration] assert(err == count);
dirtycode/alsa-lib/src/mixer/mixer.c (line 804)
[declaration] return list_entry(mixer->elems.next, snd_mixer_elem_t, list);
dirtycode/alsa-lib/src/mixer/mixer.c (line 817)
[declaration] return list_entry(mixer->elems.prev, snd_mixer_elem_t, list);
dirtycode/alsa-lib/src/mixer/mixer.c (line 830)
[declaration] return list_entry(elem->list.next, snd_mixer_elem_t, list);
dirtycode/alsa-lib/src/mixer/mixer.c (line 843)
[declaration] return list_entry(elem->list.prev, snd_mixer_elem_t, list);
dirtycode/alsa-lib/src/mixer/mixer.c (line 962)
[declaration] return sizeof(snd_mixer_class_t);
dirtycode/alsa-lib/src/mixer/simple.c (line 68)
[function] if (options != NULL) {
dirtycode/alsa-lib/src/mixer/simple.c (line 76)
[declaration] return snd_mixer_simple_basic_register(mixer, options, classp);
dirtycode/alsa-lib/src/mixer/simple.c (line 199)
[declaration] return COND_CAPS(elem, SM_CAP_GVOLUME);
dirtycode/alsa-lib/src/mixer/simple.c (line 210)
[declaration] return COND_CAPS(elem, SM_CAP_GSWITCH);
dirtycode/alsa-lib/src/mixer/simple.c (line 247)
[declaration] return sm_selem_ops(elem)->is(elem, SM_PLAY, SM_OPS_IS_ACTIVE, 0);
dirtycode/alsa-lib/src/mixer/simple.c (line 258)
[declaration] return sm_selem_ops(elem)->is(elem, SM_PLAY, SM_OPS_IS_MONO, 0);
dirtycode/alsa-lib/src/mixer/simple.c (line 270)
[declaration] return sm_selem_ops(elem)->is(elem, SM_PLAY, SM_OPS_IS_CHANNEL,
(int)channel);
dirtycode/alsa-lib/src/mixer/simple.c (line 284)
[declaration] return sm_selem_ops(elem)->get_range(elem, SM_PLAY, min, max);
dirtycode/alsa-lib/src/mixer/simple.c (line 298)
[declaration] return sm_selem_ops(elem)->get_dB_range(elem, SM_PLAY, min, max);
dirtycode/alsa-lib/src/mixer/simple.c (line 313)
[declaration] return sm_selem_ops(elem)->set_range(elem, SM_PLAY, min, max);
dirtycode/alsa-lib/src/mixer/simple.c (line 324)
[declaration] return COND_CAPS(elem, SM_CAP_PVOLUME);

```

```

dirtycode/alsa-lib/src/mixer/simple.c (line 335)
[declaration] return COND_CAPS(elem, SM_CAP_PVOLUME_JOIN);
dirtycode/alsa-lib/src/mixer/simple.c (line 346)
[declaration] return COND_CAPS(elem, SM_CAP_PSWITCH);
dirtycode/alsa-lib/src/mixer/simple.c (line 357)
[declaration] return COND_CAPS(elem, SM_CAP_PSWITCH_JOIN);
dirtycode/alsa-lib/src/mixer/simple.c (line 371)
[declaration] return sm_selem_ops(elem)->ask_vol_dB(elem, SM_PLAY, value, dBvalue);
dirtycode/alsa-lib/src/mixer/simple.c (line 387)
[declaration] return sm_selem_ops(elem)->ask_dB_vol(elem, SM_PLAY, dBvalue, value,
dir);
dirtycode/alsa-lib/src/mixer/simple.c (line 401)
[declaration] return sm_selem_ops(elem)->get_volume(elem, SM_PLAY, channel, value);
dirtycode/alsa-lib/src/mixer/simple.c (line 421)
[declaration] return sm_selem_ops(elem)->get_dB(elem, SM_PLAY, channel, value);
dirtycode/alsa-lib/src/mixer/simple.c (line 435)
[declaration] return sm_selem_ops(elem)->get_switch(elem, SM_PLAY, channel, value);
dirtycode/alsa-lib/src/mixer/simple.c (line 449)
[declaration] return sm_selem_ops(elem)->set_volume(elem, SM_PLAY, channel, value);
dirtycode/alsa-lib/src/mixer/simple.c (line 465)
[declaration] return sm_selem_ops(elem)->set_dB(elem, SM_PLAY, channel, value, dir);
dirtycode/alsa-lib/src/mixer/simple.c (line 527)
[declaration] return sm_selem_ops(elem)->set_switch(elem, SM_PLAY, channel, value);
dirtycode/alsa-lib/src/mixer/simple.c (line 563)
[declaration] return sm_selem_ops(elem)->is(elem, SM_CAPT, SM_OPS_IS_MONO, 0);
dirtycode/alsa-lib/src/mixer/simple.c (line 576)
[declaration] return sm_selem_ops(elem)->is(elem, SM_CAPT, SM_OPS_IS_CHANNEL,
channel);
dirtycode/alsa-lib/src/mixer/simple.c (line 590)
[declaration] return sm_selem_ops(elem)->get_range(elem, SM_CAPT, min, max);
dirtycode/alsa-lib/src/mixer/simple.c (line 604)
[declaration] return sm_selem_ops(elem)->get_dB_range(elem, SM_CAPT, min, max);
dirtycode/alsa-lib/src/mixer/simple.c (line 619)
[declaration] return sm_selem_ops(elem)->set_range(elem, SM_CAPT, min, max);
dirtycode/alsa-lib/src/mixer/simple.c (line 630)
[declaration] return COND_CAPS(elem, SM_CAP_CVOLUME);
dirtycode/alsa-lib/src/mixer/simple.c (line 641)
[declaration] return COND_CAPS(elem, SM_CAP_CVOLUME_JOIN);
dirtycode/alsa-lib/src/mixer/simple.c (line 652)
[declaration] return COND_CAPS(elem, SM_CAP_CSWITCH);
dirtycode/alsa-lib/src/mixer/simple.c (line 663)
[declaration] return COND_CAPS(elem, SM_CAP_CSWITCH_JOIN);
dirtycode/alsa-lib/src/mixer/simple.c (line 674)
[declaration] return COND_CAPS(elem, SM_CAP_CSWITCH_EXCL);
dirtycode/alsa-lib/src/mixer/simple.c (line 703)
[declaration] return sm_selem_ops(elem)->ask_vol_dB(elem, SM_CAPT, value, dBvalue);
dirtycode/alsa-lib/src/mixer/simple.c (line 719)
[declaration] return sm_selem_ops(elem)->ask_dB_vol(elem, SM_CAPT, dBvalue, value,
dir);

```

```

dirtycode/alsa-lib/src/mixer/simple.c (line 733)
[declaration] return sm_selem_ops(elem)->get_volume(elem, SM_CAPT, channel, value);
dirtycode/alsa-lib/src/mixer/simple.c (line 747)
[declaration] return sm_selem_ops(elem)->get_db(elem, SM_CAPT, channel, value);
dirtycode/alsa-lib/src/mixer/simple.c (line 761)
[declaration] return sm_selem_ops(elem)->get_switch(elem, SM_CAPT, channel, value);
dirtycode/alsa-lib/src/mixer/simple.c (line 775)
[declaration] return sm_selem_ops(elem)->set_volume(elem, SM_CAPT, channel, value);
dirtycode/alsa-lib/src/mixer/simple.c (line 791)
[declaration] return sm_selem_ops(elem)->set_db(elem, SM_CAPT, channel, value, dir);
dirtycode/alsa-lib/src/mixer/simple.c (line 853)
[declaration] return sm_selem_ops(elem)->set_switch(elem, SM_CAPT, channel, value);
dirtycode/alsa-lib/src/mixer/simple.c (line 887)
[declaration] return sm_selem_ops(elem)->is(elem, SM_PLAY, SM_OPS_IS_ENUMERATED, 0);
dirtycode/alsa-lib/src/mixer/simple.c (line 898)
[declaration] return sm_selem_ops(elem)->is(elem, SM_PLAY, SM_OPS_IS_ENUMERATED, 1);
dirtycode/alsa-lib/src/mixer/simple.c (line 909)
[declaration] return sm_selem_ops(elem)->is(elem, SM_CAPT, SM_OPS_IS_ENUMERATED, 1);
dirtycode/alsa-lib/src/mixer/simple.c (line 921)
[declaration] return sm_selem_ops(elem)->is(elem, SM_PLAY, SM_OPS_IS_ENUMCNT, 0);
dirtycode/alsa-lib/src/mixer/simple.c (line 938)
[declaration] return sm_selem_ops(elem)->enum_item_name(elem, item, maxlen, buf);
dirtycode/alsa-lib/src/mixer/simple.c (line 954)
[declaration] return sm_selem_ops(elem)->get_enum_item(elem, channel, itemp);
dirtycode/alsa-lib/src/mixer/simple.c (line 970)
[declaration] return sm_selem_ops(elem)->set_enum_item(elem, channel, item);
dirtycode/alsa-lib/src/mixer/simple.c (line 979)
[declaration] return sizeof(snd_mixer_selem_id_t);
dirtycode/alsa-lib/src/mixer/simple.c (line 1079)
[function] while (*str && *str != ',') {
dirtycode/alsa-lib/src/mixer/simple.c (line 1080)
[function] if (size < (int)sizeof(buf)) {
dirtycode/alsa-lib/src/mixer/simple.c (line 1088)
[function] while (*str && *str != c) {
dirtycode/alsa-lib/src/mixer/simple.c (line 1089)
[function] if (size < (int)sizeof(buf)) {
dirtycode/alsa-lib/src/mixer/simple_abst.c (line 56)
[declaration] typedef int (*snd_mixer_sbasic_init_t)(snd_mixer_class_t *class);
dirtycode/alsa-lib/src/mixer/simple_abst.c (line 85)
[declaration] snd_error(MIXER, "Unable to open library '%s' (%s)", xlib, errbuf);
dirtycode/alsa-lib/src/mixer/simple_abst.c (line 86)
[declaration] free(xlib);
dirtycode/alsa-lib/src/mixer/simple_abst.c (line 92)
[declaration] snd_error(MIXER, "Symbol 'alsa_mixer_simple_event' was not found in '%s'", xlib);
dirtycode/alsa-lib/src/mixer/simple_abst.c (line 97)
[function] if (init_func == NULL) {

```

```
dirtycode/alsa-lib/src/mixer/simple_abst.c (line 98)
[declaration] snd_error(MIXER, "Symbol 'alsa_mixer_simple_init' was not found in
's'", xlib);
dirtycode/alsa-lib/src/mixer/simple_abst.c (line 132)
[declaration] snd_error(MIXER, "Unable to open library '%s'", xlib);
dirtycode/alsa-lib/src/mixer/simple_abst.c (line 133)
[declaration] free(xlib);
dirtycode/alsa-lib/src/mixer/simple_abst.c (line 139)
[declaration] snd_error(MIXER, "Symbol 'alsa_mixer_simple_event' was not found in
's'", xlib);
dirtycode/alsa-lib/src/mixer/simple_abst.c (line 144)
[function] if (init_func == NULL) {
dirtycode/alsa-lib/src/mixer/simple_abst.c (line 145)
[declaration] snd_error(MIXER, "Symbol 'alsa_mixer_simple_finit' was not found in
's'", xlib);
dirtycode/alsa-lib/src/mixer/simple_abst.c (line 163)
[declaration] return try_open(class, lib);
dirtycode/alsa-lib/src/mixer/simple_abst.c (line 167)
[declaration] return try_open(class, lib);
dirtycode/alsa-lib/src/mixer/simple_abst.c (line 217)
[function] snd_config_for_each(j, jnext, n) {
dirtycode/alsa-lib/src/mixer/simple_abst.c (line 221)
[function] if (!strcmp(id, "search1")) {
dirtycode/alsa-lib/src/mixer/simple_abst.c (line 227)
[function] if (!strcmp(id, "lib")) {
dirtycode/alsa-lib/src/mixer/simple_abst.c (line 250)
[declaration] snd_dlclose(priv->dlhandle);
dirtycode/alsa-lib/src/mixer/simple_abst.c (line 252)
[declaration] snd_ctl_card_info_free(priv->info);
dirtycode/alsa-lib/src/mixer/simple_abst.c (line 255)
[declaration] snd_mixer_detach_hctl(snd_mixer_class_get_mixer(class), priv->hctl);
dirtycode/alsa-lib/src/mixer/simple_abst.c (line 256)
[declaration] snd_hctl_close(priv->hctl);
dirtycode/alsa-lib/src/mixer/simple_abst.c (line 258)
[declaration] snd_ctl_close(priv->ctl);
dirtycode/alsa-lib/src/mixer/simple_abst.c (line 284)
[declaration] free(priv);
dirtycode/alsa-lib/src/mixer/simple_abst.c (line 288)
[declaration] free(priv);
dirtycode/alsa-lib/src/mixer/simple_abst.c (line 293)
[declaration] free(priv);
dirtycode/alsa-lib/src/mixer/simple_abst.c (line 294)
[declaration] snd_mixer_class_free(class);
dirtycode/alsa-lib/src/mixer/simple_abst.c (line 304)
[declaration] sprintf(s, "%s/smixer.conf", topdir);
dirtycode/alsa-lib/src/mixer/simple_abst.c (line 312)
[declaration] snd_error(MIXER, "unable to open simple mixer configuration file
's'", file);
```

dirtycode/alsa-lib/src/mixer/simple_abst.c (line 318)

[declaration] snd_error(MIXER, "%s may be old or corrupted: consider to remove or fix it", file);

dirtycode/alsa-lib/src/mixer/simple_abst.c (line 326)

[declaration] snd_error(MIXER, "unable to open control device '%s': %s", priv->device, snd_strerror(err));

dirtycode/alsa-lib/src/mixer/simple_abst.c (line 350)

[declaration] snd_config_delete(top);

dirtycode/alsa-lib/src/mixer/simple_abst.c (line 352)

[declaration] snd_mixer_class_free(class);

dirtycode/alsa-lib/src/mixer/simple_none.c (line 122)

[function] if (!strcmp(*name, *names, strlen(*names))) {

dirtycode/alsa-lib/src/mixer/simple_none.c (line 248)

[declaration] snd_ctl_elem_value_get_integer(&ctl, idx1));

dirtycode/alsa-lib/src/mixer/simple_none.c (line 313)

[declaration] snd_ctl_elem_value_get_enumerated(&ctl, idx1);

dirtycode/alsa-lib/src/mixer/simple_none.c (line 477)

[declaration] from_user(s, dir, c, s->str[dir].vol[idx]);

dirtycode/alsa-lib/src/mixer/simple_none.c (line 508)

[declaration] snd_ctl_elem_value_set_integer(&ctl, idx, !!val);

dirtycode/alsa-lib/src/mixer/simple_none.c (line 523)

[declaration] snd_ctl_elem_value_set_integer(&ctl, idx, 0);

dirtycode/alsa-lib/src/mixer/simple_none.c (line 568)

[declaration] return elem_write_enum(s);

dirtycode/alsa-lib/src/mixer/simple_none.c (line 571)

[declaration] return elem_write_enum(s);

dirtycode/alsa-lib/src/mixer/simple_none.c (line 574)

[declaration] return elem_write_enum(s);

dirtycode/alsa-lib/src/mixer/simple_none.c (line 664)

[declaration] snd_mixer_selem_id_free(simple->selem.id);

dirtycode/alsa-lib/src/mixer/simple_none.c (line 690)

[function] if (ctl->type == SND_CTL_ELEM_TYPE_INTEGER) {

dirtycode/alsa-lib/src/mixer/simple_none.c (line 918)

[declaration] return strlen("Capture");

dirtycode/alsa-lib/src/mixer/simple_none.c (line 922)

[declaration] return strlen("Capture");

dirtycode/alsa-lib/src/mixer/simple_none.c (line 928)

[function] if (nlen > slen) {

dirtycode/alsa-lib/src/mixer/simple_none.c (line 947)

[declaration] return strlen(name);

dirtycode/alsa-lib/src/mixer/simple_none.c (line 950)

[function] if (strstr(name, "Depth")) {

dirtycode/alsa-lib/src/mixer/simple_none.c (line 952)

[declaration] return strlen(name);

dirtycode/alsa-lib/src/mixer/simple_none.c (line 957)

[declaration] return strlen(name);

dirtycode/alsa-lib/src/mixer/simple_none.c (line 995)

[function] if (!(s->str[dir].sw & (1 << channel))) {

```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 1000)
[function] if (s->str[dir].sw & (1 << channel)) {
dirtycode/alsa-lib/src/mixer/simple_none.c (line 1029)
[function] if (val == 1) {
dirtycode/alsa-lib/src/mixer/simple_none.c (line 1042)
[function] if ( (s->selem.caps & (SM_CAP_CENUM | SM_CAP_PENUM)) == (SM_CAP_CENUM |
SM_CAP_PENUM) ) {
dirtycode/alsa-lib/src/mixer/simple_none.c (line 1098)
[declaration] static int init_db_range(snd_hctl_elem_t *ctl, struct selem_str *rec);
dirtycode/alsa-lib/src/mixer/simple_none.c (line 1177)
[declaration] return snd_tlv_get_db_range(rec->db_info, rec->min, rec->max, min,
max);
dirtycode/alsa-lib/src/mixer/simple_none.c (line 1191)
[declaration] return get_db_range(c->elem, &s->str[dir], min, max);
dirtycode/alsa-lib/src/mixer/simple_none.c (line 1266)
[declaration] return selem_write(elem);
dirtycode/alsa-lib/src/mixer/simple_none.c (line 1281)
[declaration] return convert_from_db(c->elem, &s->str[dir], dbValue, value, xdir);
dirtycode/alsa-lib/src/mixer/simple_none.c (line 1301)
[declaration] return set_volume_ops(elem, dir, channel, value);
dirtycode/alsa-lib/src/mixer/simple_none.c (line 1322)
[declaration] return selem_write(elem);
dirtycode/alsa-lib/src/mixer/simple_none.c (line 1408)
[declaration] return snd_hctl_elem_write(helem, &ctl);
dirtycode/alsa-lib/src/mixer/simple_none.c (line 1461)
[function] if (ctype == SND_CTL_ELEM_TYPE_ENUMERATED) {
dirtycode/alsa-lib/src/mixer/simple_none.c (line 1488)
[function] if (ctype == SND_CTL_ELEM_TYPE_ENUMERATED) {
dirtycode/alsa-lib/src/mixer/simple_none.c (line 1503)
[function] if (ctype == SND_CTL_ELEM_TYPE_ENUMERATED) {
dirtycode/alsa-lib/src/mixer/simple_none.c (line 1526)
[declaration] assert(0);
dirtycode/alsa-lib/src/mixer/simple_none.c (line 1537)
[function] if (!simple) {
dirtycode/alsa-lib/src/mixer/simple_none.c (line 1538)
[declaration] snd_mixer_selem_id_free(id);
dirtycode/alsa-lib/src/mixer/simple_none.c (line 1548)
[function] if (err < 0) {
dirtycode/alsa-lib/src/mixer/simple_none.c (line 1549)
[declaration] snd_mixer_selem_id_free(id);
dirtycode/alsa-lib/src/mixer/simple_none.c (line 1550)
[declaration] free(simple);
dirtycode/alsa-lib/src/mixer/simple_none.c (line 1556)
[declaration] snd_mixer_selem_id_free(id);
dirtycode/alsa-lib/src/mixer/simple_none.c (line 1567)
[declaration] snd_hctl_elem_get_subdevice(helem));
dirtycode/alsa-lib/src/mixer/simple_none.c (line 1581)
[function] if (ctype == SND_CTL_ELEM_TYPE_INTEGER) {

```

```

dirtycode/alsa-lib/src/mixer/simple_none.c (line 1583)
[declaration] snd_ctl_elem_info_get_min(&info);
dirtycode/alsa-lib/src/mixer/simple_none.c (line 1585)
[declaration] snd_ctl_elem_info_get_max(&info);
dirtycode/alsa-lib/src/mixer/simple_none.c (line 1618)
[declaration] snd_mixer_elem_free(melem);
dirtycode/alsa-lib/src/mixer/simple_none.c (line 1636)
[declaration] assert(err >= 0);
dirtycode/alsa-lib/src/mixer/simple_none.c (line 1643)
[declaration] snd_ctl_elem_info_set_item(&info, k);
dirtycode/alsa-lib/src/mixer/simple_none.c (line 1662)
[declaration] return simple_add1(class, ename, helem, type, 0);
dirtycode/alsa-lib/src/mixer/simple_none.c (line 1681)
[declaration] return snd_mixer_elem_remove(melem);
dirtycode/alsa-lib/src/mixer/simple_none.c (line 1683)
[declaration] return snd_mixer_elem_info(melem);
dirtycode/alsa-lib/src/mixer/simple_none.c (line 1691)
[declaration] return simple_event_remove(helem, melem);
dirtycode/alsa-lib/src/mixer/simple_none.c (line 1710)
[function] if (err) {
dirtycode/alsa-lib/src/mixer/simple_none.c (line 1739)
[declaration] free(class);
dirtycode/alsa-lib/src/output.c (line 139)
[declaration] fclose(stderr->fp);
dirtycode/alsa-lib/src/output.c (line 147)
[declaration] return vfprintf(stderr->fp, format, args);
dirtycode/alsa-lib/src/output.c (line 153)
[declaration] return fputs(str, stderr->fp);
dirtycode/alsa-lib/src/output.c (line 159)
[declaration] return putc(c, stderr->fp);
dirtycode/alsa-lib/src/output.c (line 165)
[declaration] return fflush(stderr->fp);
dirtycode/alsa-lib/src/output.c (line 198)
[declaration] free(stderr);
dirtycode/alsa-lib/src/output.c (line 228)
[declaration] fclose(fp);
dirtycode/alsa-lib/src/output.c (line 393)
[declaration] free(buffer);
dirtycode/alsa-lib/src/pcm/interval.c (line 93)
[declaration] assert(n > 0);
dirtycode/alsa-lib/src/pcm/interval.c (line 119)
[function] if (i->openmin) {
dirtycode/alsa-lib/src/pcm/interval.c (line 125)
[declaration] snd_interval_none(i);
dirtycode/alsa-lib/src/pcm/interval.c (line 145)
[function] if (i->openmax) {
dirtycode/alsa-lib/src/pcm/interval.c (line 151)
[declaration] snd_interval_none(i);

```

```

dirtycode/alsa-lib/src/pcm/interval.c (line 184)
[function] if (i->openmin) {
dirtycode/alsa-lib/src/pcm/interval.c (line 188)
[function] if (i->openmax) {
dirtycode/alsa-lib/src/pcm/interval.c (line 195)
[declaration] snd_interval_none(i);
dirtycode/alsa-lib/src/pcm/interval.c (line 240)
[declaration] return snd_interval_refine(i, &t);
dirtycode/alsa-lib/src/pcm/interval.c (line 246)
[declaration] snd_interval_none(c);
dirtycode/alsa-lib/src/pcm/interval.c (line 260)
[declaration] snd_interval_none(c);
dirtycode/alsa-lib/src/pcm/interval.c (line 274)
[declaration] snd_interval_none(c);
dirtycode/alsa-lib/src/pcm/interval.c (line 289)
[declaration] snd_interval_none(c);
dirtycode/alsa-lib/src/pcm/interval.c (line 297)
[function] if (r) {
dirtycode/alsa-lib/src/pcm/interval.c (line 315)
[declaration] snd_interval_none(d);
dirtycode/alsa-lib/src/pcm/interval.c (line 336)
[declaration] snd_interval_none(c);
dirtycode/alsa-lib/src/pcm/interval.c (line 357)
[declaration] snd_interval_none(c);
dirtycode/alsa-lib/src/pcm/interval.c (line 365)
[function] if (r) {
dirtycode/alsa-lib/src/pcm/interval.c (line 380)
[declaration] snd_output_printf(out, "NONE");
dirtycode/alsa-lib/src/pcm/interval.c (line 383)
[declaration] snd_output_printf(out, "ALL");
dirtycode/alsa-lib/src/pcm/interval.c (line 385)
[declaration] snd_output_printf(out, "%u", snd_interval_value(i));
dirtycode/alsa-lib/src/pcm/interval.c (line 413)
[declaration] assert(*c > INT_MIN);
dirtycode/alsa-lib/src/pcm/interval.c (line 416)
[declaration] assert(*c < INT_MAX);
dirtycode/alsa-lib/src/pcm/interval.c (line 445)
[declaration] return boundary_lt(dmin, dmindir, dmax, dmaxdir);
dirtycode/alsa-lib/src/pcm/pcm.c (line 782)
[declaration] snd_async_del_handler(h);
dirtycode/alsa-lib/src/pcm/pcm.c (line 785)
[declaration] snd_pcm_drop(pcm);
dirtycode/alsa-lib/src/pcm/pcm.c (line 791)
[declaration] snd_pcm_munmap(pcm);
dirtycode/alsa-lib/src/pcm/pcm.c (line 1006)
[declaration] snd_check(PCM, "PCM not set up");
dirtycode/alsa-lib/src/pcm/pcm.c (line 1010)
[declaration] snd_check(PCM, "params->avail_min is 0");

```

```

dirtycode/alsa-lib/src/pcm/pcm.c (line 1019)
[declaration] snd_check(PCM, "params->avail_min problem for start_threshold");
dirtycode/alsa-lib/src/pcm/pcm.c (line 1029)
[declaration] __snd_pcm_unlock(pcm->op_arg);
dirtycode/alsa-lib/src/pcm/pcm.c (line 1113)
[declaration] snd_check(PCM, "PCM not set up");
dirtycode/alsa-lib/src/pcm/pcm.c (line 1158)
[declaration] snd_check(PCM, "PCM not set up");
dirtycode/alsa-lib/src/pcm/pcm.c (line 1187)
[declaration] snd_check(PCM, "PCM not set up");
dirtycode/alsa-lib/src/pcm/pcm.c (line 1216)
[declaration] snd_check(PCM, "PCM not set up");
dirtycode/alsa-lib/src/pcm/pcm.c (line 1241)
[declaration] snd_check(PCM, "PCM not set up");
dirtycode/alsa-lib/src/pcm/pcm.c (line 1271)
[declaration] snd_check(PCM, "PCM not set up");
dirtycode/alsa-lib/src/pcm/pcm.c (line 1296)
[declaration] snd_check(PCM, "PCM not set up");
dirtycode/alsa-lib/src/pcm/pcm.c (line 1327)
[declaration] snd_check(PCM, "PCM not set up");
dirtycode/alsa-lib/src/pcm/pcm.c (line 1331)
[declaration] P_STATE(SUSPENDED), 0);
dirtycode/alsa-lib/src/pcm/pcm.c (line 1364)
[declaration] snd_check(PCM, "PCM not set up");
dirtycode/alsa-lib/src/pcm/pcm.c (line 1398)
[declaration] snd_check(PCM, "PCM not set up");
dirtycode/alsa-lib/src/pcm/pcm.c (line 1431)
[declaration] snd_check(PCM, "PCM not set up");
dirtycode/alsa-lib/src/pcm/pcm.c (line 1462)
[declaration] snd_check(PCM, "PCM not set up");
dirtycode/alsa-lib/src/pcm/pcm.c (line 1497)
[declaration] snd_check(PCM, "PCM not set up");
dirtycode/alsa-lib/src/pcm/pcm.c (line 1532)
[declaration] snd_check(PCM, "PCM not set up");
dirtycode/alsa-lib/src/pcm/pcm.c (line 1576)
[declaration] snd_check(PCM, "PCM not set up");
dirtycode/alsa-lib/src/pcm/pcm.c (line 1580)
[declaration] snd_check(PCM, "invalid access type %s",
snd_pcm_access_name(pcm->access));
dirtycode/alsa-lib/src/pcm/pcm.c (line 1586)
[declaration] return _snd_pcm_writei(pcm, buffer, size);
dirtycode/alsa-lib/src/pcm/pcm.c (line 1615)
[declaration] snd_check(PCM, "PCM not set up");
dirtycode/alsa-lib/src/pcm/pcm.c (line 1619)
[declaration] snd_check(PCM, "invalid access type %s",
snd_pcm_access_name(pcm->access));
dirtycode/alsa-lib/src/pcm/pcm.c (line 1625)
[declaration] return _snd_pcm_writen(pcm, bufs, size);

```

```

dirtycode/alsa-lib/src/pcm/pcm.c (line 1654)
[declaration] snd_check(PCM, "PCM not set up");
dirtycode/alsa-lib/src/pcm/pcm.c (line 1658)
[declaration] snd_check(PCM, "invalid access type %s",
snd_pcm_access_name(pcm->access));
dirtycode/alsa-lib/src/pcm/pcm.c (line 1664)
[declaration] return _snd_pcm_readi(pcm, buffer, size);
dirtycode/alsa-lib/src/pcm/pcm.c (line 1693)
[declaration] snd_check(PCM, "PCM not set up");
dirtycode/alsa-lib/src/pcm/pcm.c (line 1697)
[declaration] snd_check(PCM, "invalid access type %s",
snd_pcm_access_name(pcm->access));
dirtycode/alsa-lib/src/pcm/pcm.c (line 1703)
[declaration] return _snd_pcm_readn(pcm, bufs, size);
dirtycode/alsa-lib/src/pcm/pcm.c (line 1777)
[declaration] snd_check(PCM, "poll_fd < 0");
dirtycode/alsa-lib/src/pcm/pcm.c (line 2182)
[function] strcasecmp(name, snd_pcm_format_names[format]) == 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 2186)
[function] strcasecmp(name, snd_pcm_format_aliases[format]) == 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 2192)
[function] strcasecmp(name, snd_pcm_format_descriptions[format]) == 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 2343)
[declaration] snd_check(PCM, "PCM not set up");
dirtycode/alsa-lib/src/pcm/pcm.c (line 2373)
[declaration] snd_check(PCM, "PCM not set up");
dirtycode/alsa-lib/src/pcm/pcm.c (line 2452)
[declaration] snd_check(PCM, "PCM not set up");
dirtycode/alsa-lib/src/pcm/pcm.c (line 2468)
[declaration] snd_check(PCM, "PCM not set up");
dirtycode/alsa-lib/src/pcm/pcm.c (line 2484)
[declaration] snd_check(PCM, "PCM not set up");
dirtycode/alsa-lib/src/pcm/pcm.c (line 2500)
[declaration] snd_check(PCM, "PCM not set up");
dirtycode/alsa-lib/src/pcm/pcm.c (line 2532)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 2533)
[declaration] snd_async_del_handler(h);
dirtycode/alsa-lib/src/pcm/pcm.c (line 2549)
[declaration] snd_check(PCM, "invalid handler type %d", handler->type);
dirtycode/alsa-lib/src/pcm/pcm.c (line 2585)
[declaration] snd_error(PCM, "Invalid type for PCM %s%sdefinition (id: %s, value:
%s)", name ? name : "", name ? " " : "", id, val);
dirtycode/alsa-lib/src/pcm/pcm.c (line 2586)
[declaration] free(val);
dirtycode/alsa-lib/src/pcm/pcm.c (line 2591)
[declaration] snd_error(PCM, "type is not defined");
dirtycode/alsa-lib/src/pcm/pcm.c (line 2596)
[declaration] snd_error(PCM, "unable to get id");

```

```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2601)
[declaration] snd_error(PCM, "Invalid type for %s", id);
dirtycode/alsa-lib/src/pcm/pcm.c (line 2606)
[function] if (snd_config_get_type(type_conf) != SND_CONFIG_TYPE_COMPOUND) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 2607)
[declaration] snd_error(PCM, "Invalid type for PCM type %s definition", str);
dirtycode/alsa-lib/src/pcm/pcm.c (line 2611)
[function] snd_config_for_each(i, next, type_conf) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 2618)
[function] if (strcmp(id, "lib") == 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 2620)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 2621)
[declaration] snd_error(PCM, "Invalid type for %s", id);
dirtycode/alsa-lib/src/pcm/pcm.c (line 2626)
[function] if (strcmp(id, "open") == 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 2628)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 2629)
[declaration] snd_error(PCM, "Invalid type for %s", id);
dirtycode/alsa-lib/src/pcm/pcm.c (line 2634)
[declaration] snd_error(PCM, "Unknown field %s", id);
dirtycode/alsa-lib/src/pcm/pcm.c (line 2641)
[function] if (buf == NULL) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 2646)
[declaration] sprintf(buf, "_snd_pcm_%s_open", str);
dirtycode/alsa-lib/src/pcm/pcm.c (line 2650)
[function] while (*build_in) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 2655)
[function] if (*build_in == NULL) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 2657)
[function] if (buf1 == NULL) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 2662)
[declaration] sprintf(buf1, "libasound_module_pcm_%s.so", str);
dirtycode/alsa-lib/src/pcm/pcm.c (line 2669)
[declaration] SND_DLSYM_VERSION(SND_PCM_DLSYM_VERSION), 1);
dirtycode/alsa-lib/src/pcm/pcm.c (line 2672)
[function] if (err >= 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 2673)
[function] if ((*pcmp)->open_func) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 2675)
[declaration] snd_dlobj_cache_put(open_func);
dirtycode/alsa-lib/src/pcm/pcm.c (line 2681)
[declaration] snd_dlobj_cache_put(open_func);
dirtycode/alsa-lib/src/pcm/pcm.c (line 2688)
[function] if (err >= 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 2690)
[function] if (snd_config_get_integer(tmp, &i) >= 0) {

```

```

dirtycode/alsa-lib/src/pcm/pcm.c (line 2701)
[declaration] snd_config_get_integer(tmp, &(*pcmp)->minperiodtime);
dirtycode/alsa-lib/src/pcm/pcm.c (line 2706)
[declaration] snd_config_delete(type_conf);
dirtycode/alsa-lib/src/pcm/pcm.c (line 2722)
[declaration] snd_error(PCM, "Unknown PCM %s", name);
dirtycode/alsa-lib/src/pcm/pcm.c (line 2729)
[declaration] snd_config_set_hop(pcm_conf, hop);
dirtycode/alsa-lib/src/pcm/pcm.c (line 2779)
[declaration] return snd_pcm_open_noupdate(pcmp, lconf, name, stream, mode, 0);
dirtycode/alsa-lib/src/pcm/pcm.c (line 2800)
[declaration] free((*pcmp)->name);
dirtycode/alsa-lib/src/pcm/pcm.c (line 2847)
[function] if (do_lock_enable == -1) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 2885)
[declaration] return snd_pcm_open_conf(pcmp, name, root, conf, stream, mode);
dirtycode/alsa-lib/src/pcm/pcm.c (line 2926)
[declaration] return snd_pcm_wait_nocheck(pcm, timeout);
dirtycode/alsa-lib/src/pcm/pcm.c (line 2975)
[declaration] snd_error(PCM, "Invalid poll_fds %d", npfds);
dirtycode/alsa-lib/src/pcm/pcm.c (line 2983)
[declaration] snd_check(PCM, "invalid poll descriptors %d", err);
dirtycode/alsa-lib/src/pcm/pcm.c (line 2991)
[declaration] snd_check(PCM, "invalid snd_pcm_wait timeout argument %d", timeout);
dirtycode/alsa-lib/src/pcm/pcm.c (line 2993)
[declaration] __snd_pcm_unlock(pcm->fast_op_arg);
dirtycode/alsa-lib/src/pcm/pcm.c (line 2995)
[declaration] __snd_pcm_lock(pcm->fast_op_arg);
dirtycode/alsa-lib/src/pcm/pcm.c (line 2996)
[function] if (err_poll < 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 3006)
[function] if (revents & (POLLERR | POLLNVAL)) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 3015)
[declaration] __snd_pcm_hwsync(pcm);
dirtycode/alsa-lib/src/pcm/pcm.c (line 3017)
[function] if (avail_update < (snd_pcm_sframes_t)pcm->avail_min) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 3018)
[declaration] printf("*** snd_pcm_wait() FATAL ERROR!!!\n");
dirtycode/alsa-lib/src/pcm/pcm.c (line 3019)
[declaration] printf("avail_min = %li, avail_update = %li\n", pcm->avail_min,
avail_update);
dirtycode/alsa-lib/src/pcm/pcm.c (line 3081)
[declaration] snd_check(PCM, "PCM not set up");
dirtycode/alsa-lib/src/pcm/pcm.c (line 3114)
[declaration] snd_check(PCM, "PCM not set up");
dirtycode/alsa-lib/src/pcm/pcm.c (line 3128)
[function] if (sf < 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 3190)
[function] while (samples-- > 0) {

```

```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3191)
[function] if (dstbit) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 3200)
[function] if (dstbit == 8) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 3209)
[function] while (samples-- > 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 3217)
[function] while (samples-- > 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 3224)
[function] while (samples-- > 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 3240)
[function] while (samples-- > 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 3247)
[function] while (samples-- > 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 3254)
[declaration] snd_check(PCM, "invalid format width %d", width);
dirtycode/alsa-lib/src/pcm/pcm.c (line 3280)
[function] while (1) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 3290)
[function] if (chns > 1 && chns * width == step) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 3332)
[declaration] return snd_pcm_area_silence(dst_area, dst_offset, samples, format);
dirtycode/alsa-lib/src/pcm/pcm.c (line 3344)
[declaration] assert(src < dst || src >= dst + bytes);
dirtycode/alsa-lib/src/pcm/pcm.c (line 3345)
[declaration] assert(dst < src || dst >= src + bytes);
dirtycode/alsa-lib/src/pcm/pcm.c (line 3346)
[declaration] memcpy(dst, src, bytes);
dirtycode/alsa-lib/src/pcm/pcm.c (line 3358)
[function] while (samples-- > 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 3371)
[function] if (srcbit == 8) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 3377)
[function] if (dstbit == 8) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 3385)
[function] while (samples-- > 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 3393)
[function] while (samples-- > 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 3401)
[function] while (samples-- > 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 3410)
[function] while (samples-- > 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 3418)
[function] while (samples-- > 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 3426)
[declaration] snd_check(PCM, "invalid format width %d", width);
dirtycode/alsa-lib/src/pcm/pcm.c (line 3451)
[declaration] snd_check(PCM, "invalid channels %d", channels);

```

```

dirtycode/alsa-lib/src/pcm/pcm.c (line 3455)
[declaration] snd_check(PCM, "invalid frames %ld", frames);
dirtycode/alsa-lib/src/pcm/pcm.c (line 3466)
[function] while (dst_areas->step == step) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 3479)
[function] if (chns > 1 && chns * width == step) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 3574)
[declaration] dump_one_param(params, k, out);
dirtycode/alsa-lib/src/pcm/pcm.c (line 3576)
[declaration] dump_one_param(params, k, out);
dirtycode/alsa-lib/src/pcm/pcm.c (line 3594)
[declaration] snd_check(PCM, "invalid PCM info field");
dirtycode/alsa-lib/src/pcm/pcm.c (line 3614)
[declaration] snd_check(PCM, "invalid PCM info field");
dirtycode/alsa-lib/src/pcm/pcm.c (line 3634)
[declaration] snd_check(PCM, "invalid PCM info field");
dirtycode/alsa-lib/src/pcm/pcm.c (line 3654)
[declaration] snd_check(PCM, "invalid PCM info field");
dirtycode/alsa-lib/src/pcm/pcm.c (line 3674)
[declaration] snd_check(PCM, "invalid PCM info field");
dirtycode/alsa-lib/src/pcm/pcm.c (line 3694)
[declaration] snd_check(PCM, "invalid PCM info field");
dirtycode/alsa-lib/src/pcm/pcm.c (line 3714)
[declaration] snd_check(PCM, "invalid PCM info field");
dirtycode/alsa-lib/src/pcm/pcm.c (line 3734)
[declaration] snd_check(PCM, "invalid PCM info field");
dirtycode/alsa-lib/src/pcm/pcm.c (line 3754)
[declaration] snd_check(PCM, "invalid PCM info field");
dirtycode/alsa-lib/src/pcm/pcm.c (line 3774)
[declaration] snd_check(PCM, "invalid PCM info field");
dirtycode/alsa-lib/src/pcm/pcm.c (line 3794)
[declaration] snd_check(PCM, "invalid PCM info field");
dirtycode/alsa-lib/src/pcm/pcm.c (line 3810)
[declaration] snd_check(PCM, "invalid PCM info field");
dirtycode/alsa-lib/src/pcm/pcm.c (line 3833)
[declaration] snd_check(PCM, "invalid PCM info field");
dirtycode/alsa-lib/src/pcm/pcm.c (line 3871)
[declaration] snd_check(PCM, "invalid PCM info field");
dirtycode/alsa-lib/src/pcm/pcm.c (line 3908)
[declaration] snd_check(PCM, "invalid rate_den value");
dirtycode/alsa-lib/src/pcm/pcm.c (line 3938)
[declaration] snd_check(PCM, "invalid msbits value");
dirtycode/alsa-lib/src/pcm/pcm.c (line 3957)
[declaration] snd_check(PCM, "invalid PCM info field");
dirtycode/alsa-lib/src/pcm/pcm.c (line 3998)
[declaration] return snd_pcm_hw_refine(pcm, params);
dirtycode/alsa-lib/src/pcm/pcm.c (line 4007)
[declaration] return sizeof(snd_pcm_access_mask_t);

```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4069)
[declaration] return snd_mask_test((const snd_mask_t *) mask, (unsigned long) val);

dirtycode/alsa-lib/src/pcm/pcm.c (line 4080)
[declaration] return snd_mask_empty((const snd_mask_t *) mask);

dirtycode/alsa-lib/src/pcm/pcm.c (line 4109)
[declaration] return sizeof(snd_pcm_format_mask_t);

dirtycode/alsa-lib/src/pcm/pcm.c (line 4171)
[declaration] return snd_mask_test((const snd_mask_t *) mask, (unsigned long) val);

dirtycode/alsa-lib/src/pcm/pcm.c (line 4182)
[declaration] return snd_mask_empty((const snd_mask_t *) mask);

dirtycode/alsa-lib/src/pcm/pcm.c (line 4212)
[declaration] return sizeof(snd_pcm_subformat_mask_t);

dirtycode/alsa-lib/src/pcm/pcm.c (line 4274)
[declaration] return snd_mask_test((const snd_mask_t *) mask, (unsigned long) val);

dirtycode/alsa-lib/src/pcm/pcm.c (line 4285)
[declaration] return snd_mask_empty((const snd_mask_t *) mask);

dirtycode/alsa-lib/src/pcm/pcm.c (line 4315)
[declaration] return sizeof(snd_pcm_hw_params_t);

dirtycode/alsa-lib/src/pcm/pcm.c (line 4381)
[declaration] return snd_pcm_hw_param_set(pcm, params, SND_TEST, SND_PCM_HW_PARAM_ACCESS, access, 0);

dirtycode/alsa-lib/src/pcm/pcm.c (line 4393)
[declaration] return snd_pcm_hw_param_set(pcm, params, SND_TRY, SND_PCM_HW_PARAM_ACCESS, access, 0);

dirtycode/alsa-lib/src/pcm/pcm.c (line 4409)
[declaration] return snd_pcm_hw_param_set_first(pcm, params, SND_PCM_HW_PARAM_ACCESS, access, NULL);

dirtycode/alsa-lib/src/pcm/pcm.c (line 4425)
[declaration] return snd_pcm_hw_param_set_last(pcm, params, SND_PCM_HW_PARAM_ACCESS, access, NULL);

dirtycode/alsa-lib/src/pcm/pcm.c (line 4437)
[declaration] return snd_pcm_hw_param_set_mask(pcm, params, SND_TRY, SND_PCM_HW_PARAM_ACCESS, (snd_mask_t *) mask);

dirtycode/alsa-lib/src/pcm/pcm.c (line 4466)
[declaration] return snd_pcm_hw_param_get(params, SND_PCM_HW_PARAM_FORMAT, (unsigned int *)format, NULL);

dirtycode/alsa-lib/src/pcm/pcm.c (line 4478)
[declaration] return snd_pcm_hw_param_set(pcm, params, SND_TEST, SND_PCM_HW_PARAM_FORMAT, format, 0);

dirtycode/alsa-lib/src/pcm/pcm.c (line 4490)
[declaration] return snd_pcm_hw_param_set(pcm, params, SND_TRY, SND_PCM_HW_PARAM_FORMAT, format, 0);

dirtycode/alsa-lib/src/pcm/pcm.c (line 4506)
[declaration] return snd_pcm_hw_param_set_first(pcm, params, SND_PCM_HW_PARAM_FORMAT, (unsigned int *)format, NULL);

dirtycode/alsa-lib/src/pcm/pcm.c (line 4522)
[declaration] return snd_pcm_hw_param_set_last(pcm, params, SND_PCM_HW_PARAM_FORMAT, (unsigned int *)format, NULL);

dirtycode/alsa-lib/src/pcm/pcm.c (line 4534)
[declaration] return snd_pcm_hw_param_set_mask(pcm, params, SND_TRY, SND_PCM_HW_PARAM_FORMAT, (snd_mask_t *) mask);

dirtycode/alsa-lib/src/pcm/pcm.c (line 4560)

```
[declaration] return snd_pcm_hw_param_get(params, SND_PCM_HW_PARAM_SUBFORMAT,  
(unsigned int *)subformat, NULL);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4572)

```
[declaration] return snd_pcm_hw_param_set(pcm, params, SND_TEST,  
SND_PCM_HW_PARAM_SUBFORMAT, subformat, 0);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4584)

```
[declaration] return snd_pcm_hw_param_set(pcm, params, SND_TRY,  
SND_PCM_HW_PARAM_SUBFORMAT, subformat, 0);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4600)

```
[declaration] return snd_pcm_hw_param_set_first(pcm, params,  
SND_PCM_HW_PARAM_SUBFORMAT, (unsigned int *)subformat, NULL);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4616)

```
[declaration] return snd_pcm_hw_param_set_last(pcm, params,  
SND_PCM_HW_PARAM_SUBFORMAT, (unsigned int *)subformat, NULL);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4628)

```
[declaration] return snd_pcm_hw_param_set_mask(pcm, params, SND_TRY,  
SND_PCM_HW_PARAM_SUBFORMAT, (snd_mask_t *) mask);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4654)

```
[declaration] return snd_pcm_hw_param_get(params, SND_PCM_HW_PARAM_CHANNELS, val,  
NULL);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4669)

```
[declaration] return snd_pcm_hw_param_get_min(params, SND_PCM_HW_PARAM_CHANNELS,  
val, NULL);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4684)

```
[declaration] return snd_pcm_hw_param_get_max(params, SND_PCM_HW_PARAM_CHANNELS,  
val, NULL);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4696)

```
[declaration] return snd_pcm_hw_param_set(pcm, params, SND_TEST,  
SND_PCM_HW_PARAM_CHANNELS, val, 0);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4708)

```
[declaration] return snd_pcm_hw_param_set(pcm, params, SND_TRY,  
SND_PCM_HW_PARAM_CHANNELS, val, 0);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4720)

```
[declaration] return snd_pcm_hw_param_set_min(pcm, params, SND_TRY,  
SND_PCM_HW_PARAM_CHANNELS, val, NULL);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4732)

```
[declaration] return snd_pcm_hw_param_set_max(pcm, params, SND_TRY,  
SND_PCM_HW_PARAM_CHANNELS, val, NULL);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4745)

```
[declaration] return snd_pcm_hw_param_set_minmax(pcm, params, SND_TRY,  
SND_PCM_HW_PARAM_CHANNELS, min, NULL, max, NULL);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4761)

```
[declaration] return snd_pcm_hw_param_set_near(pcm, params,  
SND_PCM_HW_PARAM_CHANNELS, val, NULL);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4777)

```
[declaration] return snd_pcm_hw_param_set_first(pcm, params,  
SND_PCM_HW_PARAM_CHANNELS, val, NULL);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4793)

```
[declaration] return snd_pcm_hw_param_set_last(pcm, params,  
SND_PCM_HW_PARAM_CHANNELS, val, NULL);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 4812)

[declaration] return snd_pcm_hw_param_get(params, SND_PCM_HW_PARAM_RATE, val, dir);

dirtycode/alsa-lib/src/pcm/pcm.c (line 4830)

[declaration] return snd_pcm_hw_param_get_min(params, SND_PCM_HW_PARAM_RATE, val, dir);

dirtycode/alsa-lib/src/pcm/pcm.c (line 4848)

[declaration] return snd_pcm_hw_param_get_max(params, SND_PCM_HW_PARAM_RATE, val, dir);

dirtycode/alsa-lib/src/pcm/pcm.c (line 4863)

[declaration] return snd_pcm_hw_param_set(pcm, params, SND_TEST, SND_PCM_HW_PARAM_RATE, val, dir);

dirtycode/alsa-lib/src/pcm/pcm.c (line 4878)

[declaration] return snd_pcm_hw_param_set(pcm, params, SND_TRY, SND_PCM_HW_PARAM_RATE, val, dir);

dirtycode/alsa-lib/src/pcm/pcm.c (line 4893)

[declaration] return snd_pcm_hw_param_set_min(pcm, params, SND_TRY, SND_PCM_HW_PARAM_RATE, val, dir);

dirtycode/alsa-lib/src/pcm/pcm.c (line 4908)

[declaration] return snd_pcm_hw_param_set_max(pcm, params, SND_TRY, SND_PCM_HW_PARAM_RATE, val, dir);

dirtycode/alsa-lib/src/pcm/pcm.c (line 4925)

[declaration] return snd_pcm_hw_param_set_minmax(pcm, params, SND_TRY, SND_PCM_HW_PARAM_RATE, min, mindir, max, maxdir);

dirtycode/alsa-lib/src/pcm/pcm.c (line 4944)

[declaration] return snd_pcm_hw_param_set_near(pcm, params, SND_PCM_HW_PARAM_RATE, val, dir);

dirtycode/alsa-lib/src/pcm/pcm.c (line 4963)

[declaration] return snd_pcm_hw_param_set_first(pcm, params, SND_PCM_HW_PARAM_RATE, val, dir);

dirtycode/alsa-lib/src/pcm/pcm.c (line 4982)

[declaration] return snd_pcm_hw_param_set_last(pcm, params, SND_PCM_HW_PARAM_RATE, val, dir);

dirtycode/alsa-lib/src/pcm/pcm.c (line 5000)

[declaration] return snd_pcm_hw_refine(pcm, params);

dirtycode/alsa-lib/src/pcm/pcm.c (line 5032)

[declaration] return snd_pcm_hw_refine(pcm, params);

dirtycode/alsa-lib/src/pcm/pcm.c (line 5083)

[declaration] return snd_pcm_hw_refine(pcm, params);

dirtycode/alsa-lib/src/pcm/pcm.c (line 5120)

[declaration] return snd_pcm_hw_refine(pcm, params);

dirtycode/alsa-lib/src/pcm/pcm.c (line 5152)

[declaration] return snd_pcm_hw_param_get(params, SND_PCM_HW_PARAM_PERIOD_TIME, val, dir);

dirtycode/alsa-lib/src/pcm/pcm.c (line 5170)

[declaration] return snd_pcm_hw_param_get_min(params, SND_PCM_HW_PARAM_PERIOD_TIME, val, dir);

dirtycode/alsa-lib/src/pcm/pcm.c (line 5188)

[declaration] return snd_pcm_hw_param_get_max(params, SND_PCM_HW_PARAM_PERIOD_TIME, val, dir);

dirtycode/alsa-lib/src/pcm/pcm.c (line 5203)

[declaration] return snd_pcm_hw_param_set(pcm, params, SND_TEST, SND_PCM_HW_PARAM_PERIOD_TIME, val, dir);

dirtycode/alsa-lib/src/pcm/pcm.c (line 5218)

```
[declaration] return snd_pcm_hw_param_set(pcm, params, SND_TRY,  
SND_PCM_HW_PARAM_PERIOD_TIME, val, dir);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5234)

```
[declaration] return snd_pcm_hw_param_set_min(pcm, params, SND_TRY,  
SND_PCM_HW_PARAM_PERIOD_TIME, val, dir);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5249)

```
[declaration] return snd_pcm_hw_param_set_max(pcm, params, SND_TRY,  
SND_PCM_HW_PARAM_PERIOD_TIME, val, dir);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5266)

```
[declaration] return snd_pcm_hw_param_set_minmax(pcm, params, SND_TRY,  
SND_PCM_HW_PARAM_PERIOD_TIME, min, mindir, max, maxdir);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5285)

```
[declaration] return snd_pcm_hw_param_set_near(pcm, params,  
SND_PCM_HW_PARAM_PERIOD_TIME, val, dir);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5304)

```
[declaration] return snd_pcm_hw_param_set_first(pcm, params,  
SND_PCM_HW_PARAM_PERIOD_TIME, val, dir);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5321)

```
[declaration] return snd_pcm_hw_param_set_last(pcm, params,  
SND_PCM_HW_PARAM_PERIOD_TIME, val, dir);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5403)

```
[declaration] return snd_pcm_hw_param_set(pcm, params, SND_TEST,  
SND_PCM_HW_PARAM_PERIOD_SIZE, val, dir);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5418)

```
[declaration] return snd_pcm_hw_param_set(pcm, params, SND_TRY,  
SND_PCM_HW_PARAM_PERIOD_SIZE, val, dir);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5558)

```
[declaration] return snd_pcm_hw_param_set_integer(pcm, params, SND_TRY,  
SND_PCM_HW_PARAM_PERIOD_SIZE);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5577)

```
[declaration] return snd_pcm_hw_param_get(params, SND_PCM_HW_PARAM_PERIODS, val,  
dir);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5595)

```
[declaration] return snd_pcm_hw_param_get_min(params, SND_PCM_HW_PARAM_PERIODS, val,  
dir);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5613)

```
[declaration] return snd_pcm_hw_param_get_max(params, SND_PCM_HW_PARAM_PERIODS, val,  
dir);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5628)

```
[declaration] return snd_pcm_hw_param_set(pcm, params, SND_TEST,  
SND_PCM_HW_PARAM_PERIODS, val, dir);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5643)

```
[declaration] return snd_pcm_hw_param_set(pcm, params, SND_TRY,  
SND_PCM_HW_PARAM_PERIODS, val, dir);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5658)

```
[declaration] return snd_pcm_hw_param_set_min(pcm, params, SND_TRY,  
SND_PCM_HW_PARAM_PERIODS, val, dir);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5673)

```
[declaration] return snd_pcm_hw_param_set_max(pcm, params, SND_TRY,  
SND_PCM_HW_PARAM_PERIODS, val, dir);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5690)

```
[declaration] return snd_pcm_hw_param_set_minmax(pcm, params, SND_TRY,  
SND_PCM_HW_PARAM_PERIODS, min, mindir, max, maxdir);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5709)

```
[declaration] return snd_pcm_hw_param_set_near(pcm, params,  
SND_PCM_HW_PARAM_PERIODS, val, dir);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5728)

```
[declaration] return snd_pcm_hw_param_set_first(pcm, params,  
SND_PCM_HW_PARAM_PERIODS, val, dir);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5747)

```
[declaration] return snd_pcm_hw_param_set_last(pcm, params,  
SND_PCM_HW_PARAM_PERIODS, val, dir);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5758)

```
[declaration] return snd_pcm_hw_param_set_integer(pcm, params, SND_TRY,  
SND_PCM_HW_PARAM_PERIODS);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5777)

```
[declaration] return snd_pcm_hw_param_get(params, SND_PCM_HW_PARAM_BUFFER_TIME, val,  
dir);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5795)

```
[declaration] return snd_pcm_hw_param_get_min(params, SND_PCM_HW_PARAM_BUFFER_TIME,  
val, dir);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5813)

```
[declaration] return snd_pcm_hw_param_get_max(params, SND_PCM_HW_PARAM_BUFFER_TIME,  
val, dir);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5828)

```
[declaration] return snd_pcm_hw_param_set(pcm, params, SND_TEST,  
SND_PCM_HW_PARAM_BUFFER_TIME, val, dir);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5843)

```
[declaration] return snd_pcm_hw_param_set(pcm, params, SND_TRY,  
SND_PCM_HW_PARAM_BUFFER_TIME, val, dir);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5858)

```
[declaration] return snd_pcm_hw_param_set_min(pcm, params, SND_TRY,  
SND_PCM_HW_PARAM_BUFFER_TIME, val, dir);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5873)

```
[declaration] return snd_pcm_hw_param_set_max(pcm, params, SND_TRY,  
SND_PCM_HW_PARAM_BUFFER_TIME, val, dir);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5890)

```
[declaration] return snd_pcm_hw_param_set_minmax(pcm, params, SND_TRY,  
SND_PCM_HW_PARAM_BUFFER_TIME, min, mindir, max, maxdir);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5909)

```
[declaration] return snd_pcm_hw_param_set_near(pcm, params,  
SND_PCM_HW_PARAM_BUFFER_TIME, val, dir);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5928)

```
[declaration] return snd_pcm_hw_param_set_first(pcm, params,  
SND_PCM_HW_PARAM_BUFFER_TIME, val, dir);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 5947)

```
[declaration] return snd_pcm_hw_param_set_last(pcm, params,  
SND_PCM_HW_PARAM_BUFFER_TIME, val, dir);
```

dirtycode/alsa-lib/src/pcm/pcm.c (line 6021)

```
[declaration] return snd_pcm_hw_param_set(pcm, params, SND_TEST,  
SND_PCM_HW_PARAM_BUFFER_SIZE, val, 0);
```

```

dirtycode/alsa-lib/src/pcm/pcm.c (line 6035)
[declaration] return snd_pcm_hw_param_set(pcm, params, SND_TRY,
SND_PCM_HW_PARAM_BUFFER_SIZE, val, 0);
dirtycode/alsa-lib/src/pcm/pcm.c (line 6400)
[declaration] snd_check(PCM, "PCM not set up");
dirtycode/alsa-lib/src/pcm/pcm.c (line 6435)
[declaration] return sizeof(snd_pcm_sw_params_t);
dirtycode/alsa-lib/src/pcm/pcm.c (line 6503)
[declaration] snd_check(PCM, "invalid start mode value %d", val);
dirtycode/alsa-lib/src/pcm/pcm.c (line 6551)
[declaration] snd_check(PCM, "invalid xrun mode value %d", val);
dirtycode/alsa-lib/src/pcm/pcm.c (line 6592)
[declaration] snd_check(PCM, "invalid tstamp_mode value %d", val);
dirtycode/alsa-lib/src/pcm/pcm.c (line 6627)
[declaration] snd_check(PCM, "invalid tstamp_type value %d", val);
dirtycode/alsa-lib/src/pcm/pcm.c (line 6994)
[declaration] return sizeof(snd_pcm_status_t);
dirtycode/alsa-lib/src/pcm/pcm.c (line 7209)
[declaration] return sizeof(snd_pcm_info_t);
dirtycode/alsa-lib/src/pcm/pcm.c (line 7491)
[declaration] return __snd_pcm_mmap_begin_generic(pcm, areas, offset, frames);
dirtycode/alsa-lib/src/pcm/pcm.c (line 7519)
[declaration] error(avail);
dirtycode/alsa-lib/src/pcm/pcm.c (line 7527)
[function] while (size > 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 7532)
[declaration] error(err);
dirtycode/alsa-lib/src/pcm/pcm.c (line 7534)
[declaration] generate_sine(areas, offset, frames, &phase);
dirtycode/alsa-lib/src/pcm/pcm.c (line 7537)
[declaration] error(commitres >= 0 ? -EPIPE : commitres);
dirtycode/alsa-lib/src/pcm/pcm.c (line 7580)
[declaration] snd_pcm_mmap_avail(pcm));
dirtycode/alsa-lib/src/pcm/pcm.c (line 7645)
[function] switch (state) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 7666)
[function] if (avail < 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 7670)
[function] if (avail == 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 7673)
[function] if (pcm->mode & SND_PCM_NONBLOCK) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 7718)
[function] switch (state) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 7734)
[function] if (avail < 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 7740)
[function] if (snd_pcm_may_wait_for_avail_min(pcm, avail)) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 7741)
[function] if (pcm->mode & SND_PCM_NONBLOCK) {

```

```

dirtycode/alsa-lib/src/pcm/pcm.c (line 7755)
[function] if (avail < 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 7769)
[function] if (state == SND_PCM_STATE_PREPARED) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 7838)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 7839)
[declaration] snd_error(PCM, "Invalid slave definition");
dirtycode/alsa-lib/src/pcm/pcm.c (line 7845)
[declaration] snd_error(PCM, "Invalid slave definition");
dirtycode/alsa-lib/src/pcm/pcm.c (line 7864)
[function] if (strcmp(id, "pcm") == 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 7866)
[declaration] snd_config_delete(pcm_conf);
dirtycode/alsa-lib/src/pcm/pcm.c (line 7874)
[declaration] assert(idx < SND_PCM_HW_PARAM_LAST_INTERVAL);
dirtycode/alsa-lib/src/pcm/pcm.c (line 7875)
[declaration] assert(names[idx]);
dirtycode/alsa-lib/src/pcm/pcm.c (line 7878)
[function] switch (idx) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 7883)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 7885)
[declaration] snd_error(PCM, "invalid type for %s", id);
dirtycode/alsa-lib/src/pcm/pcm.c (line 7889)
[function] strcasecmp(str, "unchanged") == 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 7894)
[function] if (f == SND_PCM_FORMAT_UNKNOWN) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 7895)
[declaration] snd_error(PCM, "unknown format %s", str);
dirtycode/alsa-lib/src/pcm/pcm.c (line 7903)
[function] if ((fields[k].flags & SCNF_UNCHANGED)) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 7906)
[function] strcasecmp(str, "unchanged") == 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 7922)
[declaration] snd_error(PCM, "Unknown field %s", id);
dirtycode/alsa-lib/src/pcm/pcm.c (line 7927)
[declaration] snd_error(PCM, "missing field pcm");
dirtycode/alsa-lib/src/pcm/pcm.c (line 7932)
[function] if ((fields[k].flags & SCNF_MANDATORY) && !fields[k].present) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 7933)
[declaration] snd_error(PCM, "missing field %s", names[fields[k].index]);
dirtycode/alsa-lib/src/pcm/pcm.c (line 7943)
[declaration] snd_config_delete(pcm_conf);
dirtycode/alsa-lib/src/pcm/pcm.c (line 7945)
[declaration] snd_config_delete(conf);
dirtycode/alsa-lib/src/pcm/pcm.c (line 7982)
[function] if (a[idx] == NULL) {

```

```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8012)
[function] if (a[idx] == pcm) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 8068)
[declaration] default_symbol_version(__##name, name, new); \
dirtycode/alsa-lib/src/pcm/pcm.c (line 8069)
[declaration] symbol_version(__old_##name, name, what);
dirtycode/alsa-lib/src/pcm/pcm.c (line 8074)
[declaration] use_default_symbol_version(__##name, name, new);
dirtycode/alsa-lib/src/pcm/pcm.c (line 8337)
[declaration] free(*p);
dirtycode/alsa-lib/src/pcm/pcm.c (line 8499)
[function] if (i > 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 8517)
[function] if (map->pos[i] & SND_CHMAP_PHASE_INVERSE) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 8548)
[declaration] assert(chmap_names[val]);
dirtycode/alsa-lib/src/pcm/pcm.c (line 8573)
[declaration] return str_to_chmap(str, strlen(str));
dirtycode/alsa-lib/src/pcm/pcm.c (line 8604)
[function] if (*str == '[') {
dirtycode/alsa-lib/src/pcm/pcm.c (line 8605)
[function] if (!strncmp(str, "[INV]", 5)) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 8649)
[declaration] free(maps);
dirtycode/alsa-lib/src/pcm/pcm.c (line 8683)
[function] if (!maps[i]) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 8684)
[declaration] snd_pcm_free_chmaps(maps);
dirtycode/alsa-lib/src/pcm/pcm.c (line 8687)
[declaration] memcpy(maps[i], src[i], (src[i]->map.channels + 2) * sizeof(int));
dirtycode/alsa-lib/src/pcm/pcm.c (line 8700)
[declaration] return _snd_pcm_copy_chmap(&(*p)->map);
dirtycode/alsa-lib/src/pcm/pcm.c (line 8738)
[function] if (_copy_to_fixed_query_map(maps + nums, chmap)) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 8739)
[declaration] free(chmap);
dirtycode/alsa-lib/src/pcm/pcm.c (line 8742)
[declaration] free(chmap);
dirtycode/alsa-lib/src/pcm/pcm.c (line 8787)
[declaration] snd_error(PCM, "%s occurred", s);
dirtycode/alsa-lib/src/pcm/pcm.c (line 8789)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 8790)
[declaration] snd_error(PCM, "cannot recovery from %s, prepare failed: %s", s,
snd_strerror(err));
dirtycode/alsa-lib/src/pcm/pcm.c (line 8799)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 8801)
[function] if (err < 0) {

```

```
dirtycode/alsa-lib/src/pcm/pcm.c (line 8802)
[declaration] snd_error(PCM, "cannot recovery from suspend, prepare failed: %s",
snd_strerror(err));
dirtycode/alsa-lib/src/pcm/pcm.c (line 8850)
[declaration] s, snd_strerror(err));
dirtycode/alsa-lib/src/pcm/pcm.c (line 8858)
[declaration] s, snd_strerror(err));
dirtycode/alsa-lib/src/pcm/pcm.c (line 8866)
[declaration] s, snd_strerror(err));
dirtycode/alsa-lib/src/pcm/pcm.c (line 8874)
[declaration] channels, s, snd_strerror(err));
dirtycode/alsa-lib/src/pcm/pcm.c (line 8884)
[declaration] rate, snd_strerror(err));
dirtycode/alsa-lib/src/pcm/pcm.c (line 8905)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 8907)
[declaration] period_time, s, snd_strerror(err));
dirtycode/alsa-lib/src/pcm/pcm.c (line 8913)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 8915)
[declaration] s, snd_strerror(err));
dirtycode/alsa-lib/src/pcm/pcm.c (line 8922)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 8924)
[declaration] buffer_size, s, snd_strerror(err));
dirtycode/alsa-lib/src/pcm/pcm.c (line 8930)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 8932)
[declaration] s, snd_strerror(err));
dirtycode/alsa-lib/src/pcm/pcm.c (line 8940)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 8942)
[declaration] s, snd_strerror(err));
dirtycode/alsa-lib/src/pcm/pcm.c (line 8948)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 8950)
[declaration] s, snd_strerror(err));
dirtycode/alsa-lib/src/pcm/pcm.c (line 8958)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 8960)
[declaration] period_time, s, snd_strerror(err));
dirtycode/alsa-lib/src/pcm/pcm.c (line 8966)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm.c (line 8968)
[declaration] s, snd_strerror(err));
dirtycode/alsa-lib/src/pcm/pcm.c (line 8977)
[declaration] s, snd_strerror(err));
dirtycode/alsa-lib/src/pcm/pcm.c (line 8986)
[declaration] s, snd_strerror(err));
```

```

dirtycode/alsa-lib/src/pcm/pcm.c (line 8998)
[declaration] s, snd_strerror(err));
dirtycode/alsa-lib/src/pcm/pcm.c (line 9009)
[declaration] s, snd_strerror(err));
dirtycode/alsa-lib/src/pcm/pcm.c (line 9017)
[declaration] s, snd_strerror(err));
dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 139)
[function] if (diff >= step) {
dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 189)
[function] if (code & i) {
dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 244)
[function] while (frames1-- > 0) {
dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 259)
[function] if (srcbit == 8) {
dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 298)
[function] while (frames1-- > 0) {
dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 313)
[function] if (dstbit == 8) {
dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 425)
[function] if (adpcm->sformat == SND_PCM_FORMAT_IMA_ADPCM) {
dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 433)
[function] if (adpcm->sformat == SND_PCM_FORMAT_IMA_ADPCM) {
dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 453)
[declaration] return snd_pcm_hw_free(adpcm->plug.gen.slave);
dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 511)
[declaration] snd_pcm_format_name(adpcm->sformat));
dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 513)
[declaration] snd_output_printf(out, "Its setup is:\n");
dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 514)
[declaration] snd_pcm_dump_setup(pcm, out);
dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 573)
[declaration] free(adpcm);
dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 649)
[function] if (strcmp(id, "slave") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 653)
[declaration] snd_error(PCM, "Unknown field %s", id);
dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 657)
[declaration] snd_error(PCM, "slave is not defined");
dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 666)
[declaration] snd_config_delete(sconf);
dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 667)
[declaration] snd_error(PCM, "invalid slave format");
dirtycode/alsa-lib/src/pcm/pcm_adpcm.c (line 676)
[declaration] snd_pcm_close(spcm);
dirtycode/alsa-lib/src/pcm/pcm_alaw.c (line 168)
[function] while (frames1-- > 0) {
dirtycode/alsa-lib/src/pcm/pcm_alaw.c (line 206)
[function] while (frames1-- > 0) {

```

```

dirtycode/alsa-lib/src/pcm/pcm_alaw.c (line 322)
[function] if (alaw->sformat == SND_PCM_FORMAT_A_LAW) {
dirtycode/alsa-lib/src/pcm/pcm_alaw.c (line 330)
[function] if (alaw->sformat == SND_PCM_FORMAT_A_LAW) {
dirtycode/alsa-lib/src/pcm/pcm_alaw.c (line 385)
[declaration] snd_pcm_format_name(alaw->sformat));
dirtycode/alsa-lib/src/pcm/pcm_alaw.c (line 387)
[declaration] snd_output_printf(out, "Its setup is:\n");
dirtycode/alsa-lib/src/pcm/pcm_alaw.c (line 388)
[declaration] snd_pcm_dump_setup(pcm, out);
dirtycode/alsa-lib/src/pcm/pcm_alaw.c (line 448)
[declaration] free(alaw);
dirtycode/alsa-lib/src/pcm/pcm_alaw.c (line 524)
[function] if (strcmp(id, "slave") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_alaw.c (line 528)
[declaration] snd_error(PCM, "Unknown field %s", id);
dirtycode/alsa-lib/src/pcm/pcm_alaw.c (line 532)
[declaration] snd_error(PCM, "slave is not defined");
dirtycode/alsa-lib/src/pcm/pcm_alaw.c (line 541)
[declaration] snd_config_delete(sconf);
dirtycode/alsa-lib/src/pcm/pcm_alaw.c (line 542)
[declaration] snd_error(PCM, "invalid slave format");
dirtycode/alsa-lib/src/pcm/pcm_alaw.c (line 551)
[declaration] snd_pcm_close(spcm);
dirtycode/alsa-lib/src/pcm/pcm_asym.c (line 91)
[function] if (strcmp(id, "playback") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_asym.c (line 96)
[function] if (strcmp(id, "capture") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_asym.c (line 101)
[declaration] snd_error(PCM, "Unknown field %s", id);
dirtycode/alsa-lib/src/pcm/pcm_copy.c (line 148)
[declaration] snd_output_printf(out, "Its setup is:\n");
dirtycode/alsa-lib/src/pcm/pcm_copy.c (line 149)
[declaration] snd_pcm_dump_setup(pcm, out);
dirtycode/alsa-lib/src/pcm/pcm_copy.c (line 204)
[declaration] free(copy);
dirtycode/alsa-lib/src/pcm/pcm_copy.c (line 277)
[function] if (strcmp(id, "slave") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_copy.c (line 281)
[declaration] snd_error(PCM, "Unknown field %s", id);
dirtycode/alsa-lib/src/pcm/pcm_copy.c (line 285)
[declaration] snd_error(PCM, "slave is not defined");
dirtycode/alsa-lib/src/pcm/pcm_copy.c (line 297)
[declaration] snd_pcm_close(spcm);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 78)
[function] if (semctl(dmixmap->semid, i, IPC_STAT, s) < 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 80)
[declaration] snd_pcm_direct_semaphore_discard(dmixmap);

```

```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 85)
[declaration] semctl(dmix->semid, i, IPC_SET, s);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 131)
[declaration] snd_pcm_direct_shm_discard(dmix);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 137)
[declaration] snd_pcm_direct_shm_discard(dmix);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 141)
[declaration] memset(dmix->shmptr, 0, sizeof(snd_pcm_direct_share_t));
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 142)
[function] if (dmix->ipc_gid >= 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 144)
[declaration] shmctl(dmix->shmid, IPC_SET, &buf);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 149)
[function] if (dmix->shmptr->magic != snd_pcm_direct_magic(dmix)) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 150)
[declaration] snd_pcm_direct_shm_discard(dmix);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 188)
[declaration] return _snd_pcm_direct_shm_discard(dmix);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 215)
[declaration] snd_erronum(PCM, "socket failed");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 220)
[declaration] unlink(filename);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 226)
[function] if (bind(sock, (struct sockaddr *) addr, size) < 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 228)
[declaration] snd_erronum(PCM, "bind failed: %s", filename);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 229)
[declaration] close(sock);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 232)
[function] if (chmod(filename, ipc_perm) < 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 234)
[declaration] snd_erronum(PCM, "chmod failed: %s", filename);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 235)
[declaration] close(sock);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 236)
[declaration] unlink(filename);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 239)
[function] if (chown(filename, -1, ipc_gid) < 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 242)
[declaration] snd_erronum(PCM, "chown failed: %s", filename);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 243)
[declaration] close(sock);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 244)
[declaration] unlink(filename);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 250)
[function] if (connect(sock, (struct sockaddr *) addr, size) < 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 252)
[declaration] snd_erronum(PCM, "connect failed: %s", filename);

```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 253)

[declaration] close(sock);

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 343)

[declaration] close(i);

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 355)

[declaration] server_printf("DIRECT SERVER: poll ret = %i, revents[0] = 0x%x, errno = %i\n", ret, pfd[0].revents, errno);

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 356)

[function] if (ret < 0) {

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 362)

[function] if (ret == 0 || (pfd[0].revents & (POLLERR | POLLHUP))) { /* timeout or error? */

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 364)

[declaration] snd_pcm_direct_semaphore_down(dmix, DIRECT_IPC_SEM_CLIENT);

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 365)

[function] if (shmctl(dmix->shmid, IPC_STAT, &buf) < 0) {

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 366)

[declaration] _snd_pcm_direct_shm_discard(dmix);

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 367)

[declaration] snd_pcm_direct_semaphore_up(dmix, DIRECT_IPC_SEM_CLIENT);

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 370)

[declaration] server_printf("DIRECT SERVER: nattch = %i\n", (int)buf.shm_nattch);

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 373)

[declaration] snd_pcm_direct_semaphore_up(dmix, DIRECT_IPC_SEM_CLIENT);

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 376)

[function] if (pfd[0].revents & POLLIN) {

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 379)

[function] if (sck >= 0) {

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 380)

[declaration] server_printf("DIRECT SERVER: new connection %i\n", sck);

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 381)

[function] if (current == max) {

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 382)

[declaration] close(sck);

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 387)

[declaration] _snd_send_fd(sck, &buf, 1, dmix->hw_fd);

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 388)

[declaration] server_printf("DIRECT SERVER: fd sent ok\n");

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 396)

[declaration] server_printf("client %i revents = 0x%x\n", pfd->fd, pfd->revents);

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 397)

[function] if (pfd->revents & (POLLERR | POLLHUP)) {

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 399)

[declaration] close(pfd->fd);

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 410)

[function] if (pfd[i+1].fd < 0) {

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 412)

[declaration] memcpy(&pfd[i+1], &pfd[i+2], sizeof(struct pollfd) * (max - i - 1));

```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 442)
[declaration] close(dmixon->server_fd);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 448)
[declaration] close(dmixon->server_fd);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 453)
[declaration] server_job(dmixon);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 454)
[declaration] _exit(EXIT_SUCCESS);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 456)
[declaration] waitpid(ret, NULL, 0);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 471)
[declaration] close(dmixon->server_fd);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 494)
[declaration] close(dmixon->comm_fd);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 506)
[declaration] close(dmixon->comm_fd);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 525)
[declaration] return snd_timer_async(dmixon->timer, sig, pid);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 533)
[function] while (poll(&dmixon->timer_fd, 1, 0) > 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 536)
[function] if (dmixon->tread) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 538)
[declaration] snd_timer_read(dmixon->timer, rbuf, sizeof(rbuf));
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 541)
[declaration] snd_timer_read(dmixon->timer, &rbuf, sizeof(rbuf));
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 545)
[function] if (dmixon->tread) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 588)
[declaration] snd_error(PCM, "SEMDOWN FAILED with err %d", semerr);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 597)
[function] if (semerr < 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 598)
[declaration] snd_error(PCM, "SEMUP FAILED with err %d", semerr);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 616)
[declaration] snd_pcm_resume(direct->spcm);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 617)
[declaration] snd_pcm_drop(direct->spcm);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 618)
[declaration] snd_pcm_direct_timer_stop(direct);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 619)
[declaration] snd_pcm_direct_clear_timer_queue(direct);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 624)
[declaration] snd_error(PCM, "recover: unable to prepare slave");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 627)
[function] if (semerr < 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 628)
[declaration] snd_error(PCM, "SEMUP FAILED with err %d", semerr);

```

```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 644)
[declaration] snd_error(PCM, "recover: unable to start slave");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 647)
[function] if (semerr < 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 648)
[declaration] snd_error(PCM, "SEMUP FAILED with err %d", semerr);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 656)
[declaration] snd_error(PCM, "SEMUP FAILED with err %d", semerr);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 694)
[declaration] gettimeofday(&direct->trigger_tstamp, pcm->tstamp_type);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 697)
[declaration] * snd_pcm_direct_clear_timer_queue(direct);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 699)
[function] if (direct->recoveries & RECOVERIES_FLAG_SUSPENDED) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 730)
[declaration] snd_check(PCM, "poll_fd < 0");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 763)
[declaration] __snd_pcm_avail_update(pcm);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 764)
[function] if (pcm->stream == SND_PCM_STREAM_PLAYBACK) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 775)
[function] snd_pcm_state(dmix->spcm) == SND_PCM_STATE_SETUP) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 778)
[function] if (empty) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 791)
[function] switch (__snd_pcm_state(pcm)) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 811)
[declaration] return snd_pcm_info(dmix->spcm, info);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 818)
[declaration] snd_strncpy((char *)info->id, pcm->name, sizeof(info->id));
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 819)
[declaration] snd_strncpy((char *)info->name, pcm->name, sizeof(info->name));
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 820)
[declaration] snd_strncpy((char *)info->subname, pcm->name, sizeof(info->subname));
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 848)
[declaration] snd_error(PCM, "dshare interval %i empty?", (int)var);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 866)
[declaration] return hw_param_interval_refine_one(params, var, &t);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 912)
[function] if (snd_mask_empty(hw_param_mask(params, SND_PCM_HW_PARAM_ACCESS))) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 913)
[declaration] snd_error(PCM, "dshare access mask empty?");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 920)
[function] if (snd_mask_empty(hw_param_mask(params, SND_PCM_HW_PARAM_FORMAT))) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 921)
[declaration] snd_error(PCM, "dshare format mask empty?");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 930)
[function] if (snd_interval_empty(hw_param_interval(params,
SND_PCM_HW_PARAM_CHANNELS))) {

```

```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 931)
[declaration] snd_error(PCM, "dshare channels mask empty?");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 979)
[function] if (dshare->var_periodsize) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 981)
[function] if (dshare->shmptr->hw.buffer_size.max / 2 > period_size.max) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 985)
[function] if (dshare->shmptr->hw.buffer_time.max / 2 > period_time.max) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1060)
[declaration] return snd_pcm_channel_info_shm(pcm, info, -1);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1080)
[declaration] return snd_pcm_query_chmaps(dmix->spcm);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1087)
[declaration] free(maps);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1092)
[declaration] snd_pcm_free_chmaps(maps);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1109)
[declaration] return snd_pcm_get_chmap(dmix->spcm);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1115)
[declaration] return snd_pcm_set_chmap(dmix->spcm, map);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1130)
[declaration] snd_pcm_start(dmix->spcm);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1142)
[declaration] return snd_pcm_direct_set_timer_params(dmix);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1206)
[declaration] snd_error(PCM, "unable to find a valid configuration for slave");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1211)
[declaration] snd_error(PCM, "snd_pcm_hw_params_any failed");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1219)
[function] if (ret < 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1220)
[declaration] snd_error(PCM, "slave plugin does not support mmap interleaved or mmap
noninterleaved access");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1251)
[function] if (ret < 0 && dmix->type != SND_PCM_TYPE_DMIX) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1256)
[function] if (ret < 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1257)
[declaration] snd_error(PCM, "requested or auto-format is not available");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1265)
[declaration] snd_error(PCM, "requested count of channels is not available");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1271)
[declaration] snd_error(PCM, "requested rate is not available");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1279)
[function] if (ret < 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1280)
[declaration] snd_error(PCM, "unable to set buffer time");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1286)
[function] if (ret < 0) {

```

```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1287)
[declaration] snd_error(PCM, "unable to set buffer size");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1297)
[function] if (ret < 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1298)
[declaration] snd_error(PCM, "unable to set period_time");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1305)
[function] if (ret < 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1306)
[declaration] snd_error(PCM, "unable to set period_size");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1315)
[function] if (ret < 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1316)
[declaration] snd_error(PCM, "unable to set requested periods");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1319)
[function] if (params->periods == 1) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1321)
[function] if (params->period_time > 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1328)
[declaration] snd_error(PCM, "unable to use stream with periods == 1");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1335)
[declaration] snd_error(PCM, "unable to install hw params");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1359)
[declaration] snd_error(PCM, "unable to get current sw_params");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1365)
[declaration] snd_error(PCM, "unable to get boundary");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1370)
[declaration] snd_error(PCM, "unable to set stop threshold");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1381)
[declaration] snd_error(PCM, "unable to tstamp mode MMAP");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1388)
[function] if (ret < 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1389)
[declaration] snd_error(PCM, "unable to set tstamp type");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1400)
[declaration] snd_error(PCM, "unable to set silence threshold");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1405)
[declaration] snd_error(PCM, "unable to set silence threshold (please upgrade to
0.9.0rc8+ driver)");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1413)
[declaration] snd_error(PCM, "unable to install sw params (please upgrade to
0.9.0rc8+ driver)");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1426)
[declaration] snd_error(PCM, "unable to start PCM stream");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1431)
[declaration] snd_error(PCM, "unable to use hardware pcm with fd more than one!!!");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1454)
[declaration] ioctl(spcm->poll_fd, SNDRV_PCM_IOCTL_PVERSION, &ver);

```

```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1479)
[declaration] snd_error(PCM, "unable to info for slave pcm");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1486)
[declaration] snd_pcm_info_get_subdevice(&info) * 2 + capture);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1493)
[function] if (ret < 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1494)
[declaration] snd_error(PCM, "unable to open timer '%s'", name);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1500)
[declaration] snd_error(PCM, "unable to use timer '%s' with more than one fd!",
name);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1516)
[declaration] ioctl(dmix->poll_fd, SNDRV_TIMER_IOCTL_PVERSION, &ver);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1527)
[function] if (ver < SNDRV_PROTOCOL_VERSION(2, 0, 5)) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1601)
[declaration] snd_error(PCM, "unable to open hardware");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1619)
[declaration] snd_error(PCM, "unable to mmap channels");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1646)
[declaration] snd_error(PCM, "unable to mmap channels");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1660)
[declaration] snd_timer_params_set_early_event(&params, 1);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1665)
[declaration] INTERNAL(snd_timer_params_set_filter)(&params, filter);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1669)
[declaration] snd_error(PCM, "unable to set timer parameters");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1732)
[declaration] snd_error(PCM, "invalid type for bindings");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1742)
[function] if (err < 0 || cchannel < 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1743)
[declaration] snd_error(PCM, "invalid client channel in binding: %s", id);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1752)
[declaration] snd_error(PCM, "client channel out of range");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1767)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1768)
[declaration] free(bindings);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1771)
[function] if (snd_config_get_integer(n, &schannel) < 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1772)
[declaration] snd_error(PCM, "unable to get slave channel (should be integer type)
in binding: %s", id);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1773)
[declaration] free(bindings);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1776)
[function] if (schannel < 0 || schannel >= params->channels) {

```

```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1780)
[declaration] free(bindings);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1792)
[function] if (bindings[chn] == dmix->bindings[chn1]) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1793)
[declaration] snd_error(PCM, "unable to route channels %d,%d to same destination
%d", chn, chn1, bindings[chn]);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1794)
[declaration] free(bindings);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1821)
[function] if (hop > SND_CONF_MAX_HOPS) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1822)
[declaration] snd_error(PCM, "Too many definition levels (looped?)");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1826)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1827)
[declaration] snd_error(PCM, "Unknown slave PCM %s", str);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1833)
[declaration] snd_config_delete(pcm_conf);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1840)
[declaration] snd_output_stdio_attach(&out, stderr, 0);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1841)
[declaration] snd_config_save(sconf, out);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1842)
[declaration] snd_output_close(out);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1847)
[function] if (snd_config_search(pcm_conf, "pcm", &pcm_conf) >= 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1861)
[declaration] snd_config_delete(pcm_conf);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1864)
[declaration] snd_config_delete(pcm_conf);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1874)
[function] if (strcmp(id, "type") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1876)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1877)
[declaration] snd_error(PCM, "Invalid value for PCM type definition");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1880)
[function] if (strcmp(str, "hw")) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1881)
[declaration] snd_error(PCM, "Invalid type '%s' for slave PCM", str);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1886)
[function] if (strcmp(id, "card") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1893)
[function] if (strcmp(id, "device") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1895)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1896)
[declaration] snd_error(PCM, "Invalid type for %s", id);

```

```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1901)
[function] if (strcmp(id, "subdevice") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1903)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1904)
[declaration] snd_error(PCM, "Invalid type for %s", id);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1929)
[declaration] return _snd_pcm_direct_get_slave_ipc_offset(root, sconf, direction,
0);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1971)
[function] if (strcmp(id, "ipc_key") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1974)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1975)
[declaration] snd_error(PCM, "The field ipc_key must be an integer type");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1982)
[function] if (strcmp(id, "ipc_perm") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1985)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1986)
[declaration] snd_error(PCM, "Invalid type for %s", id);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1989)
[function] if ((perm & ~0777) != 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1990)
[declaration] snd_error(PCM, "The field ipc_perm must be a valid file permission");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1996)
[function] if (strcmp(id, "hw_ptr_alignment") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 1999)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2000)
[declaration] snd_error(PCM, "Invalid type for %s", id);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2012)
[declaration] snd_error(PCM, "The field hw_ptr_alignment is invalid : %s", str);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2018)
[function] if (strcmp(id, "tstamp_type") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2021)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2022)
[declaration] snd_error(PCM, "Invalid type for %s", id);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2034)
[declaration] snd_error(PCM, "The field tstamp_type is invalid : %s", str);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2039)
[function] if (strcmp(id, "ipc_gid") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2043)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2044)
[declaration] snd_error(PCM, "The field ipc_gid must be a valid group");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2047)
[function] if (! *group) {

```

```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2049)
[declaration] free(group);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2052)
[function] if (isdigit(*group) == 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2060)
[function] if (st != 0 || !pgrp) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2061)
[declaration] snd_error(PCM, "The field ipc_gid must be a valid group (create group
%s)", group);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2062)
[declaration] free(buffer);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2066)
[declaration] free(buffer);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2070)
[declaration] free(group);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2073)
[function] if (strcmp(id, "ipc_key_add_uid") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2074)
[function] if ((err = snd_config_get_bool(n)) < 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2075)
[declaration] snd_error(PCM, "The field ipc_key_add_uid must be a boolean type");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2081)
[function] if (strcmp(id, "slave") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2085)
[function] if (strcmp(id, "bindings") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2089)
[function] if (strcmp(id, "slowptr") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2096)
[function] if (strcmp(id, "max_periods") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2104)
[function] if (strcmp(id, "var_periodsize") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2111)
[function] if (strcmp(id, "direct_memory_access") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2118)
[declaration] snd_error(PCM, "Unknown field %s", id);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2122)
[declaration] snd_error(PCM, "slave is not defined");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2126)
[declaration] snd_error(PCM, "Unique IPC key is not defined");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2171)
[declaration] free(dmix);
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2190)
[function] if (ret < 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2191)
[declaration] snd_error(PCM, "unable to create IPC semaphore");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2195)
[function] if (ret < 0) {
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2196)
[declaration] snd_pcm_direct_semaphore_discard(dmix);

```

```

dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2206)
[declaration] snd_error(PCM, "unable to create IPC shm instance");
dirtycode/alsa-lib/src/pcm/pcm_direct.c (line 2207)
[declaration] snd_pcm_direct_semaphore_up(dmixon, DIRECT_IPC_SEM_CLIENT);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 63)
[declaration] static int shm_sum_discard(snd_pcm_direct_t *dmixon);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 76)
[declaration] sizeof(signed int);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 93)
[declaration] shm_sum_discard(dmixon);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 98)
[declaration] shmctl(dmixon->u.dmix.shmid_sum, IPC_SET, &buf);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 103)
[declaration] shm_sum_discard(dmixon);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 204)
[declaration] sizeof(signed int));
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 272)
[declaration] sizeof(signed int));
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 299)
[declaration] snd_pcm_direct_semaphore_down(dmixon, DIRECT_IPC_SEM_CLIENT);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 305)
[declaration] snd_pcm_direct_semaphore_up(dmixon, DIRECT_IPC_SEM_CLIENT);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 374)
[declaration] mix_areas(dmixon, src_areas, dst_areas, appl_ptr, slave_appl_ptr,
transfer);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 412)
[declaration] snd_timer_stop(dmixon->timer);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 413)
[declaration] gettimeofday(&dmixon->trigger_tstamp, pcm->tstamp_type);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 414)
[function] if (dmixon->state == SND_PCM_STATE_RUNNING) {
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 420)
[declaration] snd_pcm_direct_clear_timer_queue(dmixon);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 432)
[declaration] snd_pcm_hwsync(dmixon->spcm);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 438)
[declaration] return snd_pcm_dmix_sync_ptr0(pcm, slave_hw_ptr);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 465)
[declaration] snd_pcm_dmix_sync_ptr0(pcm, status->hw_ptr);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 516)
[declaration] return snd_pcm_dmix_sync_ptr(pcm);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 568)
[declaration] snd_pcm_dmix_sync_area(pcm);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 602)
[declaration] snd_pcm_dmix_start(pcm);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 604)
[declaration] snd_pcm_dmix_drop(pcm);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 610)
[declaration] snd_pcm_dmix_drop(pcm);

```

```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 620)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 621)
[declaration] snd_pcm_dmix_drop(pcm);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 624)
[function] if (dmix->state == SND_PCM_STATE_DRAINING) {
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 625)
[declaration] snd_pcm_dmix_sync_area(pcm);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 626)
[function] if ((pcm->mode & SND_PCM_NONBLOCK) == 0) {
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 627)
[declaration] snd_pcm_wait_nocheck(pcm, SND_PCM_WAIT_DRAIN);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 628)
[declaration] snd_pcm_direct_clear_timer_queue(dmix); /* force poll to wait */
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 631)
[function] switch (snd_pcm_state(dmix->spcm)) {
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 639)
[function] if (pcm->mode & SND_PCM_NONBLOCK) {
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 640)
[function] if (dmix->state == SND_PCM_STATE_DRAINING) {
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 668)
[declaration] return snd_pcm_mmap_playback_hw_rewindable(pcm);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 733)
[declaration] remix_areas(dmix, src_areas, dst_areas, appl_ptr, slave_appl_ptr,
transfer);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 755)
[declaration] return snd_pcm_mmap_avail(pcm);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 784)
[declaration] snd_timer_close(dmix->timer);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 788)
[declaration] snd_pcm_direct_server_discard(dmix);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 790)
[declaration] snd_pcm_direct_client_discard(dmix);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 794)
[declaration] snd_pcm_direct_semaphore_final(dmix, DIRECT_IPC_SEM_CLIENT);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 796)
[declaration] snd_pcm_direct_semaphore_final(dmix, DIRECT_IPC_SEM_CLIENT);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 828)
[declaration] snd_pcm_dmix_sync_area(pcm);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 831)
[declaration] snd_pcm_direct_clear_timer_queue(dmix);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 849)
[declaration] return snd_pcm_mmap_playback_avail(pcm);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 863)
[declaration] snd_pcm_dmix_sync_ptr(pcm);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 878)
[declaration] snd_pcm_dmix_sync_area(pcm);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 879)
[declaration] return snd_pcm_direct_poll_revents(pcm, pfd, nfd, revents);

```

```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 889)
[declaration] snd_output_printf(out, "Its setup is:\n");
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 890)
[declaration] snd_pcm_dump_setup(pcm, out);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 893)
[declaration] snd_pcm_dump(dmix->spcm, out);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 973)
[declaration] snd_error(PCM, "The dmix plugin supports only playback stream");
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 999)
[function] if (ret < 0) {
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1000)
[declaration] snd_error(PCM, "unable to open slave");
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1004)
[function] if (snd_pcm_type(spcm) != SND_PCM_TYPE_HW) {
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1005)
[declaration] snd_error(PCM, "dmix plugin can be only connected to hw plugin");
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1011)
[function] if (ret < 0) {
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1012)
[declaration] snd_error(PCM, "unable to initialize slave");
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1018)
[function] if (dmix->shmptr->use_server) {
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1022)
[function] if (ret < 0) {
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1023)
[declaration] snd_error(PCM, "unable to create server");
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1030)
[function] if (dmix->shmptr->use_server) {
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1032)
[declaration] snd_pcm_direct_semaphore_up(dmix, DIRECT_IPC_SEM_CLIENT);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1034)
[function] if (ret < 0) {
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1035)
[declaration] snd_error(PCM, "unable to connect client");
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1039)
[declaration] snd_pcm_direct_semaphore_down(dmix, DIRECT_IPC_SEM_CLIENT);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1049)
[function] if (ret < 0) {
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1053)
[function] if (ret == -EBADF) {
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1057)
[declaration] snd_error(PCM, "unable to open slave");
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1060)
[function] if (snd_pcm_type(spcm) != SND_PCM_TYPE_HW) {
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1061)
[declaration] snd_error(PCM, "dmix plugin can be only connected to hw plugin");
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1067)
[function] if (ret < 0) {

```

```

dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1068)
[declaration] snd_error(PCM, "unable to initialize slave");
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1078)
[declaration] snd_error(PCM, "unable to initialize sum ring buffer");
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1084)
[declaration] snd_error(PCM, "unable to initialize poll_fd");
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1107)
[declaration] snd_timer_close(dmix->timer);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1109)
[declaration] snd_pcm_direct_server_discard(dmix);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1111)
[declaration] snd_pcm_direct_client_discard(dmix);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1113)
[declaration] snd_pcm_close(spcm);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1115)
[declaration] shm_sum_discard(dmix);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1118)
[declaration] snd_pcm_direct_semaphore_final(dmix, DIRECT_IPC_SEM_CLIENT);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1120)
[declaration] snd_pcm_direct_semaphore_up(dmix, DIRECT_IPC_SEM_CLIENT);
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1326)
[function] else if (!(dmix_supported_format & (1ULL << params.format))) {
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1328)
[declaration] snd_error(PCM, "Unsupported format");
dirtycode/alsa-lib/src/pcm/pcm_dmix.c (line 1329)
[declaration] snd_config_delete(sconf);
dirtycode/alsa-lib/src/pcm/pcm_dmix_generic.c (line 62)
[declaration] ARCH_ADD(sum, sample);
dirtycode/alsa-lib/src/pcm/pcm_dmix_generic.c (line 93)
[declaration] ARCH_ADD(sum, sample);
dirtycode/alsa-lib/src/pcm/pcm_dmix_generic.c (line 141)
[function] if (! *dst) {
dirtycode/alsa-lib/src/pcm/pcm_dmix_generic.c (line 173)
[function] if (! *dst) {
dirtycode/alsa-lib/src/pcm/pcm_dmix_generic.c (line 204)
[function] if (! *dst) {
dirtycode/alsa-lib/src/pcm/pcm_dmix_generic.c (line 238)
[function] if (! *dst) {
dirtycode/alsa-lib/src/pcm/pcm_dmix_generic.c (line 271)
[function] if (! *dst) {
dirtycode/alsa-lib/src/pcm/pcm_dmix_generic.c (line 303)
[function] if (! *dst) {
dirtycode/alsa-lib/src/pcm/pcm_dmix_generic.c (line 334)
[function] if (! *dst) {
dirtycode/alsa-lib/src/pcm/pcm_dmix_generic.c (line 368)
[function] if (! *dst) {
dirtycode/alsa-lib/src/pcm/pcm_dmix_generic.c (line 402)
[function] if (!(dst[0] | dst[1] | dst[2])) {

```

```

dirtycode/alsa-lib/src/pcm/pcm_dmix_generic.c (line 435)
[function] if (!(dst[0] | dst[1] | dst[2])) {
dirtycode/alsa-lib/src/pcm/pcm_dmix_generic.c (line 466)
[function] if (*dst == 0x80) {
dirtycode/alsa-lib/src/pcm/pcm_dmix_generic.c (line 495)
[function] if (*dst == 0x80) {
dirtycode/alsa-lib/src/pcm/pcm_dmix_i386.c (line 91)
[declaration] generic_mix_select_callbacks(dmix);
dirtycode/alsa-lib/src/pcm/pcm_dmix_i386.c (line 96)
[declaration] generic_mix_select_callbacks(dmix);
dirtycode/alsa-lib/src/pcm/pcm_dmix_i386.c (line 106)
[function] if (in) {
dirtycode/alsa-lib/src/pcm/pcm_dmix_i386.c (line 107)
[function] while (!feof(in) && (fgets(line, sizeof(line), in) != NULL)) {
dirtycode/alsa-lib/src/pcm/pcm_dmix_i386.c (line 110)
[function] else if (!strncmp(line, "flags", 5)) {
dirtycode/alsa-lib/src/pcm/pcm_dmix_i386.c (line 117)
[declaration] fclose(in);
dirtycode/alsa-lib/src/pcm/pcm_dmix_x86_64.c (line 75)
[declaration] generic_mix_select_callbacks(dmix);
dirtycode/alsa-lib/src/pcm/pcm_dmix_x86_64.c (line 80)
[declaration] generic_mix_select_callbacks(dmix);
dirtycode/alsa-lib/src/pcm/pcm_dmix_x86_64.c (line 85)
[declaration] generic_mix_select_callbacks(dmix);
dirtycode/alsa-lib/src/pcm/pcm_dmix_x86_64.c (line 95)
[function] if (in) {
dirtycode/alsa-lib/src/pcm/pcm_dmix_x86_64.c (line 96)
[function] while (!feof(in) && (fgets(line, sizeof(line), in) != NULL)) {
dirtycode/alsa-lib/src/pcm/pcm_dmix_x86_64.c (line 100)
[declaration] fclose(in);
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 153)
[declaration] share_areas(dshare, src_areas, dst_areas, appl_ptr, slave_appl_ptr,
transfer);
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 191)
[declaration] snd_timer_stop(dshare->timer);
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 192)
[declaration] do_silence(pcm);
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 193)
[declaration] gettimeofday(&dshare->trigger_tstamp, pcm->tstamp_type);
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 194)
[function] if (dshare->state == SND_PCM_STATE_RUNNING) {
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 200)
[declaration] snd_pcm_direct_clear_timer_queue(dshare);
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 212)
[declaration] snd_pcm_hwsync(dshare->spcm);
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 218)
[declaration] return snd_pcm_dshare_sync_ptr0(pcm, slave_hw_ptr);
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 225)
[declaration] static snd_pcm_state_t snd_pcm_dshare_state(snd_pcm_t *pcm);

```

```

dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 237)
[declaration] snd_pcm_dshare_sync_ptr0(pcm, status->hw_ptr);
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 296)
[declaration] return snd_pcm_dshare_sync_ptr(pcm);
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 348)
[declaration] snd_pcm_dshare_sync_area(pcm);
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 385)
[declaration] snd_pcm_dshare_start(pcm);
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 387)
[declaration] snd_pcm_dshare_drop(pcm);
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 393)
[declaration] snd_pcm_dshare_drop(pcm);
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 403)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 404)
[declaration] snd_pcm_dshare_drop(pcm);
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 407)
[function] if (dshare->state == SND_PCM_STATE_DRAINING) {
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 408)
[declaration] snd_pcm_dshare_sync_area(pcm);
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 409)
[declaration] snd_pcm_wait_nocheck(pcm, SND_PCM_WAIT_DRAIN);
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 410)
[declaration] snd_pcm_direct_clear_timer_queue(dshare); /* force poll to wait */
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 412)
[function] switch (snd_pcm_state(dshare->spcm)) {
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 441)
[declaration] return snd_pcm_mmap_playback_hw_rewindable(pcm);
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 457)
[declaration] return snd_pcm_mmap_playback_avail(pcm);
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 486)
[declaration] snd_timer_close(dshare->timer);
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 488)
[declaration] do_silence(pcm);
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 493)
[declaration] snd_pcm_direct_server_discard(dshare);
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 495)
[declaration] snd_pcm_direct_client_discard(dshare);
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 498)
[declaration] snd_pcm_direct_semaphore_final(dshare, DIRECT_IPC_SEM_CLIENT);
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 500)
[declaration] snd_pcm_direct_semaphore_final(dshare, DIRECT_IPC_SEM_CLIENT);
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 533)
[declaration] snd_pcm_dshare_sync_area(pcm);
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 536)
[declaration] snd_pcm_direct_clear_timer_queue(dshare);
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 554)
[declaration] return snd_pcm_mmap_playback_avail(pcm);

```

```

dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 568)
[declaration] snd_pcm_dshare_sync_ptr(pcm);
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 585)
[declaration] snd_output_printf(out, "Its setup is:\n");
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 586)
[declaration] snd_pcm_dump_setup(pcm, out);
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 589)
[declaration] snd_pcm_dump(dshare->spcm, out);
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 675)
[declaration] snd_error(PCM, "The dshare plugin supports only playback stream");
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 705)
[function] if (ret < 0) {
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 706)
[declaration] snd_error(PCM, "unable to open slave");
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 710)
[function] if (snd_pcm_type(spcm) != SND_PCM_TYPE_HW) {
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 711)
[declaration] snd_error(PCM, "dshare plugin can be only connected to hw plugin");
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 716)
[function] if (ret < 0) {
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 717)
[declaration] snd_error(PCM, "unable to initialize slave");
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 723)
[function] if (dshare->shmptr->use_server) {
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 725)
[function] if (ret < 0) {
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 726)
[declaration] snd_error(PCM, "unable to create server");
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 733)
[function] if (dshare->shmptr->use_server) {
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 735)
[declaration] snd_pcm_direct_semaphore_up(dshare, DIRECT_IPC_SEM_CLIENT);
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 737)
[function] if (ret < 0) {
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 738)
[declaration] snd_error(PCM, "unable to connect client");
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 742)
[declaration] snd_pcm_direct_semaphore_down(dshare, DIRECT_IPC_SEM_CLIENT);
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 753)
[function] if (ret < 0) {
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 757)
[function] if (ret == -EBADFD) {
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 761)
[declaration] snd_error(PCM, "unable to open slave");
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 764)
[function] if (snd_pcm_type(spcm) != SND_PCM_TYPE_HW) {
dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 765)
[declaration] snd_error(PCM, "dshare plugin can be only connected to hw plugin");

```

dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 771)

[function] if (ret < 0) {

dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 772)

[declaration] snd_error(PCM, "unable to initialize slave");

dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 786)

[declaration] snd_error(PCM, "destination channel specified in bindings is already used");

dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 795)

[declaration] snd_error(PCM, "unable to initialize poll_fd");

dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 815)

[declaration] snd_timer_close(dshare->timer);

dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 817)

[declaration] snd_pcm_direct_server_discard(dshare);

dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 819)

[declaration] snd_pcm_direct_client_discard(dshare);

dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 821)

[declaration] snd_pcm_close(spcm);

dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 824)

[declaration] snd_pcm_direct_semaphore_final(dshare, DIRECT_IPC_SEM_CLIENT);

dirtycode/alsa-lib/src/pcm/pcm_dshare.c (line 826)

[declaration] snd_pcm_direct_semaphore_up(dshare, DIRECT_IPC_SEM_CLIENT);

dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 100)

[declaration] snd_pcm_area_copy(&dst_areas[chn], dst_ofs, &src_areas[schn], src_ofs, size, format);

dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 125)

[declaration] snoop_areas(dsnoop, src_areas, dst_areas, slave_hw_ptr, hw_ptr, transfer);

dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 144)

[declaration] snd_pcm_hwsync(dsnoop->spcm);

dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 161)

[declaration] gettimeofday(&dsnoop->trigger_tstamp, pcm->tstamp_type);

dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 183)

[declaration] snd_pcm_dsnoop_sync_ptr(pcm);

dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 242)

[declaration] return snd_pcm_dsnoop_sync_ptr(pcm);

dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 310)

[declaration] __snd_pcm_wait_in_lock(pcm, SND_PCM_WAIT_DRAIN);

dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 313)

[declaration] return snd_pcm_dsnoop_drop(pcm);

dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 333)

[declaration] return snd_pcm_mmap_capture_hw_rewindable(pcm);

dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 349)

[declaration] return snd_pcm_mmap_capture_avail(pcm);

dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 378)

[declaration] snd_timer_close(dsnoop->timer);

dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 382)

[declaration] snd_pcm_direct_server_discard(dsnoop);

dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 384)

[declaration] snd_pcm_direct_client_discard(dsnoop);

```

dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 387)
[declaration] snd_pcm_direct_semaphore_final(dsnoop, DIRECT_IPC_SEM_CLIENT);
dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 389)
[declaration] snd_pcm_direct_semaphore_final(dsnoop, DIRECT_IPC_SEM_CLIENT);
dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 414)
[declaration] snd_pcm_direct_clear_timer_queue(dsnoop);
dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 431)
[declaration] return snd_pcm_mmap_capture_avail(pcm);
dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 445)
[declaration] snd_pcm_dsnoop_sync_ptr(pcm);
dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 462)
[declaration] snd_output_printf(out, "Its setup is:\n");
dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 463)
[declaration] snd_pcm_dump_setup(pcm, out);
dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 466)
[declaration] snd_pcm_dump(dsnoop->spcm, out);
dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 546)
[declaration] snd_error(PCM, "The dsnoop plugin supports only capture stream");
dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 571)
[function] if (ret < 0) {
dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 572)
[declaration] snd_error(PCM, "unable to open slave");
dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 576)
[function] if (snd_pcm_type(spcm) != SND_PCM_TYPE_HW) {
dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 577)
[declaration] snd_error(PCM, "dsnoop plugin can be only connected to hw plugin");
dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 582)
[function] if (ret < 0) {
dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 583)
[declaration] snd_error(PCM, "unable to initialize slave");
dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 589)
[function] if (dsnoop->shmptr->use_server) {
dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 591)
[function] if (ret < 0) {
dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 592)
[declaration] snd_error(PCM, "unable to create server");
dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 599)
[function] if (dsnoop->shmptr->use_server) {
dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 601)
[declaration] snd_pcm_direct_semaphore_up(dsnoop, DIRECT_IPC_SEM_CLIENT);
dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 603)
[function] if (ret < 0) {
dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 604)
[declaration] snd_error(PCM, "unable to connect client");
dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 608)
[declaration] snd_pcm_direct_semaphore_down(dsnoop, DIRECT_IPC_SEM_CLIENT);
dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 619)
[function] if (ret < 0) {

```

```

dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 623)
[function] if (ret == -EBADF) {
dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 627)
[declaration] snd_error(PCM, "unable to open slave");
dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 630)
[function] if (snd_pcm_type(spcm) != SND_PCM_TYPE_HW) {
dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 631)
[declaration] snd_error(PCM, "dsnoop plugin can be only connected to hw plugin");
dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 637)
[function] if (ret < 0) {
dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 638)
[declaration] snd_error(PCM, "unable to initialize slave");
dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 648)
[declaration] snd_error(PCM, "unable to initialize poll_fd");
dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 669)
[declaration] snd_timer_close(dsnoop->timer);
dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 671)
[declaration] snd_pcm_direct_server_discard(dsnoop);
dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 673)
[declaration] snd_pcm_direct_client_discard(dsnoop);
dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 675)
[declaration] snd_pcm_close(spcm);
dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 678)
[declaration] snd_pcm_direct_semaphore_final(dsnoop, DIRECT_IPC_SEM_CLIENT);
dirtycode/alsa-lib/src/pcm/pcm_dsnoop.c (line 680)
[declaration] snd_pcm_direct_semaphore_up(dsnoop, DIRECT_IPC_SEM_CLIENT);
dirtycode/alsa-lib/src/pcm/pcm_empty.c (line 94)
[function] if (strcmp(id, "slave") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_empty.c (line 98)
[declaration] snd_error(PCM, "Unknown field %s", id);
dirtycode/alsa-lib/src/pcm/pcm_empty.c (line 102)
[declaration] snd_error(PCM, "slave is not defined");
dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 121)
[function] if (ival->min <= list[imin]) {
dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 133)
[function] if (ival->max >= list[imax]) {
dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 155)
[declaration] return snd_interval_list(ival, parm->num_list, parm->list);
dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 158)
[declaration] memset(&t, 0, sizeof(t));
dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 159)
[declaration] snd_interval_set_minmax(&t, parm->min, parm->max);
dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 161)
[declaration] return snd_interval_refine(ival, &t);
dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 180)
[declaration] return snd_mask_refine(mask, &bits);
dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 265)
[declaration] return _snd_pcm_hw_params_refine(sparams, links, params);

```

```

dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 275)
[declaration] return _snd_pcm_hw_params_refine(params, links, sparams);
dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 397)
[declaration] snd_output_printf(out, "%s\n", ext->data->name);
dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 399)
[declaration] snd_output_printf(out, "External PCM Plugin\n");
dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 400)
[function] if (pcm->setup) {
dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 401)
[declaration] snd_output_printf(out, "Its setup is:\n");
dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 402)
[declaration] snd_pcm_dump_setup(pcm, out);
dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 413)
[declaration] snd_ext_parm_clear(&ext->params[i]);
dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 414)
[declaration] snd_ext_parm_clear(&ext->sparams[i]);
dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 437)
[declaration] return snd_pcm_generic_query_chmaps(pcm);
dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 447)
[declaration] return snd_pcm_generic_get_chmap(pcm);
dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 457)
[declaration] return snd_pcm_generic_set_chmap(pcm, map);
dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 542)
[function] if (strcmp(id, "slave") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 546)
[function] if (strcmp(id, "my_own_parameter") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 550)
[declaration] snd_error(PCM, "Unknown field %s", id);
dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 555)
[declaration] snd_error(PCM, "No slave defined for myplug");
dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 571)
[declaration] myplug_free(myplug);
dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 726)
[declaration] free(ext);
dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 753)
[declaration] return snd_pcm_close(extplug->pcm);
dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 785)
[declaration] snd_error(PCM, "EXTPLUG: invalid parameter type %d", type);
dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 788)
[declaration] return snd_ext_parm_set_list(&ext->sparams[type], num_list, list);
dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 807)
[declaration] snd_error(PCM, "EXTPLUG: invalid parameter type %d", type);
dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 811)
[declaration] snd_error(PCM, "EXTPLUG: invalid parameter type %d", type);
dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 814)
[declaration] return snd_ext_parm_set_minmax(&ext->sparams[type], min, max);
dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 833)
[declaration] snd_error(PCM, "EXTPLUG: invalid parameter type %d", type);

```

dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 836)

```
[declaration] return snd_ext_parm_set_list(&ext->params[type], num_list, list);
```

dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 855)

```
[declaration] snd_error(PCM, "EXTPLUG: invalid parameter type %d", type);
```

dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 859)

```
[declaration] snd_error(PCM, "EXTPLUG: invalid parameter type %d", type);
```

dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 862)

```
[declaration] return snd_ext_parm_set_minmax(&ext->params[type], min, max);
```

dirtycode/alsa-lib/src/pcm/pcm_extplug.c (line 881)

```
[declaration] snd_error(PCM, "EXTPLUG: invalid parameter type %d", type);
```

dirtycode/alsa-lib/src/pcm/pcm_file.c (line 107)

```
[function] if (r < 0) {
```

dirtycode/alsa-lib/src/pcm/pcm_file.c (line 171)

```
[function] switch (*(++old_index_ch)) {
```

dirtycode/alsa-lib/src/pcm/pcm_file.c (line 202)

```
[declaration] snd_pcm_format_name(pcm->format));
```

dirtycode/alsa-lib/src/pcm/pcm_file.c (line 241)

```
[function] if (!pipe) {
```

dirtycode/alsa-lib/src/pcm/pcm_file.c (line 257)

```
[function] if (fd < 0) {
```

dirtycode/alsa-lib/src/pcm/pcm_file.c (line 271)

```
[function] if (fd >= 0) {
```

dirtycode/alsa-lib/src/pcm/pcm_file.c (line 272)

```
[declaration] free(file->final_fname);
```

dirtycode/alsa-lib/src/pcm/pcm_file.c (line 277)

```
[function] if (fd < 0) {
```

dirtycode/alsa-lib/src/pcm/pcm_file.c (line 281)

```
[declaration] free(tmpfname);
```

dirtycode/alsa-lib/src/pcm/pcm_file.c (line 308)

```
[declaration] snd_erronum(PCM, "requested more frames than pcm buffer");
```

dirtycode/alsa-lib/src/pcm/pcm_file.c (line 317)

```
[declaration] snd_erronum(PCM, "read from file failed, error: %d", bytes);
```

dirtycode/alsa-lib/src/pcm/pcm_file.c (line 381)

```
[declaration] snd_erronum(PCM, "%s write header failed, file data may be corrupt",  
file->fname);
```

dirtycode/alsa-lib/src/pcm/pcm_file.c (line 383)

```
[declaration] snd_error(PCM, "%s write header incomplete, file data may be corrupt",  
file->fname);
```

dirtycode/alsa-lib/src/pcm/pcm_file.c (line 429)

```
[function] if (err < 0) {
```

dirtycode/alsa-lib/src/pcm/pcm_file.c (line 442)

```
[function] if (err < 0) {
```

dirtycode/alsa-lib/src/pcm/pcm_file.c (line 445)

```
[declaration] snd_erronum(PCM, "%s write failed, file data may be corrupt",  
file->fname);
```

dirtycode/alsa-lib/src/pcm/pcm_file.c (line 492)

```
[function] if (file->wbuf_used_bytes > file->buffer_bytes) {
```

dirtycode/alsa-lib/src/pcm/pcm_file.c (line 497)

```
[declaration] assert(file->wbuf_used_bytes < file->wbuf_size_bytes);
```

```

dirtycode/alsa-lib/src/pcm/pcm_file.c (line 507)
[declaration] fixup_wav_header(pcm);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 508)
[declaration] free((void *)file->fname);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 509)
[function] if (file->pipe) {
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 510)
[declaration] pclose(file->pipe);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 512)
[declaration] close(file->fd);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 516)
[declaration] free((void *)file->ifname);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 517)
[declaration] close(file->ifd);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 519)
[declaration] return snd_pcm_generic_close(pcm);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 528)
[declaration] snd_pcm_file_write_bytes(pcm, file->wbuf_used_bytes);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 529)
[declaration] assert(file->wbuf_used_bytes == 0);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 540)
[declaration] snd_pcm_file_write_bytes(pcm, file->wbuf_used_bytes);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 541)
[declaration] assert(file->wbuf_used_bytes == 0);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 552)
[declaration] __snd_pcm_lock(pcm);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 553)
[declaration] snd_pcm_file_write_bytes(pcm, file->wbuf_used_bytes);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 554)
[declaration] assert(file->wbuf_used_bytes == 0);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 555)
[declaration] __snd_pcm_unlock(pcm);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 639)
[declaration] snd_pcm_areas_from_buf(pcm, areas, (void*) buffer);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 640)
[declaration] __snd_pcm_lock(pcm);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 641)
[function] if (snd_pcm_file_add_frames(pcm, areas, 0, n) < 0) {
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 642)
[declaration] __snd_pcm_unlock(pcm);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 645)
[declaration] __snd_pcm_unlock(pcm);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 657)
[declaration] snd_pcm_areas_from_bufs(pcm, areas, bufs);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 658)
[declaration] __snd_pcm_lock(pcm);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 659)
[function] if (snd_pcm_file_add_frames(pcm, areas, 0, n) < 0) {

```

```

dirtycode/alsa-lib/src/pcm/pcm_file.c (line 660)
[declaration] __snd_pcm_unlock(pcm);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 663)
[declaration] __snd_pcm_unlock(pcm);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 683)
[declaration] __snd_pcm_unlock(pcm);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 707)
[declaration] __snd_pcm_unlock(pcm);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 730)
[declaration] assert(ofs == offset && siz == size);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 732)
[function] if (result > 0) {
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 774)
[declaration] return snd_pcm_hw_free(file->gen.slave);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 797)
[declaration] snd_pcm_file_hw_free(pcm);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 802)
[declaration] snd_pcm_file_hw_free(pcm);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 811)
[declaration] snd_pcm_file_hw_free(pcm);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 823)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 824)
[declaration] snd_errormsg(PCM, "failed opening output file %s", file->fname);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 843)
[declaration] snd_output_printf(out, "File PCM (file=%s)\n", file->fname);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 845)
[declaration] snd_output_printf(out, "File PCM (fd=%d)\n", file->fd);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 851)
[declaration] snd_output_printf(out, "Its setup is:\n");
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 852)
[declaration] snd_pcm_dump_setup(pcm, out);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 947)
[declaration] snd_error(PCM, "file format %s is unknown", fmt);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 968)
[function] if (ifd < 0 || file->ifname == NULL) {
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 970)
[declaration] close(ifd);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 972)
[declaration] snd_errormsg(PCM, "open %s for reading failed", ifname);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 973)
[declaration] free(file->ifname);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 974)
[declaration] free(file->fname);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 975)
[declaration] free(file);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 988)
[declaration] close(file->ifd);

```

```

dirtycode/alsa-lib/src/pcm/pcm_file.c (line 989)
[declaration] free(file->fname);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 990)
[declaration] free(file->ifname);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 991)
[declaration] free(file);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 1092)
[function] if (strcmp(id, "slave") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 1096)
[function] if (strcmp(id, "format") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 1098)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 1099)
[declaration] snd_error(PCM, "Invalid type for %s", id);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 1104)
[function] if (strcmp(id, "file") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 1106)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 1108)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 1109)
[declaration] snd_error(PCM, "Invalid type for %s", id);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 1115)
[function] if (strcmp(id, "infile") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 1117)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 1119)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 1120)
[declaration] snd_error(PCM, "Invalid type for %s", id);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 1126)
[function] if (strcmp(id, "perm") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 1128)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 1129)
[declaration] snd_error(PCM, "Invalid type for %s", id);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 1132)
[function] if ((perm & ~0777) != 0) {
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 1133)
[declaration] snd_error(PCM, "The field perm must be a valid file permission");
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 1138)
[function] if (strcmp(id, "truncate") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 1145)
[declaration] snd_error(PCM, "Unknown field %s", id);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 1151)
[function] if (snd_config_search(root, "defaults.pcm.file_format", &n) >= 0) {
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 1153)
[function] if (err < 0) {

```

dirtycode/alsa-lib/src/pcm/pcm_file.c (line 1154)
[declaration] snd_error(PCM, "Invalid file format");
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 1160)
[declaration] snd_error(PCM, "slave is not defined");
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 1167)
[declaration] snd_config_delete(sconf);
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 1168)
[declaration] snd_error(PCM, "file is not defined");
dirtycode/alsa-lib/src/pcm/pcm_file.c (line 1178)
[declaration] snd_pcm_close(spcm);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 49)
[declaration] return snd_pcm_nonblock(generic->slave, nonblock);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 55)
[declaration] return snd_pcm_async(generic->slave, sig, pid);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 61)
[declaration] return snd_pcm_poll_descriptors_count(generic->slave);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 67)
[declaration] return snd_pcm_poll_descriptors(generic->slave, pfds, space);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 73)
[declaration] return snd_pcm_poll_descriptors_revents(generic->slave, pfds, nfds, revents);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 79)
[declaration] return snd_pcm_info(generic->slave, info);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 85)
[declaration] return snd_pcm_hw_free(generic->slave);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 91)
[declaration] return snd_pcm_sw_params(generic->slave, params);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 97)
[declaration] return snd_pcm_hw_refine(generic->slave, params);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 103)
[declaration] return _snd_pcm_hw_params_internal(generic->slave, params);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 109)
[declaration] return snd_pcm_prepare(generic->slave);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 120)
[declaration] return snd_pcm_channel_info(generic->slave, info);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 123)
[declaration] return snd_pcm_channel_info_shm(pcm, info, -1);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 130)
[declaration] return snd_pcm_status(generic->slave, status);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 136)
[declaration] return snd_pcm_state(generic->slave);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 142)
[declaration] return snd_pcm_hwsync(generic->slave);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 148)
[declaration] return snd_pcm_reset(generic->slave);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 154)
[declaration] return snd_pcm_start(generic->slave);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 160)
[declaration] return snd_pcm_drop(generic->slave);

```

dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 166)
[declaration] return snd_pcm_drain(generic->slave);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 172)
[declaration] return snd_pcm_pause(generic->slave, enable);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 178)
[declaration] return snd_pcm_resume(generic->slave);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 184)
[declaration] return snd_pcm_delay(generic->slave, delayp);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 190)
[declaration] return snd_pcm_forwardable(generic->slave);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 196)
[declaration] return INTERNAL(snd_pcm_forward)(generic->slave, frames);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 202)
[declaration] return snd_pcm_rewindable(generic->slave);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 208)
[declaration] return snd_pcm_rewind(generic->slave, frames);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 238)
[declaration] return _snd_pcm_writei(generic->slave, buffer, size);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 244)
[declaration] return _snd_pcm_writen(generic->slave, bufs, size);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 250)
[declaration] return _snd_pcm_readi(generic->slave, buffer, size);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 256)
[declaration] return _snd_pcm_readn(generic->slave, bufs, size);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 264)
[declaration] return snd_pcm_mmap_commit(generic->slave, offset, size);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 270)
[declaration] return snd_pcm_avail_update(generic->slave);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 277)
[declaration] return snd_pcm_htimestamp(generic->slave, avail, tstamp);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 296)
[declaration] gettimestamp(tstamp, pcm->tstamp_type);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 328)
[declaration] return snd_pcm_query_chmaps(generic->slave);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 334)
[declaration] return snd_pcm_get_chmap(generic->slave);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 340)
[declaration] return snd_pcm_set_chmap(generic->slave, map);
dirtycode/alsa-lib/src/pcm/pcm_generic.c (line 346)
[declaration] return snd_pcm_may_wait_for_avail_min(generic->slave,
snd_pcm_mmap_avail(generic->slave));
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 94)
[function] while (!list_empty(hooks)) {
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 98)
[declaration] snd_pcm_hook_remove(hook);
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 103)
[declaration] hook_remove_dlobj(list_entry(pos, struct snd_pcm_hook_dlist, list));
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 148)
[declaration] snd_output_printf(out, "Its setup is:\n");

```

```

dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 149)
[declaration] snd_pcm_dump_setup(pcm, out);
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 229)
[declaration] INIT_LIST_HEAD(&h->hooks[k]);
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 234)
[declaration] free(h);
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 353)
[declaration] snd_error(PCM, "Invalid hook definition");
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 363)
[function] if (strcmp(id, "type") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 367)
[function] if (strcmp(id, "hook_args") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 371)
[declaration] snd_error(PCM, "Unknown field %s", id);
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 375)
[declaration] snd_error(PCM, "type is not defined");
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 380)
[declaration] snd_error(PCM, "unable to get id");
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 385)
[declaration] snd_error(PCM, "Invalid type for %s", id);
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 390)
[function] if (snd_config_get_type(type) != SND_CONFIG_TYPE_COMPOUND) {
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 391)
[declaration] snd_error(PCM, "Invalid type for PCM type %s definition", str);
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 395)
[function] snd_config_for_each(i, next, type) {
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 402)
[function] if (strcmp(id, "lib") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 404)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 405)
[declaration] snd_error(PCM, "Invalid type for %s", id);
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 410)
[function] if (strcmp(id, "install") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 412)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 413)
[declaration] snd_error(PCM, "Invalid type for %s", id);
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 418)
[declaration] snd_error(PCM, "Unknown field %s", id);
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 425)
[declaration] snprintf(buf, sizeof(buf), "_snd_pcm_hook_%s_install", str);
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 439)
[declaration] snd_dlclose(h);
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 444)
[declaration] snd_config_delete(type);
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 451)
[declaration] snd_error(PCM, "unknown hook_args %s", str);

```

dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 454)
[declaration] snd_config_delete(args);
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 463)
[declaration] snd_dlclose(h);
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 498)
[function] if (strcmp(id, "slave") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 502)
[function] if (strcmp(id, "hooks") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 503)
[function] if (snd_config_get_type(n) != SND_CONFIG_TYPE_COMPOUND) {
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 504)
[declaration] snd_error(PCM, "Invalid type for %s", id);
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 510)
[declaration] snd_error(PCM, "Unknown field %s", id);
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 514)
[declaration] snd_error(PCM, "slave is not defined");
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 526)
[declaration] snd_pcm_close(spcm);
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 534)
[function] if (snd_config_get_string(n, &str) >= 0) {
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 536)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 537)
[declaration] snd_error(PCM, "unknown pcm_hook %s", str);
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 540)
[declaration] snd_config_delete(n);
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 544)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 545)
[declaration] snd_pcm_close(rpcm);
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 645)
[declaration] return snd_sctl_install(h);
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 651)
[declaration] return snd_sctl_remove(h);
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 686)
[declaration] snd_error(PCM, "No card for this PCM");
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 692)
[declaration] snd_error(PCM, "Cannot open CTL %s", ctl_name);
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 717)
[declaration] snd_pcm_hook_remove(h_hw_params);
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 719)
[declaration] snd_pcm_hook_remove(h_hw_free);
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 721)
[declaration] snd_pcm_hook_remove(h_close);
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 723)
[declaration] snd_sctl_free(sctl);
dirtycode/alsa-lib/src/pcm/pcm_hooks.c (line 725)
[declaration] snd_config_delete(pcm_conf);

```

dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 81)
[declaration] static int use_old_hw_params_ioctl(int fd, unsigned int cmd,
snd_pcm_hw_params_t *params);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 82)
[declaration] static snd_pcm_sframes_t snd_pcm_hw_avail_update(snd_pcm_t *pcm);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 148)
[declaration] snd_checknum(PCM, "SNDRV_PCM_IOCTL_SYNC_PTR failed (%i)", err);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 160)
[declaration] return sync_ptr1(hw, SNDRV_PCM_SYNC_PTR_APPL);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 169)
[declaration] return sync_ptr1(hw, SNDRV_PCM_SYNC_PTR_AVAIL_MIN);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 218)
[function] while (poll(&hw->period_timer_pfd, 1, 0) > 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 220)
[declaration] snd_timer_read(hw->period_timer, rbuf, sizeof(rbuf));
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 224)
[declaration] snd_timer_read(hw->period_timer, rbuf, sizeof(rbuf));
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 256)
[declaration] snd_pcm_hw_clear_timer_queue(hw);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 271)
[declaration] snd_checknum(PCM, "F_GETFL failed (%i)", err);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 280)
[declaration] snd_checknum(PCM, "F_SETFL for O_NONBLOCK failed (%i)", err);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 294)
[declaration] snd_checknum(PCM, "F_GETFL failed (%i)", err);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 303)
[declaration] snd_checknum(PCM, "F_SETFL for O_ASYNC failed (%i)", err);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 310)
[declaration] snd_checknum(PCM, "F_SETSIG failed (%i)", err);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 315)
[declaration] snd_checknum(PCM, "F_SETOWN failed (%i)", err);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 327)
[declaration] snd_checknum(PCM, "SNDRV_PCM_IOCTL_INFO failed (%i)", err);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 332)
[declaration] return __snd_pcm_info_eld_fixup(info);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 340)
[declaration] return ioctl(pcm_hw->fd, SNDRV_PCM_IOCTL_HW_REFINE, params);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 341)
[declaration] return use_old_hw_params_ioctl(pcm_hw->fd,
SND_PCM_IOCTL_HW_REFINE_OLD, params);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 398)
[function] if (snd_mask_single(m)) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 414)
[declaration] snd_checknum(PCM, "SNDRV_PCM_IOCTL_HW_PARAMS failed (%i)", err);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 422)
[declaration] return query_status_data(hw);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 428)
[declaration] snd_timer_close(hw->period_timer);

```

```

dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 447)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 456)
[function] if (snd_timer_poll_descriptors_count(hw->period_timer) != 1) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 457)
[declaration] snd_pcm_hw_close_timer(hw);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 473)
[declaration] snd_warn(PCM, "unable to get protocol version");
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 485)
[function] if (ver < SNDRV_PROTOCOL_VERSION(2, 0, 5)) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 490)
[declaration] snd_timer_params_set_auto_start(&params, 1);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 491)
[declaration] snd_timer_params_set_ticks(&params, 1);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 495)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 496)
[declaration] snd_pcm_hw_close_timer(hw);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 500)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 501)
[declaration] snd_pcm_hw_close_timer(hw);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 506)
[declaration] snd_pcm_hw_close_timer(hw);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 520)
[declaration] snd_checknum(PCM, "SNDRV_PCM_IOCTL_HW_FREE failed (%i)", err);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 546)
[declaration] snd_checknum(PCM, "Kernel doesn't support
SND_PCM_TSTAMP_TYPE_MONOTONIC_RAW");
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 552)
[declaration] snd_checknum(PCM, "Kernel doesn't support
SND_PCM_TSTAMP_TYPE_MONOTONIC");
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 558)
[declaration] snd_checknum(PCM, "SNDRV_PCM_IOCTL_SW_PARAMS failed (%i)", err);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 563)
[function] if (hw->version < SNDRV_PROTOCOL_VERSION(2, 0, 12)) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 566)
[function] if (ioctl(fd, SNDRV_PCM_IOCTL_TSTAMP, &on) < 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 568)
[declaration] snd_check(PCM, "TSTAMP failed");
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 594)
[declaration] snd_checknum(PCM, "SNDRV_PCM_IOCTL_CHANNEL_INFO failed (%i)", err);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 612)
[function] if (ioctl(fd, SNDRV_PCM_IOCTL_STATUS, status) < 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 614)
[declaration] snd_checknum(PCM, "SNDRV_PCM_IOCTL_STATUS failed (%i)", err);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 618)
[function] if (ioctl(fd, SNDRV_PCM_IOCTL_STATUS_EXT, status) < 0) {

```

```

dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 620)
[declaration] snd_checknum(PCM, "SNDRV_PCM_IOCTL_STATUS_EXT failed (%i)", err);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 647)
[declaration] snd_checknum(PCM, "SNDRV_PCM_IOCTL_DELAY failed (%i)", err);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 658)
[function] if (hw->mmap_status_fallbacked) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 663)
[function] if (ioctl(fd, SNDRV_PCM_IOCTL_HWSYNC) < 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 665)
[declaration] snd_checknum(PCM, "SNDRV_PCM_IOCTL_HWSYNC failed (%i)", err);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 672)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 673)
[function] switch (FAST_PCM_STATE(hw)) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 692)
[declaration] snd_pcm_sw_params_current_no_lock(pcm, &sw_params);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 693)
[function] if (ioctl(hw->fd, SNDRV_PCM_IOCTL_SW_PARAMS, &sw_params) < 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 695)
[declaration] snd_checknum(PCM, "SNDRV_PCM_IOCTL_SW_PARAMS failed (%i)", err);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 702)
[declaration] snd_checknum(PCM, "SNDRV_PCM_IOCTL_PREPARE failed (%i)", err);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 705)
[declaration] return query_status_and_control_data(hw);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 714)
[declaration] snd_checknum(PCM, "SNDRV_PCM_IOCTL_RESET failed (%i)", err);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 717)
[declaration] return query_status_and_control_data(hw);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 726)
[declaration] snd_pcm_mmap_playback_hw_avail(pcm) > 0);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 731)
[declaration] snd_checknum(PCM, "SNDRV_PCM_IOCTL_START failed (%i)", err);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 734)
[declaration] snd_error(PCM, "PCM state = %s",
snd_pcm_state_name(snd_pcm_hw_state(pcm)));
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 747)
[declaration] snd_checknum(PCM, "SNDRV_PCM_IOCTL_DROP failed (%i)", err);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 795)
[function] if (ioctl(hw->fd, SNDRV_PCM_IOCTL_SW_PARAMS, &sw_params) < 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 797)
[declaration] snd_checknum(PCM, "SNDRV_PCM_IOCTL_SW_PARAMS failed (%i)", err);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 805)
[declaration] snd_checknum(PCM, "SNDRV_PCM_IOCTL_DRAIN failed (%i)", err);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 817)
[declaration] snd_checknum(PCM, "SNDRV_PCM_IOCTL_PAUSE failed (%i)", err);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 825)
[declaration] return snd_pcm_mmap_hw_rewindable(pcm);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 834)
[declaration] snd_checknum(PCM, "SNDRV_PCM_IOCTL_REWIND failed (%i)", err);

```

```

dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 845)
[declaration] return snd_pcm_mmap_avail(pcm);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 853)
[function] if (ioctl(hw->fd, SNDRV_PCM_IOCTL_FORWARD, &frames) < 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 855)
[declaration] snd_checknum(PCM, "SNDRV_PCM_IOCTL_FORWARD failed (%i)", err);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 865)
[function] switch (FAST_PCM_STATE(hw)) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 881)
[declaration] snd_pcm_mmap_appl_forward(pcm, frames);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 892)
[declaration] snd_checknum(PCM, "SNDRV_PCM_IOCTL_RESUME failed (%i)", err);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 903)
[declaration] snd_checknum(PCM, "SNDRV_PCM_IOCTL_LINK failed (%i)", -errno);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 912)
[declaration] snd_checknum(PCM, "Invalid type for SNDRV_PCM_IOCTL_LINK (%i)",
master->type);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 915)
[declaration] return hw_link(master, pcm);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 925)
[declaration] return hw_link(pcm1, pcm2);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 933)
[declaration] snd_checknum(PCM, "SNDRV_PCM_IOCTL_UNLINK failed (%i)", -errno);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 956)
[declaration] return snd_pcm_check_error(pcm, err);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 977)
[declaration] return snd_pcm_check_error(pcm, err);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 998)
[declaration] return snd_pcm_check_error(pcm, err);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1019)
[declaration] return snd_pcm_check_error(pcm, err);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1087)
[declaration] map_status_data(hw, sync_ptr, force_fallback);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1089)
[declaration] map_control_data(hw, sync_ptr, force_fallback);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1095)
[declaration] free(sync_ptr);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1109)
[declaration] offsetof(struct snd_pcm_mmap_status, hw_ptr));
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1133)
[declaration] snd_checknum(PCM, "status munmap failed (%u)", errno);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1142)
[declaration] snd_checknum(PCM, "control munmap failed (%u)", errno);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1152)
[declaration] free(hw->sync_ptr);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1177)
[declaration] snd_checknum(PCM, "close failed (%i)", err);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1209)
[function] if (avail >= pcm->stop_threshold) {

```

```

dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1211)
[function] if (SNDRV_PROTOCOL_VERSION(2, 0, 1) <= hw->version) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1252)
[declaration] snd_ctl_elem_id_set_name(id, "Playback Channel Map");
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1254)
[declaration] snd_ctl_elem_id_set_name(id, "Capture Channel Map");
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1300)
[declaration] snd_checknum(PCM, "Cannot open the associated CTL");
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1308)
[declaration] snd_checknum(PCM, "Cannot read Channel Map TLV");
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1314)
[declaration] fprintf(stderr, "%02x: %08x\n", i, tlv[i]);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1321)
[function] if (!is_chmap_type(type)) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1322)
[declaration] snd_checknum(PCM, "Invalid TLV type %d", type);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1334)
[function] if (!is_chmap_type(p[0])) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1335)
[declaration] snd_checknum(PCM, "Invalid TLV type %d", p[0]);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1348)
[function] if (!map[i]) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1349)
[declaration] snd_pcm_free_chmaps(map);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1354)
[declaration] memcpy(map[i]->map.pos, start + 2, start[1]);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1387)
[declaration] return _snd_pcm_copy_chmap_query(hw->chmap_override);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1395)
[declaration] chmap_caps_set_ok(hw, CHMAP_CTL_QUERY);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1397)
[declaration] chmap_caps_set_error(hw, CHMAP_CTL_QUERY);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1412)
[declaration] return _snd_pcm_choose_fixed_chmap(pcm, hw->chmap_override);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1427)
[declaration] snd_pcm_state_name(FAST_PCM_STATE(hw)));
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1437)
[declaration] free(map);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1438)
[declaration] snd_checknum(PCM, "Cannot open the associated CTL");
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1439)
[declaration] chmap_caps_set_error(hw, CHMAP_CTL_GET);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1447)
[declaration] free(map);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1448)
[declaration] snd_checknum(PCM, "Cannot read Channel Map ctl");
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1449)
[declaration] chmap_caps_set_error(hw, CHMAP_CTL_GET);

```

```

dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1474)
[declaration] snd_checknum(PCM, "Invalid number of channels %d", map->channels);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1479)
[declaration] snd_pcm_state_name(FAST_PCM_STATE(hw)));
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1485)
[declaration] snd_checknum(PCM, "Cannot open the associated CTL");
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1486)
[declaration] chmap_caps_set_error(hw, CHMAP_CTL_SET);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1493)
[declaration] snd_ctl_elem_value_set_integer(&val, i, map->pos[i]);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1497)
[declaration] chmap_caps_set_ok(hw, CHMAP_CTL_SET);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1498)
[function] else if (ret == -ENOENT || ret == -EPERM || ret == -ENXIO) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1499)
[declaration] chmap_caps_set_error(hw, CHMAP_CTL_SET);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1503)
[declaration] snd_checknum(PCM, "Cannot write Channel Map ctl");
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1513)
[declaration] snd_error(PCM, "cannot get card name");
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1520)
[declaration] snd_output_printf(out, "Its setup is:\n");
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1521)
[declaration] snd_pcm_dump_setup(pcm, out);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1522)
[declaration] snd_output_printf(out, " appl_ptr : %li\n",
hw->mmap_control->appl_ptr);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1523)
[declaration] snd_output_printf(out, " hw_ptr : %li\n", hw->mmap_status->hw_ptr);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1634)
[declaration] snd_checknum(PCM, "SNDRV_PCM_IOCTL_INFO failed (%i)", ret);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1635)
[declaration] close(fd);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1642)
[declaration] close(fd);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1655)
[declaration] snd_checknum(PCM, "SNDRV_PCM_IOCTL_PVERSION failed (%i)", ret);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1656)
[declaration] close(fd);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1665)
[function] if (ioctl(fd, SNDRV_PCM_IOCTL_USER_PVERSION, &user_ver) < 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1667)
[declaration] snd_check(PCM, "USER_PVERSION failed");
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1675)
[function] if (clock_gettime(CLOCK_MONOTONIC, &timespec) == 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1676)
[function] if (!(mode & SND_PCM_APPEND)) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1678)
[function] if (ioctl(fd, SNDRV_PCM_IOCTL_TTSTAMP, &on) < 0) {

```

```

dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1680)
[declaration] snd_check(PCM, "TTSTAMP failed");
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1688)
[function] if (SNDRV_PROTOCOL_VERSION(2, 0, 5) <= ver && !(mode & SND_PCM_APPEND)) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1690)
[function] if (ioctl(fd, SNDRV_PCM_IOCTL_TSTAMP, &on) < 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1692)
[declaration] snd_check(PCM, "TSTAMP failed");
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1699)
[declaration] close(fd);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1715)
[declaration] free(hw);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1716)
[declaration] close(fd);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1733)
[declaration] snd_pcm_close(pcm);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1784)
[declaration] snd_error(PCM, "invalid stream %d", stream);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1807)
[declaration] snd_checknum(PCM, "open '%s' failed (%i)", filename, ret);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1811)
[declaration] memset(&info, 0, sizeof(info));
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1812)
[function] if (ioctl(fd, SNDRV_PCM_IOCTL_INFO, &info) < 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1814)
[declaration] snd_checknum(PCM, "SNDRV_PCM_IOCTL_INFO failed (%i)", ret);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1817)
[function] if (info.subdevice != (unsigned int) subdevice) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1818)
[declaration] close(fd);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1824)
[declaration] return snd_pcm_hw_open_fd(pcmp, name, fd, sync_ptr_ioctl);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1827)
[declaration] close(fd);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1912)
[function] if (strcmp(id, "card") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1919)
[function] if (strcmp(id, "device") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1921)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1922)
[declaration] snd_error(PCM, "Invalid type for %s", id);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1927)
[function] if (strcmp(id, "subdevice") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1929)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1930)
[declaration] snd_error(PCM, "Invalid type for %s", id);

```

```

dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1935)
[function] if (strcmp(id, "sync_ptr_ioctl") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1942)
[function] if (strcmp(id, "nonblock") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1949)
[function] if (strcmp(id, "rate") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1952)
[function] snd_config_is_array(n)) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1955)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1956)
[declaration] snd_error(PCM, "array expected for rate compound");
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1960)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1961)
[declaration] snd_error(PCM, "Invalid type for rate.0");
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1966)
[function] if (err >= 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1968)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1969)
[declaration] snd_error(PCM, "Invalid type for rate.0");
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1976)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1977)
[declaration] snd_error(PCM, "Invalid type for %s", id);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1984)
[function] if (strcmp(id, "min_rate") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1987)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1988)
[declaration] snd_error(PCM, "Invalid type for %s", id);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1994)
[function] if (strcmp(id, "max_rate") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1997)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 1998)
[declaration] snd_error(PCM, "Invalid type for %s", id);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 2004)
[function] if (strcmp(id, "format") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 2006)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 2007)
[declaration] snd_error(PCM, "invalid type for %s", id);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 2013)
[function] if (strcmp(id, "channels") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 2016)
[function] if (err < 0) {

```

```

dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 2017)
[declaration] snd_error(PCM, "Invalid type for %s", id);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 2023)
[function] if (strcmp(id, "chmap") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 2024)
[declaration] snd_pcm_free_chmaps(chmap);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 2026)
[function] if (!chmap) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 2027)
[declaration] snd_error(PCM, "Invalid channel map for %s", id);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 2033)
[function] if (strcmp(id, "drain_silence") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 2036)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 2037)
[declaration] snd_error(PCM, "Invalid type for %s", id);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 2043)
[declaration] snd_error(PCM, "Unknown field %s", id);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 2048)
[declaration] snd_error(PCM, "card is not defined");
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 2053)
[declaration] snd_error(PCM, "min_rate - max_rate configuration invalid");
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 2064)
[declaration] snd_pcm_hw_nonblock(*pcmp, 0);
dirtycode/alsa-lib/src/pcm/pcm_hw.c (line 2137)
[function] if (params->masks[i].bits[j]) {
dirtycode/alsa-lib/src/pcm/pcm_iec958.c (line 169)
[function] while (frames1-- > 0) {
dirtycode/alsa-lib/src/pcm/pcm_iec958.c (line 218)
[function] while (frames1-- > 0) {
dirtycode/alsa-lib/src/pcm/pcm_iec958.c (line 369)
[function] if ((iec->status[3] & IEC958_AES3_CON_FS) == IEC958_AES3_CON_FS_NOTID) {
dirtycode/alsa-lib/src/pcm/pcm_iec958.c (line 377)
[function] switch (rate) {
dirtycode/alsa-lib/src/pcm/pcm_iec958.c (line 417)
[function] if ((iec->status[4] & IEC958_AES4_CON_WORDLEN) ==
IEC958_AES4_CON_WORDLEN_NOTID) {
dirtycode/alsa-lib/src/pcm/pcm_iec958.c (line 419)
[function] switch (snd_pcm_format_width(format)) {
dirtycode/alsa-lib/src/pcm/pcm_iec958.c (line 493)
[declaration] snd_pcm_format_name(iec->sformat));
dirtycode/alsa-lib/src/pcm/pcm_iec958.c (line 495)
[declaration] snd_output_printf(out, "Its setup is:\n");
dirtycode/alsa-lib/src/pcm/pcm_iec958.c (line 496)
[declaration] snd_pcm_dump_setup(pcm, out);
dirtycode/alsa-lib/src/pcm/pcm_iec958.c (line 601)
[declaration] memcpy(iec->status, status_bits, sizeof(iec->status));
dirtycode/alsa-lib/src/pcm/pcm_iec958.c (line 603)
[declaration] memcpy(iec->status, default_status_bits, sizeof(default_status_bits));

```

```

dirtycode/alsa-lib/src/pcm/pcm_iec958.c (line 611)
[declaration] free(iec);
dirtycode/alsa-lib/src/pcm/pcm_iec958.c (line 709)
[function] if (strcmp(id, "slave") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_iec958.c (line 713)
[function] if (strcmp(id, "status") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_iec958.c (line 714)
[function] if (snd_config_get_type(n) != SND_CONFIG_TYPE_COMPOUND) {
dirtycode/alsa-lib/src/pcm/pcm_iec958.c (line 715)
[declaration] snd_error(PCM, "Invalid type for %s", id);
dirtycode/alsa-lib/src/pcm/pcm_iec958.c (line 721)
[function] if (strcmp(id, "preamble") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_iec958.c (line 722)
[function] if (snd_config_get_type(n) != SND_CONFIG_TYPE_COMPOUND) {
dirtycode/alsa-lib/src/pcm/pcm_iec958.c (line 723)
[declaration] snd_error(PCM, "Invalid type for %s", id);
dirtycode/alsa-lib/src/pcm/pcm_iec958.c (line 729)
[function] if (strcmp(id, "hdmi_mode") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_iec958.c (line 736)
[declaration] snd_error(PCM, "Unknown field %s", id);
dirtycode/alsa-lib/src/pcm/pcm_iec958.c (line 743)
[function] snd_config_for_each(i, inext, status) {
dirtycode/alsa-lib/src/pcm/pcm_iec958.c (line 746)
[function] if (snd_config_get_type(n) != SND_CONFIG_TYPE_INTEGER) {
dirtycode/alsa-lib/src/pcm/pcm_iec958.c (line 747)
[declaration] snd_error(PCM, "invalid IEC958 status bits");
dirtycode/alsa-lib/src/pcm/pcm_iec958.c (line 751)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_iec958.c (line 752)
[declaration] snd_error(PCM, "invalid IEC958 status bits");
dirtycode/alsa-lib/src/pcm/pcm_iec958.c (line 764)
[function] snd_config_for_each(i, inext, preamble) {
dirtycode/alsa-lib/src/pcm/pcm_iec958.c (line 778)
[declaration] snd_error(PCM, "invalid IEC958 preamble type %s", id);
dirtycode/alsa-lib/src/pcm/pcm_iec958.c (line 782)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_iec958.c (line 783)
[declaration] snd_error(PCM, "invalid IEC958 preamble value");
dirtycode/alsa-lib/src/pcm/pcm_iec958.c (line 790)
[declaration] snd_error(PCM, "slave is not defined");
dirtycode/alsa-lib/src/pcm/pcm_iec958.c (line 800)
[declaration] snd_config_delete(sconf);
dirtycode/alsa-lib/src/pcm/pcm_iec958.c (line 801)
[declaration] snd_error(PCM, "invalid slave format");
dirtycode/alsa-lib/src/pcm/pcm_iec958.c (line 812)
[declaration] snd_pcm_close(spcm);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 50)
[declaration] static int snd_pcm_ioplug_drop(snd_pcm_t *pcm);

```

```

dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 51)
[declaration] static int snd_pcm_ioplug_poll_descriptors_count(snd_pcm_t *pcm);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 52)
[declaration] static int snd_pcm_ioplug_poll_descriptors(snd_pcm_t *pcm, struct
pollfd *pfd, unsigned int space);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 53)
[declaration] static int snd_pcm_ioplug_poll_revents(snd_pcm_t *pcm, struct pollfd
*pfd, unsigned int nfds, unsigned short *revents);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 75)
[declaration] snd_pcm_mmap_hw_forward(io->data->pcm, delta);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 77)
[function] if (io->data->state == SND_PCM_STATE_DRAINING) {
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 80)
[declaration] snd_pcm_ioplug_drop(pcm);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 85)
[declaration] snd_pcm_ioplug_drop(pcm);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 97)
[declaration] snd_strncpy((char *)info->id, pcm->name, sizeof(info->id));
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 98)
[declaration] snd_strncpy((char *)info->name, pcm->name, sizeof(info->name));
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 99)
[declaration] snd_strncpy((char *)info->subname, pcm->name, sizeof(info->subname));
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 107)
[declaration] return snd_pcm_channel_info_shm(pcm, info, -1);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 118)
[declaration] snd_pcm_ioplug_hw_ptr_update(pcm);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 174)
[declaration] snd_pcm_unlock(pcm); /* to avoid deadlock */
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 176)
[declaration] snd_pcm_lock(pcm);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 201)
[declaration] hw_param_interval(params, b), &t);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 202)
[declaration] return snd_interval_refine(hw_param_interval(params, x), &t);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 211)
[declaration] hw_param_interval(params, b), &t);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 212)
[declaration] return snd_interval_refine(hw_param_interval(params, x), &t);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 221)
[declaration] hw_param_interval(params, b), k, &t);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 222)
[declaration] return snd_interval_refine(hw_param_interval(params, x), &t);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 231)
[declaration] hw_param_interval(params, b), &t);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 232)
[declaration] return snd_interval_refine(hw_param_interval(params, x), &t);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 408)
[function] switch (p->list[i]) {

```

```

dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 446)
[declaration] INTERNAL(snd_pcm_hw_params_get_access)(params, &io->data->access);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 447)
[declaration] INTERNAL(snd_pcm_hw_params_get_format)(params, &io->data->format);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 448)
[declaration] INTERNAL(snd_pcm_hw_params_get_channels)(params, &io->data->channels);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 449)
[declaration] INTERNAL(snd_pcm_hw_params_get_rate)(params, &io->data->rate, 0);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 450)
[declaration] INTERNAL(snd_pcm_hw_params_get_period_size)(params,
&io->data->period_size, 0);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 451)
[declaration] INTERNAL(snd_pcm_hw_params_get_buffer_size)(params,
&io->data->buffer_size);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 519)
[declaration] snd_pcm_ioplug_hw_ptr_update(pcm);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 543)
[declaration] snd_pcm_unlock(pcm);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 546)
[function] if (pcm->stream == SND_PCM_STREAM_PLAYBACK) {
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 547)
[function] if (!io->data->callback->drain) {
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 563)
[function] if (io->data->callback->drain) {
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 564)
[declaration] snd_pcm_unlock(pcm); /* let plugin own locking */
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 566)
[declaration] snd_pcm_lock(pcm);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 574)
[declaration] snd_pcm_ioplug_drop(pcm);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 602)
[declaration] return snd_pcm_mmap_hw_rewindable(pcm);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 613)
[declaration] return snd_pcm_mmap_avail(pcm);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 648)
[declaration] snd_pcm_mmap_appl_forward(pcm, result);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 655)
[declaration] return snd_pcm_mmap_writei(pcm, buffer, size);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 658)
[declaration] snd_pcm_areas_from_buf(pcm, areas, (void*)buffer);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 667)
[declaration] return snd_pcm_mmap_writen(pcm, bufs, size);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 670)
[declaration] snd_pcm_areas_from_bufs(pcm, areas, bufs);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 679)
[declaration] return snd_pcm_mmap_readi(pcm, buffer, size);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 682)
[declaration] snd_pcm_areas_from_buf(pcm, areas, buffer);

```

```

dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 691)
[declaration] return snd_pcm_mmap_readn(pcm, bufs, size);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 694)
[declaration] snd_pcm_areas_from_bufs(pcm, areas, bufs);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 728)
[declaration] return __snd_pcm_mmap_begin_generic(pcm, areas, offset, frames);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 729)
[declaration] return snd_pcm_ioplug_mmap_begin_capture(pcm, areas, offset, frames);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 742)
[declaration] __snd_pcm_mmap_begin_generic(pcm, &areas, &ofs, &frames);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 745)
[declaration] return ioplug_priv_transfer_areas(pcm, areas, offset, frames);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 781)
[declaration] snd_pcm_unlock(pcm); /* to avoid deadlock */
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 783)
[declaration] snd_pcm_lock(pcm);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 794)
[declaration] snd_pcm_unlock(pcm); /* to avoid deadlock */
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 796)
[declaration] snd_pcm_lock(pcm);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 816)
[declaration] snd_pcm_unlock(pcm); /* to avoid deadlock */
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 818)
[declaration] snd_pcm_lock(pcm);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 881)
[declaration] snd_output_printf(out, "%s\n", io->data->name);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 883)
[declaration] snd_output_printf(out, "IO-PCM Plugin\n");
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 884)
[function] if (pcm->setup) {
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 885)
[declaration] snd_output_printf(out, "Its setup is:\n");
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 886)
[declaration] snd_pcm_dump_setup(pcm, out);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 895)
[declaration] snd_ext_parm_clear(&io->params[i]);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 1105)
[declaration] free(io);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 1129)
[declaration] return snd_pcm_close(ioplug->pcm);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 1160)
[declaration] snd_error(PCM, "IOPLUG: invalid parameter type %d", type);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 1165)
[declaration] return snd_ext_parm_set_list(&io->params[type], num_list, list);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 1184)
[declaration] snd_error(PCM, "IOPLUG: invalid parameter type %d", type);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 1188)
[declaration] snd_error(PCM, "IOPLUG: invalid parameter type %d", type);

```

```

dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 1193)
[declaration] return snd_ext_parm_set_minmax(&io->params[type], min, max);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 1228)
[declaration] return snd_pcm_mmap_areas(ioplug->pcm);
dirtycode/alsa-lib/src/pcm/pcm_ioplug.c (line 1260)
[declaration] return __snd_pcm_avail(ioplug->pcm, hw_ptr, appl_ptr);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 132)
[function] if ((lplug->desc->PortDescriptors[idx] & pdesc) == pdesc) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 133)
[function] if (port_idx == 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 185)
[declaration] snd_pcm_ladspa_free_io(&plugin->input);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 186)
[declaration] snd_pcm_ladspa_free_io(&plugin->output);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 188)
[declaration] dlclose(plugin->dl_handle);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 189)
[declaration] free(plugin->filename);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 190)
[declaration] list_del(&plugin->list);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 191)
[declaration] free(plugin);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 202)
[declaration] free(ladspa->zero[idx]);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 213)
[declaration] return snd_pcm_generic_close(pcm);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 250)
[declaration] _snd_pcm_hw_param_set(sparams, SND_PCM_HW_PARAM_CHANNELS,
ladspa->channels, 0);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 327)
[function] list_for_each_safe(pos1, next1, &plugin->instances) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 331)
[function] if (cleanup) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 334)
[function] if (instance->input.m_data) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 336)
[declaration] free(instance->input.m_data[idx]);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 337)
[declaration] free(instance->input.m_data);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 339)
[function] if (instance->output.m_data) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 341)
[declaration] free(instance->output.m_data[idx]);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 342)
[declaration] free(instance->output.m_data);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 344)
[declaration] free(instance->input.data);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 345)
[declaration] free(instance->output.data);

```

```

dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 346)
[declaration] list_del(&(instance->list));
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 347)
[declaration] snd_pcm_ladspa_free_eps(&instance->input);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 348)
[declaration] snd_pcm_ladspa_free_eps(&instance->output);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 349)
[declaration] free(instance);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 355)
[function] if (cleanup) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 356)
[declaration] assert(list_empty(&plugin->instances));
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 416)
[declaration] snd_pcm_ladspa_count_ports(plugin, io->pdesc | LADSPA_PORT_AUDIO);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 422)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 423)
[declaration] snd_error(PCM, "unable to find audio %s port %u plugin '%s'",
io->pdesc & LADSPA_PORT_INPUT ? "input" : "output", idx, plugin->desc->Name);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 430)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 431)
[declaration] snd_error(PCM, "unable to add channel %u for audio %s plugin '%s'",
idx, io->pdesc & LADSPA_PORT_INPUT ? "input" : "output", plugin->desc->Name);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 435)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 436)
[declaration] snd_error(PCM, "unable to add port %u for audio %s plugin '%s'", port,
io->pdesc & LADSPA_PORT_INPUT ? "input" : "output", plugin->desc->Name);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 471)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 472)
[declaration] snd_error(PCM, "unable to find audio %s port %u plugin '%s'",
io->pdesc & LADSPA_PORT_INPUT ? "input" : "output", (unsigned int)0,
plugin->desc->Name);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 478)
[declaration] snd_error(PCM, "unable to add channel %u for audio %s plugin '%s'",
idx, io->pdesc & LADSPA_PORT_INPUT ? "input" : "output", plugin->desc->Name);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 483)
[declaration] snd_error(PCM, "unable to add port %u for audio %s plugin '%s'", port,
io->pdesc & LADSPA_PORT_INPUT ? "input" : "output", plugin->desc->Name);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 516)
[function] if (LADSPA_IS_HINT_LOGARITHMIC(hdesc)) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 527)
[function] if (LADSPA_IS_HINT_LOGARITHMIC(hdesc)) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 537)
[function] if (LADSPA_IS_HINT_LOGARITHMIC(hdesc)) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 575)
[function] if ((plugin->desc->PortDescriptors[idx] & (io->pdesc |
LADSPA_PORT_CONTROL)) == (io->pdesc | LADSPA_PORT_CONTROL)) {

```

```

dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 576)
[function] if (io->controls_size > midx) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 578)
[declaration] snd_pcm_ladspa_get_default_cvalue(plugin->desc, idx,
&io->controls[midx]);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 597)
[function] if ((plugin->desc->PortDescriptors[idx] & (io->pdesc |
LADSPA_PORT_AUDIO)) == (io->pdesc | LADSPA_PORT_AUDIO)) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 598)
[function] if (eps->channels.array[midx] == NO_ASSIGN) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 599)
[declaration] snd_error(PCM, "%s port for plugin %s depth %u is not connected",
io->pdesc & LADSPA_PORT_INPUT ? "input" : "output", plugin->desc->Name, depth);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 605)
[declaration] snd_error(PCM, "%i connection errors total", err);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 629)
[function] if (plugin->policy == SND_PCM_LADSPA_POLICY_DUPLICATE) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 642)
[function] if (instance->handle == NULL) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 643)
[declaration] snd_error(PCM, "Unable to create instance of LADSPA plugin '%s'",
plugin->desc->Name);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 644)
[declaration] free(instance);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 647)
[declaration] list_add_tail(&instance->list, &plugin->instances);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 648)
[function] if (plugin->policy == SND_PCM_LADSPA_POLICY_DUPLICATE) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 650)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 651)
[declaration] snd_error(PCM, "Unable to connect duplicate port of plugin '%s'
channel %u depth %u", plugin->desc->Name, idx, instance->depth);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 656)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 657)
[declaration] snd_error(PCM, "Unable to connect plugin '%s' depth %u",
plugin->desc->Name, depth);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 662)
[declaration] assert(err >= 0);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 664)
[declaration] assert(err >= 0);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 711)
[function] list_for_each(pos1, &plugin->instances) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 716)
[declaration] assert(instance->input.ports.array[idx] != NO_ASSIGN);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 722)
[declaration] assert(instance->output.ports.array[idx] != NO_ASSIGN);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 726)
[function] if (nchannels != channels) {

```

```

dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 728)
[function] if (npchannels == NULL) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 729)
[declaration] free(pchannels);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 736)
[declaration] assert(instance->input.data == NULL);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 737)
[declaration] assert(instance->input.m_data == NULL);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 738)
[declaration] assert(instance->output.data == NULL);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 739)
[declaration] assert(instance->output.m_data == NULL);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 748)
[declaration] free(pchannels);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 753)
[function] if (pchannels[chn] == NULL && chn < ichannels) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 758)
[function] if (instance->input.data[idx] == NULL) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 760)
[function] if (instance->input.data[idx] == NULL) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 761)
[declaration] free(pchannels);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 771)
[function] if (instance->output.data[idx] == NULL) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 772)
[declaration] free(pchannels);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 785)
[function] list_for_each(pos1, &plugin->instances) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 789)
[function] if (instance->output.data[idx] == pchannels[chn]) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 790)
[declaration] free(instance->output.m_data[idx]);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 792)
[function] if (chn < ochannels) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 796)
[function] if (instance->output.data[idx] == NULL) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 797)
[declaration] free(pchannels);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 810)
[function] list_for_each(pos1, &plugin->instances) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 813)
[declaration] printf("%i:alloc-input%i: data = %p, m_data = %p\n", instance->depth,
idx, instance->input.data[idx], instance->input.m_data[idx]);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 815)
[declaration] printf("%i:alloc-output%i: data = %p, m_data = %p\n", instance->depth,
idx, instance->output.data[idx], instance->output.m_data[idx]);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 831)
[declaration] snd_pcm_ladspa_free_instances(pcm, ladspa, 1);

```

```

dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 836)
[declaration] snd_pcm_ladspa_free_instances(pcm, ladspa, 1);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 847)
[declaration] return snd_pcm_generic_hw_free(pcm);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 877)
[function] list_for_each(pos, &ladspa->pplugins) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 879)
[function] list_for_each(pos1, &plugin->instances) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 884)
[function] if (data == NULL) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 893)
[function] if (data == NULL) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 938)
[function] list_for_each(pos, &ladspa->cplugins) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 940)
[function] list_for_each(pos1, &plugin->instances) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 945)
[function] if (data == NULL) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 954)
[function] if (data == NULL) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 983)
[declaration] snd_output_printf(out, " %i -> NONE\n", idx);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 985)
[declaration] snd_output_printf(out, " %i -> %i\n", idx, io->port_bindings[idx]);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 992)
[function] if ((plugin->desc->PortDescriptors[idx] & (io->pdesc |
LADSPA_PORT_CONTROL)) == (io->pdesc | LADSPA_PORT_CONTROL)) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 993)
[declaration] snd_output_printf(out, " %i \"%s\" = %.8f\n", idx,
plugin->desc->PortNames[idx], io->controls[midx]);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1007)
[function] if (idx > 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1008)
[declaration] snd_output_putc(out, ',');
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1009)
[declaration] snd_output_putc(out, ' ');
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1013)
[declaration] snd_output_putc(out, '-');
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1015)
[declaration] snd_output_printf(out, "%u", val);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1017)
[declaration] snd_output_printf(out, " \"%s\"", plugin->desc->PortNames[val]);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1027)
[declaration] snd_output_printf(out, " Policy: %s\n", plugin->policy ==
SND_PCM_LADSPA_POLICY_NONE ? "none" : "duplicate");
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1028)
[declaration] snd_output_printf(out, " Filename: %s\n", plugin->filename);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1029)
[declaration] snd_output_printf(out, " Plugin Name: %s\n", plugin->desc->Name);

```

```

dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1030)
[declaration] snd_output_printf(out, " Plugin Label: %s\n", plugin->desc->Label);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1031)
[declaration] snd_output_printf(out, " Plugin Unique ID: %lu\n",
plugin->desc->UniqueID);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1032)
[declaration] snd_output_printf(out, " Instances:\n");
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1033)
[function] list_for_each(pos2, &plugin->instances) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1035)
[declaration] snd_output_printf(out, " Depth: %i\n", in->depth);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1036)
[declaration] snd_output_printf(out, " InChannels: ");
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1037)
[declaration] snd_pcm_ladspa_dump_array(out, &in->input.channels, NULL);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1038)
[declaration] snd_output_printf(out, "\n InPorts: ");
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1039)
[declaration] snd_pcm_ladspa_dump_array(out, &in->input.ports, plugin);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1040)
[declaration] snd_output_printf(out, "\n OutChannels: ");
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1041)
[declaration] snd_pcm_ladspa_dump_array(out, &in->output.channels, NULL);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1042)
[declaration] snd_output_printf(out, "\n OutPorts: ");
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1043)
[declaration] snd_pcm_ladspa_dump_array(out, &in->output.ports, plugin);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1044)
[declaration] snd_output_printf(out, "\n");
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1046)
[declaration] snd_pcm_ladspa_dump_direction(plugin, &plugin->input, out);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1047)
[declaration] snd_pcm_ladspa_dump_direction(plugin, &plugin->output, out);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1061)
[declaration] snd_output_printf(out, "Its setup is:\n");
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1062)
[declaration] snd_pcm_dump_setup(pcm, out);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1097)
[function] if (fcn) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1110)
[function] if (label != NULL) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1113)
[function] if (labellocale == NULL) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1114)
[declaration] dlclose(handle);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1117)
[declaration] strcpy (labellocale, label);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1120)
[function] if (strcmp(label, d->Label) && strcmp(labellocale, d->Label)) {

```

```
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1121)
[declaration] free(labellocale);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1124)
[declaration] free (labellocale);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1130)
[function] if (plugin->filename == NULL) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1131)
[declaration] dlclose(handle);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1139)
[declaration] dlclose(handle);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1165)
[function] if (!dirent) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1166)
[declaration] closedir(dir);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1171)
[function] if (filename == NULL) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1172)
[declaration] closedir(dir);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1175)
[declaration] strcpy(filename, path);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1177)
[declaration] strcat(filename, "/");
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1178)
[declaration] strcat(filename, dirent->d_name);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1180)
[declaration] free(filename);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1181)
[function] if (err < 0 && err != -ENOENT) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1182)
[declaration] closedir(dir);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1185)
[function] if (err > 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1186)
[declaration] closedir(dir);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1206)
[declaration] memcpy(name, c, 1);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1212)
[declaration] free(fullpath);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1241)
[declaration] free(array);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1259)
[declaration] snd_error(PCM, "controls definition must be a compound");
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1272)
[function] if (err >= 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1277)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1278)
[declaration] snd_error(PCM, "Unable to find an control port (%s)", id);
```

```

dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1281)
[function] if (snd_config_get_ireal(n, &dval) < 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1282)
[declaration] snd_error(PCM, "Control port %s has not an float or integer value",
id);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1286)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1287)
[declaration] snd_error(PCM, "internal error");
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1307)
[declaration] snd_error(PCM, "bindings definition must be a compound");
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1317)
[function] if (err < 0 || channel < 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1318)
[declaration] snd_error(PCM, "Invalid channel number: %s", id);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1321)
[function] if (lplug->policy == SND_PCM_LADSPA_POLICY_DUPLICATE && channel > 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1322)
[declaration] snd_error(PCM, "Wrong channel specification for duplicate policy");
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1332)
[declaration] memset(array, 0xff, count * sizeof(unsigned int));
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1335)
[function] snd_config_for_each(i, next, bindings) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1342)
[function] if (err < 0 || channel < 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1343)
[declaration] assert(0); /* should never happen */
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1347)
[function] if (err >= 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1349)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1350)
[declaration] snd_error(PCM, "Unable to find an audio port (%li) for channel %s",
port, id);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1356)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1357)
[declaration] snd_error(PCM, "Invalid LADSPA port field type for %s", id);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1361)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1362)
[declaration] snd_error(PCM, "Unable to find an audio port (%s) for channel %s",
sport, id);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1382)
[declaration] snd_error(PCM, "error adding default controls");
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1391)
[declaration] snd_error(PCM, "input or output definition must be a compound");
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1399)
[function] if (strcmp(id, "bindings") == 0) {

```

```

dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1403)
[function] if (strcmp(id, "controls") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1444)
[function] if (strcmp(id, "label") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1450)
[function] if (strcmp(id, "id") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1456)
[function] if (strcmp(id, "filename") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1462)
[function] if (strcmp(id, "input") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1466)
[function] if (strcmp(id, "output") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1470)
[function] if (strcmp(id, "policy") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1473)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1474)
[declaration] snd_error(PCM, "policy field must be a string");
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1482)
[declaration] snd_error(PCM, "unknown policy definition");
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1489)
[declaration] snd_error(PCM, "no plugin label or id");
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1501)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1502)
[declaration] snd_error(PCM, "Unable to load plugin '%s' ID %li, filename '%s'",
label, ladspa_id, filename);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1503)
[declaration] free(lplug);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1508)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1509)
[declaration] snd_error(PCM, "Unable to find or load plugin '%s' ID %li, path '%s'",
label, ladspa_id, path);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1510)
[declaration] free(lplug);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1515)
[declaration] list_add_tail(&lplug->list, list);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1517)
[declaration] list_add(&lplug->list, list);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1539)
[declaration] snd_error(PCM, "plugins must be defined inside a compound");
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1544)
[function] snd_config_for_each(i, next, plugins) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1551)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1552)
[declaration] snd_error(PCM, "id of field %s is not an integer", id);

```

```

dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1555)
[function] if (i == idx) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1565)
[declaration] snd_error(PCM, "empty plugin list is not accepted");
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1619)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1620)
[declaration] snd_pcm_ladspa_free(ladspa);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1628)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1629)
[declaration] snd_pcm_ladspa_free(ladspa);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1636)
[declaration] snd_pcm_ladspa_free(ladspa);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1751)
[function] if (strcmp(id, "slave") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1755)
[function] if (strcmp(id, "path") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1761)
[function] if (strcmp(id, "channels") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1771)
[function] if (strcmp(id, "plugins") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1775)
[function] if (strcmp(id, "playback_plugins") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1779)
[function] if (strcmp(id, "capture_plugins") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1783)
[declaration] snd_error(PCM, "Unknown field %s", id);
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1787)
[declaration] snd_error(PCM, "slave is not defined");
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1791)
[function] if (pplugins || cplugins) {
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1792)
[declaration] snd_error(PCM, "'plugins' definition cannot be combined with
'playback_plugins' or 'capture_plugins'");
dirtycode/alsa-lib/src/pcm/pcm_ladspa.c (line 1807)
[declaration] snd_pcm_close(spcm);
dirtycode/alsa-lib/src/pcm/pcm_lfloat.c (line 86)
[declaration] return snd_pcm_lfloat_get_s32_index(format);
dirtycode/alsa-lib/src/pcm/pcm_lfloat.c (line 123)
[function] while (frames1-- > 0) {
dirtycode/alsa-lib/src/pcm/pcm_lfloat.c (line 168)
[function] while (frames1-- > 0) {
dirtycode/alsa-lib/src/pcm/pcm_lfloat.c (line 343)
[declaration] snd_pcm_format_name(lfloat->sformat));
dirtycode/alsa-lib/src/pcm/pcm_lfloat.c (line 345)
[declaration] snd_output_printf(out, "Its setup is:\n");
dirtycode/alsa-lib/src/pcm/pcm_lfloat.c (line 346)
[declaration] snd_pcm_dump_setup(pcm, out);

```

```

dirtycode/alsa-lib/src/pcm/pcm_lfloat.c (line 406)
[declaration] free(lfloat);
dirtycode/alsa-lib/src/pcm/pcm_lfloat.c (line 482)
[function] if (strcmp(id, "slave") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_lfloat.c (line 486)
[declaration] snd_error(PCM, "Unknown field %s", id);
dirtycode/alsa-lib/src/pcm/pcm_lfloat.c (line 490)
[declaration] snd_error(PCM, "slave is not defined");
dirtycode/alsa-lib/src/pcm/pcm_lfloat.c (line 498)
[function] snd_pcm_format_float(sformat) != 1) {
dirtycode/alsa-lib/src/pcm/pcm_lfloat.c (line 499)
[declaration] snd_config_delete(sconf);
dirtycode/alsa-lib/src/pcm/pcm_lfloat.c (line 500)
[declaration] snd_error(PCM, "slave format is not linear integer or linear float");
dirtycode/alsa-lib/src/pcm/pcm_lfloat.c (line 509)
[declaration] snd_pcm_close(spcm);
dirtycode/alsa-lib/src/pcm/pcm_linear.c (line 58)
[declaration] snd_pcm_format_signed(dst_format));
dirtycode/alsa-lib/src/pcm/pcm_linear.c (line 82)
[declaration] snd_pcm_format_signed(dst_format));
dirtycode/alsa-lib/src/pcm/pcm_linear.c (line 93)
[function] switch (width) {
dirtycode/alsa-lib/src/pcm/pcm_linear.c (line 116)
[declaration] snd_pcm_format_signed(dst_format));
dirtycode/alsa-lib/src/pcm/pcm_linear.c (line 127)
[function] switch (width) {
dirtycode/alsa-lib/src/pcm/pcm_linear.c (line 168)
[function] while (frames1-- > 0) {
dirtycode/alsa-lib/src/pcm/pcm_linear.c (line 204)
[function] while (frames1-- > 0) {
dirtycode/alsa-lib/src/pcm/pcm_linear.c (line 313)
[declaration] snd_pcm_format_width(linear->sformat) == 20);
dirtycode/alsa-lib/src/pcm/pcm_linear.c (line 315)
[function] if (pcm->stream == SND_PCM_STREAM_PLAYBACK) {
dirtycode/alsa-lib/src/pcm/pcm_linear.c (line 387)
[declaration] snd_pcm_format_name(linear->sformat));
dirtycode/alsa-lib/src/pcm/pcm_linear.c (line 389)
[declaration] snd_output_printf(out, "Its setup is:\n");
dirtycode/alsa-lib/src/pcm/pcm_linear.c (line 390)
[declaration] snd_pcm_dump_setup(pcm, out);
dirtycode/alsa-lib/src/pcm/pcm_linear.c (line 450)
[declaration] free(linear);
dirtycode/alsa-lib/src/pcm/pcm_linear.c (line 525)
[function] if (strcmp(id, "slave") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_linear.c (line 529)
[declaration] snd_error(PCM, "Unknown field %s", id);
dirtycode/alsa-lib/src/pcm/pcm_linear.c (line 533)
[declaration] snd_error(PCM, "slave is not defined");

```

```

dirtycode/alsa-lib/src/pcm/pcm_linear.c (line 541)
[declaration] snd_config_delete(sconf);
dirtycode/alsa-lib/src/pcm/pcm_linear.c (line 542)
[declaration] snd_error(PCM, "slave format is not linear");
dirtycode/alsa-lib/src/pcm/pcm_linear.c (line 551)
[declaration] snd_pcm_close(spcm);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 122)
[declaration] assert((snd_pcm_uframes_t) frames <= pcm->buffer_size);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 127)
[declaration] pthread_mutex_unlock(&meter->update_mutex);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 145)
[declaration] atomic_dec(&meter->reset);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 153)
[declaration] assert((snd_pcm_uframes_t) frames <= pcm->buffer_size);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 197)
[declaration] snd_pcm_scope_enable(scope);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 203)
[declaration] pthread_mutex_lock(&meter->running_mutex);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 205)
[declaration] assert(err >= 0);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 209)
[function] if (meter->running) {
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 210)
[function] list_for_each(pos, &meter->scopes) {
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 218)
[declaration] pthread_mutex_unlock(&meter->running_mutex);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 221)
[declaration] pthread_mutex_unlock(&meter->running_mutex);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 222)
[function] if (pcm->stream == SND_PCM_STREAM_PLAYBACK) {
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 236)
[function] while (atomic_read(&meter->reset)) {
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 238)
[declaration] atomic_dec(&meter->reset);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 241)
[function] if (reset) {
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 242)
[function] list_for_each(pos, &meter->scopes) {
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 249)
[function] if (!meter->running) {
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 250)
[function] list_for_each(pos, &meter->scopes) {
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 257)
[function] list_for_each(pos, &meter->scopes) {
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 262)
[declaration] nanosleep(&meter->delay, NULL);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 267)
[declaration] snd_pcm_scope_disable(scope);

```

```

dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 285)
[declaration] snd_pcm_scope_remove(scope);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 288)
[declaration] snd_dlclose(meter->dl_handle);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 326)
[declaration] pthread_cond_signal(&meter->running_cond);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 359)
[declaration] snd_pcm_meter_add_frames(pcm, snd_pcm_mmap_areas(pcm), old_rptr,
result);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 372)
[declaration] snd_pcm_meter_update_main(pcm);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 422)
[declaration] return snd_pcm_hw_refine(meter->gen.slave, params);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 428)
[declaration] return _snd_pcm_hw_params_internal(meter->gen.slave, params);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 468)
[declaration] free(meter->buf);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 497)
[declaration] return snd_pcm_hw_free(meter->gen.slave);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 505)
[declaration] snd_output_printf(out, "Its setup is:\n");
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 506)
[declaration] snd_pcm_dump_setup(pcm, out);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 589)
[declaration] free(meter);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 626)
[declaration] snd_error(PCM, "Invalid type for scope %s", str);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 632)
[declaration] snd_error(PCM, "type is not defined");
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 637)
[declaration] snd_error(PCM, "unable to get id");
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 642)
[declaration] snd_error(PCM, "Invalid type for %s", id);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 647)
[function] snd_config_for_each(i, next, type_conf) {
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 654)
[function] if (strcmp(id, "lib") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 656)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 657)
[declaration] snd_error(PCM, "Invalid type for %s", id);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 662)
[function] if (strcmp(id, "open") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 664)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 665)
[declaration] snd_error(PCM, "Invalid type for %s", id);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 670)
[declaration] snd_error(PCM, "Unknown field %s", id);

```

```

dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 677)
[declaration] snprintf(buf, sizeof(buf), "_snd_pcm_scope_%s_open", str);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 683)
[declaration] snd_error(PCM, "Cannot open shared library %s (%s)", lib, errbuf);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 686)
[declaration] snd_error(PCM, "symbol %s is not defined inside %s", open_name, lib);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 687)
[declaration] snd_dlcclose(h);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 692)
[declaration] snd_config_delete(type_conf);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 696)
[declaration] snd_dlcclose(h);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 777)
[function] if (strcmp(id, "slave") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 781)
[function] if (strcmp(id, "frequency") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 783)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 784)
[declaration] snd_error(PCM, "Invalid type for %s", id);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 789)
[function] if (strcmp(id, "scopes") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 790)
[function] if (snd_config_get_type(n) != SND_CONFIG_TYPE_COMPOUND) {
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 791)
[declaration] snd_error(PCM, "Invalid type for %s", id);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 797)
[declaration] snd_error(PCM, "Unknown field %s", id);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 801)
[declaration] snd_error(PCM, "slave is not defined");
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 813)
[declaration] snd_pcm_close(spcm);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 823)
[function] if (snd_config_get_string(n, &str) >= 0) {
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 825)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 826)
[declaration] snd_error(PCM, "unknown pcm_scope %s", str);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 829)
[declaration] snd_config_delete(n);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 833)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 834)
[declaration] snd_pcm_close(*pcmp);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 1056)
[declaration] free(s16->adpcm_states);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 1061)
[declaration] free(s16->buf);

```

```

dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 1062)
[declaration] free(s16->adpcm_states);
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 1117)
[function] switch (spcm->format) {
dirtycode/alsa-lib/src/pcm/pcm_meter.c (line 1196)
[declaration] free(scope);
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 199)
[declaration] return snd_pcm_format_little_endian(format);
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 201)
[declaration] return snd_pcm_format_big_endian(format);
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 400)
[declaration] assert(0);
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 506)
[declaration] return bswap_64(u.i);
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 519)
[declaration] return bswap_64(u.i);
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 530)
[declaration] return bswap_64(u.i);
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 543)
[declaration] return bswap_64(u.i);
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 561)
[declaration] assert(0);
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 618)
[declaration] memset(data, silence, samples1);
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 623)
[declaration] memset(data, silence, samples);
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 630)
[declaration] memset(data, 0, samples * 2);
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 641)
[declaration] memset(data, 0, samples * 3);
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 643)
[function] while (samples-- > 0) {
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 661)
[declaration] memset(data, 0, samples * 4);
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 672)
[declaration] memset(data, 0, samples * 8);
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 680)
[declaration] assert(0);
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 719)
[function] switch (width) {
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 734)
[function] switch (width) {
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 796)
[function] if (strcmp(id, "card") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 803)
[function] if (strcmp(id, "iface") == 0 || strcmp(id, "interface") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 805)
[function] if ((err = snd_config_get_string(n, &ptr)) < 0) {

```

```

dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 806)
[declaration] snd_error(PCM, "field %s is not a string", id);
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 809)
[function] if ((err = snd_config_get_ctl_iface_ascii(ptr)) < 0) {
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 810)
[declaration] snd_error(PCM, "Invalid value for '%s'", id);
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 816)
[function] if (strcmp(id, "name") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 817)
[function] if ((err = snd_config_get_string(n, &name)) < 0) {
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 818)
[declaration] snd_error(PCM, "field %s is not a string", id);
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 823)
[function] if (strcmp(id, "index") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 824)
[function] if ((err = snd_config_get_integer(n, &index)) < 0) {
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 825)
[declaration] snd_error(PCM, "field %s is not an integer", id);
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 830)
[function] if (strcmp(id, "device") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 831)
[function] if ((err = snd_config_get_integer(n, &device)) < 0) {
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 832)
[declaration] snd_error(PCM, "field %s is not an integer", id);
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 837)
[function] if (strcmp(id, "subdevice") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 838)
[function] if ((err = snd_config_get_integer(n, &subdevice)) < 0) {
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 839)
[declaration] snd_error(PCM, "field %s is not an integer", id);
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 844)
[function] if (cchannelsp && strcmp(id, "count") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 846)
[function] if ((err = snd_config_get_integer(n, &v)) < 0) {
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 847)
[declaration] snd_error(PCM, "field %s is not an integer", id);
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 850)
[function] if (v < 1 || v > 2) {
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 851)
[declaration] snd_error(PCM, "Invalid count %ld", v);
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 857)
[function] if (hwctlp && strcmp(id, "hwctl") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 858)
[function] if ((err = snd_config_get_bool(n)) < 0) {
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 859)
[declaration] snd_error(PCM, "The field %s must be a boolean type", id);
dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 865)
[declaration] snd_error(PCM, "Unknown field %s", id);

```

dirtycode/alsa-lib/src/pcm/pcm_misc.c (line 869)

[declaration] snd_error(PCM, "Missing control name");

dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 77)

**[declaration] snd_check(PCM, "too short avail %ld to size %ld",
snd_pcm_mmap_playback_avail(pcm), size);**

dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 86)

[declaration] __snd_pcm_mmap_begin(pcm, &pcm_areas, &pcm_offset, &frames);

dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 109)

**[declaration] snd_check(PCM, "too short avail %ld to size %ld",
snd_pcm_mmap_capture_avail(pcm), size);**

dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 118)

[declaration] __snd_pcm_mmap_begin(pcm, &pcm_areas, &pcm_offset, &frames);

dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 247)

[declaration] snd_check(PCM, "invalid access type %d", pcm->access);

dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 266)

[declaration] snd_check(PCM, "PCM not set up");

dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 270)

[declaration] snd_check(PCM, "Already mmaped");

dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 286)

[declaration] free(pcm->mmap_channels);

dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 294)

[function] if (err < 0) {

dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 295)

[declaration] free(pcm->mmap_channels);

dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 296)

[declaration] free(pcm->running_areas);

dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 308)

[function] if (i->addr) {

dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 320)

[function] switch (i1->type) {

dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 333)

[declaration] assert(0);

dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 341)

[function] switch (i->type) {

dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 344)

[function] if (ptr == MAP_FAILED) {

dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 345)

[declaration] snd_errormsg(PCM, "mmap failed");

dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 352)

[function] if (i->u.shm.shmid < 0) {

dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 356)

[function] if (id < 0) {

dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 357)

[declaration] snd_errormsg(PCM, "shmget failed");

dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 362)

[function] if (ptr == (void *) -1) {

dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 363)

[declaration] snd_errormsg(PCM, "shmat failed");

```

dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 367)
[function] if (shmctl(id, IPC_RMID, NULL) < 0){
dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 368)
[declaration] snd_erronum(PCM, "shmctl mark remove failed");
dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 372)
[function] if (i->u.shm.area == NULL) {
dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 373)
[declaration] snd_erronum(PCM, "snd_shm_area_create failed");
dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 381)
[function] if (i1->u.shm.shmid < 0) {
dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 389)
[function] if (ptr == (void*) -1) {
dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 390)
[declaration] snd_erronum(PCM, "shmat failed");
dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 397)
[declaration] snd_erronum(PCM, "shm support not available");
dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 402)
[function] if (ptr == NULL) {
dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 403)
[declaration] snd_erronum(PCM, "malloc failed");
dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 409)
[declaration] assert(0);
dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 415)
[function] switch (i1->type) {
dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 431)
[declaration] assert(0);
dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 448)
[declaration] snd_check(PCM, "Not mmapped");
dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 475)
[function] switch (i->type) {
dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 478)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 479)
[declaration] snd_erronum(PCM, "mmap failed");
dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 486)
[function] if (i->u.shm.area) {
dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 487)
[declaration] snd_shm_area_destroy(i->u.shm.area);
dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 494)
[function] if (i1->u.shm.area) {
dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 495)
[declaration] snd_shm_area_destroy(i1->u.shm.area);
dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 503)
[declaration] snd_erronum(PCM, "shm support not available");
dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 507)
[declaration] free(i->addr);
dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 510)
[declaration] assert(0);

```

```

dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 540)
[function] switch (pcm->access) {
dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 545)
[declaration] snd_pcm_unlock(pcm); /* to avoid deadlock */
dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 547)
[declaration] snd_pcm_lock(pcm);
dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 562)
[declaration] snd_pcm_unlock(pcm); /* to avoid deadlock */
dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 564)
[declaration] snd_pcm_lock(pcm);
dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 570)
[declaration] snd_check(PCM, "invalid access type %d", pcm->access);
dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 596)
[function] switch (pcm->access) {
dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 601)
[declaration] snd_pcm_unlock(pcm); /* to avoid deadlock */
dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 603)
[declaration] snd_pcm_lock(pcm);
dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 618)
[declaration] snd_pcm_unlock(pcm); /* to avoid deadlock */
dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 620)
[declaration] snd_pcm_lock(pcm);
dirtycode/alsa-lib/src/pcm/pcm_mmap.c (line 626)
[declaration] snd_check(PCM, "invalid access type %d", pcm->access);
dirtycode/alsa-lib/src/pcm/pcm_mmap_emul.c (line 178)
[declaration] snd_pcm_access_mask_set(pmask, SND_PCM_ACCESS_RW_INTERLEAVED);
dirtycode/alsa-lib/src/pcm/pcm_mmap_emul.c (line 216)
[declaration] return snd_pcm_generic_sw_params(pcm, params);
dirtycode/alsa-lib/src/pcm/pcm_mmap_emul.c (line 259)
[declaration] snd_pcm_mmap_appl_backward(pcm, frames);
dirtycode/alsa-lib/src/pcm/pcm_mmap_emul.c (line 268)
[declaration] snd_pcm_mmap_appl_forward(pcm, frames);
dirtycode/alsa-lib/src/pcm/pcm_mmap_emul.c (line 312)
[declaration] snd_pcm_mmap_hw_forward(pcm, size);
dirtycode/alsa-lib/src/pcm/pcm_mmap_emul.c (line 325)
[declaration] return snd_pcm_mmap_commit(slave, offset, size);
dirtycode/alsa-lib/src/pcm/pcm_mmap_emul.c (line 327)
[declaration] sync_slave_write(pcm);
dirtycode/alsa-lib/src/pcm/pcm_mmap_emul.c (line 339)
[declaration] sync_slave_read(pcm);
dirtycode/alsa-lib/src/pcm/pcm_mmap_emul.c (line 340)
[declaration] return snd_pcm_mmap_avail(pcm);
dirtycode/alsa-lib/src/pcm/pcm_mmap_emul.c (line 349)
[declaration] snd_output_printf(out, "Its setup is:\n");
dirtycode/alsa-lib/src/pcm/pcm_mmap_emul.c (line 350)
[declaration] snd_pcm_dump_setup(pcm, out);
dirtycode/alsa-lib/src/pcm/pcm_mmap_emul.c (line 423)
[declaration] free(map);

```

```

dirtycode/alsa-lib/src/pcm/pcm_mmap_emul.c (line 488)
[function] if (strcmp(id, "slave") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_mmap_emul.c (line 492)
[declaration] snd_error(PCM, "Unknown field %s", id);
dirtycode/alsa-lib/src/pcm/pcm_mmap_emul.c (line 496)
[declaration] snd_error(PCM, "slave is not defined");
dirtycode/alsa-lib/src/pcm/pcm_mmap_emul.c (line 508)
[declaration] snd_pcm_close(spcm);
dirtycode/alsa-lib/src/pcm/pcm_mulaw.c (line 184)
[function] while (frames1-- > 0) {
dirtycode/alsa-lib/src/pcm/pcm_mulaw.c (line 222)
[function] while (frames1-- > 0) {
dirtycode/alsa-lib/src/pcm/pcm_mulaw.c (line 338)
[function] if (mulaw->sformat == SND_PCM_FORMAT_MU_LAW) {
dirtycode/alsa-lib/src/pcm/pcm_mulaw.c (line 346)
[function] if (mulaw->sformat == SND_PCM_FORMAT_MU_LAW) {
dirtycode/alsa-lib/src/pcm/pcm_mulaw.c (line 401)
[declaration] snd_pcm_format_name(mulaw->sformat));
dirtycode/alsa-lib/src/pcm/pcm_mulaw.c (line 403)
[declaration] snd_output_printf(out, "Its setup is:\n");
dirtycode/alsa-lib/src/pcm/pcm_mulaw.c (line 404)
[declaration] snd_pcm_dump_setup(pcm, out);
dirtycode/alsa-lib/src/pcm/pcm_mulaw.c (line 463)
[declaration] free(mulaw);
dirtycode/alsa-lib/src/pcm/pcm_mulaw.c (line 539)
[function] if (strcmp(id, "slave") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_mulaw.c (line 543)
[declaration] snd_error(PCM, "Unknown field %s", id);
dirtycode/alsa-lib/src/pcm/pcm_mulaw.c (line 547)
[declaration] snd_error(PCM, "slave is not defined");
dirtycode/alsa-lib/src/pcm/pcm_mulaw.c (line 556)
[declaration] snd_config_delete(sconf);
dirtycode/alsa-lib/src/pcm/pcm_mulaw.c (line 557)
[declaration] snd_error(PCM, "invalid slave format");
dirtycode/alsa-lib/src/pcm/pcm_mulaw.c (line 566)
[declaration] snd_pcm_close(spcm);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 74)
[function] if (slave->close_slave) {
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 95)
[declaration] return snd_pcm_async(slave_0, sig, pid);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 102)
[declaration] return snd_pcm_poll_descriptors_count(slave_0);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 122)
[declaration] return snd_pcm_poll_descriptors(slave_0, pfd, pfd, space);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 129)
[declaration] return snd_pcm_poll_descriptors_revents(slave_0, pfd, pfd, nfds, revents);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 199)
[declaration] snd_pcm_access_mask_any(&saccess_mask);

```

```

dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 200)
[declaration] snd_pcm_access_mask_reset(&saccess_mask,
SND_PCM_ACCESS_MMAP_NONINTERLEAVED);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 232)
[declaration] snd_pcm_access_mask_reset(&access_mask,
SND_PCM_ACCESS_MMAP_NONINTERLEAVED);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 235)
[declaration] snd_pcm_access_mask_reset(&access_mask, SND_PCM_ACCESS_MMAP_COMPLEX);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 253)
[declaration] return snd_pcm_hw_refine(slave, sparams);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 268)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 269)
[declaration] snd_error(PCM, "Slave PCM #%d not usable", k);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 280)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 281)
[declaration] snd_pcm_multi_hw_refine_cchange(pcm, k, params, &sparams[k]);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 327)
[declaration] snd_pcm_unlink(multi->slaves[i].linked);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 344)
[declaration] assert(err >= 0);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 346)
[declaration] assert(err >= 0);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 348)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 349)
[declaration] snd_pcm_multi_hw_refine_cchange(pcm, i, params, &sparams[i]);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 391)
[declaration] static snd_pcm_sframes_t snd_pcm_multi_avail_update(snd_pcm_t *pcm);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 412)
[declaration] return snd_pcm_state(slave);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 426)
[function] if (avail > last_avail) {
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 436)
[function] if (avail < last_avail) {
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 498)
[declaration] return snd_pcm_htimestamp(slave, avail, tstamp);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 543)
[declaration] return snd_pcm_start(multi->slaves[0].linked);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 560)
[declaration] return snd_pcm_drop(multi->slaves[0].linked);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 577)
[declaration] return snd_pcm_drain(multi->slaves[0].linked);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 594)
[declaration] return snd_pcm_pause(multi->slaves[0].linked, enable);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 674)
[function] if (f > 0) {

```

```

dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 705)
[function] if (f > 0) {
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 723)
[declaration] return snd_pcm_resume(multi->slaves[0].linked);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 744)
[declaration] snd_pcm_unlink(multi->slaves[i].pcm);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 747)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 748)
[declaration] reset_links(multi);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 775)
[declaration] snd_pcm_unlink(multi->slaves[i].linked);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 817)
[declaration] sizeof(pcm->mmap_channels[0]));
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 819)
[declaration] sizeof(pcm->running_areas[0]));
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 821)
[declaration] snd_pcm_multi_munmap(pcm);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 829)
[function] if (chan->slave_idx < 0) {
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 830)
[declaration] snd_pcm_multi_munmap(pcm);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 885)
[function] if ((*p)->map.channels == slave_channels) {
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 897)
[declaration] snd_pcm_free_chmaps(slave_maps[i]);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 901)
[declaration] snd_pcm_free_chmaps(maps);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 936)
[declaration] free(slave_maps[i]);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 939)
[declaration] free(map);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 958)
[declaration] sizeof(int));
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 959)
[function] if (!slave_maps[i]) {
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 981)
[declaration] free(slave_maps[i]);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1000)
[declaration] snd_output_printf(out, "Its setup is:\n");
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1001)
[declaration] snd_pcm_dump_setup(pcm, out);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1004)
[declaration] snd_output_printf(out, "Slave #%d: ", k);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1005)
[declaration] snd_pcm_dump(multi->slaves[k].pcm, out);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1105)
[declaration] free(multi);

```

```

dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1111)
[declaration] free(multi->slaves);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1112)
[declaration] free(multi);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1117)
[declaration] assert(slaves_pcm[i]->stream == stream);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1124)
[declaration] assert(sidxs[i] < (int)slaves_count);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1125)
[declaration] assert(schannels[i] < schannels_count[sidxs[i]]);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1136)
[declaration] free(multi->slaves);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1137)
[declaration] free(multi->channels);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1138)
[declaration] free(multi);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1268)
[function] if (strcmp(id, "slaves") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1269)
[function] if (snd_config_get_type(n) != SND_CONFIG_TYPE_COMPOUND) {
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1270)
[declaration] snd_error(PCM, "Invalid type for %s", id);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1276)
[function] if (strcmp(id, "bindings") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1277)
[function] if (snd_config_get_type(n) != SND_CONFIG_TYPE_COMPOUND) {
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1278)
[declaration] snd_error(PCM, "Invalid type for %s", id);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1284)
[function] if (strcmp(id, "master") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1285)
[function] if (snd_config_get_integer(n, &master_slave) < 0) {
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1286)
[declaration] snd_error(PCM, "Invalid type for %s", id);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1291)
[declaration] snd_error(PCM, "Unknown field %s", id);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1295)
[declaration] snd_error(PCM, "slaves is not defined");
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1299)
[declaration] snd_error(PCM, "bindings is not defined");
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1306)
[declaration] snd_error(PCM, "Master slave is out of range (0-%u)", slaves_count-1);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1316)
[function] if (err < 0 || cchannel < 0) {
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1317)
[declaration] snd_error(PCM, "Invalid channel number: %s", id);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1324)
[declaration] snd_error(PCM, "No channels defined");

```

```

dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1367)
[function] if (err < 0 || cchannel < 0) {
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1368)
[declaration] snd_error(PCM, "Invalid channel number: %s", id);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1372)
[function] snd_config_for_each(j, jnext, m) {
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1379)
[function] if (strcmp(id, "slave") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1383)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1385)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1386)
[declaration] snd_error(PCM, "Invalid value for %s", id);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1389)
[declaration] sprintf(buf, "%ld", val);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1398)
[function] if (strcmp(id, "channel") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1400)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1401)
[declaration] snd_error(PCM, "Invalid type for %s", id);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1406)
[declaration] snd_error(PCM, "Unknown field %s", id);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1410)
[function] if (slave < 0 || (unsigned int)slave >= slaves_count) {
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1411)
[declaration] snd_error(PCM, "Invalid or missing idx for channel %s", id);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1417)
[declaration] snd_error(PCM, "Invalid or missing schannel for channel %s", id);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1431)
[declaration] snd_config_delete(slaves_conf[idx]);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1443)
[declaration] snd_pcm_close(slaves_pcm[idx]);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1449)
[declaration] snd_config_delete(slaves_conf[idx]);
dirtycode/alsa-lib/src/pcm/pcm_multi.c (line 1451)
[declaration] free(slaves_conf);
dirtycode/alsa-lib/src/pcm/pcm_null.c (line 74)
[declaration] snd_strncpy((char *)info->id, pcm->name, sizeof(info->id));
dirtycode/alsa-lib/src/pcm/pcm_null.c (line 75)
[declaration] snd_strncpy((char *)info->name, pcm->name, sizeof(info->name));
dirtycode/alsa-lib/src/pcm/pcm_null.c (line 76)
[declaration] snd_strncpy((char *)info->subname, pcm->name, sizeof(info->subname));
dirtycode/alsa-lib/src/pcm/pcm_null.c (line 88)
[declaration] return snd_pcm_mmap_avail(pcm);
dirtycode/alsa-lib/src/pcm/pcm_null.c (line 135)
[declaration] return snd_pcm_null_reset(pcm);

```

```

dirtycode/alsa-lib/src/pcm/pcm_null.c (line 197)
[declaration] snd_pcm_mmap_hw_backward(pcm, frames);
dirtycode/alsa-lib/src/pcm/pcm_null.c (line 200)
[declaration] snd_pcm_mmap_appl_backward(pcm, frames);
dirtycode/alsa-lib/src/pcm/pcm_null.c (line 212)
[declaration] snd_pcm_mmap_hw_forward(pcm, frames);
dirtycode/alsa-lib/src/pcm/pcm_null.c (line 215)
[declaration] snd_pcm_mmap_appl_forward(pcm, frames);
dirtycode/alsa-lib/src/pcm/pcm_null.c (line 239)
[declaration] return snd_pcm_write_areas(pcm, NULL, 0, size,
snd_pcm_null_xfer_areas);
dirtycode/alsa-lib/src/pcm/pcm_null.c (line 244)
[declaration] return snd_pcm_write_areas(pcm, NULL, 0, size,
snd_pcm_null_xfer_areas);
dirtycode/alsa-lib/src/pcm/pcm_null.c (line 249)
[declaration] return snd_pcm_read_areas(pcm, NULL, 0, size,
snd_pcm_null_xfer_areas);
dirtycode/alsa-lib/src/pcm/pcm_null.c (line 254)
[declaration] return snd_pcm_read_areas(pcm, NULL, 0, size,
snd_pcm_null_xfer_areas);
dirtycode/alsa-lib/src/pcm/pcm_null.c (line 261)
[declaration] return snd_pcm_null_forward(pcm, size);
dirtycode/alsa-lib/src/pcm/pcm_null.c (line 303)
[declaration] return _snd_pcm_copy_chmap_query(null->chmap);
dirtycode/alsa-lib/src/pcm/pcm_null.c (line 312)
[declaration] return _snd_pcm_choose_fixed_chmap(pcm, null->chmap);
dirtycode/alsa-lib/src/pcm/pcm_null.c (line 320)
[declaration] snd_output_printf(out, "Its setup is:\n");
dirtycode/alsa-lib/src/pcm/pcm_null.c (line 321)
[declaration] snd_pcm_dump_setup(pcm, out);
dirtycode/alsa-lib/src/pcm/pcm_null.c (line 388)
[function] if (fd < 0) {
dirtycode/alsa-lib/src/pcm/pcm_null.c (line 389)
[declaration] snd_erronum(PCM, "Cannot open /dev/null");
dirtycode/alsa-lib/src/pcm/pcm_null.c (line 394)
[function] if (fd < 0) {
dirtycode/alsa-lib/src/pcm/pcm_null.c (line 395)
[declaration] snd_erronum(PCM, "Cannot open /dev/full");
dirtycode/alsa-lib/src/pcm/pcm_null.c (line 401)
[declaration] close(fd);
dirtycode/alsa-lib/src/pcm/pcm_null.c (line 409)
[declaration] close(fd);
dirtycode/alsa-lib/src/pcm/pcm_null.c (line 410)
[declaration] free(null);
dirtycode/alsa-lib/src/pcm/pcm_null.c (line 480)
[function] if (strcmp(id, "chmap") == 0) {
dirtycode/alsa-lib/src/pcm/pcm_null.c (line 481)
[declaration] snd_pcm_free_chmaps(chmap);
dirtycode/alsa-lib/src/pcm/pcm_null.c (line 483)
[function] if (!chmap) {

```

```

dirtycode/alsa-lib/src/pcm/pcm_null.c (line 484)
[declaration] snd_error(PCM, "Invalid channel map for %s", id);
dirtycode/alsa-lib/src/pcm/pcm_null.c (line 489)
[declaration] snd_error(PCM, "Unknown field %s", id);
dirtycode/alsa-lib/src/pcm/pcm_null.c (line 490)
[declaration] snd_pcm_free_chmaps(chmap);
dirtycode/alsa-lib/src/pcm/pcm_null.c (line 495)
[declaration] snd_pcm_free_chmaps(chmap);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 57)
[declaration] type, snd_pcm_hw_param_name(var), s, snd_strerror(err), buf);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 60)
[declaration] type, snd_pcm_hw_param_name(var), val, snd_strerror(err), buf);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 98)
[declaration] snd_mask_any(hw_param_mask(params, var));
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 104)
[declaration] snd_interval_any(hw_param_interval(params, var));
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 116)
[declaration] return snd_pcm_hw_refine(pcm, params);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 124)
[declaration] _snd_pcm_hw_param_any(params, k);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 126)
[declaration] _snd_pcm_hw_param_any(params, k);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 167)
[declaration] assert(!snd_mask_empty(m));
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 175)
[declaration] assert(!snd_interval_empty(i));
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 192)
[declaration] assert(!snd_mask_empty(m));
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 200)
[declaration] assert(!snd_interval_empty(i));
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 218)
[declaration] return hw_param_mask_c(params, var);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 228)
[declaration] return hw_param_interval_c(params, var);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 251)
[declaration] snd_mask_none(hw_param_mask(params, var));
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 255)
[declaration] snd_interval_none(hw_param_interval(params, var));
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 259)
[declaration] assert(0);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 298)
[declaration] assert(0);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 325)
[declaration] assert(0);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 355)
[declaration] return snd_pcm_hw_param_get(params, var, rval, dir);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 367)
[declaration] assert(0);

```

```

dirtycode/alsa-lib/src/pcm/pcm_params.c (line 397)
[declaration] return snd_pcm_hw_param_get(params, var, rval, dir);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 406)
[function] if (dir > 0) {
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 409)
[function] if (val > 0) {
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 420)
[declaration] assert(0);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 451)
[declaration] assert(0);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 461)
[function] if (snd_pcm_hw_param_empty(params, var)) {
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 466)
[declaration] return snd_pcm_hw_param_get_min(params, var, val, dir);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 471)
[declaration] dump_hw_params(params, "set_min", var, *val, err);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 481)
[function] if (dir < 0) {
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 489)
[function] if (val == 0 && openmax) {
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 490)
[declaration] snd_mask_none(hw_param_mask(params, var));
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 497)
[declaration] assert(0);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 528)
[declaration] assert(0);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 538)
[function] if (snd_pcm_hw_param_empty(params, var)) {
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 543)
[declaration] return snd_pcm_hw_param_get_max(params, var, val, dir);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 548)
[declaration] dump_hw_params(params, "set_max", var, *val, err);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 560)
[function] if (mindir > 0) {
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 563)
[function] if (min > 0) {
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 570)
[function] if (maxdir < 0) {
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 579)
[function] if (max == 0 && openmax) {
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 580)
[declaration] snd_mask_none(mask);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 595)
[function] else if (hw_is_interval(var)) {
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 608)
[declaration] assert(0);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 641)
[declaration] assert(0);

```

```

dirtycode/alsa-lib/src/pcm/pcm_params.c (line 657)
[declaration] return snd_pcm_hw_param_get_max(params, var, max, maxdir);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 662)
[declaration] dump_hw_params(params, "set_minmax", var, *min, err);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 672)
[function] if (val == 0 && dir < 0) {
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 674)
[declaration] snd_mask_none(m);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 684)
[function] if (val == 0 && dir < 0) {
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 686)
[declaration] snd_interval_none(i);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 695)
[function] if (dir < 0) {
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 705)
[declaration] assert(0);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 736)
[declaration] assert(0);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 752)
[declaration] dump_hw_params(params, "set", var, val, err);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 793)
[declaration] assert(0);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 850)
[declaration] dump_hw_params(params, "set_near", var, *val, err);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 862)
[function] if (boundary_nearer(max, maxdir, best, valdir, min, mindir)) {
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 869)
[function] if (err < 0) {
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 870)
[declaration] dump_hw_params(params, "set_near", var, *val, err);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 881)
[declaration] dump_hw_params(params, "set_near", var, *val, err);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 935)
[function] if (boundary_nearer(max, maxdir, best, bestdir, min, mindir)) {
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 967)
[declaration] return snd_pcm_hw_param_set_near(pcm, params, var, &min, mindir);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1019)
[function] if (it->min <= cur && ! (it->min == cur && st->openmin)) {
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1029)
[declaration] return snd_pcm_hw_param_refine_near(pcm, params, var, src);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1038)
[declaration] return snd_mask_empty(hw_param_mask_c(params, var));
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1040)
[declaration] return snd_interval_empty(hw_param_interval_c(params, var));
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1051)
[declaration] hw_param_mask_c(params1, var));
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1054)
[declaration] hw_param_interval_c(params1, var));

```

```

dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1065)
[declaration] hw_param_mask_c(params1, var));
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1068)
[declaration] hw_param_interval_c(params1, var));
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1122)
[declaration] snd_pcm_hw_param_set_min(pcm, params, SND_CHANGE,
SND_PCM_HW_PARAM_PERIOD_TIME, &min, &dir);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1165)
[declaration] return snd_mask_count(mask);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1205)
[declaration] snd_mask_copy(d, s);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1213)
[declaration] snd_interval_copy(d, s);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1228)
[declaration] snd_output_puts(out, "NONE");
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1230)
[declaration] snd_output_puts(out, "ALL");
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1235)
[function] if (snd_mask_test(mask, k)) {
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1237)
[function] switch (var) {
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1248)
[declaration] assert(0);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1251)
[function] if (s) {
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1253)
[declaration] snd_output_putc(out, ' ');
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1254)
[declaration] snd_output_puts(out, s);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1263)
[declaration] snd_interval_print(hw_param_interval_c(params, var), out);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1376)
[function] if (badness >= 0) {
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1377)
[function] if ((unsigned int) badness <= badness_min) {
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1421)
[declaration] assert(choices != 0);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1497)
[declaration] return snd_pcm_hw_param_set_near(pcm, params, var, p->best, dir);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1499)
[declaration] return snd_pcm_hw_param_set_next(pcm, params, var, p->best, 0, value,
dir);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1534)
[function] if (p->choices[k].value == (unsigned int) value) {
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1570)
[declaration] free(data);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1649)
[declaration] _snd_pcm_hw_param_copy(&i, var, fail);

```

```

dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1654)
[declaration] snd_output_printf(out, "%s: ", snd_pcm_hw_param_name(var));
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1655)
[declaration] snd_pcm_hw_param_dump(fail, var, out);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1656)
[declaration] snd_output_putc(out, '\n');
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1676)
[declaration] snd_output_printf(out, "%s is empty\n", snd_pcm_hw_param_name(var));
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1684)
[declaration] snd_output_printf(out, "Configuration is virtually correct\n");
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1688)
[declaration] snd_pcm_hw_params_any(pcm, &any);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1691)
[declaration] return snd_pcm_hw_params_try_explain_failure1(pcm, fail, success,
depth, out);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1713)
[declaration] hw_param_interval_c(params, rule->deps[1]), &t);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1714)
[declaration] return snd_interval_refine(hw_param_interval(params, rule->var), &t);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1722)
[declaration] hw_param_interval_c(params, rule->deps[1]), &t);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1723)
[declaration] return snd_interval_refine(hw_param_interval(params, rule->var), &t);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1733)
[declaration] return snd_interval_refine(hw_param_interval(params, rule->var), &t);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1742)
[declaration] hw_param_interval_c(params, rule->deps[1]), &t);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1743)
[declaration] return snd_interval_refine(hw_param_interval(params, rule->var), &t);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1760)
[function] if (!snd_interval_test(i, (unsigned int) bits)) {
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 1761)
[declaration] snd_pcm_format_mask_reset(mask, k);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 2133)
[function] if (vstamps[r->deps[d]] > rstamps[k]) {
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 2141)
[declaration] snd_output_printf(log, "Rule %d (%p): ", k, r->func);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 2142)
[function] if (r->var >= 0) {
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 2143)
[declaration] snd_output_printf(log, "%s=", snd_pcm_hw_param_name(r->var));
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 2144)
[declaration] snd_pcm_hw_param_dump(params, r->var, log);
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 2145)
[declaration] snd_output_puts(log, " -> ");
dirtycode/alsa-lib/src/pcm/pcm_params.c (line 2151)
[declaration] snd_pcm_hw_param_dump(params, r->var, log);

```

(List truncated to 4000 entries to keep the document size manageable. Use --max-items-per-section to change this.)